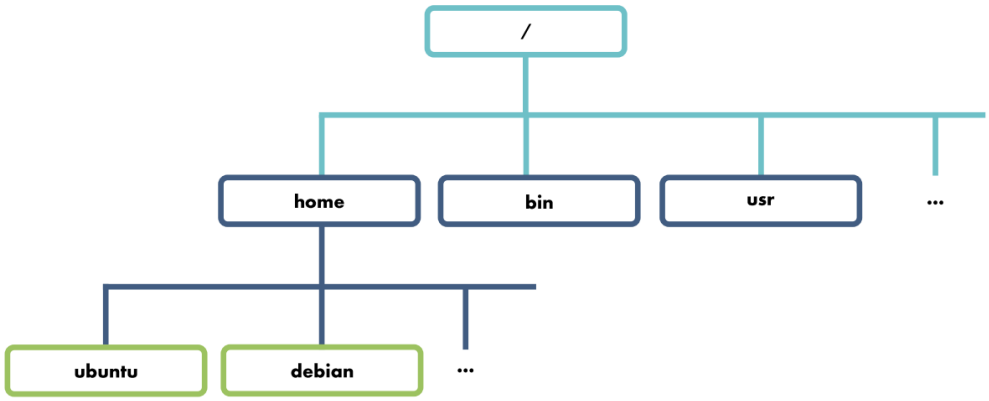
Laborator 02 - Lucrul cu fișiere (partea 1)

Căi în ierarhia de fișiere

Atunci când folosim un terminal este important să știm unde ne aflăm în ierarhie. Dacă vrem să jucăm un joc, știm că vrem să ajungem în directorul `Games`, unde avem jocul preferat, `Warcraft3`. Pentru a ajunge în directorul `Games`, trebuie să știm unde suntem și apoi să navigăm în sistemul de fișiere până acolo.

O ierarhie de fișiere și directoare este o structură de organizare a acestora. Structura ierarhică permite gruparea acestora în directoare, ce conțin la rândul lor alte directoare sau fișiere. În acest fel, putem lucra doar cu un subset de fișiere și directoare din totalitatea acestora de pe sistem.

Un astfel de exemplu este următorul:



În Linux, începutul acestei ierarhii este `/`, directorul rădăcină al sistemului de fișiere (*root directory*). Dacă privim sistemul de fișiere ca un arbore întors, directorul rădăcină este chiar rădăcina acestui arbore. De acolo cresc ramurile și frunzele sistemului de fișiere, formând ceea ce vom numi **căi** (*paths*).

Putem vedea mai jos începutul ierarhiei din Linux:

```
student@uso:/$ tree -F -L 1 /
/
|-- bin/
|-- boot/
|-- dev/
|-- etc/
|-- home/
|-- initrd.img -> /boot/initrd.img-3.16.0-4-586
|-- lib/
|-- lib64/
|-- libx32/
|-- lost+found/
|-- media/
|-- mnt/
|-- opt/
|-- proc/
|-- root/
|-- run/
|-- sbin/
|-- srv/
|-- sys/
|-- tmp/
|-- usr/
|-- var/
``-- vmlinuz -> boot/vmlinuz-3.16.0-4-586
```

Observăm `/` (directorul rădăcină) la începutul ierarhiei. Pe nivelul următor, adică ramuri / copii ai directorului rădăcină, sunt directoarele `bin`, `boot`, `dev` etc.

Observăm conținutul directorul `home`:

```
student@uso:/home$ tree -F -L 1 /home
.
``-- student/
```

Există un singur director, numit `student`.

Observăm cum se schimbă promptul de terminal:

- în primul exemplu `student@uso:/$ tree -F -L 1 /` eram în directorul `/`.
- în al doilea exemplu `student@uso:/home$ tree -F -L 1` eram în directorul `/home`.

În continuare detaliem conceptul de cale.

Calea curentă

Pornim de la următoarea analogie: Avem patru puncte pe o dreaptă: A, B, C și D. Pentru a ajunge de la A la D, trebuie să trecem prin B și C. Astfel, avem următoarele segmente: AB, BC, CD.

Similar, pentru sistemul de fișiere, pornim de la directorul rădăcină (`/`) și coborâm pe arbore.

Lista de directoare intermediare de la rădăcina ierarhiei de fișiere (`/`) până la un anumit fișier sau director se numeste **cale**.

Calea curentă este pozitia noastră în sistemul de fișiere.

În lina de comandă afișăm calea până la directorul în care ne aflăm (*calea curentă*) folosind comanda `pwd` (*print working directory*) :

```
student@uso:~$ pwd
/home/student
```

Aceasta afișează poziția noastră în sistemul de fișiere.

Ne mutăm la calea `/usr/bin` folosind comanda `cd` (*change directory*):

```
student@uso:~$ pwd
/home/student
student@uso:~$ cd /usr/bin/
student@uso:/usr/bin$ pwd
/usr/bin
```

Acum ne aflăm la poziția `/usr/bin` în sistemul de fișiere. Observăm cum calea curentă se schimbă atunci când schimbăm directorul în care ne aflăm. Vom vorbi despre comanda `cd` în laboratorul *Îmbunătățirea lucrului în linia de comandă*.

Cale relativă și cale absolută

Atunci când vrem să citim, scriem, ștergem un fișier sau director trebuie să ne referim la acel fișier sau director. Practic, *orice operație în sistemul de fișiere (arhivare, criptare, ștergere, editare) folosește căi*. Este impropriu să spunem "deschide fișierul `/home/student/a.txt`"; tehnic corect este "deschide fișierul de la calea `/home/student/a.txt`".

În sistemul de fișiere, ne putem referi la un fișier prin 2 modalități: folosind *calea relativă* sau folosind *calea absolută*.

Spre exemplu, luăm un serial pe Netflix. Ne referim la un episod în două moduri:

- Față de început - primul episod (în mod absolut): *În episodul 213 s-a întâmplat ceva.*
- Față de episodul la care ne aflăm (în mod relativ): *În urmă cu 3 episoade s-a întâmplat ceva.*

Similar, ne referim la un fișier:

- Pornind din rădăcina ierarhiei de fișiere (*root*), reprezentat prin simbolul /, numită *cale absolută*.
- Pornind de la directorul în care ne aflăm (*director curent*), numită *cale relativă*.

Cale absolută

După cum am precizat mai sus, calea absolută începe întotdeauna cu caracterul / (slash), denumit și *director rădăcină*. Acesta indică începutul ierarhiei de fișiere.

Afișam fișierele `/usr/bin/touch` și `/etc/passwd` prin cale absolută, folosind `ls`:

```
student@uso:~$ ls -l /usr/bin/touch
lrwxrwxrwx 1 root root 10 Mar 14  2015 /usr/bin/touch -> /bin/touch
student@uso:~$ ls -l /etc/passwd
-rw-r--r-- 1 root root 1768 Sep  9 14:07 /etc/passwd
```

Am folosit opțiunea *-l* a utilitarului `ls` pentru a afișa informații suplimentare ale fisierelor.

Observăm că la începutul căilor este întotdeauna caracterul **'/'** (directorul rădăcină).

Cale relativă

Prin calea relativă ne referim la un fișier în funcție de calea curentă.

```
student@uso:~$ ls -l uso.git/lab02/
total 12
drwxr-xr-x 5 student student 4096 aug 20  2018 anul_1
drwxr-xr-x 2 student student 4096 aug 20  2018 anul_2
drwxr-xr-x 4 student student 4096 aug 20  2018 demo
```

În exemplul de mai sus, ne aflăm la calea `/home/student` și am coborât în jos în ierarhia de fișiere către directorul `uso.git`, apoi directorul `lab02`.

Același rezultat îl obținem folosind calea absolută:

```
student@uso:~$ ls -l /home/student/uso.git/lab02/
total 12
drwxr-xr-x 5 student student 4096 aug 20  2018 anul_1
drwxr-xr-x 2 student student 4096 aug 20  2018 anul_2
drwxr-xr-x 4 student student 4096 aug 20  2018 demo
```

Calea `/home/student/uso.git/lab02/` este validă oricare ar fi calea curentă (oriunde ne-am afla în ierarhia de fișiere).

În acest caz, este mai eficient să folosim *calea relativă*. În cazul în care putem ajunge mai repede de la directorul rădăcina / la directorul țintă, este mai utilă *calea absolută*.

Intrarea '..' din sistemul de fișiere

Referim aceleași fișiere (`/usr/bin/touch` și `/etc/passwd`) prin cale relativă, folosind `ls`:

```
student@uso:~$ pwd
/home/student
student@uso:~$ ls -l ../../usr/bin/touch
lrwxrwxrwx 1 root root 10 Mar 14  2015 ../../usr/bin/touch -> /bin/touch
student@uso:~$ ls -l ../../etc/passwd
-rw-r--r-- 1 root root 1768 Sep  9 14:07 ../../etc/passwd
```

Observăm că ne aflăm la calea `/home/student` și urcăm spre vârful ierarhiei de fișiere folosind caracterele `..`.

Odată ajunși la *directorul rădăcină* /, coborâm spre fișierul `touch`, trecând prin directoarele `usr` și `bin`.

Este bine să ne referim la `..` ca *punct punct*, nu două puncte *:* pentru a elimina confuziile.

Exerciții

- Afișați calea curentă folosind utilitarul `pwd`.
- Referiți fișirele/directoarele `/lib/`, `/home`, `/bin/cat` atât prin cale relativă cât și prin cale absolută.
- Referiți directorul părinte (un nivel mai sus față de cel curent).

Schimbarea căii curente

Plasarea noastră într-un alt director decât cel curent înseamnă schimbarea căii curente. Facem acest lucru folosind comanda `cd` (*change directory*):

```
student@uso:/etc$ pwd
/etc
student@uso:/etc$ cd /usr
student@uso:/usr$ pwd
/usr
```

Am schimbat calea curentă de la `/etc` la `/usr`. Această comandă poate fi folosită oriunde ne-am afla deoarece este o *cale absolută*.

În general, comanda `cd` schimbă promptul terminalului; aceasta este o indicație a directorului curent în care ne aflăm:

- în cazul `student@uso:/etc$ pwd` ne aflăm în `etc`
- în cazul `student@uso:/usr$ pwd` ne aflăm în `usr`

La fel ca toate comenzile de lucru cu sistemul de fișiere, comanda `cd` primește ca parametru o cale. Aceasta poate fi atât *relativă* cât și *absolută*.

Schimbăm calea curentă în `/home/student`, folosind `cd`:

```
student@uso:/usr$ cd /home/student
student@uso:~$ pwd
/home/student
```

Fiind la calea `/usr`, am folosit utilitarul `cd` împreună cu o cale *absolută* `/home/student` pentru a schimba calea curentă. Cea de-a doua comanda `pwd` confirmă că am schimbat calea curentă.

Acum ne aflăm la calea `/home/student`. O alternativă mai puțin eficientă este să facem schimbarea căii curente în doi pași, astfel:

```
student@uso:~$ cd /home
student@uso:/home$ pwd
/home
student@uso:/home$ cd student/
student@uso:~$ pwd
/home/student
```

Schimbăm calea curentă la `/usr/bin` folosind cale relativă:

```
student@uso:~$ pwd
/home/student
student@uso:~$ cd ../../usr/bin
student@uso:/usr/bin$ pwd
/usr/bin
```

Una dintre cele mai importate informații când lucrăm în sistemul de operare (în special în linia de comandă) este unde ne aflăm. Dacă știm unde ne aflăm, ne putem orienta către destinație. Știm unde ne aflăm folosind comanda `pwd`; ne deplasăm în altă parte folosind comanda `cd`.

Observăm cum am urcat în ierarhie folosind `..` până la `/` (*directorul rădăcină*) iar apoi am coborât spre `usr` și apoi `bin`.

În exemplul de mai jos, comanda de mai sus nu este valabilă:

```
student@uso:~/Desktop$ pwd
/home/student/Desktop
student@uso:~/Desktop$ cd ../../usr/bin
bash: cd: ../../usr/bin: No such file or directory
```

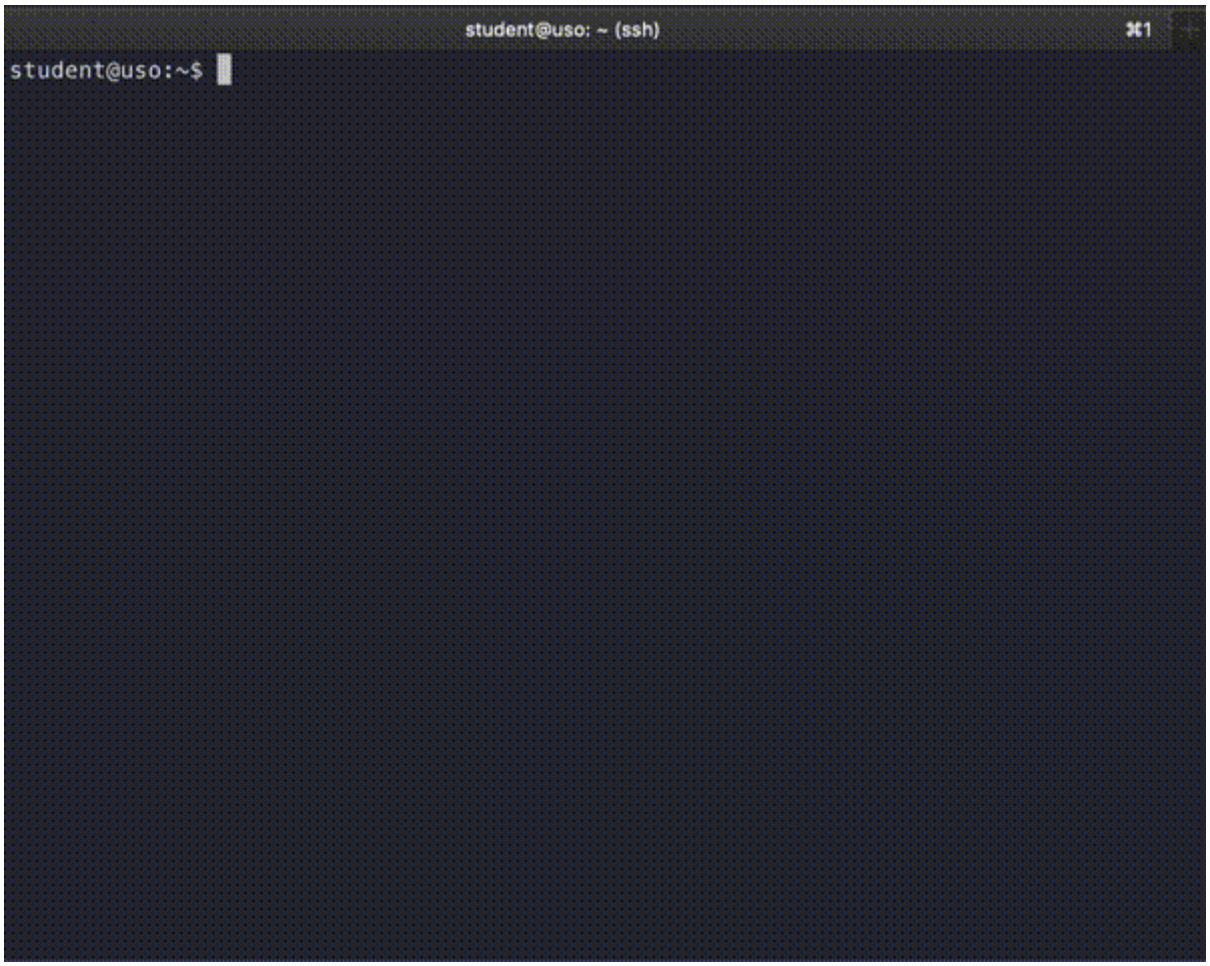
Observăm că suntem la calea `/home/student/Desktop`. Executând comanda `cd ../../usr/bin` de mai sus, am ajunge la calea `/home/usr/bin` care nu există.

Exerciții - schimbarea directorului curent

- Schimbați directorul curent cu directorul de la calea `/usr/bin`. Folosiți cale absolută.
- Schimbați directorul curent cu directorul de la calea `/etc/`. Folosiți cale relativă.
- Schimbați directorul curent cu directorul de la calea `/etc/network/`. Folosiți cale relativă.

TAB completion

Funcția de *tab completion* este probabil una dintre cele mai utile funcții expuse de către shell. Prin simpla apăsare a tastei `Tab` în timp ce scriem numele unei comenzi, al unei opțiuni a unei comenzi sau calea către un director sau fișier, shellul va completa în mod automat textul sau va sugera opțiuni de auto-complete.



Observăm că atunci când calea pe care o scriem se termină cu / (slash), apăsând de 2 ori pe tasta `Tab` ne arată fișierele și directoarele din calea respectivă. Atunci vrem să ne auto-completeze numele, apăsăm o singură dată pe tasta `Tab`.

Tasta `Tab` este descrisă în detaliu în laboratorul *Îmbunătățirea lucrului în linia de comandă*.

Scurtături de căi

Pentru o eficiență mai bună, putem folosi simboluri pentru interacțiunea cu utilitarul `cd`.

Folosim caracterul (~) pentru a referi *directorul home* a utilizatorului.

```
student@uso:/usr/bin$ cd /tmp/
student@uso:/tmp$ pwd
/tmp
student@uso:/tmp$ cd ~
student@uso:~$ pwd
/home/student
student@uso:~$ cd /usr/bin/X11/
student@uso:/usr/bin/X11$ pwd
/usr/bin/X11
student@uso:/usr/bin/X11$ cd ~
student@uso:~$ pwd
/home/student
```

Atunci când suntem în directorul home, promptul conține caracterul ~ pentru a indica acest lucru.

Interacțiunea cu fișiere în linia de comandă

Fișierul este unitatea de bază folosită pentru a stoca informații. Fie că este vorba despre un document (tema la mate) sau un joc, fie că este vorba de o configurare în sistemul de operare, sau chiar sistemul de operare, folosim fișiere pentru stocare.

Putem interacționa cu fișiere folosind programe specifice:

- Microsoft Word [https://www.microsoft.com/download/office.aspx] pentru documente;
- browser (de exemplu, Firefox [https://www.mozilla.org/ro/firefox/new/]) pentru o pagină web;
- player audio (de exemplu, Winamp) pentu melodii;
- player video (de exemplu, vlc [https://www.videolan.org/vlc/]) pentru filme.

Acestea au în comun o interfață grafică. O altă opțiune este să interacționăm cu fișierele în *linia de comandă*. Acesta este subiectul acestei secțiuni.

Folosim linia de comandă pentru interacțiunea cu fișierele în mai multe cazuri:

- Atunci când sistemul pe care ne aflăm are **doar** linie de comandă;
- Atunci când putem să facem anumite operații **mai repede** decât în interfața grafică; putem folosi automatizarea unor sarcini (taskuri): scriem un script (un fișier de automatizare) o dată și îl executăm de fiecare dată când este nevoie. Există un capitol întreg *Îmbunătățirea lucrului în linia de comandă* unde vom afla mai multe detalii.

Un motiv în plus pentru a folosi linia de comandă este că folosește mult mai puține resurse decât interfața grafică. Astfel, putem să facem mai multe cu mai puține resurse.

Componenta ce se ocupă de interpretarea comenzilor se numește *shell*. Shellul citește comenzile date de utilizator, le interpretează și comunică sistemului de operare ce are de făcut.

În acest capitol discutăm despre cum interacționăm cu fișierele în linia de comandă.

Pentru exercițiile din acest capitol, asigurați-vă că sunteți în directorul `/home/student/uso_lab02`. Pentru a face acest lucru, **rulati urmatoarele comenzi**:

```
student@uso:~$ mkdir /home/student/uso_lab02/
student@uso:~$ cd /home/student/uso_lab02/
student@uso:~/uso_lab02$ pwd
/home/student/uso_lab02
```

Lucrul cu fișiere și directoare

În această secțiune vom învăța să lucrăm cu fișierele și directoarele în linia de comandă:

- afișarea conținutului fișierelor și directoarelor;
- crearea de fișiere și directoare;
- ștergerea de fișiere și directoare;
- redenumirea fișierelor și directoarelor;
- fișiere și directoare ascunse;
- legături (linkuri);
- execuția programelor.

Afișarea conținutului unui director

Așa cum un dosar are mai multe foi în el, la fel și directoarele au mai multe fișiere și directoare în ele.

Pentru a afișa conținutul unui director folosim comanda `ls`:

```
student@uso:~$ ls
Desktop    Downloads  Pictures  Templates  examples.desktop  vm-actions-log.txt
Documents  Music      Public    Videos     uso.git
```

Vedem mai multe detalii despre fișierele și directoarele din directorul `/home/student` (adică directorul **home** al utilizatorului **student**, **~**) folosind comanda `ls -l`:

```
student@uso:~$ student@uso:~$ ls -l
total 56
drwxr-xr-x  2 student student 4096 aug  6  2018 Desktop
drwxr-xr-x  3 student student 4096 aug 20  2018 Documents
drwxr-xr-x  2 student student 4096 aug  6  2018 Downloads
drwxr-xr-x  2 student student 4096 aug  6  2018 Music
drwxr-xr-x  2 student student 4096 aug  6  2018 Pictures
drwxr-xr-x  2 student student 4096 aug  6  2018 Public
drwxr-xr-x  2 student student 4096 aug  6  2018 Templates
drwxr-xr-x  2 student student 4096 aug  6  2018 Videos
-rw-r--r--  1 student student 8980 aug  6  2018 examples.desktop
drwxr-xr-x 14 student student 4096 aug 20  2018 uso.git
-rw-r--r--  1 student student 4827 aug 21  2018 vm-actions-log.txt
```

Interpretăm rezultatele de mai sus:

1. Primul caracter `-` ne arată că `examples.desktop` este un fișier obișnuit. Primul caracter `d` ne arată că `Desktop` este director.
2. Șirurile `rw-r--r--` (pentru `examples.desktop`) și `rwxr-xr-x` (pentru `Desktop`) se referă la permisiuni de acces. Vorbim despre permisiuni de acces în capitolul *Îmbunătățirea lucrului în linia de comandă*.
3. Următorul număr îl ignorăm pentru moment.
4. Următoarele două cuvinte, `student`, se referă tot la permisiuni și ownership; le vom discuta într-un capitol ulterior.
5. Numărul `8980` arată dimensiunea fișierului `examples.desktop`. Observăm că directorul `Desktop` are dimensiunea de 4096 octeți (*bytes*);
6. Urmează 3 coloane ce arată momentul ultimei modificări. O accesare poate înseamna creare sau modificare. Citirea nu schimbă această dată.
7. La final este afișat numele fișierului sau a directorului.

Exerciții - afișarea conținutului directoarelor

- Afișați conținutul directoarelor **/usr**, **/tmp**, **/etc**, **/home**.

Crearea fișierelor

Există situații când vrem să creăm fișiere. De exemplu vrem să scriem cod sau să luăm notițe la un curs.

Fișierele se creează folosind utilitarul `touch`.

Creăm un fișier astfel:

```
student@uso:~$ cd /home/student/uso_lab02/
student@uso:~/uso_lab02$ pwd
/home/student/uso_lab02
student@uso:~/uso_lab02$ ls cities
ls: cannot access 'cities': No such file or directory
student@uso:~/uso_lab02$ touch cities
student@uso:~/uso_lab02$ ls cities
cities
```

Dacă vrem să creăm un fișier cu numele `orașe romania`, adică vrem ca numele fișierului să conțină caracterul *spațiu (space)*, folosim tot comanda `touch` ca în exemplul de mai jos:

```
student@uso:~/uso_lab02$ touch cities romania
student@uso:~/uso_lab02$ ls -l
total 0
-rw-rw-r-- 1 student student 0 oct 12 01:05 cities
-rw-rw-r-- 1 student student 0 oct 12 01:05 romania
```

Observăm că în comanda de mai sus am încercat să creăm un fișier numit `cities romania`, însă au fost create două fișiere. Pentru a crea un fișier cu *spații (space)* inclus în nume este nevoie să folosim ghilimelele la începutul și sfârșitul numelui astfel: `touch "cities romania"`.

```
student@uso:~/uso_lab02$ touch "cities in romania"
student@uso:~/uso_lab02$ ls -l
total 0
-rw-rw-r-- 1 student student 0 oct 12 01:05  cities
-rw-rw-r-- 1 student student 0 oct 12 01:05 'cities in romania'
-rw-rw-r-- 1 student student 0 oct 12 01:05  romania
```

Acum avem în directorul `~/uso_lab02` 3 fișiere în plus: `cities`, `cities in romania`, `romania`. Zero-ul *(0)* din linia `-rw-r--r-- 1 student student 0 Oct 13 13:02 cities` ne arată că fișierul cities are 0 bytes, adică este gol. Toate cele 3 fișiere sunt goale.

Crearea directoarelor

Este util să creăm directoarele atunci când vrem să păstrăm fișierele din sistem într-un mod organizat. De exemplu, putem crea câte un director pentru fiecare an de licență. Pentru fiecare an/director, cream câte un director pentru fiecare materie si pentru fiecare materie putem să facem directoare pentru teme, laboratoare, cursuri etc.

Creăm directoarele folosind utilitarul `mkdir`. Etimologia comenzii provine din limba engleză - *make directory*.

Creăm un directoarele `GameOfThrones` și `Avengers` în directorul `~` ca în exemplul de mai jos:

```
student@uso:~$ cd ~/uso_lab02/
student@uso:~/uso_lab02$ mkdir GameOfThrones
student@uso:~/uso_lab02$ mkdir Avengers
student@uso:~/uso_lab02$ ls -l
total 8
drwxrwxr-x 2 student student 4096 oct 12 01:07  Avengers
drwxrwxr-x 2 student student 4096 oct 12 01:07  GameOfThrones
(...)
```

Am creat 2 directoare noi: `Avengers` și `GameOfThrones`.

Exerciții - creare fișiere și directoare

Pentru următoarele exerciții, asigurați-vă că sunteți în directorul `/home/student/uso_lab02` folosind comanda:

```
student@uso:~$ cd /home/student/uso_lab02/
student@uso:~/uso_lab02$ pwd
/home/student/uso_lab02
```

- Creați un director numit "WinnieThePooh" și unul numit "TomSiJerry".
- Creați fișierele "Iron Man", "Hulk", "Thor", "Captain America" (**cu spațiu**) în directorul "Avengers"; afișați fișierele din director;
- Creați fișierele "Arya", "Daenerys Targaryen" (**cu spațiu**), "Jon Snow", "Tyrion Lannister" în directorul "GameOfThrones"; afișați fișierele din director.

Este important de reținut: orice comandă executați, aceasta trebuie succedată de o verificare. De exemplu, la crearea unui fișier (`touch`) sau director (`mkdir`), executăm comanda de verificare `ls`.

Afișarea conținutului unui fișier

Afișăm rapid conținutul fișierului `Arya` din directorul `GameOfThrones` folosind comanda `cat`:

```
student@uso:~/uso_lab02$ cat GameOfThrones/Arya
student@uso:~/uso_lab02$
```

Comanda `cat` nu a afișat nimic pentru că fișierul `Arya` este gol.

Scriem textul `A girl has no name` în fișierul `Arya` folosind comanda `echo`:

```
student@uso:~/uso_lab02$ echo "A girl has no name" > "GameOfThrones/Arya"
student@uso:~/uso_lab02$ cat "GameOfThrones/Arya"
A girl has no name
```

O altă metodă este de a folosi un editor de text: **vim**, **gedit**, **emacs**, **nano**, **sublime**, etc. Vom detalia utilizarea unui editor de text în subsectiunea *Editor de text*.

Ștergerea fișierelor

Ștergem fișierul "Hulk" din directorul "Avengers" folosind utilitarul `rm`:

```
student@uso:~/uso_lab02$ ls Avengers/
Captain America  Hulk  Iron Man  Thor
student@uso:~/uso_lab02$ rm Avengers/Hulk
student@uso:~/uso_lab02$ ls Avengers/
Captain America  Iron Man  Thor
```

La prima comandă am afișat ce fișiere sunt în directorul `Avengers`. Am folosit utiltarul `rm` pentru a șterge fișierul `Hulk` din directorul `Avengers`. În final am verificat că fișierul `Hulk` nu mai există în directorul `Avengers`.

Ștergerea directoarelor

Directoarele se șterg folosind comanda `rmdir` sau comanda `rm -r`. Etimologia comenzii provine din engleză - *remove directory*. Incercam sa stergem directorul `Avengers` folosind ambele comenzi:

```
student@uso:~/uso_lab02$ ls -l
total 60
drwxr-xr-x 2 student student 4096 Sep 29 18:02 Avengers
(...)
drwxr-xr-x 2 student student 4096 Sep 29 17:44 GameOfThrones
(...)
student@uso:~/uso_lab02$ rmdir Avengers/
rmdir: failed to remove 'Avengers/': Directory not empty
```

Comanda `rmdir` pe un director care nu este gol (care conține cel puțin un alt fișier sau director) nu funcționează.

Pentru a șterge un director care **nu** este gol, adică un director care are cel puțin un fișier sau director în el, folosim utilitarul `rm` cu opțiunea de recursivitate `-r`: `rm -r`. Aceasta permite ștergerea întregii ierarhii de fișiere din director, pe scurt: ștergerea întregului director.

```
student@uso:~/uso_lab02$ rm -r Avengers/
student@uso:~/uso_lab02$ ls -l
total 52
(...)
drwxr-xr-x 2 student student 4096 Sep 29 17:44 GameOfThrones
(...)
```

După executarea comenzii, verificăm corectitudinea operației. Trebuie să verificăm și să nu presupunem că o comandă s-a executat. Lipsa unei verifică poate duce la erori și mult timp pierdut din partea noastră.

Redenumirea și mutarea fișierelor și directoarelor

Fișierele și directoarele se redenumesc în mod similar, folosind comanda `mv` astfel: `mv nume_actual nume_nou`.

Redenumim fișierele si directoarele folosind comanda `mv`:

```
student@uso:~/uso_lab02$ ls
GameOfThrones
student@uso:~/uso_lab02$ mv GameOfThrones ThroneOfGames
student@uso:~/uso_lab02$ ls
ThroneOfGames
```

Am redenumit fișierul `GameOfThrones` în `ThroneOfGames`.

Un alt rol al comenzii `mv` este de a muta fișierele și directoarele, în ierarhia de fișiere, dintr-un loc în altul.

Mutăm directorul `GameOfThrones` (cu tot conținutul acestuia) la calea `/tmp/` folosind comanda `mv`:

```
student@uso:~/uso_lab02$ ls
Avengers  GameOfThrones
student@uso:~/uso_lab02$ mv GameOfThrones/ /tmp/
student@uso:~/uso_lab02$ ls /tmp/
GameOfThrones  ssh-ApUMKI3HSJ
student@uso:~/uso_lab02$ ls /tmp/
GameOfThrones  ssh-ApUMKI3HSJ
student@uso:~/uso_lab02$ ls
Avengers
```

Acum directorul `GameOfThrones` se află în calea `/tmp/GameOfThrones`. Am verificat folosind `ls` că nu se mai află în directorul curent și că există în directorul `/tmp/`.

Mutăm înapoi directorul `GameOfThrones` (cu tot conținutul acestuia) folosind cale relativă, pentru a indica directorul, și cale absolută, pentru a indica locul unde vrem să ajungă directorul folosind `mv`:

```
student@uso:~/uso_lab02$ mv ../../../tmp/GameOfThrones/ /home/student/
student@uso:~/uso_lab02$ ls
Avengers  GameOfThrones
```

Am verificat că directorul `GameOfThrones` se află la calea indicată (calea curentă).

Observăm că putem folosi atât căi relative, cât și căi absolute ca argumente pentru comanda `mv` de mutare / redenumire de fișiere și directoare.

Exerciții - redenumire și mutare

1. - Creați un director `Vikings` în directorul de la calea `/home/student/uso_lab02`.
   - Creați fișierele `Ragnar`, `Rollo`, `Lagertha` în directorul `Vikings`.

- Verificați că directorul și fișierele au fost create.
- Mutati directorul (împreună cu întreaga ierarhie de fișiere) la locația `/tmp/`.
- Verificați că directorul și ierarhia de fișiere au fost mutate.

2.
- Creați un director `NBA` în directorul de la calea `/home/student/uso_lab02`.
- Creați fișierele `MichaelJordan`, `LeBronJames`, `DwayneWade` și `KobeBryant` în directorul `NBA`.
- Verificați că directorul și fișierele au fost create.
- Mutați directorul `NBA` (împreună cu întreaga ierarhie de fișiere) în directorul `Vikings`.
- Verificați că directorul și ierarhia de fișiere au fost mutate.

3.
- Redenumiți directorul `GameOfThrones` în `ThronesInTheGame`.
- Mutați fișierele din interiorul directorului `ThronesInTheGame` în `/tmp`.
- Verificați operațiile.
- Mutați directorul înapoi la locatia inițială folosind atât căi relative cât și căi absolute.

Copierea fișierelor și directoarelor

Copiem fișierul `Thor` în directorul `/tmp/` folosind `cp`:

```
student@uso:~/uso_lab02$ cp Avengers/Thor /tmp/
student@uso:~/uso_lab02$ ls /tmp/
Thor  ssh-ApUMKI3HSJ
```

Sintaxa este similară comenzii `mv`. Acum fișierul `Thor` este atât în `/home/student/Avengers/Thor`, cât și în `/tmp/Thor`.

```
student@uso:~/uso_lab02$ cp Avengers/ /tmp/
cp: omitting directory 'Avengers/'
```

Observăm că nu se poate copia un director ce conține alte fișiere sau directoare.

Pentru a copia în altă parte un director care nu este gol, trebuie să folosim comanda `cp -r`:

```
student@uso:~/uso_lab02$ cp -r Avengers/ /tmp/
student@uso:~/uso_lab02$ ls -l /tmp/
total 8
drwxr-xr-x 2 student student 4096 Sep 29 19:06 Avengers
-rw-r--r-- 1 student student    0 Sep 29 19:04 Thor
drwx------ 2 student student 4096 Sep 29 13:45 ssh-ApUMKI3HSJ
student@uso:~/uso_lab02$ ls -l /tmp/Avengers/
total 0
-rw-r--r-- 1 student student 0 Sep 29 19:06 Captain America
-rw-r--r-- 1 student student 0 Sep 29 19:06 Iron Man
-rw-r--r-- 1 student student 0 Sep 29 19:06 Thor
```

Am copiat directorul `Avengers` și conținutul acestuia din calea curentă în directorul `/tmp`. Observăm că s-a copiat întreaga ierarhie de fișiere/directoare de sub directorul `Avengers`. Acesta se află acum în ambele locuri.

Fișiere și directoare ascunse

Un fișier sau director este ascuns atunci când nu apare în mod normal în ierarhia de fișiere. De exemplu, atunci când executăm într-un director comanda `ls`, acesta nu apare.

Există câteva motive pentru care vrem să ascundem un fișier sau director:

- Există fișiere pe care nu vrem să le vedem în mod normal (fișiere de configurare; ex: `.profile`);
- Există comenzi care șterg toate fișierele dintr-un director fără a șterge directorul (`rm Avengers/*`). Acestea nu au efect asupra fișierelor ascunse;
- Vrem să facem anumite fișiere puțin mai greu de găsit.

Vedem fișiere și directoare ascunse folosind comanda `ls -a`:

```
student@uso:~/uso_lab02$ ls -al
total 56
drwxr-xr-x 5 student student 4096 Sep 29 18:41 .
drwxr-xr-x 4 student student 4096 Sep 29 18:35 ..
drwxr-xr-x 2 student student 4096 Sep 29 18:20 Avengers
(...)
drwxr-xr-x 2 student student 4096 Sep 29 17:44 GameOfThrones
(...)
```

Opțiunea `-a` vine de la *all*, adică vrem să vedem toate fișierele: și cele vizibile și cele ascunse.

Observăm două directoare noi `.` și `...` Directorul `.` este o referință la directorul curent, iar `..` este o referință la directorul părinte.

Directorul `..` este în cazul nostru echivalent cu `/home/student`.

Creăm fișiere ascunse punând un punct *(.)* în fața numelui:

```
student@uso:~/uso_lab02/Avengers$ ls -l
total 0
-rw-r--r-- 1 student student 0 Sep 29 18:20 Captain America
-rw-r--r-- 1 student student 0 Sep 29 18:20 Iron Man
-rw-r--r-- 1 student student 0 Sep 29 18:20 Thor
student@uso:~/uso_lab02/Avengers$ touch .Hulk
student@uso:~/uso_lab02/Avengers$ ls -l
total 0
-rw-r--r-- 1 student student 0 Sep 29 18:20 Captain America
-rw-r--r-- 1 student student 0 Sep 29 18:20 Iron Man
-rw-r--r-- 1 student student 0 Sep 29 18:20 Thor
student@uso:~/uso_lab02/Avengers$ ls -al
total 8
drwxr-xr-x 2 student student 4096 Oct  6 16:44 .
drwxr-xr-x 5 student student 4096 Sep 29 19:14 ..
-rw-r--r-- 1 student student    0 Oct  6 16:44 .Hulk
-rw-r--r-- 1 student student    0 Sep 29 18:20 Captain America
-rw-r--r-- 1 student student    0 Sep 29 18:20 Iron Man
-rw-r--r-- 1 student student    0 Sep 29 18:20 Thor
```

Am creat fișierul ascuns `.Hulk`. Observăm că acesta nu apare la execuția `ls -l`, dar apare la execuția comenzii `ls -al`.

Similar, creăm directoare ascunse punând un *(.)* în fața numelui:

```
student@uso:~/uso_lab02$ mkdir .LordOfTheRings
student@uso:~/uso_lab02$ ls
Avengers  GameOfThrones
student@uso:~$ ls -al
total 20
drwxr-xr-x 5 student student 4096 Sep 29 18:41 .
drwxr-xr-x 4 student student 4096 Sep 29 18:35 ..
drwxr-xr-x 2 student student 4096 Sep 29 18:41 .LordOfTheRings
drwxr-xr-x 2 student student 4096 Sep 29 18:20 Avengers
(...)
drwxr-xr-x 2 student student 4096 Sep 29 17:44 GameOfThrones
(...)
```

Am creat directorul ascuns `.LordOfTheRings`.

Exerciții - creare fișiere/directoare ascunse

1.
   - Creați un director cu numele `.LordOfTheRings` în directorul de la calea `/home/student/uso_lab02`.
   - Intrați în directorul creat.
   - Creați 3 fișiere ascunse cu numele `Aragorn`, `Legolas`, `Frodo Baggins`.
   - Verificați operațiile de creare (director și fișiere).

2. - Creați un director cu numele `stiri` în directorul de la calea `/home/student/uso_lab02`.
   - Creați fișierele `hotnews`, `biziday`, `digi24`.
   - Creați fișierul ascuns `.cancan`.
   - Afișați **toate** fișierele din director.
   - Afișați fișierele din director care **nu** sunt ascunse.
3. - Creați un director `tv` în directorul de la calea `/home/student/uso_lab02`.
   - Creați fișierele `ProTV`, `Digi24`, `Eurosport`.
   - Creați fișierele ascunse `Antena3`, `Romania24`.
   - Afișați **toate** fișierele din director.
   - Afișați fișierele din director care **nu** sunt ascunse.
   - Copiați directorul `stiri` împreună cu fișierele de sub acesta în directorul `tv`.
   - Mutați fișierele `.Antena3` și `.Romania24` în directorul `/tmp`.

Exerciții suplimentare - ierarhii de fișiere și directoare

1. Afișați ierarhia de fișiere pornind de la directorul vostru home (`~`).
2. Creați următoarea ierarhie pornind de la directorul `/home/student/uso_lab02` (**Notă: Numele cu extensie din structura sunt <u>fișiere</u>, restul sunt <u>directoare</u>**):

```
student@uso:~$ tree
.
├── prime_video
├── Hulu
│   └── Favorite.txt
├── Netflix
│   ├── Filme
│   │   ├── filme_2020.txt
│   │   ├── filme_2019.txt
│   │   ├── filme_vechi
│   └── Seriale
│       ├── seriale_2020
│       ├── seriale_2019.txt
│       └── seriale_vechi
├── HBO_GO
│   ├── Modern_Family.txt
│   └── Harry_Potter
└── YouTubeTV
```

1. Copiați întreaga ierarhie în calea `/tmp`. Folosiți comanda `cp`. (Hint: recursivitate).
2. Confirmați copierea prin afișarea directorului părinte.
3. Ștergeți fișierul `Favorite.txt` din directorul `Hulu`. Confirmați ștergerea prin afișarea directorului părinte.
4. Ștergeți directorul `prime video`.
5. Confirmați ștergerea prin afișarea directorului părinte.
6. Ștergeți directorul `HBO GO`.
7. Găsiți parametrii corespunzători ștergerii unui director care nu este gol.
8. Confirmați ștergerea prin afișarea directorului părinte.
9. Ștergeți ierarhia de directoare începând cu directorul `Netflix`.
10. Confirmați ștergerea prin afișarea directorului părinte.
11. Creați ierarhia de directoare de mai jos.
12. Căutați parametrul necesar pentru `mkdir` pentru a crea toată ierarhia *dintr-o singură executare a comenzii*:

```
student@uso:~/uso_lab02$ tree
.
└── Cale
    └── Lungă
```

```
        └── De
            └── Directoare
```

13. Mutați ierarhia copiată anterior în `/tmp` în directorul `Directoare`.

Legături (Links)

O legătură este o scurtătură către un fișier sau un director. Acestea sunt necesare atunci când nu vrem să parcurgem toată ierarhia de fișiere. De exemplu, punem executabilul jocului *Warcraft3* pe Desktop pentru a-l accesa rapid. Mai multe legături pot referi același director/fișier.

Caracteristici:

- Similar *Shortcut* din Windows;
- Orice modificare în fisierul țintă, se vede și în fișierul legătură;
- Este o legătură către numele fișierului și nu către conținut;
- Dacă fișierul este mutat sau redenumit, legătura se pierde;
- Putem crea legături către directoare.

Creăm o legătură simbolică către directorul `/home/student/uso_lab02/Avengers/Thor` folosind comanda `ln -s`:

```
student@uso:~/uso_lab02$ ln -s /home/student/uso_lab02/Avengers/Thor thor
student@uso:~/uso_lab02$ ls -l
(...)
lrwxrwxrwx 1 student student 28 Oct  6 17:58 thor -> /home/student/uso_lab02/Avengers/Thor
(...)
```

Observăm că fișierul creat este *link* (are primul caracter `l`). Mai mult, observăm că acest link este o scurtătură către calea `/home/student/uso_lab02/Avengers/Thor`.

Scriem textul "Hello" în fișierul `Avengers/Thor`:

```
student@uso:~/uso_lab02$ echo "Hello" > Avengers/Thor
student@uso:~/uso_lab02$ cat Avengers/Thor
Hello
student@uso:~/uso_lab02$ cat thor
Hello
```

Vedem conținutul fișierului atât folosind o cale (relativă sau absolută), cât și folosind legătura simbolică (shortcutul).

Vedem în continuare ce se întâmplă dacă ștergem un fișier.

```
student@uso:~$ cd /tmp
student@uso:/tmp$ touch /home/student/uso_lab02/test
student@uso:/tmp$ ls -l /home/student/uso_lab02
(...)
drwxr-xr-x 2 student student 4096 Sep 29 18:20 Avengers
drwxr-xr-x 2 student student 4096 Sep 29 17:44 GameOfThrones
(...)
-rw-r--r-- 1 student student    0 Sep 29 19:04 test
(...)
student@uso:/tmp$ ln -s /home/student/uso_lab02/test my_link
student@uso:/tmp$ ls -l
(...)
lrwxrwxrwx 1 student student    28 oct 12 01:33 my_link -> /home/student/uso_lab02/test
(...)
student@uso:/tmp$ rm my_link
student@uso:/tmp$ ls
student@uso:/tmp$ ls -l /home/student/uso_lab02
(...)
drwxr-xr-x 2 student student 4096 Sep 29 18:20 Avengers
drwxr-xr-x 2 student student 4096 Sep 29 17:44 GameOfThrones
(...)
```

Observăm că dacă am șters un fișier accesându-l prin legătura, s-a șters și la destinație.

Exerciții - legături (linkuri)

- Creați o legătură simbolică în directorul de la calea `/home/student/uso_lab02` către directorul `Netflix`.
- Verificați crearea legăturii prin afișarea conținutului directorului părinte.
- Afișați conținutul legăturii.
- Ștergeți directorul `Netflix`.
- Afișați conținutul legăturii.

Execuția programelor

Execuția unui fișier/program se face folosind calea către fișier în locul unei comenzi pe care am rula-o în mod normal. Putem rula un executabil folosind o care relativă cum ar fi `./executabil` sau `./director/executabil`, ori folosind o cale absoluta cum ar fi `/usr/bin/firefox`.

Creăm un executabil (adică un program) numit `my_ls` care face același lucru ca și comanda `ls`:

```
student@uso:~$ cd /home/student/uso_lab02
student@uso:~/uso_lab02$ cp /bin/ls my_ls
```

Executăm executabilul creat de noi ca mai jos:

```
student@uso:~/uso_lab02$ ./my_ls
(...) my_ls (...)
```

Putem folosi atât calea relativă cât și absolută:

```
student@uso:~/uso_lab02$ ls -l
(...)
-rwxr-xr-x 1 student student 133792 Oct 13 14:01  my_ls
-rw-rw-r-- 1 student student      0 Oct 13 13:05  romania
(...)
student@uso:~/uso_lab02$ ./my_ls
GameOfThrones  cities    my_ls  'cities in romania'   romania
(...)
student@uso:~$ /home/student/my_ls
GameOfThrones  cities    my_ls  'cities in romania'   romania
(...)
```

Observăm cum se rulează un executabil atât cu cale relativă (în primul caz) cât și cu cale absolută (în al doilea caz).

Editor de text

O acțiune foarte des întâlnită este modificarea unor fișiere. Fie că scriem în jurnal, facem un TODO list sau scriem cod, trebuie să scriem într-un fișier. Pentru a modifica un fișier, folosim un program numit **editor de text**. Există două tipuri de editoare de text: **editoare în mod grafic** sau **editoare în linie de comandă**.

Nu există un editor bun sau prost. Există un editor potrivit sau nepotrivit.

Editor în linie de comandă

Editoarele în linia de comandă sunt făcute pentru interacțiunea cu terminalul. Acestea asigură funcționalitați restrânse de formatare în comparație cu editoarele grafice. Nu există butoane fizice; de obicei comenzile se dau prin combinații de taste.

Un alt caz în care folosim editoarele în linie de comandă este lucrul la distanță. Atunci când ne conectăm remote la un server și nu avem interfață grafică, utilizarea unui editor în linie de comandă

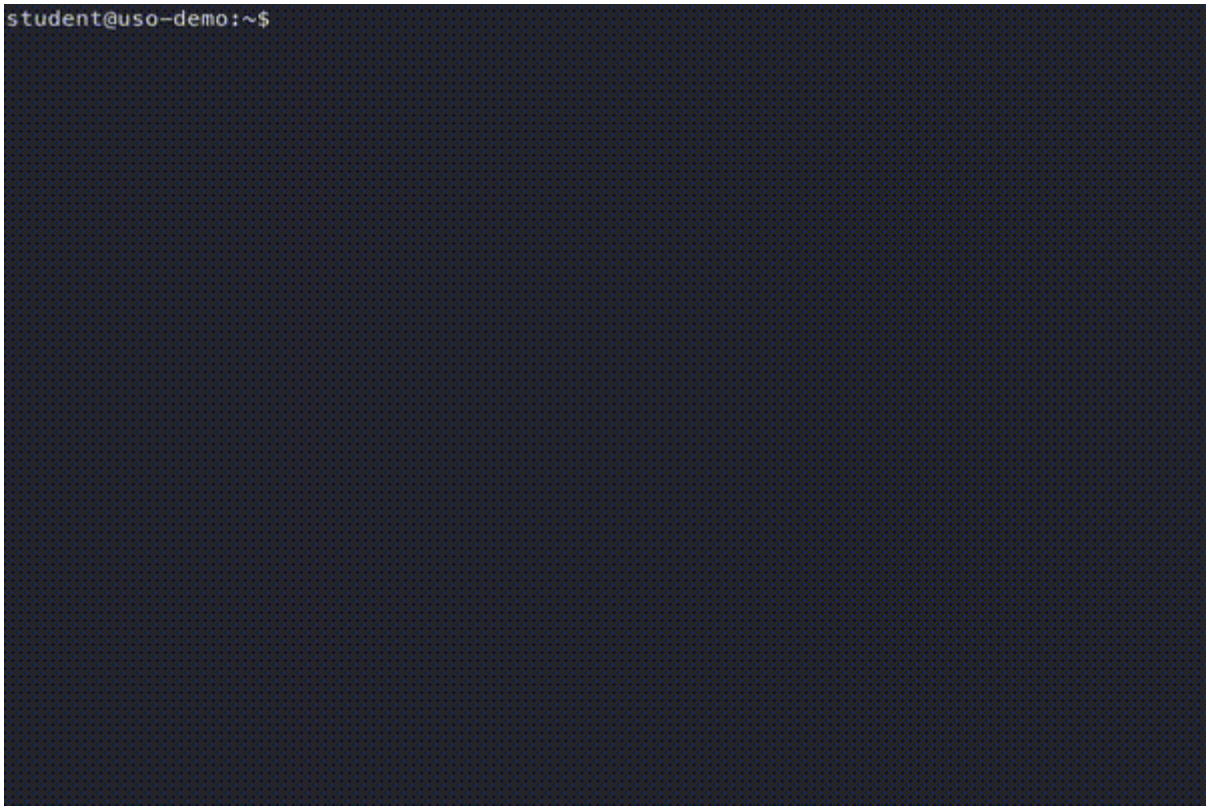
este necesară.

Există editoare mai puternice vim [https://www.vim.org/], emacs [https://www.gnu.org/software/emacs/] care permit automatizarea unor sarcini precum cele de mai sus.

Un editor în linie de comandă uzual folosit este nano [https://www.nano-editor.org/]. Acesta are funcționalități de bază și este ușor de folosit.

Interacțiunea cu fișiere cu ajutorul editoarelor în linie de comandă

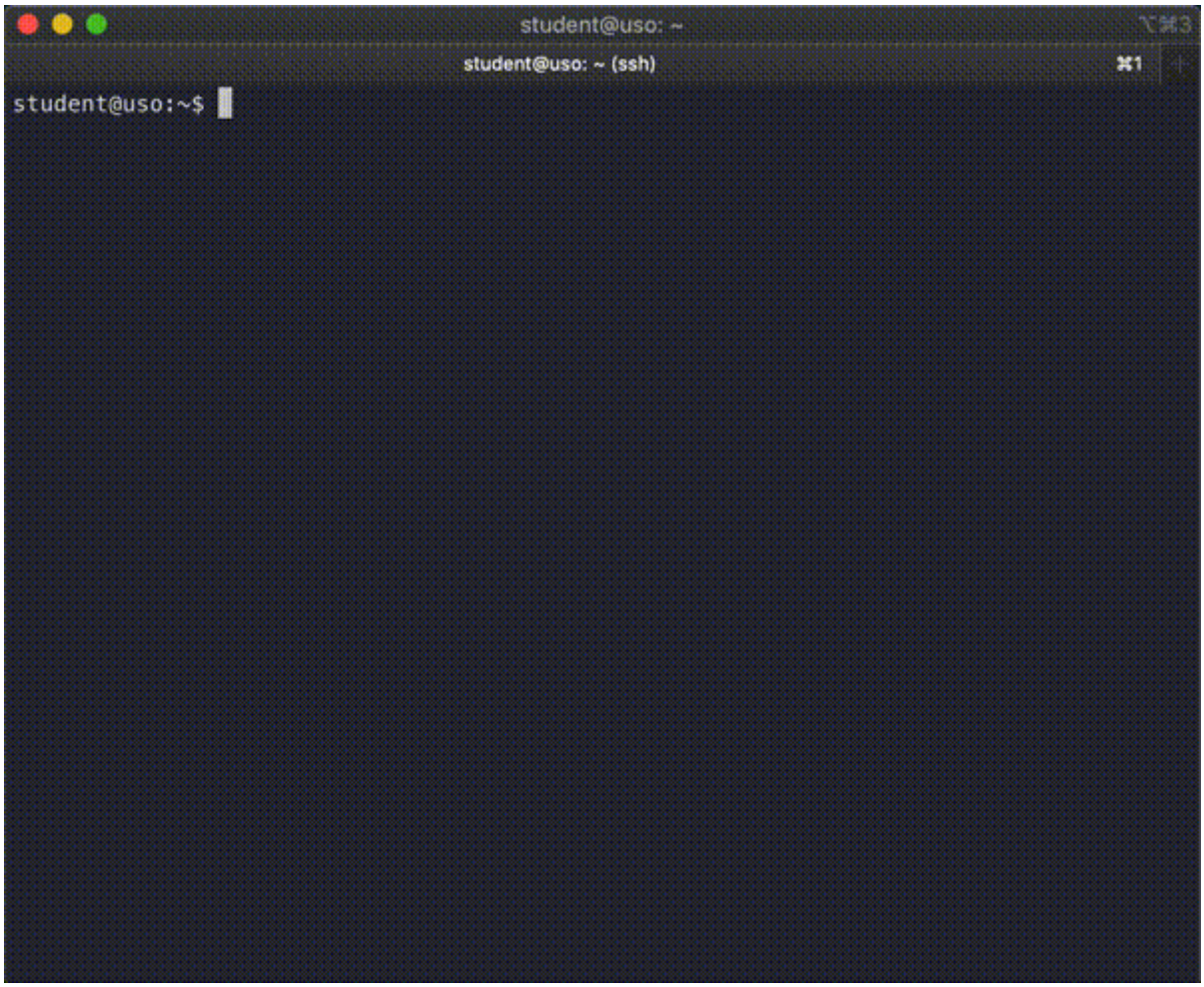
În continuare vom folosi `nano` pentru a interacționa cu un fișier.



În exemplul de mai sus am deschis fișierul `avengers` cu `nano`. În urma comenzii, s-a creat fișierul dacă acesta nu exista. Am adăugat numele a 4 supereroi și am salvat folosind combinația de taste `Ctrl+x` după care a apărut întrebarea dacă vrem să salvăm modificările. În partea de jos apare optiunea de Y es și N o. Apăsând pe tasta Y, ne cere să trecem numele fișierului, după care să confirmăm cu tasta **Enter**.

O listă de comenzi utile găsim la cheatsheet nano [https://www.nano-editor.org/dist/latest/cheatsheet.html].

În continuare vom exemplifica scurtături cu ajutorul combinațiilor de taste pentru a spori eficiența.



Exerciții - editor în linie de comandă

1. Deschideți fișierul `US Presidents` și completați fișierul cu cel puțin 3 președinți ai Statelor Unite are Americii.
2. Salvati fișierul în directorul vostru `home`.
3. Duplicați textul scris anterior (copiat/lipit) de 5 ori.
4. Indentați fiecare linie de două ori.
5. Comentați liniile 2-8.
6. Salvati fișierul în directorul vostru **home**.
7. Duplicați textul scris anterior (copiat/lipit) de 5 ori.
8. Adăugați 4 spații la început de fiecare rând.
9. Adăugați caracterul **#** la început de rând pentru liniile 2-8.
10. Scrieti cod C (similar exemplului de la editor grafic) pentru a afișa textul `Make USO Great Again!`.
11. Folositi scurtăturile pentru indentare.
12. Compilați folosind scurtătura `Ctrl+b`.

Sumar - Cheatsheet

Căi în ierarhia de fișiere

Calea curentă

```
student@uso:~$ pwd
/home/student
```

Cale relativă și cale absolută

Cale relativă:

```
student@uso:~$ ls -l snap/spotify/
total 8
drwxr-xr-x 5 student student 4096 Oct  2 03:11 42
drwxr-xr-x 3 student student 4096 Oct  2 03:10 common
lrwxrwxrwx 1 student student    2 Oct  2 03:10 current -> 42
```

Cale absolută:

```
student@uso:~$ ls -l /home/student/snap/spotify
total 8
drwxr-xr-x 5 student student 4096 Oct  2 03:11 42
drwxr-xr-x 3 student student 4096 Oct  2 03:10 common
lrwxrwxrwx 1 student student    2 Oct  2 03:10 current -> 42
```

Schimbarea căii curente

```
student@uso:/etc$ pwd
/etc
student@uso:/etc$ cd /usr
student@uso:/usr$ pwd
/usr
```

Scurtături de căi

Caracterul -:

```
student@uso:~$ pwd
/home/student
student@uso:~$ cd /usr/bin/
student@uso:/usr/bin$ pwd
/usr/bin
student@uso:/usr/bin$ cd -
/home/student
student@uso:~$ cd -
/usr/bin
student@uso:/usr/bin$ cd -
/home/student
student@uso:~$ cd -
/usr/bin
```

Caracterul ~:

```
student@uso:/usr/bin$ cd /tmp/
student@uso:/tmp$ pwd
/tmp
student@uso:/tmp$ cd ~
student@uso:~$ pwd
/home/student
student@uso:~$ cd /usr/bin/X11/
student@uso:/usr/bin/X11$ pwd
/usr/bin/X11
student@uso:/usr/bin/X11$ cd ~
student@uso:~$ pwd
/home/student
```

Interacțiunea cu fișiere în linia de comandă

Ierarhie de fișiere

```
student@uso:~$ tree -L 1 /
/
|-- bin
```

```
|-- boot
|-- dev
|-- etc
|-- home
|-- initrd.img -> /boot/initrd.img-3.16.0-4-586
|-- lib
|-- lib64
|-- libx32
|-- lost+found
|-- media
|-- mnt
|-- opt
|-- proc
|-- root
|-- run
|-- sbin
|-- srv
|-- sys
|-- tmp
|-- usr
|-- var
``-- vmlinuz -> boot/vmlinuz-3.16.0-4-586
```

Afișarea conținutului unui director

```
student@uso:~/Avengers$ ls
Captain America  Iron Man  Thor
```

```
student@uso:~$ ls -l
(...)
-rw-rw-r-- 1 student student    0 Oct 13 13:02  cities
-rw-rw-r-- 1 student student    0 Oct 13 13:03 'cities in romania'
-rw-r--r-- 1 student student 8980 Sep 24 09:00  examples.desktop
-rw-rw-r-- 1 student student    0 Oct 13 13:02  romania
drwxr-xr-x 4 student student 4096 Oct  2 03:11  snap
```

Crearea fișierelor

```
student@uso:~$ ls cities
ls: cannot access cities: No such file or directory
student@uso:~$ touch cities
student@uso:~$ ls cities
cities
```

Crearea directoarelor

```
student@uso:~$ mkdir GameOfThrones
student@uso:~$ mkdir Avengers
student@uso:~$ ls -l
(...)
drwxr-xr-x 2 student student 4096 Sep 29 17:43 Avengers
(...)
drwxr-xr-x 2 student student 4096 Sep 29 17:43 GameOfThrones
(...)
```

Afișarea conținutului unui fișier

```
student@uso:~$ cat GameOfThrones/Arya
A girl has no name
```

Ștergerea fișierelor

```
student@uso:~$ ls Avengers/
Captain America  Hulk  Iron Man  Thor
student@uso:~$ rm Avengers/Hulk
student@uso:~$ ls Avengers/
Captain America  Iron Man  Thor
```

Ștergerea directoarelor

```
student@uso:~$ mkdir LordOfTheRings
student@uso:~$ ls -l
total 12
drwxr-xr-x 2 student student 4096 Sep 29 18:02 Avengers
drwxr-xr-x 2 student student 4096 Sep 29 17:44 GameOfThrones
drwxr-xr-x 2 student student 4096 Sep 29 18:09 LordOfTheRings
student@uso:~$ rmdir "LordOfTheRings"
student@uso:~$ ls -l
total 8
drwxr-xr-x 2 student student 4096 Sep 29 18:02 Avengers
drwxr-xr-x 2 student student 4096 Sep 29 17:44 GameOfThrones
```

Redenumirea și mutarea fișierelor și directoarelor

```
student@uso:~$ ls
GameOfThrones
student@uso:~$ mv GameOfThrones ThroneOfGames
student@uso:~$ ls
ThroneOfGames
```

Copierea fișierelor și directoarelor

```
student@uso:~$ cp Avengers/Thor /tmp/
student@uso:~$ ls /tmp/
Thor  ssh-ApUMKI3HSJ
```

Fișiere și directoare ascunse

```
student@uso:~$ ls -al
total 8
drwxr-xr-x 5 student student 4096 Sep 29 18:41 .
drwxr-xr-x 4 student student 4096 Sep 29 18:35 ..
drwxr-xr-x 2 student student 4096 Sep 29 18:20 Avengers
drwxr-xr-x 2 student student 4096 Sep 29 17:44 GameOfThrones
```

```
student@uso:~/Avengers$ ls -l
total 0
-rw-r--r-- 1 student student 0 Sep 29 18:20 Captain America
-rw-r--r-- 1 student student 0 Sep 29 18:20 Iron Man
-rw-r--r-- 1 student student 0 Sep 29 18:20 Thor
student@uso:~/Avengers$ touch .Hulk
student@uso:~/Avengers$ ls -l
total 0
-rw-r--r-- 1 student student 0 Sep 29 18:20 Captain America
-rw-r--r-- 1 student student 0 Sep 29 18:20 Iron Man
-rw-r--r-- 1 student student 0 Sep 29 18:20 Thor
student@uso:~/Avengers$ ls -al
total 8
drwxr-xr-x 2 student student 4096 Oct  6 16:44 .
drwxr-xr-x 5 student student 4096 Sep 29 19:14 ..
-rw-r--r-- 1 student student    0 Oct  6 16:44 .Hulk
-rw-r--r-- 1 student student    0 Sep 29 18:20 Captain America
-rw-r--r-- 1 student student    0 Sep 29 18:20 Iron Man
-rw-r--r-- 1 student student    0 Sep 29 18:20 Thor
```

Cuprins

- Laborator 02 (one page)
- Căi în ierarhia de fișiere
- Interacțiunea cu fișiere în linia de comandă
- Editor de text
- Sumar - Cheatsheet

Laborator 03 - Lucrul cu fișiere (partea a 2-a)

Pregătirea laboratorului

Pentru pregătirea suportului de laborator, rulați următoarele comenzi în ordine:

```
student@uso:~$ cd ~
student@uso:~$ wget http://elf.cs.pub.ro/uso/res/labs/lab3/lab3.zip
--2021-10-18 22:44:01--  http://elf.cs.pub.ro/uso/res/labs/lab3/lab3.zip
Resolving elf.cs.pub.ro (elf.cs.pub.ro)... 141.85.227.116
Connecting to elf.cs.pub.ro (elf.cs.pub.ro)|141.85.227.116|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 14908126 (14M) [application/zip]
Saving to: ‘lab3.zip’

lab3.zip                                            100%[=================================================================================================================>]  14,22M  7,

2021-10-18 22:44:03 (7,61 MB/s) - ‘lab3.zip’ saved [14908126/14908126]

student@uso:~$ unzip lab3.zip
Archive:  lab3.zip
   creating: archive/
  inflating: archive/inregistrare_lab03.mp4
  inflating: archive/inregistrare_lab01.mp4
  inflating: archive/inregistrare_lab04.mp4
  inflating: archive/inregistrare_lab02.mp4
   creating: redirection/
  inflating: redirection/RL
  inflating: redirection/USO
  inflating: redirection/EGC

# Când vi se cere parola pentru student, scrieți: student și apăsați enter.
# Nu va apărea pe ecran nimic în timp ce tastați.

student@uso:~$  sudo apt -y install vlc
[sudo] password for student:
```

Redirectări

Până acum am dat comenzi în terminal și am urmărit outputul (rezultatul) lor. **Outputul** (*rezultatul*) este afișat în terminal. Dacă voiam să salvăm outputul undeva (*de obicei într-un fișier*) trebuia să copiem outputul, să creăm un fișier nou sau să folosim unul deja existent, să îl deschidem, să lipim textul în el și să salvăm modificările. Trecerea prin toți acești pași durează și devenim ineficienți în lucrul în terminal.

Interpretorul de comenzi (*shellul*) are opțiunea de a direcționa afișarea rezultatului direct într-un fișier.

Rezultatul unei comenzi este format din două fluxuri de informații: informații de **ieșire** (*standard output, stdout*) și de **eroare** (*standard error, stderr*). Avem opțiunea de a direcționa (*redirecta*) doar ieșirea, doar eroarea sau ambele fluxuri într-un fișier.

Redirectări simple

Mergem în directorul `~/redirection` rulând comanda `cd ~/redirection`. Acolo avem mai multe fișiere, câte unul pentru fiecare materie. Afișăm fișierele folosind comanda `ls`:

```
student@uso:~$ cd ~/redirection
student@uso:~$ ls -lh
total 6.1M
-rw-r--r-- 1 student student 1.1M Aug 19 11:55 EGC
-rw-r--r-- 1 student student 2.1M Aug 19 11:55 RL
-rw-r--r-- 1 student student 3.1M Aug 19 11:55 USO
```

Salvăm outputul comenzii `ls -lh` de mai sus folosind caracterul `>`, astfel:

```
student@uso:~$ ls -lh > lista_materii
student@uso:~$ ls
EGC lista_materii RL USO
```

Observăm că acțiunea de mai sus a făcut două lucruri: a creat fișierul `lista_materii` și a introdus în conținutul fișierului rezultatul comenzii `ls -lh`.

Caracterul `>` așteaptă un șir de caractere după el care este numele fișierului în care vrem să scriem outputul. Șirul este, de fapt, calea către un fișier. Șirul `lista_materii` este calea relativă până la fișierul `lista_materii` care se află în directorul curent `/home/student`.

Fișierul `lista_materii` nu exista. Prin direcționarea outputului am creat fișierul `lista_materii`.

Vizualizăm conținutul fișierului `lista_materii` folosind comanda `cat`:

```
student@uso:~$ cat lista_materii
total 6.1M
-rw-r--r-- 1 student student 1.1M Aug 19 11:55 EGC
-rw-r--r-- 1 student student 224K Aug 19 12:00 lista_materii
-rw-r--r-- 1 student student 2.1M Aug 19 11:55 RL
-rw-r--r-- 1 student student 3.1M Aug 19 11:55 USO
```

Odată direcționat într-un fișier, fluxul de informații nu mai apare la ecran.

Ștergem fișierul `lista_materii` folosind comanda `rm lista_materii` Îl vom crea din nou în următoarele momente.

Folosirea caracterului `>` suprascrie conținutul fișierului:

```
student@uso:~$ rm lista_materii
student@uso:~$ cd comenzi/mkdir
student@uso:~/comenzi/mkdir$ cat lista_materii
cat: lista_materii: No such file or directory
student@uso:~$ ls -l /usr/ > lista_materii
student@uso:~$ cat lista_materii
total 72
drwxr-xr-x   2 root root  61440 Sep  9 14:07 bin
drwxr-xr-x   2 root root   4096 Aug 31  2015 games
drwxr-xr-x  34 root root  16384 Sep  9 14:06 include
drwxr-xr-x  74 root root   4096 Sep  9 14:07 lib
drwxr-xr-x   3 root root   4096 Nov 21  2015 lib32
drwxr-xr-x   2 root root   4096 Nov 21  2015 libexec
```

```
drwxr-xr-x   3 root root  12288 Nov 21  2015 libx32
drwxrwsr-x  10 root root   4096 Nov 21  2015 local
drwxr-xr-x   2 root root  12288 Sep  9 14:07 sbin
drwxr-xr-x 137 root root  12288 Sep  9 14:07 share
drwxr-xr-x   2 root root   4096 Aug 31  2015 src
student@uso:~$ echo "cats" > lista_materii
student@uso:~$ cat lista_materii
cats
```

Fișierul `lista_materii` nu era creat inițial. În urma comenzii `ls -l /usr/ > lista_materii`, fișierul `lista_materii` a fost creat și populat cu ieșirea comenzii `ls -l /usr/`. Comanda `echo "cats" > lista_materii` suprascrie conținutul inițial al fișierului, înlocuindu-l cu rezultatul comenzii `echo "cats"`.

Exerciții - redirectări simple

- Folosiți `tree` pentru a afișa ierarhia de fișiere pornind de la `/home/student`;
- Folosiți `tree` pentru afișarea aceleiași ierarhii, dar de data asta direcționați-o în fișierul `ierarhie_student`;
- Inspectați fișierul pentru a demonstra corectitudinea operației.

Redirectări prin anexare

Putem direcționa rezultatul comenzii, *fără* a suprascrie fișierul. Facem acest lucru folosim caracterele >> folosind fișierul creat anterior `lista_materii`:

```
student@uso:~$ ls >> lista_materii
student@uso:~$ ls
EGC lista_materii RL USO
student@uso:~$ cat lista_materii
total 6.1M
-rw-r--r-- 1 student student 1.1M Aug 19 11:55 EGC
-rw-r--r-- 1 student student 209K Aug 19 12:00 lista_materii
-rw-r--r-- 1 student student 2.1M Aug 19 11:55 RL
-rw-r--r-- 1 student student 3.1M Aug 19 11:55 USO
EGC
lista_materii
RL
USO
```

Acum avem în fișierul `lista_materii`, pe lângă conținutul vechi, și outputul comenzii `ls`.

Exerciții - direcționări prin anexare

1. Folosiți `tree` pentru a afișa ierarhia de fișiere pornind de la `/tmp`;
   - Folosiți `tree` pentru a afișa aceeași ierarhie, dar de data asta direcționați-o prin anexare la fișierul `ierarhie_student` creat anterior;
   - Afisați conținutul fișierului `ierarhie_student` folosind comanda `cat`.
2. Folosiți `tree` pentru a afișa ierarhia de fișiere pornind de la `/usr`;
   - Repetați comanda anterioara, dar de data asta direcționați-o *fără* anexare la fișierul `ierarhie_student` folosit anterior;
   - Afisați conținutul fișierului `ierarhie_student` folosind comanda `cat`. Mai exista conținutul inițial al fișierului?

Redirectări de eroare

Atunci când direcționăm ieșirea unei comenzi folosind > sau >>, ne referim *doar* la ieșirea standard, nu și la erori.

Folosim caracterele 2> pentru a direcționa ieșirea de eroare către un fișier:

```
student@uso:~$ ls
EGC lista_materii RL USO
student@uso:~$ cat lista_materie
cat: lista_materie: No such file or directory
```

Am vrut să afișăm conținutul fișierului `lista_materie`, însă acest fișier nu există. Am primit informația *No such file or directory*, și aceasta este afișată sub formă de **eroare**.

Direcționăm conținutul fișierului SO2 în fișierul `materii`:

```
student@uso:~$ cat SO2 > materii
cat: SO2: No such file or directory
```

Vedem că dacă folosim direcționarea simplă >, eroarea tot este afișată.

Deși comanda cat eșuează, direcționarea este făcută. Fișierul materii a fost creat și este gol.

```
student@uso:~$ cat SO2 2> materii
student@uso:~$ cat materii
cat: SO2: No such file or directory
```

Observăm că nu am primit eroare la prima comandă unde am încercat să afișăm conținutul unui fișier inexistent. Informațiile de eroare au fost direcționate în fișier.

Redirectările de eroare și ieșire pot fi folosite simultan folosind caracterele specifice > pentru ieșire și 2> pentru eroare. Putem folosi sintaxa `comanda > fișier_ieșire 2> fișier_erori` pentru a face acest lucru.

```
student@uso:~$ cat USO SO2
USO is the b3st!
USO Rullz
cat: SO2: No such file or directory
```

În exemplul de mai sus încercăm să afișăm conținutul fișierelor USO și SO2, dar cu fișierul SO2 neexistent. Liniile `USO is the b3st!` și `USO Rullz` sunt conținutul fișierelului USO. Comanda `cat` ne-a arătat conținutul celor 2 fișiere. Tot comanda `cat` a încercat să afișeze conținutul fișierului SO2, însă el nu există și a afișat eroarea `No such file or directory`.

În continuare vom face o separare a fluxurilor: direcționăm ieșirea în fișierul `continut_materii` și erorile în fișierul `erori_comenzi`. Ca să redirectăm și outputul și erorile de la o comandă folosim și > și 2> ca mai jos:

```
student@uso:~$ cat USO SO2 > continut_materii 2> erori_comenzi
student@uso:~$ cat continut_materii
USO is the b3st!
```

```
USO Rullz
student@uso:~$ cat erori_comenzi
cat: SO2: No such file or directory
```

Observăm cum după executarea primei comenzi, la ecran nu mai apare nimic; atât ieșirea standard cât și erorile au fost direcționate în fișiere. În continuare am verificat conținutul acestor fișiere pentru a demonstra corectitudinea operației.

Exerciții

- Redirectați (simplu) conținutul fișierului de la calea `/etc/passwd` într-un fișier cu numele `utilizatori_si_grupuri`.
- Redirectare (simplu) fișierul de la calea `/etc/group` în același fișier.
- Repetați cele două operații de redirectare de mai sus folosind redirectare cu anexare.
- Redirectați conținutul fișierului de la calea `/etc/shadow` în fișierul `parole` și afișați conținutul fișierului. HINT: Puteți folosi sudo!

Arhive

Sunt situații în care vrem să trimitem/stocăm o ierarhie de fișiere. Pentru a ușura acest lucru, folosim arhive. Arhivele sunt o concatenare a unei ierarhii de fișiere și directoare într-un singur fișier.

Există două motive pentru care am vrea să folosim arhive:

1. Să avem o ierarhie de fișiere și directoare într-un singur fișier - *Arhivare*. De exemplu, vrem să trimitem pe mail o ierarhie de directoare cu laboratoarele și resursele materiei USO.
2. Ne dorim ca fișierele să ocupe mai puțin spațiu - *Compresie*.

Comprimarea este o arhivare mai specială pentru că micșorează dimensiunea fișierului rezultat. Rezultatul este similar: agregarea într-un singur fișier.

În acest capitol vom folosi arhive de tipul *zip* pentru arhivare comprimată.

Aceasta nu este singura modalitate de arhivare existenta. Pentru mai multe modalitati, puteti cauta pe Google. Google is your friend :) **(hint: tar)**

Comprimare - zip

Acțiunea de comprimare este compusă din doi pași: **arhivare** și **reducerea dimensiunii** (*comprimare*). Utilitarul `zip` face ambii pași simultan.

În continuare folosim utilitarul `zip` pentru arhivare și comprimare.

Crearea arhivelor

Putem crea o arhivă folosind utilitarul `zip`, astfel:

```
student@uso:~$ cd ~/archive
student@uso:~$ ls -l
total 42M
-rw-r--r-- 1 student student 6,8M oct 11 02:23 inregistrare_lab01.mp4
-rw-r--r-- 1 student student 2,5M oct 11 02:23 inregistrare_lab02.mp4
-rw-r--r-- 1 student student 4,8M oct 11 02:23 inregistrare_lab03.mp4
-rw-r--r-- 1 student student 391K oct 11 02:23 inregistrare_lab04.mp4
student@uso:~$ zip inregistrari.zip inregistrare_lab01.mp4 inregistrare_lab02.mp4 inregistrare_lab03.mp4
  adding: inregistrare_lab01.mp4 (deflated 0%)
  adding: inregistrare_lab02.mp4 (deflated 0%)
  adding: inregistrare_lab03.mp4 (deflated 0%)
student@uso:~$ ls -lh
total 56M
-rw-r--r-- 1 student student 6,8M oct 11 02:23 inregistrare_lab01.mp4
-rw-r--r-- 1 student student 2,5M oct 11 02:23 inregistrare_lab02.mp4
-rw-r--r-- 1 student student 4,8M oct 11 02:23 inregistrare_lab03.mp4
-rw-r--r-- 1 student student 391K oct 11 02:23 inregistrare_lab04.mp4
-rw-rw-r-- 1 student student  14M oct 14 00:11 inregistrari.zip
```

Sintaxa este în felul următor: `zip numele_arhivă.zip ierarhia_de_fișiere` Observăm cum după comanda de creare a arhivei `zip` am verificat corectitudinea operației.

Parametrul `-h` de la comanda `ls -lh` transformă dimensiunea fișierului în format *human-readable*, adică folosește K pentru KiloOcteți, M pentru MegaOcteți, G pentru GigaOcteți. La prima vedere, folosind comanda `ls -lh` observăm că fișierul de tipul *zip* nou creat **nu** ocupă mai puțin spațiu decât suma dimensiunilor celor trei fișiere. Motivul este că în sistemul de fișiere a apărut un nou fișier ce conține arhiva nou creată și sistemul de fișiere stochează informații despre acest fișier. Exemplu de informații stocate: dimensiune, data creare, permisiuni, utilizatorul deținător.

Mai sus afișarea dimensiunii este în format *human-readable* adică în *MegaBytes* (MegaOcteți). Afișăm dimensiunea în octeți:

```
student@uso:~$ ls -l
-rw-rw-r-- 1 student student 14517242 oct 14 00:11 inregistrari.zip
```

Arhiva `zip` are dimensiunea de 14517242 bytes.

Mergem mai departe cu un experiment. Dacă facem o arhivare și comprimare a ierarhiei de directoare `/usr/bin` și comparăm dimensiunea:

```
student@uso:~$ ls -l
-rw-r--r-- 1 student student  87282498 Sep 29 04:46 usr_bin.zip
student@uso:~$ ls -lh
-rw-r--r-- 1 student student 84M Sep 29 04:46 usr_bin.zip
```

Putem observa deja o diferență mai mare de dimensiune între cele două.

Pentru o mai bună înțelegere a comenzii de creare a arhivelor, inspectați manualul: `man zip`.

Exerciții - creare arhive

1. - Creați 3 fișiere noi care să aibă pe rând numele orașului natal, țării natale și liceului absolvit. Verificați crearea fișierelor afișând conținutul directorului părinte.
   - Creați o arhivă de tipul `zip` care să conțină cele trei fișiere și care să aibă numele `personal_data.zip`. Verificați conținutul arhivei.
   - Afișați dimensiunea arhivei.
2. - Creați o arhivă de tipul `zip` a fișierelor din interiorul directorului `/usr/include/net/` cu numele `net.zip`.

Afișarea conținutului arhivelor

Putem vizualiza conținutul arhivei astfel:

```
student@uso:~$ zip -sf inregistrari.zip
Archive contains:
  inregistrare_lab01.mp4
  inregistrare_lab02.mp4
  inregistrare_lab03.mp4
```

Pentru a vizualiza arhiva **fără** dezarhivare folosim opțiunea `-sf` (prescurtare de la *show-files*).

Exerciții - afișarea conținutului unei arhivei

- Acest exercițiu folosește arhiva creată la exercițiu anterior.
- Afișați conținutul arhivelor `personal_data.zip` și `net.zip` **fără** dezarhivare.

Dezarhivarea arhivelor

În momentul în care dezarhivăm un fișier de tip *zip*, fișierele din cadrul arhivei vor fi create în directorul curent (dacă nu se specifică altfel). Cu alte cuvinte, ierarhia de fișiere pe care am arhivat-o se păstrează, iar în momentul dezarhivării aceasta va arăta la fel ca înainte de arhivare. Pentru a vedea mai clar efectul operației de dezarhivare, ștergem fișierele `inregistrare_lab01.mp4`, `inregistrare_lab02.mp4`, `inregistrare_lab03.mp4`, adică fișierele care se află în arhiva *inregistrari.zip*.

```
student@uso:~$ ls -l
total 57168
drwxr-xr-x 3 student student    4096 oct 11 12:15 dezarhivare
-rw-r--r-- 1 student student  7032579 oct 11 02:23 inregistrare_lab01.mp4
-rw-r--r-- 1 student student  2528132 oct 11 02:23 inregistrare_lab02.mp4
-rw-r--r-- 1 student student  4979963 oct 11 02:23 inregistrare_lab03.mp4
-rw-r--r-- 1 student student   399418 oct 11 02:23 inregistrare_lab04.mp4
-rw-rw-r-- 1 student student 14517242 oct 14 00:11 inregistrari.zip
student@uso:~$ rm inregistrari.zip
```

În prima instanță, ștergem arhiva `inregistrari.zip`. În continuare, recreăm arhiva `inregistrari.zip` și ștergem fișierele `inregistrare_lab01.mp4`, `inregistrare_lab02.mp4`, `inregistrare_lab03.mp4`:

```
student@uso:~$ zip inregistrari.zip inregistrare_lab01.mp4 inregistrare_lab02.mp4  inregistrare_lab03.mp4
  adding: inregistrare_lab01.mp4 (deflated 0%)
  adding: inregistrare_lab02.mp4 (deflated 0%)
  adding: inregistrare_lab03.mp4 (deflated 0%)
student@uso:~$ ls -l
total 34828
drwxr-xr-x 3 student student    4096 oct 11 12:15 dezarhivare
-rw-r--r-- 1 student student 5242880 oct 11 02:23 inregistrare_lab01.mp4
-rw-r--r-- 1 student student 3145728 oct 11 02:23 inregistrare_lab02.mp4
-rw-r--r-- 1 student student 1048576 oct 11 02:23 inregistrare_lab03.mp4
-rw-r--r-- 1 student student 7340032 oct 11 12:35 inregistrare_lab04.mp4
-rw-r--r-- 1 student student 9439072 oct 11 18:02 inregistrari.zip
student@uso:~$ rm inregistrare_lab01.mp4 inregistrare_lab02.mp4  inregistrare_lab03.mp4
```

În momentul de față avem doar arhiva *zip* și urmează să obținem fișierele cu înregistrari, folosind utilitarul `zip`:

```
```

Observăm cum după operațiile de ștergere și dezarhivare, verificăm corectitudinea operațiilor.

Pentru a dezarhiva o arhivă folosim utilitarul `unzip`. Puteți observa că am șters fișierele existente înainte de extragere.

Putem extrage fișierele către o anumită cale:

```
student@uso:~$ mkdir dezarhivare/zip
student@uso:~$ unzip inregistrari.zip -d dezarhivare/zip/
Archive:  inregistrari.zip
  inflating: dezarhivare/zip/inregistrare_lab01.mp4
  inflating: dezarhivare/zip/inregistrare_lab02.mp4
  inflating: dezarhivare/zip/inregistrare_lab03.mp4
student@uso:~$ tree
.
|-- dezarhivare
|   ``-- zip
|       |-- inregistrare_lab01.mp4
|       |-- inregistrare_lab02.mp4
|       ``-- inregistrare_lab03.mp4
|-- inregistrare_lab01.mp4
|-- inregistrare_lab02.mp4
|-- inregistrare_lab03.mp4
|-- inregistrare_lab04.mp4
``-- inregistrari.zip

3 directories, 9 files
```

Observăm că după crearea directoarelor și după extragerea arhivelor am verificat corectitudinea operațiilor.

Am folosit comanda `unzip -d destinație` pentru a extrage o arhivă *zip* la calea *destinație*.

Putem dezarhiva *un singur* fișier/director din toată arhiva:

```
student@uso:~$ rm inregistrare_lab03.mp4
student@uso:~$ ls -lh
total 34M
drwxr-xr-x 4 student student 4.0K oct 11 18:05 dezarhivare
total 48M
drwxr-xr-x 4 student student 4.0K oct 11 18:05 dezarhivare
-rw-r--r-- 1 student student 6,8M oct 11 02:23 inregistrare_lab01.mp4
-rw-r--r-- 1 student student 2,5M oct 11 02:23 inregistrare_lab02.mp4
-rw-r--r-- 1 student student 391K oct 11 02:23 inregistrare_lab04.mp4
-rw-rw-r-- 1 student student  14M oct 14 00:11 inregistrari.zip
student@uso:~$ unzip inregistrari.zip inregistrare_lab03.mp4
Archive:  inregistrari.zip
inflating: inregistrare_lab03.mp4
student@uso:~$ ls -lh
drwxr-xr-x 4 student student 4.0K oct 11 18:05 dezarhivare
-rw-r--r-- 1 student student 6,8M oct 11 02:23 inregistrare_lab01.mp4
-rw-r--r-- 1 student student 2,5M oct 11 02:23 inregistrare_lab02.mp4
-rw-r--r-- 1 student student 4,8M oct 11 02:23 inregistrare_lab03.mp4
-rw-r--r-- 1 student student 391K oct 11 02:23 inregistrare_lab04.mp4
-rw-rw-r-- 1 student student  14M oct 14 00:11 inregistrari.zip
```

Observăm cum după comenzile de ștergere și dezarhivare s-a verificat corectitudinea operațiilor.

Puteți observa că am extras un singur fișier punând ca ultim parametru numele fișierului.

Exerciții - dezarhivarea unei arhive

1. - Creați un director cu numele `zip` în directorul `dezarhivare` ca în ierarhia de mai sus. Verificați crearea afișând conținutul directorului `dezarhivare`.
   - Dezarhivați arhiva `personal_data.zip` în directorul `dezarhivare/zip/`. Verificați operația de dezarhivare afișând conținutul directorului.
   - Dezarhivați **doar** fișierul `oraș natal` în directorul curent. Verificați operația afișând conținutul directorului curent.
2. - Creați directorul `my_net` în directorul `dezarhivare` ca în ierarhia de mai sus. Verificați crearea afișând conținutul directorului `dezarhivare`.
   - Dezarhivați arhiva `net.zip` în directorul creat anterior `my_net`. Verificați operația de dezarhivare afișând conținutul directorului.

Adăugarea de noi fișiere la arhive

Putem adăuga un nou fișier la arhivă folosind comanda `zip -u`:

```
student@uso:~$ zip -u inregistrari.zip inregistrare_lab04.mp4
  adding: inregistrare_lab04.mp4 (deflated 0%)
student@uso:~$ zip -sf inregistrari.zip
Archive contains:
  inregistrare_lab01.mp4
  inregistrare_lab02.mp4
  inregistrare_lab03.mp4
  inregistrare_lab04.mp4
Total 4 entries (16777216 bytes)
```

Observăm cum după comanda de adăugare de fișier la arhivă, am verificat corectitudinea operației folosind comanda `zip -sf`.

În exemplul de mai sus am adăugat `inregistrare_lab04.mp4` la arhivă.

Exerciții - adăugarea unui fișier în arhivă

- Creați un fișier UPB cu conținutul "Make UPB Great Again!".
- Verificați conținutul arhivei `personal_data.zip`.
- Adăugați fișierul UPB în arhivă.
- Verificați adăugarea fișierului la arhivă fără dezarhivare.
- Dezarhivați arhiva în directorul `personal_data_zip`.

Este bine de știut că în urma realizării unei operații trebuie verificată executarea acesteia.

Sumar - Cheatsheet

Redirectări simple

```
student@uso:~$ ls -lh
total 9.0M
-rw-r--r-- 1 student student 5.0M Aug 19 11:55 USO
-rw-r--r-- 1 student student 3.0M Aug 19 11:55 RL
-rw-r--r-- 1 student student 1.0M Aug 19 11:55 EGC
student@uso:~$ ls -lh > lista_materii
student@uso:~$ ls
EGC lista_materii RL USO
```

Redirectări prin anexare

```
student@uso:~$ ls >> lista_materii
student@uso:~$ ls
USO RL EGC lista_materii
student@uso:~$ cat lista_materii
total 9.2M
-rw-r--r-- 1 student student 1.0M Aug 19 11:55 EGC
-rw-r--r-- 1 student student 209K Aug 19 12:00 lista_materii
-rw-r--r-- 1 student student 5.0M Aug 19 11:55 USO
-rw-r--r-- 1 student student 3.0M Aug 19 11:55 RL
EGC
lista_materii
RL
USO
```

Redirectări de eroare

```
student@uso:~$ ls
EGC lista_materii RL USO
student@uso:~$ cat S02
cat: S02: No such file or directory
student@uso:~$ cat S02 2> erori_materii
student@uso:~$ cat erori_materii
cat: S02: No such file or directory
```

Crearea arhivelor - zip

```
student@uso:~$ zip înregistrări.zip înregistrare_lab01.mov înregistrare_lab02.mov înregistrare_lab03.mov
  adding: înregistrare_lab01.mov (deflated 0%)
  adding: înregistrare_lab02.mov (deflated 0%)
  adding: înregistrare_lab03.mov (deflated 0%)
student@uso:~$ ls
înregistrare_lab01.mov înregistrare_lab02.mov înregistrare_lab03.mov înregistrări.zip
```

Afișarea conținutului arhivelor - zip

```
student@uso:~$ zip -sf înregistrări.zip
Archive contains:
  înregistrare_lab01.mov
  înregistrare_lab02.mov
  înregistrare_lab03.mov
```

Dezarhivarea arhivelor - zip

```
student@uso:~$ ls
înregistrare_lab01.mov înregistrare_lab02.mov înregistrare_lab03.mov înregistrări.zip
student@uso:~$ rm înregistrare_lab01.mov înregistrare_lab02.mov  înregistrare_lab03.mov
student@uso:~$ unzip înregistrări.zip
Archive:  înregistrări.zip
  inflating: înregistrare_lab01.mov
  inflating: înregistrare_lab02.mov
  inflating: înregistrare_lab03.mov
student@uso:~$ ls
înregistrare_lab01.mov înregistrare_lab02.mov înregistrare_lab03.mov înregistrări.zip
```

Dezarhivarea arhivelor la o cale specifică - zip

```
student@uso:~$ mkdir -p dezarhivare/zip
student@uso:~$ unzip înregistrări.zip -d dezarhivare/zip/
Archive:  înregistrări.zip
  inflating: dezarhivare/zip/înregistrare_lab01.mov
  inflating: dezarhivare/zip/înregistrare_lab02.mov
  inflating: dezarhivare/zip/înregistrare_lab03.mov
student@uso:~$ tree
.
|-- dezarhivare
|   |-- zip
|       |-- înregistrare_lab01.mov
|       |-- înregistrare_lab02.mov
|       `-- înregistrare_lab03.mov
|-- înregistrare_lab04.mov
`-- înregistrări.zip

2 directories, 5 files
```

Adăugarea de noi fișiere la arhive - zip

```
student@uso:~$ zip -u înregistrări.zip înregistrare_lab04.mov
  adding: înregistrare_lab04.mov (deflated 0%)
student@uso:~$ zip -sf înregistrări.zip
Archive contains:
  înregistrare_lab01.mov
  înregistrare_lab02.mov
  înregistrare_lab03.mov
  înregistrare_lab04.mov
Total 4 entries (16777216 bytes)
```

Cuprins

- Laborator 03 (one page)
- Redirectări
- Arhive
- Sumar - Cheatsheet

uso/laboratoare/ac/laborator-03.txt · Last modified: 2021/10/21 12:58 by liza_elena.babu

Laborator 04 - Instalarea și utilizarea aplicațiilor

Pornirea și oprirea aplicațiilor

Pe un sistem se găsesc mai multe aplicații. Utilizatorul pornește aplicația potrivită pentru o nevoie pe care o are. De exemplu, pornește un browser web pentru a accesa Wikipedia sau pornește aplicația Spotify pentru a asculta muzică.

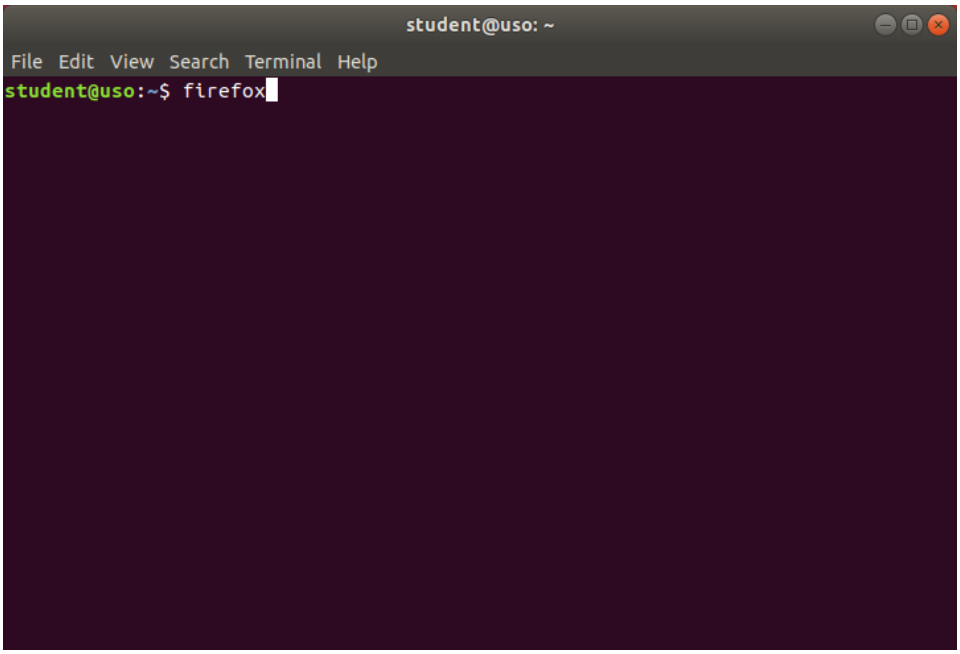
Utilizatorul poate porni aplicațiile folosind interfața grafică (*Graphical User Interface*, GUI) sau interfața în linia de comandă (*Command Line Interface*, CLI). În cazul interfeței grafice pornirea se face folosind elementele grafice (mouse, meniuri, iconuri de desktop). În cazul interfeței în linia de comandă pornirea se face introducând și rulând comenzi.

Pornirea și oprirea aplicațiilor grafice

Pornirea unei aplicații grafice folosind linia de comandă

Pe lângă cele două moduri de mai sus, pornirea unei aplicații grafice poate fi realizată din linia de comandă (CLI), folosind șirul care identifică aplicația, la fel ca în cazul folosirii `Alt+F2`. Pentru a porni o aplicație grafică din linia de comandă, avem nevoie de un terminal. Terminalul este un dispozitiv în care rulează o aplicație numită **shell**, care este interpretorul comenzilor introduse. Pe parcursul acestui capitol și a întregii cărți vom folosi interschimbabil shell și terminal.

Pornim o aplicație de terminal folosind fie iconul corespunzător, fie `Alt+F2` urmat de șirul (comanda) `gnome-terminal`. În aplicația de terminal, pornim aplicația Firefox scriind șirul (comanda) `firefox`, urmat de apăsarea tastei `Enter`, ca în imaginea de mai jos. Aplicația Firefox va porni.



În acest moment, în aplicația de terminal nu mai putem introduce noi comenzi pentru a porni alte aplicații. Va trebui să oprim aplicația Firefox. Oprim aplicația Firefox din mediul grafic (click pe butonul de închidere a ferestrei sau folosirea combinației de taste `Alt+F4`) sau din terminal, folosind combinația de taste `Ctrl+c`.

Exerciții

1. La fel ca mai sus, porniți aplicația LibreOffice folosind linia de comandă.

Menținerea accesului la terminal

Atunci când pornim o aplicație grafică din linia de comandă, aplicația "acaparează" terminalul; nu mai putem introduce noi comenzi pentru a porni alte aplicații. Putem rezolva acest lucru prin trecerea aplicației grafice în

backgroundul shellului. Adică urmăm pașii:

- Pornim, în terminal, aplicația grafică (**Firefox**) folosind comanda `firefox`. Spunem că aplicația rulează în **foreground** și controlează terminalul; adică nu permite rularea unei alte comenzi.
- În terminal, folosim combinația de taste `Ctrl+z` pentru a trece aplicația în background. În acest moment, avem din nou promptul terminalului, ca mai jos:

```
student@uso:~$ firefox
^Z
[1]+  Stopped                 firefox
```

- Dar aplicația este acum "înghețată", nu mai răspunde. Spunem că este **suspendată**.
- Folosim, în terminal, comanda `bg`, ca mai jos:

```
student@uso:~$ bg
[1]+ firefox &
```

- Astfel am "dezghețat" aplicația, care acum este interactivă.

În acest mod, avem aplicația **Firefox** în rulare și avem acces la terminal să introducem noi comenzi.

Exerciții: Folosiți pașii de mai sus pentru a porni și aplicația **LibreOffice** din linie de comandă și să mențineți accesul la terminal.

Vom afla mai multe despre background și suspendarea proceselor în `Oprirea proceselor. Semnale`.

Pornirea aplicațiilor în linia de comandă

Pornirea de aplicații CLI folosind linia de comandă

Aplicațiile CLI, numite și utilitare, sunt proiectate pentru a fi pornite și folosite în linia de comandă. Cel mai adesea numim aplicațiile în linie de comandă **utilitare** sau, pur și simplu, **comenzi**. Scriem numele utilitarului / comenzii într-un terminal și utilitarul va fi pornit[1)]. De exemplu, dacă dorim să afișăm utilizatorii prezenți în sistem, pornim o aplicație de terminal și folosim utilitarul `who`:

```
student@uso:~$ who
student  :0           2020-09-04 17:42 (:0)
student  pts/0        2020-09-19 15:57 (192.168.56.1)
```

Sau, dacă dorim să vedem câtă memorie avem (disponibilă) în sistem, folosim utilitarul `free`:

```
student@uso:~$ free
               total        used        free      shared  buff/cache   available
Mem:        2040972     1025716       83824       32916      931432      794692
Swap:        777300       37056      740244
```

Exerciții

1. Porniți utilitarul `ls` (de listare a conținutului unui director în linia de comandă (CLI)) folosind linia de comandă.
2. Porniți utilitarul `df` (de afișare a spațiului ocupat pe disc) folosind linia de comandă.

Argumente în linia de comandă

Pentru a porni aplicații / utilitare în linia de comandă, folosim comenzi care conțin numele utilitarului urmate, eventual, de argumente. Astfel, pentru a porni utilitarele `ls` sau `ps`, folosim comenzi precum cele de mai jos, simple sau cu argumente:

```
student@uso:~$ ls
Desktop  Documents  Downloads  examples.desktop  Music  Pictures  Public  snap  Templates  uso.git  Videos  vm-actions-log.txt
student@uso:~$ ls -l
total 60
drwxr-xr-x  2 student student 4096 Aug  6  2018 Desktop
drwxr-xr-x  3 student student 4096 Aug 20  2018 Documents
drwxr-xr-x  2 student student 4096 Aug  6  2018 Downloads
```

```
-rw-r--r--  1 student student 8980 Aug  6  2018 examples.desktop
drwxr-xr-x  2 student student 4096 Aug  6  2018 Music
drwxr-xr-x  2 student student 4096 Aug  8 11:52 Pictures
drwxr-xr-x  2 student student 4096 Aug  6  2018 Public
drwxr-xr-x  3 student student 4096 Aug  8 09:02 snap
drwxr-xr-x  2 student student 4096 Aug  6  2018 Templates
drwxr-xr-x 14 student student 4096 Aug 20  2018 uso.git
drwxr-xr-x  2 student student 4096 Aug  6  2018 Videos
-rw-r--r--  1 student student 4827 Aug 21  2018 vm-actions-log.txt
student@uso:~$ ps
  PID TTY          TIME CMD
 3370 pts/4    00:00:00 bash
 7979 pts/4    00:00:00 ps
student@uso:~$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
student   3370  3369  0 08:55 pts/4    00:00:00 -bash
student   7982  3370  0 13:17 pts/4    00:00:00 ps -f
```

Oprirea forțată a aplicațiilor în linia de comandă

Se poate întâmpla ca o aplicație în linia de comandă să ruleze pentru prea mult timp sau să se blocheze. Caz în care dorim să o oprim. Soluția de avarie este să închidem fereastra de terminal, lucru care, de obicei, închide și aplicația. Soluția mai bună este să închidem doar aplicația. Acest lucru îl facem folosind combinația de taste `Ctrl+c` care oprește aplicația care rulează în terminal[2)]. Acest lucru poate fi realizat și pentru aplicații grafice, așa cum am văzut mai sus.

De exemplu, dacă folosim comanda `sleep 100` care se va bloca pentru 100 de secunde, o vom opri folosind `Ctrl+c` ca mai jos:

```
student@uso:~$ sleep 100
^C
student@uso:~$
```

Exerciții: Folosiți următoarea comandă care pornește o aplicație care durează mult și opriți-o forțat:

- `watch ps`: pentru a monitoriza procesele din terminalul curent

Note de subsol

Rularea aplicațiilor

Pornirea unei aplicații înseamnă că se alocă resursele sistemului (procesor, memorie, dispozitive de intrare/ieșire) pentru a rula aplicația. O aplicație care rulează, adică folosește resursele sistemului pentru a executa cod și a prelucra date, se numește **proces**. Atunci când pornim o aplicație, se creează un proces; atunci când oprim aplicația, sau când își încheie execuția, ne referim la încheierea execuției procesului.

Procesul este pornit dintr-un fișier executabil care conține codul (instrucțiunile) și datele aplicației. Fișierul executabil mai este numit și **imaginea procesului**. Fișierul executabil este un **program**. Spunem că procesul este un program aflat în execuție[3)].

Identificarea fișierului executabil al unei aplicații

Fișierul executabil al unei aplicații este încărcat în memoria sistemului și codul este executat; din acest moment spunem că aplicația **rulează**.

De exemplu aplicația Firefox (browser web) are asociat fișierul executabil `/usr/bin/firefox`; aplicația Vim (editor) are asociat fișierul executabil `/usr/bin/vim`; aplicația LibreOffice (suită office) are asociat fișierul executabil `/usr/bin/libreoffice`. Putem identifica fișierul executabil al unei aplicații folosind comanda `which`, urmată de comanda pentru pornirea aplicației ca mai jos:

```
student@uso:~$ which firefox
/usr/bin/firefox
student@uso:~$ which vim
/usr/bin/vim
student@uso:~$ which libreoffice
/usr/bin/libreoffice
```

Exerciții: Identificați fișierul executabil al aplicațiilor / utilitarelor Shutter, Gedit, `ls`, `df`.

Investigarea proceselor

Un sistem de operare are de obicei mai multe aplicații care rulează, deci mai multe procese. Prea multe procese pot duce la o încărcare prea mare a sistemului, încetinind sau împiedicând funcționarea acestuia. Anumite procese pot consuma excesiv resurse afectând celelalte procese. De aceea, este util să investigăm procesele unui sistem și consumul de resurse al acestora.

Un proces este creat de un alt proces. De exemplu, mai sus, procesul `which` a fost creat dintr-un proces shell (`bash`). Procesul shell a fost, la rândul său, creat de un alt proces. Un proces are un proces părinte; un proces poate avea mai multe procese copil. Procesele sunt, așadar, parte dintr-o ierarhie.

În vârful ierarhiei de procese este procesul numit clasic `init`. În listarea de mai sus vedem că procesul din vârful ierarhiei este `systemd`. `systemd`[4] este implementarea de `init` prezentă în cea mai mare parte a distribuțiilor Linux curente[5].

Listarea proceselor

La nivel mai degrabă didactic, putem vizualiza lista de procese a unui sistem. Utilitarul `ps` afișează procesele curente în sistem (un *snapshot* al proceselor sistemului). La o rulare simplă, utilitarul `ps` afișează procesele din terminalul curent:

```
student@uso:~$ ps
  PID TTY          TIME CMD
14897 pts/4    00:00:00 bash
14910 pts/4    00:00:00 ps
```

În terminalul curent (indicat de coloana `TTY` din afișare, adică terminalul `pts/4`) sunt două procese:

1. procesul shell (`bash`) în care rulăm comenzi care creează noi procese;
2. procesul de listare (`ps`) pe care tocmai l-am lansat prin comanda `ps`; practic se afișează pe sine

Pentru a afișa toate procesele sistemului folosim opțiunea `-e` (pentru *everything*) a utilitarului `ps` ca în comanda de mai jos:

```
student@uso:~$ ps -e
  PID TTY          TIME CMD
    1 ?        00:00:19 systemd
    2 ?        00:00:00 kthreadd
    4 ?        00:00:00 kworker/0:0H
    6 ?        00:00:00 mm_percpu_wq
    7 ?        00:00:09 ksoftirqd/0
    8 ?        00:00:06 rcu_sched
    9 ?        00:00:00 rcu_bh
   10 ?        00:00:00 migration/0
   11 ?        00:00:00 watchdog/0
[...]
```

Atributele proceselor

Utilitarul `ps` are o afișare tabelară a proceselor, fiecare coloană corespunzând unui atribut al proceselor. La o rulare simplă, așa cum am văzut mai sus sunt afișate patru coloane:

- `PID`: reprezentând identificatorul procesului
- `TTY`: terminalul în care rulează procesul (apare `?` pentru un proces care nu are terminal - în general procesele de tip serviciu, numite și procese daemon nu au terminal)
- `TIME`: timpul de rulare pe procesor (în ore, minute, secunde)
- `CMD`: numele imaginii de proces (adică numele executabilului / programului din care a fost creat procesul)

Identificarea unui proces

`PID` (*Process Id*) este atributul esențial al procesului, un index care identifică procesul la nivelul sistemului. Un proces este identificat după PID, nu după numele executabilului (`CMD`). Putem avea mai multe procese create din același executabil, fiecare proces având PID-ul său[6].

Pentru a verifica existența mai multor procese, o să creăm mai multe procese shell. Pentru început, deschidem mai multe sesiuni de terminal, folosind, de exemplu, `Alt+F2` în mediul grafic și introducând comanda `gnome-terminal` în promptul creat. Apoi vizualizăm doar procesele create din executabilul `bash` rulând comanda:

```
student@uso:~$ ps -e | grep bash
 2181 pts/1    00:00:00 bash
 2194 pts/2    00:00:00 bash
 2205 pts/3    00:00:00 bash
14750 pts/0    00:00:00 bash
14897 pts/4    00:00:00 bash
```

Obținem un rezultat precum cel de mai sus. Sunt cinci procese, toate create din executabilul `bash`, cu cinci PID-uri diferite: `2181`, `2194`, `2205`, `14705`, `14879`.

Afișarea atributelor unui proces

Un proces are mai mult decât cele patru atribute afișate la o rulare simplă a utilitarului `ps`. Pentru a afișa mai multe atribute, folosim opțiunea `-f` (de la *full format*) sau opțiunea `-F` (de la *extra full format*), ca mai jos:

```
student@uso:~$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
student  14897 14896  0 17:12 pts/4    00:00:00 -bash
student  15026 14897  0 17:46 pts/4    00:00:00 ps -f

student@uso:~$ ps -F
UID        PID  PPID  C    SZ   RSS PSR STIME TTY          TIME CMD
student  14897 14896  0  6056  5136   0 17:12 pts/4    00:00:00 -bash
student  15027 14897  0  9728  3340   0 17:46 pts/4    00:00:00 ps -F
```

Desigur, putem să combinăm aceste opțiuni cu opțiunea `-e` de afișare a tuturor proceselor:

```
student@uso:~$ ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 Aug18 ?        00:00:19 /lib/systemd/systemd --system --deserialize 39
root         2     0  0 Aug18 ?        00:00:00 [kthreadd]
root         4     2  0 Aug18 ?        00:00:00 [kworker/0:0H]
root         6     2  0 Aug18 ?        00:00:00 [mm_percpu_wq]
root         7     2  0 Aug18 ?        00:00:09 [ksoftirqd/0]
root         8     2  0 Aug18 ?        00:00:06 [rcu_sched]
root         9     2  0 Aug18 ?        00:00:00 [rcu_bh]
root        10     2  0 Aug18 ?        00:00:00 [migration/0]
root        11     2  0 Aug18 ?        00:00:00 [watchdog/0]
[...]
```

Opțiunile `-f` și `-F` afișează și alte atribute ale procesului, precum:

- `UID`: numele utilizatorului care deține procesul
- `PPID`: identificatorul procesului părinte
- `C`: procentul de procesor ocupat
- `STIME`: timpul de pornire (*start time*)
- `RSS`: memoria RAM ocupată (*resident set size*)

Astfel de atribute sunt utile pentru a vedea care sunt procesele cele mai consumatoare de resurse (de exemplu procesor sau memorie).

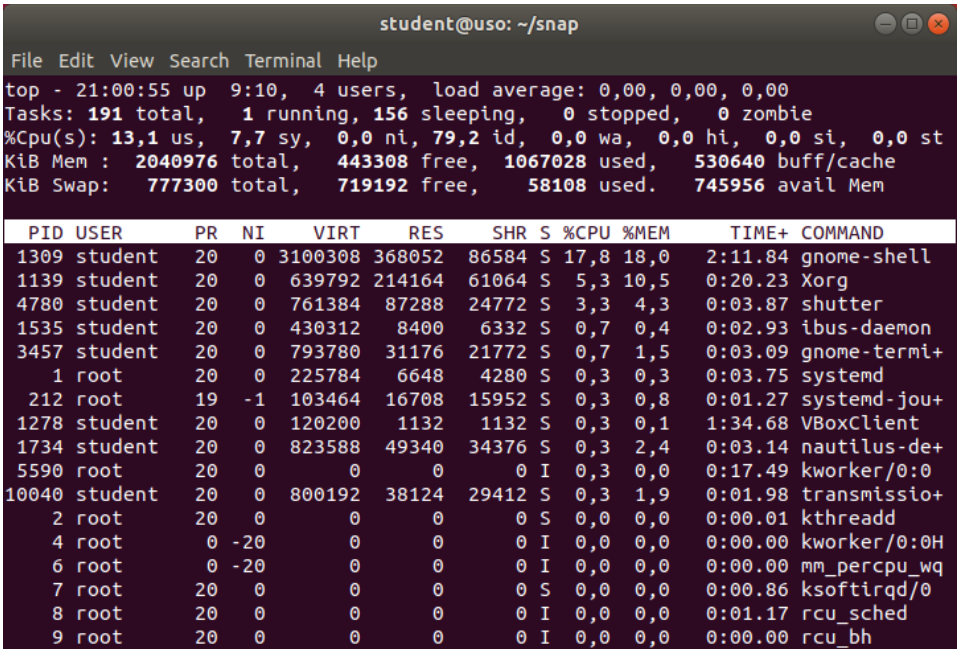
Monitorizarea proceselor

Utilitarul `ps` afișează procesele și atributele acestora la un moment dat, un *snapshot* al proceselor sistemului. De multe ori ne interesează și evoluția în timp a proceselor: schimbarea în consumul de resurse, apariția de noi procese. Adică să monitorizăm procesele. Monitorizarea proceselor înseamnă obținerea periodică de informații despre procese.

Evident, un mod simplu de monitorizare este să rulăm utilitarul `ps` periodic. Există, însă, utilitare dedicate pentru monitorizare.

Monitorizarea folosind top

Utilitarul `top` este utilitarul de bază de monitorizare a proceselor în lumea Linux. Este echivalent Task Manager din Windows. Rularea `top` duce la afișarea, în terminal, a proceselor sistemului și reîmprospătarea informației periodic (implicit 2 secunde). Imaginea de mai jos este o fereastră de terminal cu rularea `top`:



`top` afișează informații periodice despre procese **și** despre starea sistemului: consum de procesor, de memorie. La fiecare perioadă (implicit 2 secunde) informația afișată este reîmprospătată.

Ieșirea din utilitarul `top` se realizează folosind tasta `q`[7].

Oprirea proceselor. Semnale

Folosirea semnalelor pentru omorârea proceselor

Ca să terminăm forțat (omorâm) un proces folosim semnale. Un semnal este o notificare trimisă de utilizator sau de sistemul de operare către un proces. Nu este obligatoriu ca un semnal să omoare procesul care îl primește, dar este cel mai des întâlnit comportament, și principala utilizare a semnalelor.

Ca să trimitem un semnal unui proces trebuie să știm PID-ul acestuia și folosim utilitarul `kill` urmat de PID-ul procesului. Adică, dacă pornim într-un terminal un proces `sleep` folosind comanda de mai jos:

```
student@uso:~$ sleep 60
```

în alt terminal vom afla PID-ul său (folosind `pidof`):

```
student@uso:~$ pidof sleep
9486
```

și apoi îl vom omorî (folosind `kill`):

```
student@uso:~$ kill -SIGKILL 9486
```

Comanda `kill` primește ca argument PID-ul procesului de omorât, adică 9486.

Verificăm din nou dacă există un proces `sleep` folosind `pidof`:

```
student@uso:~$ pidof sleep
student@uso:~$
```

Vedem din output că nu mai există procesul `sleep`, deci a fost omorât.

În terminalul inițial, în care am rulat comanda `sleep`, apare un mesaj care indică omorârea procesului:

```
student@uso:~$ sleep 60
Killed
```

Exerciții: Oprirea proceselor

- Porniți în trei terminale diferite trei procese `sleep`. Omorâți-le pe toate cu o singură comandă. PID-ul shellului curent poate fi aflat folosind comanda:

```
student@uso:~$ echo $$
9477
```

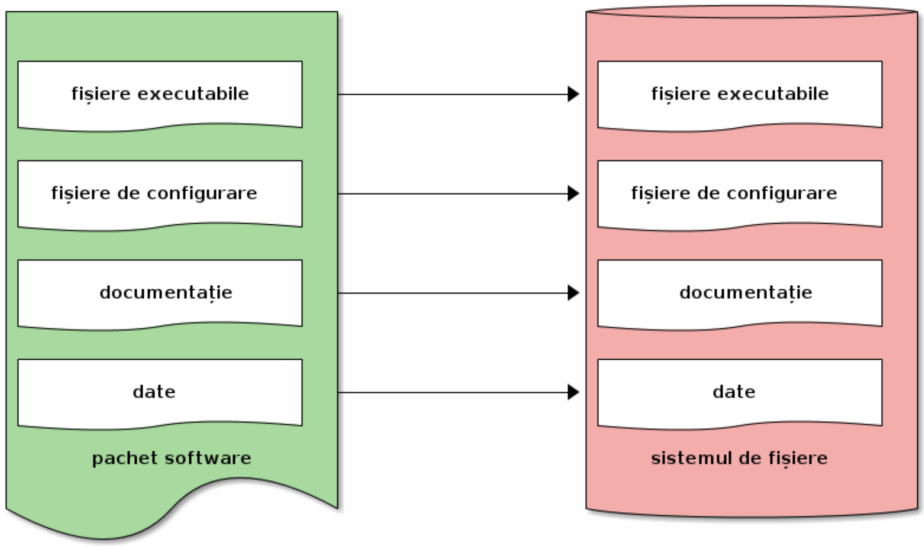
- Omorâți shellul curent.
- Porniți o aplicație `shutter`. Porniți o aplicație `libreoffice`. Omorâți aceste procese folosind utilitarul `kill`.

Note de subsol

Instalarea și dezinstalarea aplicațiilor

Instalarea unui sistem de operare duce la instalarea unui set de bază de aplicații pe acel sistem. Acest set este minimal; dacă dorim funcționalități peste acest set, instalăm noi aplicații. Instalarea unei aplicații duce la adăugarea în sistem a unui fișier executabil (sau mai multe) pentru rularea aplicației și eventuale fișiere de configurare, de documentare și de stocare a datelor aplicației. Instalarea unei aplicații presupune obținerea unui fișier specializat numit **pachet software** și apoi despachetarea acestuia în fișierele specifice aplicației. Un pachet este efectiv o arhivă cu fișierele specifice aplicației. După instalarea aplicației / pachetului, aceasta poate fi pornită și rulată de utilizator pentru nevoile sale.

Instalarea unei aplicații dintr-un pachet software este prezentată schematic mai jos:



De partea cealaltă, dezinstalarea unei aplicații / unui pachet înseamnă ștergerea din sistem a fișierelor specifice. După dezinstalarea aplicației, aceasta nu mai poate fi pornită, nemaiexistând fișierul executabil corespunzător sau celelalte fișiere specifice.

Acțiunile de instalare, dezinstalare și configurare a pachetelor într-un sistem sunt numite colectiv **gestiunea pachetelor (software)** (*(software) package management*).

În Linux, aplicațiile sunt, în general, instalate prin intermediul unei aplicații dedicate numite **manager de pachete (software)** (*(software) package manager* sau *(software) package management system*). În distribuțiile Linux bazate pe Debian / Ubuntu, managerul de pachete oferă utilitarele `apt` și `dpkg` (și altele) și aplicații precum Synaptic. În distribuțiile Linux bazate pe RedHat / Fedora, managerul de pachete oferă utilitarele `yum` sau `dnf` sau `rpm` (și altele) precum PackageKit.

Mașina virtuală de suport folosește o distribuție Ubuntu. De aceea ne vom concentra doar pe distribuțiile bazate pe Debian / Ubuntu și deci, pe utilitarele `apt` și `dpkg`.

În continuare vom prezenta acțiunile de instalare și dezinstalare a unui pachet folosind interfața în linia de comandă.

Instalarea unei aplicații cunoscute

Cel mai adesea, dorim rapid să avem o aplicație care să ne rezolve o nevoie. Să presupunem că avem nevoia să lucrăm cu fișiere în format SVG (*Scalable Vector Graphics*). Pentru aceasta vom instala aplicația Inkscape.

Instalarea din linia de comandă

În linia de comandă, folosim utilitarul `apt` pentru a instala pachetul `inkscape`:

```
student@uso:~$ sudo apt update
[sudo] password for student:
Hit:1 http://ro.archive.ubuntu.com/ubuntu bionic InRelease
[...]

student@uso:~$ sudo apt install inkscape
[...]
```

Comanda `sudo apt update` actualizează informațiile despre pachete; vom clarifica mai jos rolul său, inclusiv de ce este recomandată rularea sa (fără a fi obligatorie). Comanda `sudo apt install inkscape` instalează efectiv pachetul numit `inkscape`, care va instala aplicația Inkscape. Instalarea pachetelor în sistem este o acțiune privilegiată, care necesită permisiuni administrative. Din acest motiv cele două comenzi de mai sus sunt prefixate de comanda `sudo`.

În acest moment, aplicația Inkscape este instalată și poate fi pornită.

Exerciții: Porniți aplicația Inkscape în toate modurile descrise în secțiunea `Pornirea și oprirea aplicațiilor grafice`.

Instalarea de noi aplicații

Instalarea VLC

Avem nevoie de un player video și știm că VLC [https://www.videolan.org/vlc/index.html] este unul dintre cele mai folosite playere. Pentru a îl folosi pe sistem, instalăm pachetul `vlc`:

```
student@uso:~$ sudo apt install vlc
[sudo] password for student:
[...]
Do you want to continue? [Y/n] Y
[...]
```

Acum aplicația VLC este disponibilă în sistem.

Exerciții: Porniți aplicația VLC în modurile descrise în `Pornirea și oprirea aplicațiilor grafice`.

Mai sus nu am mai rulat comanda `sudo apt update` pentru că sistemul este deja actualizat de la instalarea pachetului `inkscape`.

Exerciții de instalare de aplicații

Instalați și porniți următoarele pachete din linia de comandă:

- `wikit`: pachet care instalează utilitarului `wikit` (în linia de comandă) care obține informații din Wikipedia
- `gimp`: pachet care instalează aplicația Gimp (cu interfață grafică), un editor de imagini

Dezinstalarea aplicațiilor

Dacă nu mai folosim o aplicație și dorim să eliberăm spațiul ocupat pe disc, putem opta pentru a dezinstala o aplicație. Dezinstalarea înseamnă că toate fișierele corespunzătoare aplicației vor fi șterse de pe disc.

Acțiunea de dezinstalare este mai rară. Beneficiul principal al dezinstalării este eliberarea spațiului ocupat pe disc de fișiere corespunzătoare aplicației. Dar pentru că spațiul pe disc este suficient, apelăm mai rar la această acțiune.

La fel ca în cazul instalării, o aplicație poate fi instalată sau dezinstalată din interfața grafică a managerului de pachete sau din cea în linia de comandă. Dezinstalarea poate fi făcută din interfața în linia de comandă chiar dacă aplicația a fost instalată din interfața grafică și invers.

La fel ca în cazul instalării, acțiunea de dezinstalare este privilegiată.

Dezinstalarea din linia de comandă

Dezinstalarea din linia de comandă se face cu o comandă precum:

```
student@uso:~$ sudo apt remove neofetch
[sudo] password for student:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages will be REMOVED:
  neofetch
0 upgraded, 0 newly installed, 1 to remove and 311 not upgraded.
After this operation, 365 kB disk space will be freed.
Do you want to continue? [Y/n] Y
(Reading database ... 199030 files and directories currently installed.)
Removing neofetch (3.4.0-1) ...
Processing triggers for man-db (2.8.3-2) ...
```

În comanda de mai sus, am dezinstalat pachetul `neofetch`. Comanda de dezinstalare (`apt remove`) a fost prefixată de comanda `sudo` pentru că este vorba de o acțiune privilegiată. La dezinstalare se cere confirmarea că dorim dezinstalarea pachetului.

În urma acestei acțiuni, fișierele corespunzătoare pachetului `neofetch` au fost șterse, incluzând fișierul executabil corespunzător. Acum nu mai avem acces la utilitar și comanda `neofetch` va eșua.

Comanda `apt remove` șterge fișierele corespunzătoare pachetului, dar lasă anumite fișiere de configurare modificate de utilizator. Aceasta pentru a permite ca o instalare ulterioară să refolosească vechea configurare. Dacă dorim ștergerea inclusiv a fișierelor de configurare modificate, folosim comanda:

```
student@uso:~$ sudo apt purge neofetch
[sudo] password for student:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages will be REMOVED:
  neofetch*
0 upgraded, 0 newly installed, 1 to remove and 311 not upgraded.
After this operation, 0 B of additional disk space will be used.
Do you want to continue? [Y/n] Y
(Reading database ... 198873 files and directories currently installed.)
Purging configuration files for neofetch (3.4.0-1) ...
```

Comanda de mai sus a șters fișierele de configurare pentru pachetul `neofetch`. Comanda `apt purge` poate fi folosită pentru a dezinstala pachete instalate, sau pentru a "curăța" (*purge*) pachete care au fost dezinstalate dar care mai au fișiere de configurare.

Exerciții de dezinstalare

Dezinstalați două dintre pachetele pe care le-ați instalat mai sus folosind linia de comandă.

Actualizarea aplicațiilor

O aplicație este, în general, dezvoltată în continuu. Dezvoltatorii aplicației adaugă noi funcționalități, sau rezolvă probleme de funcționare sau de securitate, sau fac aplicația mai robustă sau mai eficientă[8)].

La fel ca în cazul instalării și dezinstalării, actualizarea aplicațiilor duce la modificarea fișierelor din sistem, deci este o acțiune privilegiată.

Actualizarea individuală a unei aplicații

Dacă dorim actualizarea individuală a unui pachet, putem folosi interfața grafică, navigând până la zona specifică aplicației.

Altfel, putem folosi o comandă precum:

```
student@uso:~$ sudo apt install inkscape
[sudo] password for student:
Reading package lists... Done
Building dependency tree
Reading state information... Done
inkscape is already the newest version (0.92.3-1).
0 upgraded, 0 newly installed, 0 to remove and 311 not upgraded.
```

Când comanda `apt install` primește ca argument un pachet deja instalat, va verifica dacă există o versiune actualizată a acestuia. În cazul de mai sus, versiunea instalată a pachetului `inkscape` (`0.92.3-1`) este cea mai nouă și nu este nevoie de actualizare.

Actualizarea tuturor aplicațiilor

Cel mai adesea, un utilizator va opta pentru actualizarea tuturor aplicațiilor sistemului, sau pe scurt, pentru actualizarea sistemului. Acest lucru se întâmplă și pentru că sistemul notifică periodic utilizatorul de prezența unor versiuni noi de aplicații.

Când este cazul, utilizatorul poate folosi interfața grafică sau sau cea în linia de comandă a managerului de pachete pentru actualizarea sistemului. De obicei, utilizatorul va folosi interfața grafică în momentul primirii unei notificări.

Pentru actualizarea sistemului din interfața în linia de comandă, vom folosi o comandă precum:

```
student@uso:~$ sudo apt update
[...]
student@uso:~$ sudo apt upgrade
[...]
311 upgraded, 4 newly installed, 0 to remove and 0 not upgraded.
Need to get 0 B/301 MB of archives.
After this operation, 100 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
[...]
```

Folosind comanda `apt upgrade` actualizăm toate aplicațiile în sistem[9)]. Este recomandat să actualizăm informațiile interne despre pachetele sistemului folosind comanda `apt update`.

Cuprins

- Laborator 04 (one page)
- Pornirea și oprirea aplicațiilor
- Rularea aplicațiilor
- Instalarea și dezinstalarea aplicațiilor

---

1) Spre deosebire de scenariul folosirii `Alt+F2`, când folosim aplicații CLI în linia de comandă, mesajele vor fi vizibile. Acest lucru se întâmplă pentru că linia de comandă înseamnă existența unui terminal unde putem vizualiza mesajele.

2) Pentru anumite aplicații combinația de taste `Ctrl+c` poate să nu funcționeze. În acest caz putem folosi combinația de taste `Ctrl+\`, mai puternică. Dacă nici `Ctrl+\` nu funcționează, va trebui să trimitem aplicației un semnal mai puternic care să o oprească. Vom discuta despre semnale în `Oprirea proceselor. Semnale`.

3) Momentul creării unui proces dintr-un fișier executabil, prin încărcarea codului și datelor fișierului executabil în memorie, se numește **load-time**. Rularea procesului și folosirea resurselor sistemului se numește **run-time**.

4) https://freedesktop.org/wiki/Software/systemd/ [https://freedesktop.org/wiki/Software/systemd/]

5) Procesul `init` va exista în permanență în vârful ierarhiei de procese, indiferent de faptul că se va numi `systemd`, `init`, `upstart` sau altfel.

6) Este impropriu să spunem "procesul bash"; corect este "un proces creat din programul / executabilul bash" sau "procesul cu PID-ul XY". Cu toate acestea vom folosi în general expresia "procesul bash" pentru că reiese din context despre ce proces este vorba.

7) `top` este un utilitar interactiv. Adică în timpul rulării sale utilizatorul poate interacționa cu procesele sau poate modifica informațiile afișate. De exemplu, folosirea tastei `M` într-o rulare `top` afișează procesele sortate după consumul de memorie. Un sumar al comenzilor interactive din `top` se obține folosind tasta `?`.

[8] Aducerea unei aplicații la o versiune mai nouă (*update* sau *upgrade*) are plusuri și minusuri. Avantajul este că noua versiune va avea cele mai noi funcționalități și va avea rezolvate probleme vechi de configurare. Dezavantajul este că noua versiune va fi mai puțin folosită și deci mai puțin stabilă, cu posibile probleme noi de funcționare. În general este recomandat ca aplicațiile sistemului să fie actualizate (*up-to-date*) cu versiuni care au îmbunătățiri de funcționare sau securitate (*security updates*). Dacă se optează pentru cea mai recentă versiune a aplicației (posibil mai puțin stabilă), trebuie cântărit beneficiul adus de noile funcționalități față de posibilele probleme de funcționare. De obicei un utilizator obișnuit va opta pentru cea mai nouă versiune, fiind interesat în primul rând de cele mai noi funcționalități. Un administrator de sistem va opta pentru versiunile mai stabile care asigură o robustețe ridicată a sistemului.

[9] Comanda `apt upgrade` face actualizare conservatoare a sistemului. Dacă un anumit pachet ar fi dezinstalat de acțiunea de actualizare a sistemului, pachetul nu va fi atins de managerul de pachete. Dacă dorim o actualizare completă a sistemului, incluzând dezinstalarea anumitor pachete (pentru că sunt incompatibile cu alte apchete), vom folosi comanda:

```
student@uso:~$ sudo apt full-upgrade
```

Laborator 05 - Conectarea la Internet

Înainte de începerea laboratorului, rulați următoarele comenzi:

```
student@uso:~$ cd /home/student
student@uso:~$ rm -rf uso.git
student@uso:~$ git clone https://github.com/systems-cs-pub-ro/uso-lab.git
Cloning into 'uso-lab'...
remote: Enumerating objects: 996, done.
remote: Counting objects: 100% (290/290), done.
remote: Compressing objects: 100% (206/206), done.
remote: Total 996 (delta 88), reused 246 (delta 52), pack-reused 706
Receiving objects: 100% (996/996), 82.24 MiB | 8.99 MiB/s, done.
Resolving deltas: 100% (319/319), done.
```

Conectarea la rețea și la Internet

Pentru a parcurge această secțiune este recomandat să descărcați ultima versiune a respository-ului laboratorului. Pentru a descărca ultima versiune a repository-ului rulați comanda `git pull` în directorul `~/uso-lab`.

Infrastructura laboratorului este bazată pe containere docker ale căror imagini vor fi generate pe propriul calculator. Dacă nu aveți deja instalat Docker Engine pe sistem, rulați comanda `~/uso-lab/labs/03-user/lab-container/lab_prepare.sh install` pentru a vă instala aplicația.

După ce ați terminat de lucrat vă recomandăm să opriți containerele rulând comanda `./lab_prepare.sh delete` în directorul `~/uso-lab/labs/03-user/lab-container`.

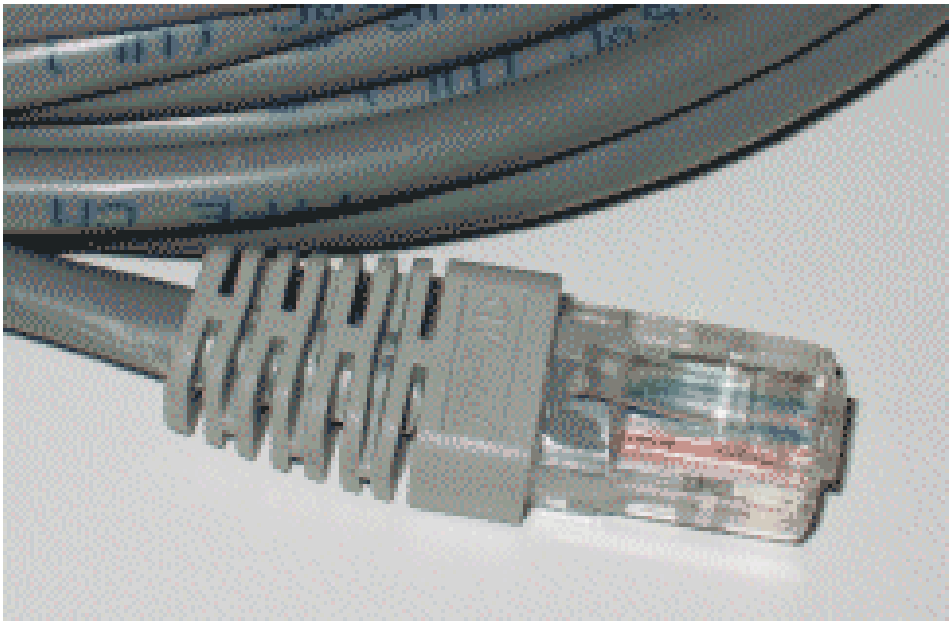
În cadrul acestei secțiuni vom învăța cum să reparăm problemele de conectivitate la rețea (sau, informal, rezolvarea problemei "nu-mi merge Internetul"). Pentru a face asta este necesar să parcurgem toate nivelurile de rețea prin care trec datele pentru a fi trimise în Internet.

În continuare vom prezenta pașii pe care îi urmăm ca să verificăm funcționalitatea nivelului de rețea și cum putem să îl configurăm sumar.

Interacţiunea cu nivelul fizic

Primul nivel cu care interacționăm este nivelul fizic, care are rolul de a trimite date sub formă de semnale prin mediul de transmisie. De exemplu, semnalele electrice sunt transmise prin fir de cupru, pulsurile luminoase prin fibră optică și undele radio prin wireless.

Așa arată un cablu de cupru de tip UTP (Unshielded Twisted Pair):



O altă componentă a nivelului fizic este placa de rețea a sistemului (în engleză, NIC - *Network Interface Card*), care va trimite datele prin mediul de transmisie.

Majoritatea timpului problemele de conexiune la Internet vin de la faptul că nu este conectat cablul de Internet la placa de rețea, sau de la faptul că avem conexiune slabă la rețeaua wireless.

La nivel fizic, putem verifica conexiunea și funcționalitatea unei plăci de rețea uitându-ne la ledurile care reprezintă conexiunea la mediul fizic. Observăm în GIF-ul de mai jos cum arată ledurile unei plăci de rețea funcționale. Dacă acestea nu sunt aprinse, atunci nu vom avea conectivitate la rețea.



Investigarea nivelului fizic al rețelei

În general, în Linux fiecare placă de rețea are asociată câte o interfață de rețea.

Pentru rularea acestui demo rulați în directorul `~/uso-lab/labs/03-user/lab-container` comanda `./lab_prepare.sh install fizic`. Pentru a ne conecta la infrastructura pentru această secțiune vom folosi comanda `./lab_prepare.sh connect fizic`

O interfață de rețea este un mijloc de realizare a configurărilor de rețea asociată de obicei unei plăci de rețea și identificată printr-un nume.

Există interfețe care nu corespund niciunei plăci de rețea fizice. De exemplu ``loopback`` este o interfață virtuală, de auto-adresare, care se adresează sistemului însuși. Este numită ``lo`` in Linux.

La nivelul sistemului de operare putem verifica dacă o placă de rețea este activă folosind comanda următoare:

```
root@fizic:~# ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
293: eth0@if294: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group default
    link/ether 02:42:0a:0a:0a:01 brd ff:ff:ff:ff:ff:ff link-netnsid 0
297: eth1@if298: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default
    link/ether 02:42:0b:0b:0b:01 brd ff:ff:ff:ff:ff:ff link-netnsid 0
299: eth2@if300: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default
    link/ether 02:42:0c:0c:0c:01 brd ff:ff:ff:ff:ff:ff link-netnsid 0
```

Starea fiecărei interfețe de rețea este reprezentată de numărul interfeței și numele ei, împreună cu parametrii săi de rulare. Majoritatea informațiilor afișate de comanda de mai sus nu sunt relevante pentru noi. O opțiune relevantă este valoarea ``state``, urmată de starea interfeței de rețea, care poate să fie ``UP``, ``DOWN`` sau ``UNKNOWN``.

Formatul pentru numele interfețelor diferă de la o distribuție la alta. În cadrul infrastructurii de laborator folosim containere de tip Docker în cadrul cărora numele interfețelor este de forma `ethX`, unde X este un număr. În funcție de distribuția pe care rulăm numele interfețelor poate să se fie într-un format diferit.

Observăm că interfața de rețea cu numele `eth0` este pornită, deoarece linia asociată interfeței conține șirul de caractere `state UP`. În același timp observăm că interfața de rețea `eth1` nu este activă deoarece pe linia sa observăm șirul de caractere `state DOWN`.

Pentru a porni interfața `eth1` vom folosi următoarea comandă:

```
root@uso:~# ip link set up dev eth1
```

Mereu, după ce rulăm o comandă, trebuie să verificăm dacă s-a efectuat cu succes, folosind o metodă de verificare. În cazul de față vom folosi tot comanda `ip link show`:

```
root@fizic:~# ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
293: eth0@if294: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group default
    link/ether 02:42:0a:0a:0a:01 brd ff:ff:ff:ff:ff:ff link-netnsid 0
297: eth1@if298: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group default
    link/ether 02:42:0b:0b:0b:01 brd ff:ff:ff:ff:ff:ff link-netnsid 0
299: eth2@if300: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default
    link/ether 02:42:0c:0c:0c:01 brd ff:ff:ff:ff:ff:ff link-netnsid 0
```

Exercițiu - Pornirea unei interfețe

Porniți interfața de rețea `eth2`.

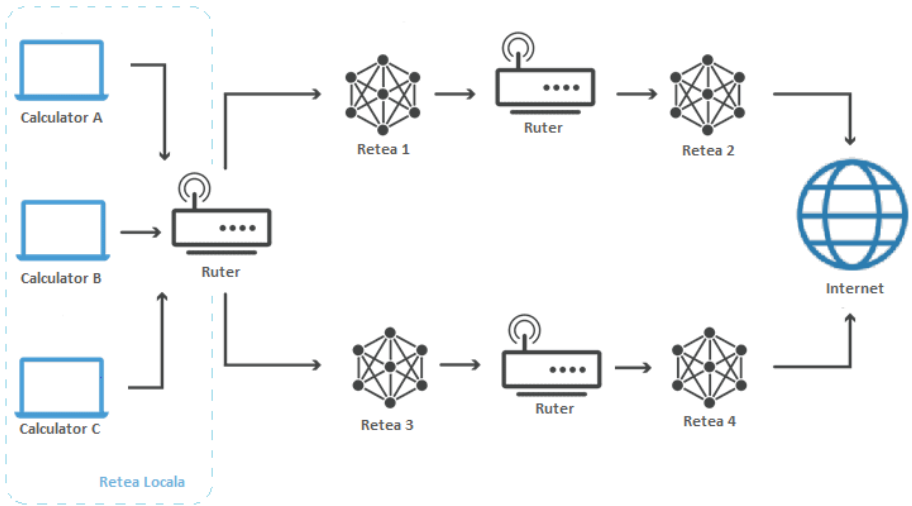
Ieșiți din docker-ul `fizic` folosind comanda `exit`.

Configurarea nivelului Internet

Identificarea adresei de Internet

Pentru rularea acestui demo rulați în directorul `~/uso-lab/labs/03-user/lab-container` comanda `./lab_prepare.sh install internet`. Pentru a ne conecta la infrastructura pentru această secțiune vom folosi comanda `./lab_prepare.sh connect internet`

Internetul este o interconectare de dispozitive, numite stații și organizate în rețele, care se extinde pe toată Planeta. Datele trimise în Internet trebuie redirecționate de la un nod la altul, așa încât să ajungă rapid de la o stație la altă stație.



Deci, pentru ca o stație să comunice cu o altă stație din Internet, trebuie ca cele doua stații să fie conectate la Internet.

Mai exact, stațiile trebuie sa aiba un punct de ieșire din rețeaua locală, conectat la restul rețelelor din Internet, care se numește default gateway și de care vom menționa mai târziu.

Mai apoi, cele două stații trebuie să se poată adresa una alteia. Adică fiecare stație are nevoie de un identificator, o adresă. Cum fiecare casă din lume are o adresă cu care poate fi identificată unic, așa și fiecare stație are o adresă unică in Internet numită adresa IP (*Internet Protocol*).

Fiecare interfață de rețea este o cale diferită către Internet, deci fiecare are nevoie de a avea configurată câte o adresă IP.

Pentru a vedea adresele IP configurate pe interfețele de rețea folosim următoarea comandă:

```
root@internet:~# ip address show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
195: eth0@if196: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:0a:0a:0a:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.10.10.2/24 brd 10.10.10.255 scope global eth0
       valid_lft forever preferred_lft forever
199: eth1@if200: <BROADCAST,MULTICAST> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:0b:0b:0b:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
203: eth2@if204: <BROADCAST,MULTICAST> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:0c:0c:0c:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
```

Există două tipuri de adrese IP (IPv4 și IPv6), dar în cadrul acestui laborator vom lucra numai cu adrese de tip IPv4. Adresele IP ale interfețelor sunt scrise pe liniile care conțin `inet`. Adresele IPv4 sunt de forma `A.B.C.D`, unde A, B, C și D sunt numere cu valori între 1 si 255.

Pentru detalii despre adresele de tip IPv6 folosiți urmăriți această[1)] explicație.

Configurarea unei adrese IP

Există două metode pentru configurarea unei adrese IP pe o interfață:

- configurare statică, prin care noi configurăm manual adresa IP pe interfața de rețea, implică să cunoaștem din ce rețea face parte interfața pe care vrem să o configurăm și ce adrese IP sunt libere;
- configurare dinamică, obținută automat, care nu presupune cunoașterea informațiilor despre rețea, deoarece acestea vor fi primite automat de pe rețea.

Vom insista pe configurarea dinamică, deoarece este mai simplă. În plus, nu avem cum să aflăm informațiile despre rețea înainte de a configura interfața de rețea.

Recapitulare - Pornirea interfețelor de rețea

Faceți modificările necesare astfel încât interfața `eth1` să fie în starea UP.

Configurarea IP-ului în mod dinamic

Pentru a obține o adresă IP în mod dinamic pe o interfață folosim comanda `dhclient`:

```
root@internet:~# dhclient eth1
mv: cannot move '/etc/resolv.conf.dhclient-new.35' to '/etc/resolv.conf': Device or resource busy
```

Linia `mv: cannot move '/etc/resolv.conf.dhclient-new.35' to '/etc/resolv.conf': Device or resource busy` apare mereu în containerele docker atunci când încercăm să obținem o adresă IP folosind comanda `dhclient`. **NU** este o problemă dacă aceasta apare.

Mai sus am rulat comanda pentru a obține o adresă IP pentru interfața `eth1`.

Comanda `dhclient` este bazată pe protocolul DHCP (*Dynamic Host Configuration Protocol*). Acesta presupune că există un server pe rețea care cunoaște ce IP-uri sunt folosite pe rețea și care poate să ofere adrese IP calculatoarelor care fac cereri pe rețea. `dhclient` face o cerere de rezervare a unei adrese IP către serverul DHCP de pe rețea.

Recapitulare - Afișarea adreselor IP configurate pe interfețele de rețea

Afișați adresele IP de pe toate interfețele.

Observați că am obținut o adresă IP pe interfața `eth1`.

Exercițiu - Configurarea dinamică a unei adrese IP

Configurați adresa IP pe interfața `eth2`.

Ștergerea unei configurații de rețea de pe o interfață

Pentru a șterge o adresă IP de pe o interfața folosim comanda `ip address flush` în felul următor:

```
root@internet:~# ip address flush eth1
root@internet:~# ip address show eth1
199: eth1@if200: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:0b:0b:0b:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
```

Putem să afișăm configurația unei singure interfețe de rețea folosind numele interfeței ca parametru pentru comanda `ip address show`

Exercițiu - Ștergerea configurației de rețea

Ștergeți configurația de rețea de pe interfețele `eth1` și `eth2`.

Verificarea conectivității la o altă stație

Pentru rularea acestui demo, comenzile vor fi rulate în cadrul mașinii virtuale USO.

Pentru a verifica conexiunea dintre două stații folosim comanda `ping`. Această comandă trimite mesaje către o stație și așteaptă un răspuns de la ea.

Atunci când testăm conexiunea la internet, vrem să verificăm câteva aspecte, odată ce am obținut o adresă IP de la serverul DHCP:

- verificăm dacă putem să ne conectăm la alte calculatoare din aceeași rețea
- verificăm dacă putem să comunicăm cu stații din afara rețelei

De exemplu, dacă vrem să verificăm conectivitatea la serverul `8.8.8.8` (un server public din Internet), folosim comanda:

```
student@uso:~# ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=61 time=23.0 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=61 time=25.7 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=61 time=24.8 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=61 time=25.2 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3092ms
rtt min/avg/max/mdev = 23.051/24.731/25.707/1.020 ms
```

Comportamentul implicit al comenzii `ping` este să trimită pachete la infinit. Am folosit opțiunea `-c 4` în exemplul de mai sus pentru a reduce numărul de pachete trimise la 4.

Atunci când nu pot fi trimise mesaje către stația identificată prin adresa IP, mesajul de eroare va arăta în felul următor:

```
student@uso:~# ping 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
From 10.10.10.3 icmp_seq=1 Destination Host Unreachable
From 10.10.10.3 icmp_seq=2 Destination Host Unreachable
From 10.10.10.3 icmp_seq=3 Destination Host Unreachable
From 10.10.10.3 icmp_seq=4 Destination Host Unreachable
^C
--- 10.10.10.10 ping statistics ---
4 packets transmitted, 0 received, +4 errors, 100% packet loss, time 3074ms
```

Pentru verificarea conectivității în interiorul rețelei trebuie să verificăm că putem să trimitem mesaje folosind utilitarul `ping` unui calculator din rețea.

În mod implicit comanda `ping` trimite mesaje de verificare a conexiunii la infinit. De data aceasta, în loc să rulăm comanda `ping` folosind opțiunea `-c 4`, am oprit rularea comenzii folosind combinația de taste `Ctrl+c`.

O țintă bună de testare pentru trimiterea mesajelor în rețea este (default) gateway-ul. Un gateway este un dispozitiv de rețea care se ocupă de interconectarea rețelelor și care primește mesaje de la toate stațiile din rețea pentru a le trimite în Internet.

Gateway-ul este configurat static sau dinamic, cum este configurată și adresa IP a unei interfețe.

Pentru a identifica gateway-ul, folosim comanda `ip route show` în felul următor:

```
student@uso:~# ip route show
default via 10.0.2.2 dev enp0s3 proto dhcp metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
169.254.0.0/16 dev enp0s3 scope link metric 1000
192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.4 metric 101
```

Observăm că adresa IP a default gateway-ului este `10.0.2.2`, deoarece acesta se află pe linia care conține șirul de caractere `default`.

Recapitulare - Afișarea adresei IP configurată pe o interfață

Aflați adresa de rețea de pe interfața `enp0s3`.

Adresa IP a gateway-ului și adresa IP a interfeței `enp0s3` sunt foarte similare. Acest lucru se întâmplă deoarece stațiile se află în aceeași rețea.

Exercițiu - Verificarea conectivității cu gateway-ul

Verificați conexiunea cu gateway-ul folosind comanda `ping`.

Pentru verificarea conexiunii la Internet este bine să verificăm cu o adresă consacrată, în care avem încredere că nu va avea probleme tehnice. Un astfel de exemplu este serverul oferit de Google de la adresa IP `1.1.1.1`.

Exercițiu - Verificarea conectivității la Internet

Verificați conexiunea la serverul `8.8.8.8` oferit de Google folosind comanda `ping`.

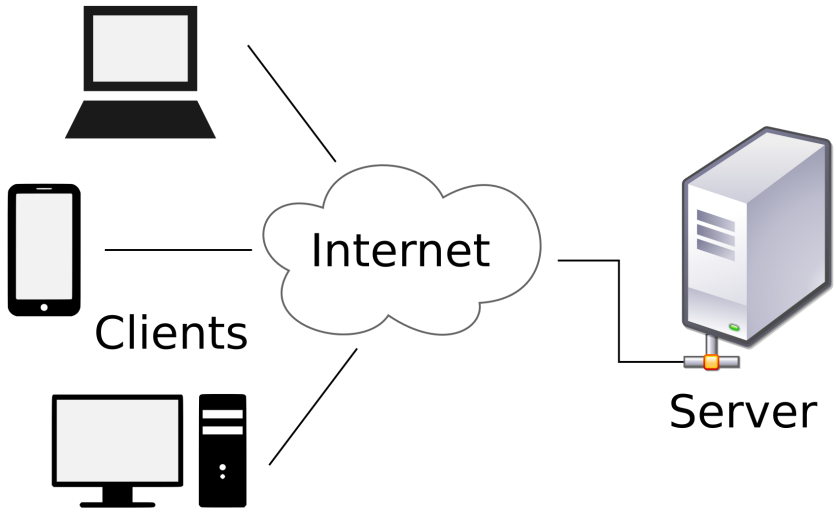
Servicii și clienți de rețea

Dispozitivele pe care le folosim noi devin din ce în ce mai mici, mai eficiente și ieftine. Asta se întâmplă deoarece multe dintre aplicațiile care până nu de curând rulau pe calculatorul propriu s-au mutat în spațiu online. De exemplu, în loc să descărcăm filme și să le urmărim de pe calculator, folosim o aplicație cum ar fi Netflix pentru a transmite prin Internet filmul pe care vrem să îl urmărim. Un alt exemplu relevant este Google Drive, care ne permite să stocăm, să replicăm și să editățm documente într-o interfață web, în loc să le păstrăm local pe calculatorul pe care îl folosim. Toată puterea de procesare și tot spațiul de stocare s-a mutat de pe calculatorul propriu pe servere aflate în Internet.

Vom numi aceste aplicații care rulează în Internet **servicii**.

Un serviciu este o aplicație care oferă o funcționalitate utilizatorilor care apelează la ele. Serviciile în domeniul calculatoarelor lucrează folosind paradigma server-client. Un avantaj major al acestei abordări este că reduce puterea de calcul necesară pentru rularea aplicațiilor de către utilizatori. Aceștia au nevoie doar de o aplicație client care știe să comunice cu serverul. Astfel, aplicația client trimite o cerere către aplicația server, serverul primește cererea, procesează cererea și servește răspunsul aplicației client care a făcut cererea.

Această paradigmă poate fi observată în schema următoare:



Atunci când noi vrem să urmărim un film pe Netflix aplicația client Netflix de pe calculator sau smart TV va trimite o cerere de descărcare a filmului de pe serverul Netflix aflat la distanță.

Clienţi web în linia de comandă

În viață de zi cu zi aplicația pe care o folosim cel mai mult este browserul web, deoarece majoritatea aplicațiilor pe care le folosim au fost transformate în pagini web cu care noi interacționăm. Browserul web este o aplicație care execută o cerere HTTP către un server web, identificat printr-o adresă, un link, prin care face o acțiune și primește un răspuns. De exemplu, când accesăm pagina `www.facebook.com` se trimite o cerere către serverul HTTP, iar acesta trimite un răspuns către browser sub forma unei pagini web, în formatul HTML, pe care browserul o afișează.

Pentru interacțiunea cu serverele web putem folosi și clienți web în linie de comandă. Clienții web folosiți în linie de comandă sunt folositori atunci când nu avem acces la o interfață GUI, sau când încercăm să automatizăm un proces. De exemplu, pentru a verifica automat starea unui site avem nevoie să descărcăm pagina site-ului.

Există mai multe implementări de clienți web în linie de comandă. Vom folosi comanda `wget` pentru descărcarea unei pagini web.

```
student@uso:~$ wget elf.cs.pub.ro
--2020-10-20 23:01:02--  http://elf.cs.pub.ro/
Resolving elf.cs.pub.ro (elf.cs.pub.ro)... 141.85.227.116
Connecting to elf.cs.pub.ro (elf.cs.pub.ro)|141.85.227.116|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 168 [text/html]
Saving to: ‘index.html’

index.html                      100%[=============================================================>]     168  --.-KB/s    in 0s

2020-10-20 23:01:02 (7,61 MB/s) - ‘index.html’ saved [168/168]

student@uso:~$ cat index.html
<html>
    <head>
        <meta name="google-site-verification" content="gTsIxyV43HSJraRPl6X1A5jzGFgQ3N__hKAcuL2QsO8" />
    </head>

    <body>
        <h1>It works!</h1>
    </body>
</html>
```

Comanda `wget` primește ca parametru link-ul către pagina pe care vrem să o descărcăm.

Am folosit comanda `cat` pentru afișarea conținutului fișierului `index.html`. Fișierul `index.html` este fișierul descărcat în mod implicit, dacă nu descărcăm o pagină specifică.

Exercițiu - Descărcarea paginilor web

- Deschideți într-un editor de text pagina web descărcată pentru a vedea conținutul HTML.
- Descărcați pagina web de la adresa `www.facebook.com`. Afișați conținutul fișierului descărcat.

Clienții web nu sunt folosiți doar pentru accesarea paginilor web. Putem folosi clienți web pentru a descărca fișiere indiferent de tipul acestora.

- Descărcați pagina web de la adresa `http://wttr.in/`. Afișați conținutul fișierului descărcat.
- Descărcați fișierul `http://elf.cs.pub.ro/uso/res/final/07-feb/heroes.csv`. Ce tip de fișier este acesta?

Accesul la distanţă în linie de comandă

Pentru rularea acestui demo rulați în directorul `~/uso-lab/labs/03-user/lab-container` comanda `./lab_prepare.sh install ssh`. Pentru a ne conecta la infrastructura pentru această secțiune vom folosi comanda `./lab_prepare.sh connect ssh`

În multe situații atunci când lucrăm cu sisteme, este necesar să rulăm aplicații pe alte stații în afara calculatorului nostru fără să avem acces fizic la stații.

Protocolul cel mai folosit pentru accesul la stații la distanță este protocolul SSH. SSH permite autentificarea la o stație pe care rulează un server SSH. Când ne conecta la o stație, trebuie să precizăm utilizatorul cu care vrem să ne logăm. Pentru autentificare introducem parola utilizatorului, sau folosim o cheie de acces la stație.

Conectarea folosind autentificare cu parolă

Pentru a rula comenzi pe o altă stație putem folosi programul SSH (*Secure Shell*) pentru a ne conecta la acesta în felul următor:

```
student@uso:~$ hostname
uso
student@uso:~$ ssh root@10.10.10.3
The authenticity of host '10.10.10.3 (10.10.10.3)' can't be established.
ECDSA key fingerprint is SHA256:I3Ybkkk7nF2FjwVHMzjkyujDnhlRlnSwPRVwUKm6OCM.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.10.3' (ECDSA) to the list of known hosts.
root@10.10.10.3's password:
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 5.4.0-51-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage
This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/\*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
root@ssh:~# ls /
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@ssh:~# hostname
ssh
```

Unde `root` este numele utilizatorului și `10.10.10.3` este adresa IP, sau hostname-ul stației la care vrem să ne conectăm.

Atunci când ne conectăm la o stație, clientul SSH va încerca să autentifice stația la care ne-am conectat. Dacă ne conectăm pentru prima oară la un sistem trebuie să spunem clientului că am verificat datele clientului la care ne-am conectat.

În mod implicit protocolul SSH va folosi autentificarea cu parolă. Parola introdusă în exemplul de mai sus este parola `root`.

Odată ce ne-am conectat la calculator, avem acces la un shell în care rulăm comenzi ca mai sus. Putem observa că ne-am autentificat pe un calculator diferit deoarece s-a schimbat promptul terminalului de la `student@uso:~$` la `root@ssh:~#` și s-a afișat un mesaj numit *Message of the Day*.

În exemplul de mai sus am rulat comanda `hostname` care afișează numele stației la care ne-am conectat pentru a ne asigura că ne-am conectat pe un nou calculator. Putem observa numele stației și din prompt.

Exercițiu - Conectarea la distanță

- Autentificați-vă la stația cu adresa IP `11.11.11.3` folosind utilizatorul `root` și parola `root`.
- Autentificați-vă la stația cu adresa IP `12.12.12.3` folosind utilizatorul `student` și parola `student`.

Observație:

Atunci când ne conectăm la o stație folosind protocolul SSH este necesar să precizăm un nume de utilizator valid. Dacă utilizatorul nu există, serverul nu va preciza faptul că utilizatorul nu există pe sistem, ci va cere parola utilizatorului, dar nu va permite autentificarea la stație. De ce serverul SSH nu specifică dacă utilizatorul exista sau nu?

Transferul fișierelor la distanţă

Pentru a transfera fișiere la distanță folosim `scp` (secure copy). Comanda `scp` se folosește de protocolul SSH pentru transferul de date între stații, astfel ne putem folosi de modelul de autentificare de la SSH, ca în comanda de mai jos:

```
student@uso:~$ scp /bin/bash student@10.10.10.3:~/
student@10.10.10.3's password:
bash                                          100% 1156KB  30.5MB/s   00:00
student@uso:~$ ssh student@10.10.10.3 ls ~
student@10.10.10.3's password:
bash
```

Fișierul `/bin/bash` a fost copiat de pe stația `uso` pe stația de la adresa IP `10.10.10.3` în directorul home al utilizatorului `student`. Am rulat comanda `ls ~` prin SSH pentru a verifica că s-a efectuat copierea cu succes.

Trimiterea fișierelor poate fi realizată în orice direcție:

- încărcarea fișierelor de la client la server
- descărcarea fișierelor de la server la client

Pentru descărcarea fișierelor de pe un server folosim comanda `scp`:

```
student@uso:~$ scp root@10.10.10.3:/etc/resolv.conf .
root@10.10.10.3's password:
resolv.conf                                   100%   38    19.3KB/s   00:00
student@uso:~$ cat resolv.conf
nameserver 127.0.0.11
options ndots:0
student@uso:~$ ssh root@10.10.10.3 cat /etc/resolv.conf
root@10.10.10.3's password:
nameserver 127.0.0.11
options ndots:0
```

Comanda rulată anterior a descărcat fișierul `resolv.conf` din directorul `/etc/` de pe stația `10.10.10.3` în directorul curent (`.`).

Exercițiu - Copierea fișierelor la distanță

Descărcați fișierul `/etc/passwd` de la adresa `10.10.10.3` folosind utilizatorul `student` și parola `student` în directorul `/home/student/Downloads`.

Copierea directoarelor la distanță

Pentru copierea unui director folosim opțiunea `-r`:

```
student@uso:~$ scp -r ./Downloads/ root@10.10.10.3:~/
root@10.10.10.3's password:
macos.txt                                     100%   18     4.2KB/s   00:00
index.html                                    100%  168   168.4KB/s   00:00
teamviewer_15.10.5_amd64.deb                  100%   14MB  48.1MB/s   00:00
```

Comanda anterioară a copiat directorul `Downloads` și conținutul său din directorul curent în directorul home al utilizatorului `root` de la adresa `10.10.10.3`.

Exercițiu - Copierea directoarelor la distanță

Copiați directorul `/usr` de pe stația de la adresa `10.10.10.3` în directorul home al utilizatorului curent. Vă veți autentifica pe stația de la distanță folosind utilizatorul `root` și parola `root`.

Cuprins

- Laborator 05 (one page)
- Conectarea la rețea și la Internet
- Servicii și clienți de rețea

[1)] https://en.wikipedia.org/wiki/IP_address#IPv6_addresses [https://en.wikipedia.org/wiki/IP_address#IPv6_addresses]

Laborator 06 - Dezvoltarea aplicațiilor

Înainte de începerea laboratorului, rulați următoarele comenzi:

```
student@uso:~$ cd /home/student/uso-lab
student@uso:~$ git stash
student@uso:~$ git pull
student@uso:~$ git stash pop
Updating d192801..bb8cbe7
Fast-forward
 labs/04-appdev/support/simple-make/Makefile | 5 ++++-
 1 file changed, 4 insertions(+), 1 deletion(-)
```

Introducere

Toate dispozitivele electronice pe care le folosim în viața de zi cu zi, de la laptopuri sau calculatoare personale, până la telefoane mobile, frigidere smart sau dispozitive inteligente, au în comun un lucru: *rulează software*. Având în vedere cât de variate sunt sistemele pe care rulează aplicațiile, trebuie să avem mereu în minte sistemul pentru care dorim să creăm o aplicație. De exemplu, dacă aplicația noastră este gândită pentru un ceas inteligent, atunci trebuie să ținem în minte că resursele vor fi limitate; apare astfel o constrângere din punctul de vedere al memoriei disponibile.

Astfel, trebuie să ținem cont de următoarele:

- ce limbaj de programare folosim
- cum scriem și compilăm codul sursă
- cum depanăm codul sursă
- cum versionăm codul sursă

Vom vorbi despre toate aceste aspecte în acest capitol.

Alegerea limbajului de programare

Există trei categorii de limbaje de programare, grupate în funcție de modul în care codul sursă scris de noi (*de limbaj înalt*) ajunge să fie rulat pe procesor: **compilate**, **interpretate** și **hibride**. Alegem limbajul de programare potrivit pentru dezvoltarea programului nostru în funcție de ce avem nevoie: să avem un program portabil sau să avem un program rapid sau să avem acces mai ușor la memorie.

În această carte vom folosi *limbajul C* ca să scriem codul sursă și *compilatorul gcc* pentru a compila codul sursă, însă mai jos descriem particularitățile fiecărui tip de limbaj de programare.

Limbaje de programare compilate

Programele scrise într-un limbaj **compilat** sunt analizate de către un program numit *compilator* înainte ca ele să poată fi rulate pe sistem. În general, limbajele compilate sunt mai rapide în execuție decât cele interpretate sau hibride, deoarece problemele de **sintaxă** (*de scriere*) sunt eliminate din faza *compilării*, înainte să rulăm programul. Neajunsul programelor compilate este faptul că programele **NU** sunt **portabile**, adică nu putem lua un program compilat pe calculator să îl rulăm pe un smartwatch; trebuie să *recompilăm* codul sursă.

Exemple de limbaje compilate sunt **C**, **C++**, **Fortran**, **Rust**, **Go**, **D**.

Limbaje de programare interpretate

Atunci când scriem un program într-un limbaj **interpretat**, programul este trecut printr-un alt program, numit *interpretor* care analizează și rulează fiecare linie de cod scrisă, *pe rând*. Acest lucru înseamnă că programul poate să ruleze 999 de linii de cod, să afișeze ce trebuie, și să se oprească la un 'print' scris greșit pe linia 1000. Dacă alegem să scriem un program într-un limbaj de programare interpretat, atunci programul nostru este **portabil**, adică putem să îl rulăm pe orice sistem pe care avem

interpretorul instalat. Din cauza faptului că programele sunt analizate și rulate linie cu linie, execuția lor poate fi mai lentă.

Exemple de limbaje interpretate sunt **PHP** și **Perl**.

Limbaje de programare hibride

Programele scrise într-un limbaj **hibrid** trec print-un proces care îmbină etapa de *compilare* și etapa de *interpretare*, scopul fiind de a avea în final un program portabil al cărui timp de execuție este mai redus decât în cazul programelor interpretate.

Exemple de limbare hibride sunt **Python**, **Java** și **C#**.

Scrierea codului sursă

Atunci când spunem că *dezvoltăm o aplicație*, spunem, de fapt, că scriem codul sursă, îl verificăm de erori, îl compilăm și reiterăm procesul până când obținem rezultatul dorit. Pentru scrierea codului sursă putem să alegem **editoare de text** sau medii de dezvoltare integrate [https://www.redhat.com/en/topics/middleware/what-is-ide] (*Integrated Development Environment*, *IDE*).

Editoarele de text sunt programe mai simple în care putem edita fișiere text, deci putem dezvolta programe. Printre cele mai cunoscute editoare de text se numără GNU Nano [https://www.nano-editor.org], Vim [https://www.vim.org], Sublime [http://sublimetext.com], Atom [https://atom.io], Visual Studio Code [https://code.visualstudio.com].

IDE-urile au anumite funcționalități avansate, multe dintre ele fiind adaptate unui singur limbaj de programare. În plus, ele au integrat un **compilator/interpretor** pentru limbajul/limbajele suportat/e. Astfel, la o simplă apăsare de buton, programul este rulat. Printre IDE-uri se număra: Microsoft Visual Studio [https://visualstudio.microsoft.com], Eclipse [https://www.eclipse.org/ide/], IntelliJ [https://www.jetbrains.com/idea/], Pycharm [https://www.jetbrains.com/pycharm/].

În această carte vom folosi *GNU Nano* ca editor de text principal. Este un editor CLI și se pornește folosind comanda `nano`, așa cum a fost prezentat în capitolul *Lucrul cu fișiere*.

Compilarea unui fișier cod sursă C

În această secțiune urmărim să învățăm pașii pentru a compila un program, adică a-l aduce de la cod sursă la executabil. Pentru aceasta vom crea un program care verifică dacă un număr citit de la tastatură este prim sau nu.

Pașii pe care îi vom face sunt:

- vom scrie într-un nou fișier cu extensia .c programul care implementează algoritmul
- vom compila codul sursă folosind compilatorul GCC [https://gcc.gnu.org]
- vom testa că programul funcționează.

Crearea unui fișier cod sursă

Creăm un fișier nou cu numele `is-prime.c` cu implementarea algoritmului de verificare. Copiem codul sursă de mai jos și îl lipim în nano ca în imaginea de mai jos:

```
#include <stdio.h>

int check_if_prime(int n)
{
    int i;

    for (i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            return 0;
        }
    }
```

```
    return 1;
}

int main(void)
{
    int n;

    printf("Please gimme a number: ");
    scanf("%d", &n);

    if (check_if_prime(n)) {
        printf("%d is prime\n", n);
    } else {
        printf("%d is not prime\n", n);
    }

    return 0;
}
```

Mai multe detalii despre folosirea editorului de text `nano` găsim în capitolul *Lucrul cu fișiere*.

Compilarea codului sursă în executabilul a.out

Avem fișierul cod sursă `is-prime.c` și vrem să obținem un program pe care să-l rulăm pe sistemul nostru. Pentru aceasta, trebuie să compilăm fișierul `is-prime.c`. Acest program este de fapt un **executabil** (*fișier binar*). Executabilele sunt fișiere care conțin instrucțiuni pe care sistemul de calcul le poate interpreta și rula.

Creăm un executabil din fișierul `is-prime.c` folosind comanda `gcc`: Astfel invocăm compilatorul GCC(GNU Compiler Collection) instalat anterior și îi dăm ca argument numele fișierului dorit.

```
student@uso:~$ gcc is-prime.c
student@uso:~$ ls -l
total 16
-rwxr-xr-x 1 student student 8448 Oct 26 06:34 a.out
-rw-r--r-- 1 student student  406 Oct 26 06:17 is-prime.c
```

Așa cum vedem în rezultatul rulări comenzii `ls -l` de mai sus, executabilul se numește `a.out`. Acesta este numele **implicit** dat de GCC. Adică `a.out` va fi numele tuturor fișierelor executabile generate cu GCC, indiferent de fișierul cod sursă. Rulăm executabilul `a.out` în felul următor și introducem de la tastatură un număr:

```
student@uso:~$ ./a.out
Please gimme a number: 13
13 is prime
```

Programul funcționează: citește un număr de la tastatură și afișează dacă acesta este prim sau nu.

Compilarea codului sursă într-un executabil cu nume diferit

Numele `a.out` este implicit, deci toate programele compilate cu GCC se vor numi `a.out`. Putem configura un nume diferit pentru executabilul obținut.

Spre exemplu, pentru programul care verifică dacă un număr este prim sau nu, numim executabilul `is-prime`. Creăm un executabil cu numele `is-prime` din fișierul `is-prime.c` folosind opțiunea `-o` a comenzii `gcc`:

```
student@uso:~$ gcc -o is-prime is-prime.c
student@uso:~$ ls -l
total 28
-rwxr-xr-x 1 student student 8448 Oct 26 06:34 a.out
-rwxr-xr-x 1 student student 8448 Oct 26 06:57 is-prime
-rw-r--r-- 1 student student  406 Oct 26 06:17 is-prime.c
```

Opțiunea `-o` urmată de numele ales pentru program (`is-prime`) transmite compilatorului ca programul să se numească `is-prime`, și nu `a.out`. Rulăm executabilul `is-prime` în felul următor și introducem de la tastatură un număr:

```
student@uso:~$ ./is-prime
Please gimme a number: 13
13 is prime
```

Vedem că, deși au nume diferit, programele `a.out` și `is-prime` au același comportament. Acest lucru este normal deoarece ele sunt 2 fișiere executabile obținute din același fișier cod sursă obținute folosind același compilator: GCC.

Exerciții

1. Creați un fișier cu numele `is-palindrome.c` care să conțină următorul conținut:

```
#include <stdio.h>

int check_if_palindrome(int n)
{
    int new_n = 0;
    int original_n = n;

    while (n > 0) {
        int r = n % 10;
        n /= 10;
        new_n = new_n * 10 + r;
    }

    return (new_n == original_n) ? 1 : 0;
}

int main(void)
{
    int n;

    printf("Please gimme a number: ");
    scanf("%d", &n);

    if (check_if_palindrome(n)) {
        printf("%d is a palindrome\n", n);
    } else {
        printf("%d is not a palindrome\n", n);
    }

    return 0;
}
```

1. Compilați fișierul `is-palindrome.c` într-un executabil cu numele `a.out` folosind `gcc`. Verificați funcționalitatea programului.
2. Compilați fișierul `is-palindrome.c` într-un executabil cu numele `is-palindrome` folosind `gcc`. Verificați funcționalitatea programului.

Introducere în utilitarul Make și fișiere Makefile

În secțiunile anterioare, am compilat fișiere cod sursă C folosind compilatorul GCC. Dezvoltarea unui program este un proces *continuu*, nu scriem tot codul dintr-o singură iterație și de multe ori ajungem să îl modificăm pe parcurs. Vrem să testăm schimbările aduse în program. Pentru aceasta trebuie să *recompilăm* fișierele pe care le-am modificat și să creăm un nou executabil.

Automatizarea procesului de compilare ne ajută să fim eficienți atunci când dezvoltăm un proiect: * În loc să dăm de fiecare dată toate comenzile pentru recompilarea fișierelor, putem să dăm o singură comandă care să le facă pe toate. * Putem să evităm compilarea unor fișiere pe care nu le-am modificat de la ultima compilare, astfel salvând mult timp. Acest proces se numește *build automation*. Există mai multe soluții de build automation[1)]. În aceast laborator vom folosi utilitarul Make [https://linux.die.net/man/1/make] împreună cu fișiere Makefile

[https://www.gnu.org/software/make/manual/make.html#Makefiles] ca să automatizăm procesul de compilare.

În secțiunile următoare vom vedea cum funcționează utilitarul `make` și cum arată un fișier Makefile. După, vom crea un fișier Makefile pentru un proiect dat.

Folosirea unui Makefile existent

În această secțiune vom compila programul Hangman [https://rextester.com/WYT71966] folosind un fișier Makefile.

Întrăm în directorul `~/uso-lab/labs/04-appdev/support/simple-make` folosind comanda `cd`:

```
student@uso:~$ cd ~/uso-lab/labs/04-appdev/support/simple-make
student@uso:~/.../04-appdev/support/simple-make$ ls
hangman.c  Makefile
```

Avem în director un fișier cod sursă C, `hangman.c`, și un fișier Makefile existent, astfel că putem utiliza comanda `make`. Aceasta va rula scriptul scris în Makefile, care, în cazul nostru, urmărește compilarea fișierului hangman.c.

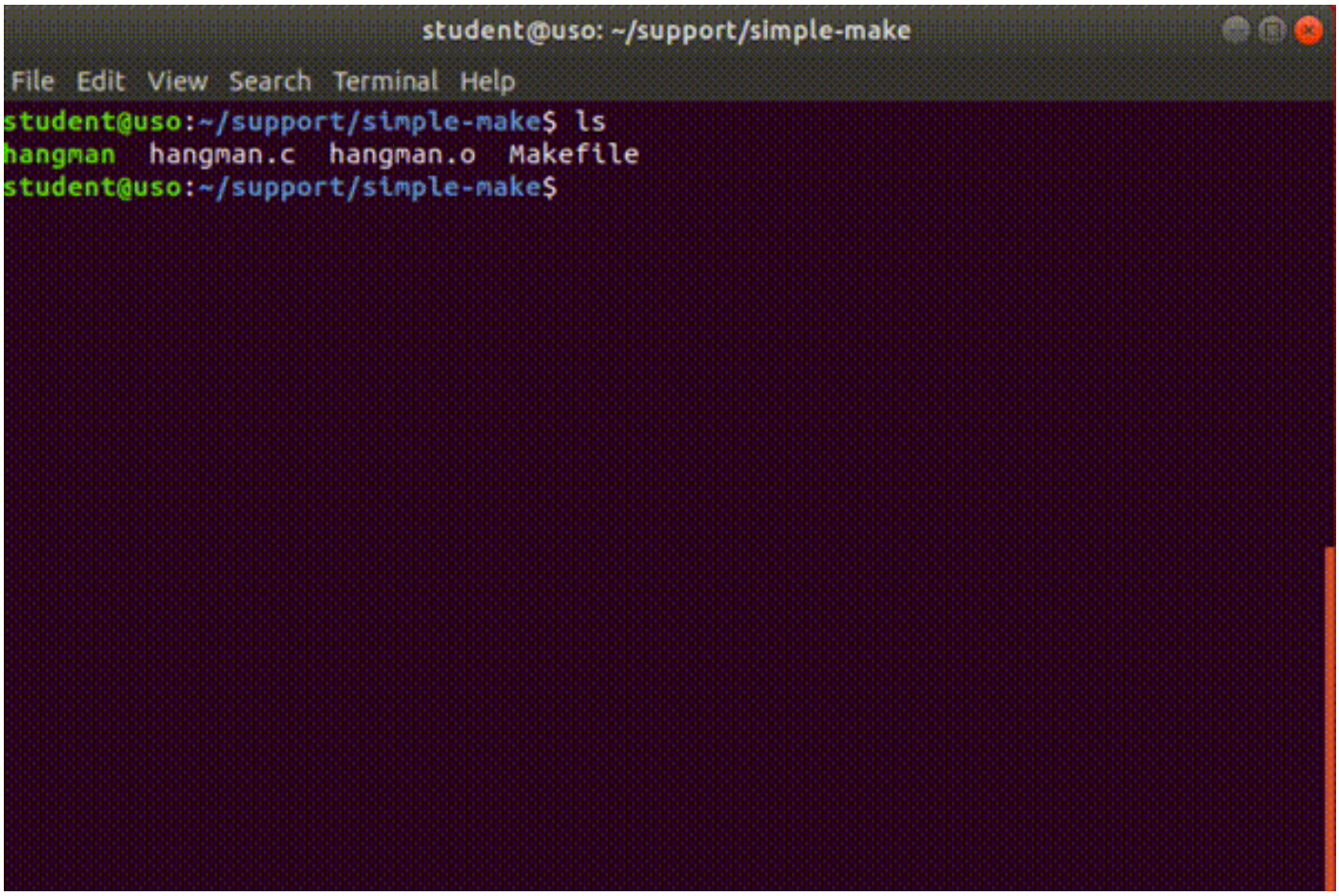
```
student@uso:~/.../04-appdev/support/simple-make$ make
gcc -c hangman.c
gcc -o hangman hangman.o
student@uso:~/.../04-appdev/support/simple-make$ ls
hangman  hangman.c  hangman.o  Makefile
```

Comanda `make` a rulat, de fapt, 2 comenzi:

1. De *compilare*: `gcc -c hangman.c`, comandă prin care am creat fișierul obiect `hangman.o`
2. De *legare*: `gcc -o hangman hangman.o`, comandă prin care am creat executabilul `hangman`.

Practic, scriind doar comanda `make`, am trecut fișierul `hangman.c` prin toate etapele compilării și am obținut executabilul final.

Rulăm executabilul `hangman` ca să vedem că funcționează, ca în imaginea de mai jos:



Înțelegerea formatului Makefile

În secțiunea anterioară, `Folosirea unui Makefile existent`, am folosit fișierul `Makefile` ca să compilăm programul Hangman. Ca să putem crea un Makefile pentru un proiect al nostru, trebuie să înțelegem formatul fișierului Makefile. În această secțiune vom folosi fișierul Makefile pe care l-am folosit anterior ca suport.

Fișierul `Makefile` folosit la programul Hangman are următorul conținut:

```
all: hangman

hangman: hangman.o
    gcc -o hangman hangman.o

hangman.o: hangman.c
    gcc -c hangman.c

clean:
    rm -rf *.o hangman
```

Liniile din fișier sunt de două tipuri:

1. **Regulă**, care are formatul `regulă: <dependență>` (`all: hangman` sau `clean:`). Regula trebuie să existe, dependența este opțională. O *regulă* din fișierul Makefile este, de fapt, un nume asociat unei comenzi.

```
student@uso:~/support/simple-make$ make clean
rm *.o hangman
student@uso:~/support/simple-make$ ls
hangman.c   Makefile
```

2. Observăm deci că putem rula `make + <nume regulă>`, dar și `make` simplu. Utilizând prima formă, din fișierul Makefile se va executa doar regula dorită, în cazul nostru fiind codul specific regulii 'clean'. Utilizând a doua formă, simplă, se va apela implicit prima regulă găsită din fișierul Makefile, în cazul

nostru regula 'all'. Aceasta la rândul ei poate trimite către una sau mai multe reguli care să fie executate în succesiune, în cazul nostru urmând regula 'hangman'.

Introducere în Git și GitHub

GitHub [http://www.github.com/] este o platformă online, pe care dezvoltatorii o pot folosi pentru a stoca și versiona codul lor sursă. **Git** este un sistem de management și versionare a codului sursă care permite lucrul eficient la un proiect software.

Astfel, Git este utilitarul folosit în terminal, iar GitHub este serverul și aplicația web pe care rulează acesta, locul în care păstrăm repository-ul remote.

Folosim Github pentru:

- Vor fi cazuri când vom *strica* o versiune a codului și vom avea nevoie să revenim la o versiune corectă, caz în care un istoric de versiuni ne-ar fi de folos. În caz contrar, va trebui să salvăm diferite versiuni a proiectului nostru că niște checkpoints pe PC-ul nostru, ocupând mult spațiu de depozitare.
- Vom putea să lucrăm la un proiect de pe un alt sistem, în afară de al nostru, sau vom vrea să cerem feedback pe codul scris de noi.

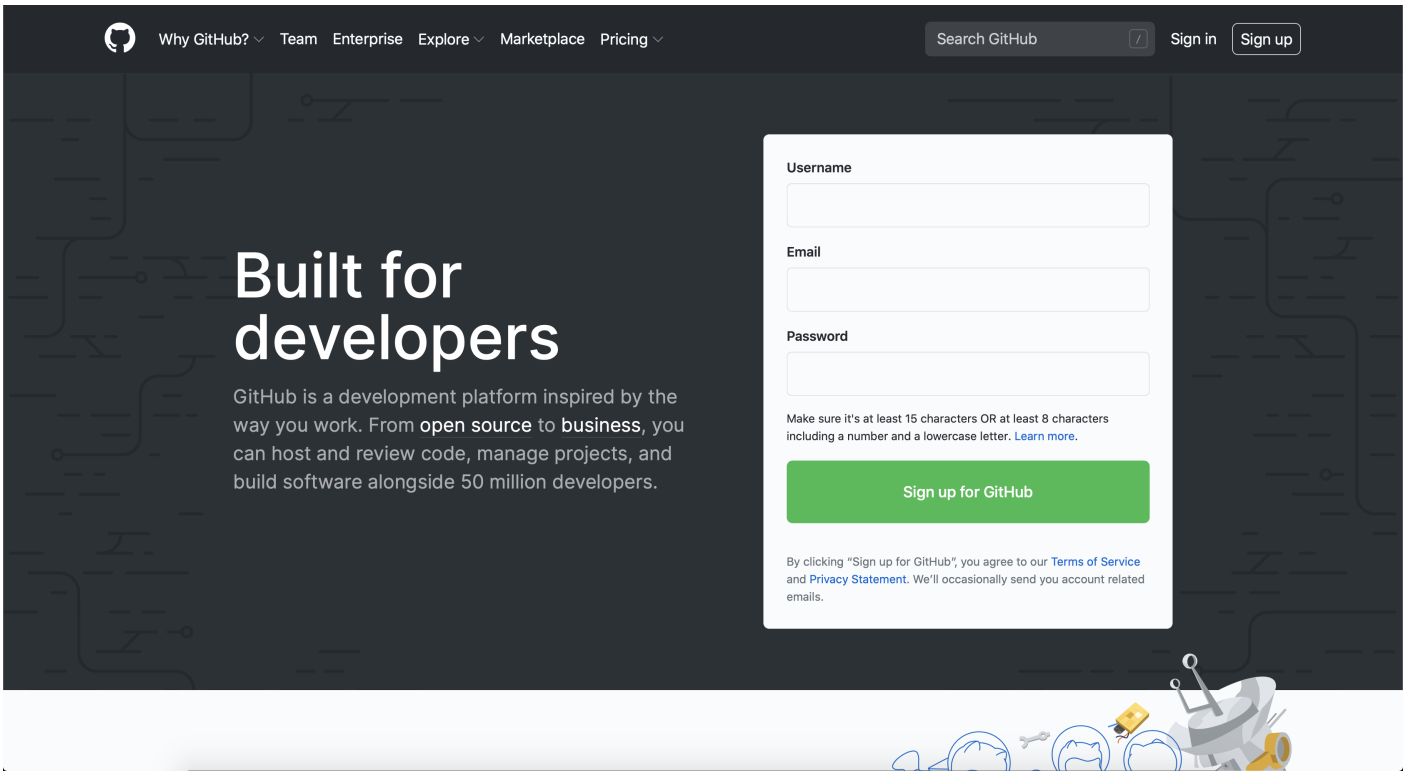
- Când vorbim despre un proiect software vrem să avem dezvoltatori, oameni care să lucreze împreună cu noi la proiect. Dezvoltatorii au nevoie de acces la codul sursă al proiectului software la care lucrăm. După ce le dăm acces, vrem ca fiecare dezvoltator să știe la ce a lucrat și la ce lucrează ceilalți; ca să nu se suprapună, ca să ajute și ca să ofere feedback.

- Pentru a putea rezolva problemele de sincronizare între doi sau mai mulți colegi de echipă care lucrează la același proiect, ne ajută să avem un **sistem de versionare a codului**, adică să avem un istoric de modificări. Fiecare modificare înseamnă o nouă versiune a proiectului; avem astfel o listă de versiuni gestionată de sistemul de versionare a codului. Pe lângă rezolvarea problemelor de sincronizare, versionarea codului aduce și alte avantaje cum ar fi revenirea la o versiune mai veche a proiectului, găsirea rapidă a autorului unei secvențe de cod sau, pur și simplu, organizarea unui proiect.

Crearea unui cont pe GitHub (dacă nu aveți deja)

Înainte de toate, ne asigurăm că avem cont pe GitHub. Dacă aveți deja un cont pe GitHub, puteți trece la subsecțiunea următoare: `Pregătirea inițială a mediului Git`.

Dacă nu aveți cont, intrați pe GitHub [http://www.github.com/]. Pagina de pornire va arăta similar cu cea din imaginea de mai jos.



Introduceți un nume de utilizator (*username*), adresa voastră de e-mail și o parolă sigură pentru cont. Pentru validarea contului, accesați-vă căsuța de e-mail. Acolo veți găsi un e-mail în care vi se explică cum se poate valida noul cont creat. Verificați și căsuța **spam** în caz că nu ați primit nimic în inbox.

Crearea unui token de autentificare (dacă nu aveți deja)

Înainte să treceți la urmatoarele secțiuni, urmați pașii de mai jos:

1. Autentificați-vă pe GitHub.
2. Mergeți pe această pagină [https://github.com/settings/tokens]. Vom genera un token de autentificare pentru GitHub.
3. Apăsați pe `Generate new token` și introduceți parola voastră de GitHub.
4. La câmpul `Note` scrieți un nume pentru token, ca să știți pentru ce îl puteți folosi ulterior. Spre exemplu, puteți scrie `uso-lab`.
5. La câmpul `Expiration` selectați `No expiration`.
6. La câmpul `Select scopes` bifați `repo Full control of private repositories` (se vor selecta automat și căsuțele din lista lui `repo`).
7. Apăsați pe `Generate token`.
8. Veți vedea o căsuță verde cu un șir de caractere, salvați-l. **Acesta va fi folosit în loc de parolă atunci când vi se va cere parola în linia de comandă.**

Pregătirea inițială a mediului Git

Ca să utilizăm Git, facem în primă fază niște pași de configurare. Adică vom configura numele și e-mail-ul nostru, ca mai jos:

```
student@uso:~$ git config --global user.name "Prenume Nume"
student@uso:~$ git config --global user.email "adresa_de_email@example.com"
```

În listingul de mai sus "Prenume Nume" și "adresa_de_email@example.com [mailto:adresa_de_email@example.com]" sunt placeholdere. Le înlocuiți cu datele voastre. De exemplu, pentru autorul acestei secțiuni, comenzile rulate sunt:

```
student@uso:~$ git config --global user.name "Liza Babu"
student@uso:~$ git config --global user.email "lizababu@example.com"
```

Această configurare trebuie făcută o singură dată pe PC / Virtual Machine. Pentru a verifica că setările s-au salvat, verificăm rulând acest lucru rulând `git config --list`.

În următoarele secțiuni vom da doar comenzi pentru Git, așadar toate comenzile date sunt precedate de `git`.

Crearea primului repository

Pentru a lucra la un proiect software, creăm un **repository software**. Un repository reprezintă o metodă de împărțire a proiectelor: poate fi o temă la o materie, poate conține chiar toate temele la o materie, în directoare diferite sau poate conține doar un simplu README.md;

Vom crea unul pe GitHub (numit de acum remote), unul local (la noi pe PC), după care le vom interconecta.

Repository software

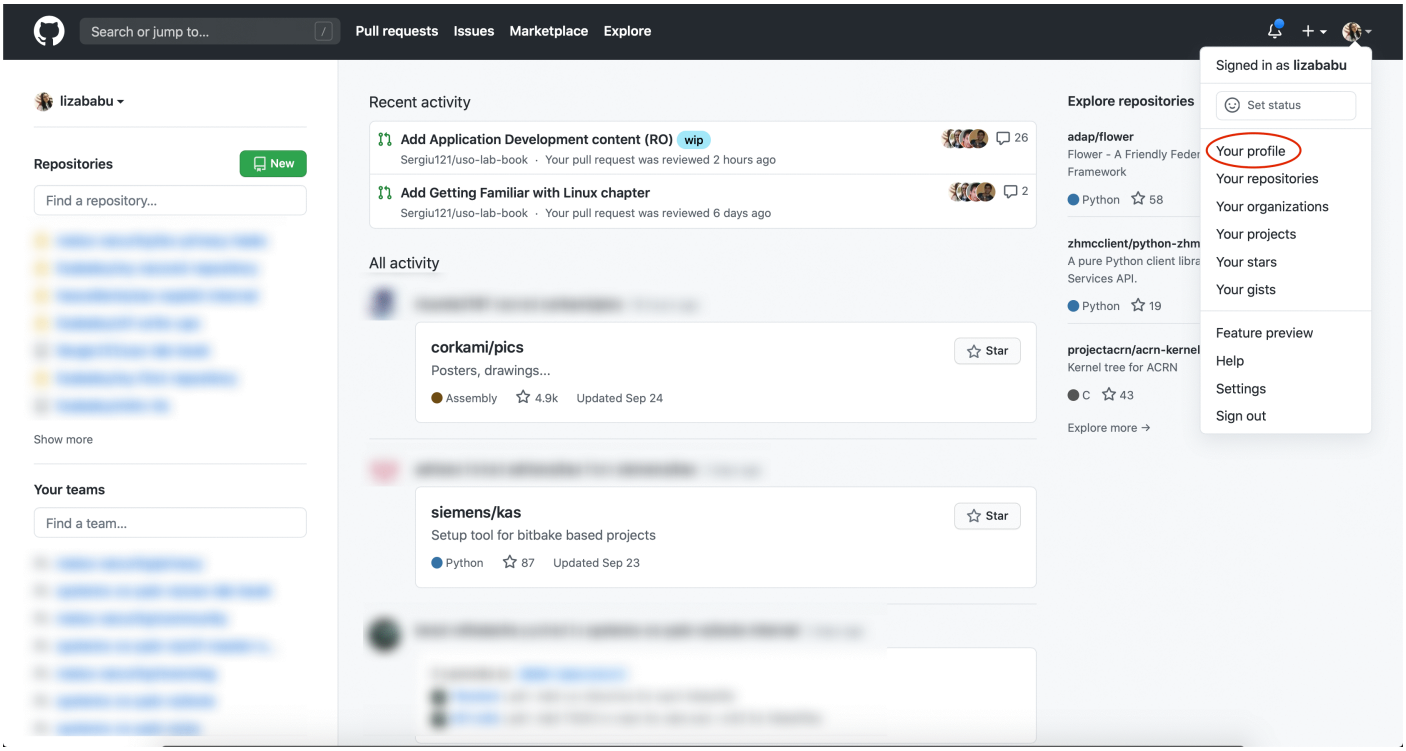
Proiectul este stocat într-un **repository software**. Repository-ul conține fișierele proiectului: codul sursă, fișiere de configurare. De obicei acesta vine însoțit și de un fișier **README.md** în care se găsesc informații despre proiect: care este scopul proiectului, cum se compilează, pe ce platforme rulează.

Repository-urile sunt de două tipuri: **locale** și **remote**. Acestea pot fi interconectate și să refere de fapt același proiect. Repository-ul local este cel pe care îl avem la noi pe calculator, pe când cel remote este unul stocat pe un server (în cazul nostru **GitHub**). Este doar o diferență de perspectivă între cele două, ele nu diferă din punct de vedere tehnic. De obicei, într-un proiect Git / GitHub există un repository central (remote) și mai multe repository-uri secundare (locale), câte unul pentru fiecare dezvoltator din echipa proiectului.

Printre cele mai importante operații cu un repository sunt: **init**, **fork**, **clone**.

Crearea unui repository gol pe GitHub

Ne autentificăm pe GitHub. Urmărim pașii prezentați în imaginea de mai jos și explicați imediat după.



1. Apăsăm pe săgeata din meniul din dreapta sus și vedem ceva similar cu imaginea de mai sus.
2. Apăsăm pe `Your profile` pentru a merge pe profilul nostru. Aici este locul în care vom putea vedea contribuțiile noastre pe GitHub. În partea de sus a ecranului vom vedea un meniu orizontal care conține 4 opțiuni: `Overview`, `Repositories`, `Projects` și `Packages`.

3. Apăsăm pe `Repositories`. Acum vom vedea întreaga listă de repository-uri pe care le avem. Pentru a crea unul nou, apăsăm pe butonul verde din dreapta sus pe care scrie `New`.
4. Acum este momentul în care vom da un nume proiectului nostru, o descriere succintă a acestuia și vom putea decide dacă să fie **public** (vizibil tuturor utilizatorilor) sau **privat** (vizibil doar pentru noi și eventualii colaboratori ai proiectului). Ne va apărea un formular similar cu cel din imaginea de mai sus. Pentru acest tutorial vom crea un repository **public**). Este indicat ca numele repository-ului să descrie bine proiectul, în cazul nostru `array-sorting-algorithms`. Descrierea proiectului este opțională, dar e recomandat să o adăugăm pentru a fi ușor de înțeles pentru cei care vor ajunge la proiectul nostru.
5. Apăsăm pe `Create repository`. Vor apărea câteva instrucțiuni pentru crearea unui repository local nou și conectarea celui nou cu cel remote. Acest lucru este acoperit în secțiunile următoare.

Acum avem un repository creat remote, pe GitHub, numit `array-sorting-algorithms`.

Crearea unui repository gol local

Creăm un director din ierarhia de fișiere în care vom inițializa repository-ul Git.

În acest tutorial creăm directorul `array-sorting-algorithms` în directorul home (adică `/home/student` sau `~`), folosind comenzile de mai jos:

```
student@uso:~$ pwd
/home/student
student@uso:~$ mkdir array-sorting-algorithms
student@uso:~$ cd array-sorting-algorithms
student@uso:~/array-sorting-algorithms$ git init
Initialized empty Git repository in /home/student/array-sorting-algorithms/.git/
student@uso:~/array-sorting-algorithms$ ls -a
./    ../   .git/
```

Mai sus am inițializat repository-ul local prin comanda `git init`, dată în directorul ales (`array-sorting-algorithms`) din directorul home al utilizatorului student (`/home/student`).

Acum avem un repository creat local, numit `array-sorting-algorithms`.

Init

Operația **init** este una locală și are rolul de a inițializa un repository gol, local. Inițializarea repository-ului local înseamnă crearea, în directorul ales, a mediului pentru a putea lucra la un proiect software versionat Git. Această operare duce la crearea unui director numit `.git` în care se vor ține ulterior date suplimentare despre repository, numite **metadatele** repository-ului.

Am folosit opțiunea `-a` (`ls -a`) pentru a afișa și fișierele și directoarele ascunse. Directorul `.git` este un director ascuns.

Conectarea celor două repository-uri

Am creat până în acest moment un repository local (pe PC-ul nostru) și unul remote (platforma GitHub). Trebuie să le interconectăm pentru a lucra cu ele.

În cazul în care suntem mai mulți membri în echipă, fiecare membru va conecta repository-ul său local, la repository-ul remote. Pentru conectarea celor două repository-uri folosim comanda de mai jos, dată în directorul unde este repository-ul local Git (în cazul nostru `/home/student/array-sorting-algorithms`):

```
student@uso:~/array-sorting-algorithms$ git remote add origin https://github.com/{username}/array-sorting-algorithms.git
```

În comanda de mai sus `{username}` este numele utilizatorului nostru de pe GitHub. De exemplu, pentru autorul acestui capitol, `{username}` se înlocuiește cu `lizababu`.

Conectarea celor două repository-uri înseamnă setarea repository-ului `origin`, adică repository-ului remote la care se conectează cel local.

Imaginea de mai jos arată cum arată cum se contectează repository-urile **remote** și **local**. Sincronizarea lor se face prin intermediul operațiilor **push** și **pull** care sunt prezentate pe parcursul secțiunii `Primele comitturi`.



Primele commituri

Odată creat repository-ul, putem să începem să lucrăm la proiect.

Lucrul la proiect înseamnă să adăugăm și să ștergem fișiere sau să modificăm fișiere existente. De obicei este vorba de fișiere text (*human-readable*), cel mai des fișiere cod sursă[2)]. Vrem să salvăm aceste adăugări și modificări; apoi să salvăm din nou alte modificări; și tot așa.

Salvarea acestor modificări înseamnă crearea unui **commit** în repository.

Același lucru îl fac și ceilalți colegi care lucrează la același proiect. Fiecare commit împachetează un set de adăugări și modificări realizate de un dezvoltator al proiectului. Având commiturile în repository putem să gestionăm mai ușor proiectul, adică:

- să revenim la un commit anterior (adesea chiar ultimul) dacă modificările cele mai recente "strică" proiectul
- să vedem cine este autorul anumitor modificări
- să creăm o ramură de dezvoltare separată pornind de la un commit anterior, pe care să încercăm o funcționalitate nouă, fără a afecta restul proiectului

Putem să ne gândim la un commit ca la un checkpoint. Când lucrăm pe repository-ul local, orice modificări aducem sunt doar la noi pe calculator. Asta înseamnă că un coleg cu care lucrezi la proiect nu va vedea ce ai lucrat tu. Astfel că, atunci când lucrăm la un proiect, periodic sau când terminăm un anumit task / o funcție importantă ce vrem să fim singuri că s-a salvat, vom face un commit. Mai exact,

salvăm tot ce am lucrat într-un checkpoint cu nume sugestiv (un mesaj de commit) la care vom putea reveni mai târziu în cazul în care stricăm ceva în program.

Git se ocupă de păstrarea și gestiunea istoricului repository-ului nostru prin păstrarea listei de commituri făcute. Adică Git păstrează un istoric de versiuni al proiectului.

Când facem un commit, acesta va fi reținut în repository-ul Git local, nu și în repository-ul Git remote.

Fără a actualiza și repository-ul remote, ceilalți colegi nu vor putea vedea schimbările făcute de noi. Vrem, așadar, ca modificările făcute local să se găsească și remote. Adică să publicăm commiturile din repository-ul local în repository-ul remote. Realizăm publicarea prin operația **push**.

Vom vedea în următoarele secțiuni care sunt pașii pentru a crea un commit și pentru a-l publica.

În următoarele secțiuni vom lucra în repository-ul `array-sorting-algorithms` creat în secțiunea `Introducere în Git și GitHub`. Vom crea, pas cu pas, un proiect software scris în limbajul de programare C, care conține mai mulți algoritmi de sortare a unui vector de elemente întregi.

Punctual, în această secțiune, vom crea fișierul `README` al proiectului și scheletul de cod pentru algoritmii de sortare **Bubble Sort**, **Merge Sort** și **Radix Sort**. Vom crea commituri în repository-ul local pentru fiecare schimbare, după care vom publica commiturile astfel încât schimbările să fie vizibile și pe GitHub, în repository-ul remote.

Adăugarea unui fișier README

O bună practică, prezentă în majoritatea proiectelor software, este să adăugăm un fișier `README` în care se află informații despre un proiect. Spre exemplu, în `README` se află informații despre ce funcționalități are proiectul nostru, cum se compilează un proiect, cum se rulează, pe ce tip de platforme poate fi rulat etc.

Un fișier `README` este un fișier text. Îl putem crea și îi putem adăuga titlul `Sorting Algorithms for Beginners` folosind un editor sau, mai simplu și mai direct, folosind comanda de mai jos:

```
student@uso:~/array-sorting-algorithms$ echo "# Sorting Algorithms for Beginners" > README.md
student@uso:~/array-sorting-algorithms$ ls -a
./         ../        .git/      README.md
```

Caracterul # din fața textului `Sorting Algorithms for Beginners` are rol de a formata textul sub formă de titlu. Nu intrăm în mai multe detalii aici pentru că nu face obiectul cărții.

Folosim extensia `.md` care semnalează un fișier de tip Markdown [https://www.markdownguide.org]. Facem acest lucru deoarece pe GitHub fișierele `README` sunt afișate în format Markdown. Acest format este simplu de înțeles, însă nu face obiectul acestei cărți, deci nu vom insista pe înțelegerea lui.

Crearea primului commit

Acum avem în repository un nou fișier: `README.md`. Vrem să reținem acest lucru în repository prin crearea unui commit, primul.

Pașii creării unui commit sunt următorii:

1. Verificăm repository-ului. Cu alte cuvinte, verificăm ce modificări au fost făcute în repository de la ultimul commit.
2. Adăugăm fișierele pe care vrem să le împachetăm într-un commit în **staging area**, adică în lista de fișiere pe care Git le organizează.
3. Alegem un mesaj de commit. Creăm commitul.
4. Publicăm commitul și pe repository-ul remote.

Detaliem acești pași în continuare.

Verificarea stării repository-ului local

Prin **starea repository-ului** înțelegem forma la care am adus proiectul prin modificările noastre. Aceasta include ce fișiere am creat, modificat sau șters de la ultimul commit. Ne interesează întotdeauna starea repository-ului pe care lucrăm.

Pentru a verifica starea repository-ului folosim comanda `git status`:

```
student@uso:~/array-sorting-algorithms$ git status
On branch master

No commits yet

Untracked files:
(use "git add <file>..." to include in what will be committed)

    README.md
```

A doua linie afișată `No commits yet` ne spune că nu am făcut până acum niciun commit, adică am pornit de la un repository gol.

În ultima parte a outputului se află o listă de fișiere `untracked`, adică lista fișierelor pe care Git le vede ca nou adăugate în repository-ul curent, dar pe care nu le monitorizează încă. Acest lucru înseamnă că, deocamdată, orice modificare vom face asupra acestor fișiere nu va fi urmărită (*tracked*) de Git. În cazul nostru, fișierul aflat în starea *untracked* este `README.md`.

Adăugarea unui fișier (în staging area)

Un commit va conține o listă de modificări: fișiere adăugate, fișiere șterse, conținut modificat. Un pas intermediar în crearea unui commit este pregătirea modificărilor ce vor fi parte din commit. Acest pas de pregătire înseamnă să adăugăm (*add*) aceste modificări într-o zonă de lucru pentru Git, numită **staging area**.

În cazul nostru, vrem să adăugăm fișierul `README.md` în *staging area*. Facem acest lucru folosind comanda `git add`:

```
student@uso:~/array-sorting-algorithms$ git add README.md
```

Verificăm ce s-a schimbat în urma adăugării fișierului `README.md` în staging folosind comanda `git status`:

```
student@uso:~/array-sorting-algorithms$ git status
On branch master

No commits yet

Changes to be committed:
(use "git rm --cached <file>..." to unstage)

    new file:   README.md
```

Primele 2 mesaje afișate au rămas neschimbate. Partea interesantă apare la ultima parte a outputului. Vedem că mesajul a devenit `Changes to be commited`. Acest lucru înseamnă că acum Git urmărește noile modificări și așteaptă ca modificările să fie adunate într-un commit.

Crearea commitului local

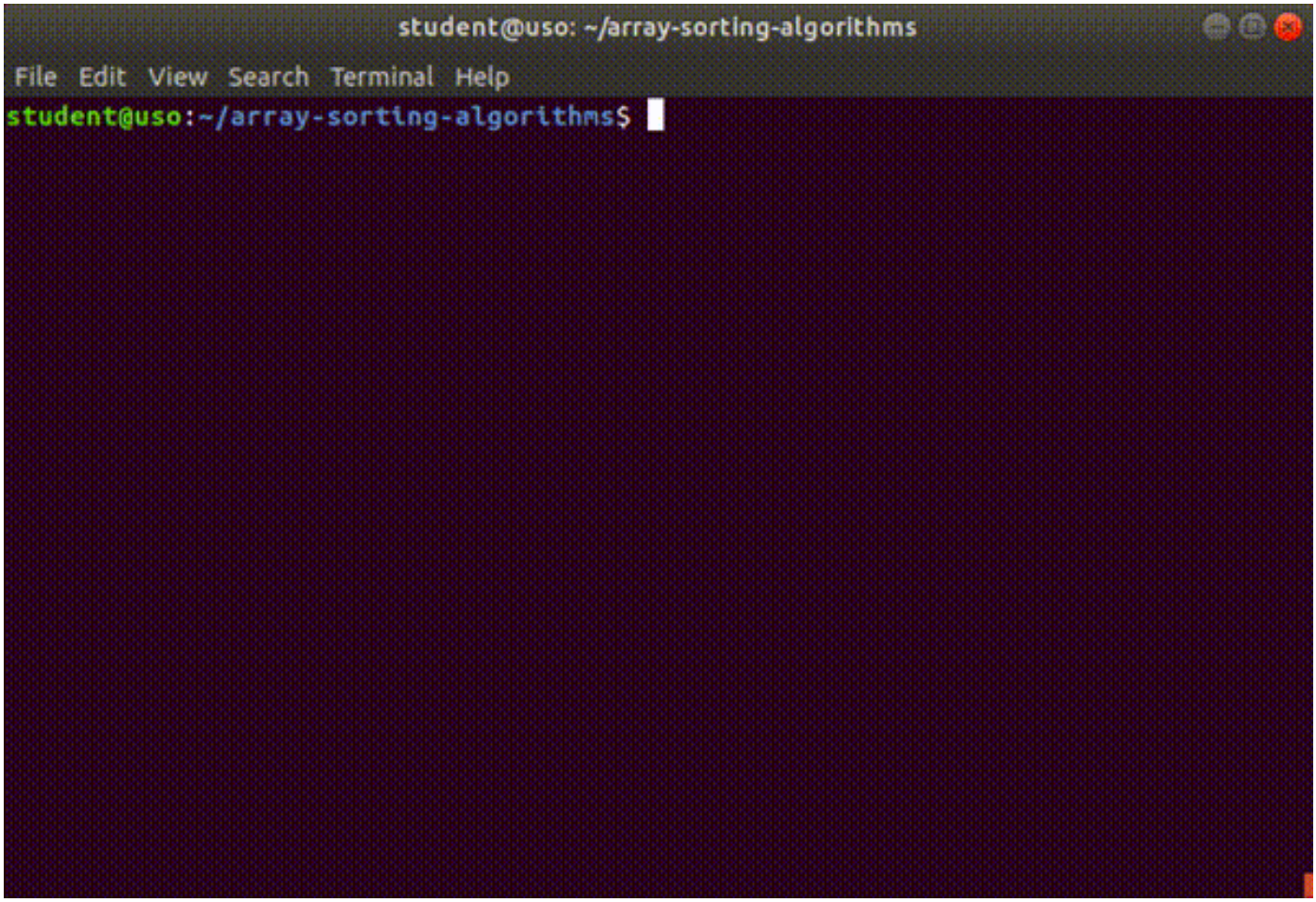
Acum vrem ca modificările de mai sus să ajungă în repository. Pentru aceasta creăm un commit folosind comanda `git commit`:

```
student@uso:~/array-sorting-algorithms$ git commit -m "Add README file"
[master (root-commit) b2a590a] Add README.md
1 file changed, 1 insertion(+)
create mode 100644 README.md
student@uso:~/array-sorting-algorithms$ git status
On branch master
nothing to commit, working tree clean
```

Am folosit descrierea `Add README file` la comanda `git commit` drept **mesaj de commit**[3]. Aceasta este o descriere succintă a modificărilor făcute prin acest commit.

Crearea unui nou commit

În continuare vom adăuga scheletul de cod pentru algoritmul Bubble Sort în repository. Vom crea un nou fișier cod sursă C `bubble-sort.c` și vom scrie în el scheletul de cod pentru algoritm. Vom crea un nou commit care va conține fișierul `bubble-sort.c`. Pentru aceasta vom folosi un editor, precum nano, ca în imaginea de mai jos:



Mai sus am creat un commit cu fișierul `bubble-sort.c` urmând pași similari cu cei din secțiunea `Crearea commitului local`:

```
student@uso:~/array-sorting-algorithms$ git status
(...)
student@uso:~/array-sorting-algorithms$ git add bubble-sort.c
student@uso:~/array-sorting-algorithms$ git status
(...)
student@uso:~/array-sorting-algorithms$ git commit -m "Add Bubble Sort skeleton"
(...)
```

Conținutul fișierului `bubble-sort.c` este:

```
#include <stdio.h>

static void sort(void)
{
    // TODO: add bubble sort algorithm here
}

int main()
{
    return 0;
}
```

Exerciții

1. Creați un nou fișier numit `radix-sort.c` cu următorul conținut:

```
#include <stdio.h>

static void sort(void)
{
    // TODO: add radix sort algorithm here
}

int main()
{
    return 0;
}
```

1. Creați un commit care să conțină fișierul `radix-sort.c`. Folosiți următorul mesaj de commit: `Add Radix Sort algorithm skeleton`.
2. Dați comanda de verificare `git log`. Detaliem outputul comenzii `git log` în subsecțiunea `Verificarea istoricului de commituri`.
3. Creați un nou fișier numit `merge-sort.c` cu următorul conținut:

```
#include <stdio.h>

static void sort(void)
{
    // TODO: add merge sort algorithm here
}

int main()
{
    return 0;
}
```

1. Creați un commit care să conțină fișierul `merge-sort.c`. Folosiți următorul mesaj de commit: `Add Merge Sort algorithm skeleton`.
2. Dați comanda de verificare `git log`.

Crearea unui commit cu modificări în fișiere existente

Până acum am creat commituri care conțineau un fișier nou creat. În această secțiune vom modifica conținutului fișierului `README.md` și vom crea un nou commit, așa cum apare în comenzile de mai jos:

```
student@uso:~/array-sorting-algorithms$ echo "We implement 3 sorting algorithms for integer arrays." >> README.md
student@uso:~/array-sorting-algorithms$ git status
On branch master
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git checkout -- <file>..." to discard changes in working directory)

    modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
student@uso:~/array-sorting-algorithms$ git add README.md
student@uso:~/array-sorting-algorithms$ git status
On branch master
Changes to be committed:
(use "git reset HEAD <file>..." to unstage)

    modified:   README.md

student@uso:~/array-sorting-algorithms$ git commit -m "Update README with project explanation"
t explanation"
[master 247b87f] Update README with project explanation
1 file changed, 1 insertion(+)
```

Spre deosebire de secțiunea `Crearea unui nou commit`, unde comanda `git status` arăta că fișierul modificat (în acel caz, `bubble-sort.c`) este nou (*new file*), acum comanda `git status` arată

că fișierul modificat (în acest caz, `README.md`) a fost modificat (*modified*). Deși apare această diferență în outputul comenzii `git status`, pașii pentru crearea unui commit care conține un fișier nou sau unul deja existent (dar modificat) sunt aceiași.

Exerciții

1. Modificați titlul fișierului `README.md` în `# Sorting Algorithm for Integer Arrays`.
2. Creați un commit care să conțină modificările la fișierul `README.md`. Folosiți următorul mesaj de commit: `Update README title`.

Verificarea istoricului de commituri

La orice pas al dezvoltării proiectului vrem să știm în ce stadiu ne aflăm ca să ne dăm seama ce am făcut deja și ce trebuie să mai facem în continuare. De aceea folosim un sistem de versionare a codului, în cazul nostru `Git`. Verificăm istoricul commiturilor folosind comanda `git log`:

```
student@uso:~/array-sorting-algorithms$ git log
commit 66b7c5fabb93b521326e6cd9ff219a06a3aec064 (HEAD -> master)
Author: Liza Babu <lizababu@example.com>
Date:   Thu Sep 24 10:40:06 2020 -0700

    Update README title

commit 247b87f3f317816a204c4512f6fd9914527a03ad
Author: Liza Babu <lizababu@example.com>
Date:   Thu Sep 24 09:50:30 2020 -0700

    Update README with project explanation

commit 3c835b0d8e7fc88ef45dfd3681867c21b75ed588
Author: Liza Babu <lizababu@example.com>
Date:   Thu Sep 24 09:45:13 2020 -0700

    Add Merge Sort algotihm skeleton

commit b92a52c0fc5d66dce4b2562114cc84ea326b2763
Author: Liza Babu <lizababu@example.com>
Date:   Thu Sep 24 09:44:36 2020 -0700

    Add Radix Sort algotihm skeleton

commit f65a7fbe8bbe1b36ba5c8e16607456a879bfb6fa
Author: Liza Babu <lizababu@example.com>
Date:   Thu Sep 24 09:21:49 2020 -0700

    Add Bubble Sort algorithm skeleton

commit b2a590a8637f1eab96e557334dbd4be14bf95833
Author: Liza Babu <lizababu@example.com>
Date:   Thu Sep 24 09:09:51 2020 -0700

    Add README file
```

Navigați prin outputul comenzii `git log` prin intermediul săgeților sus/jos. Apăsați tasta **q** când ați terminat de inspectat.

În cazul autorului acestui capitol, numele, prenumele și emailul sunt `Liza Babu <lizababu@example.com>`, așa cum apare în exemplul de mai sus: `Author: Liza Babu <lizababu@example.com>`.

Fiecare commit este identificat unic printr-un cod, numit **cod hash**[4]. Discutăm în continuare despre ultimul commit din listă. Acesta are codul hash `66b7c5fabb93b521326e6cd9ff219a06a3aec064` și mesajul de commit `Update README title`.

Acum vedem că repository-ul indică spre acest nou commit. Ne dăm seama de acest lucru pentru că HEAD se află în dreptul commitului tocmai făcut. HEAD ne indică starea repository-ului, adică ne arată care este ultimul commit pe care l-am făcut în repository.

Publicarea commiturilor în repository-ul remote

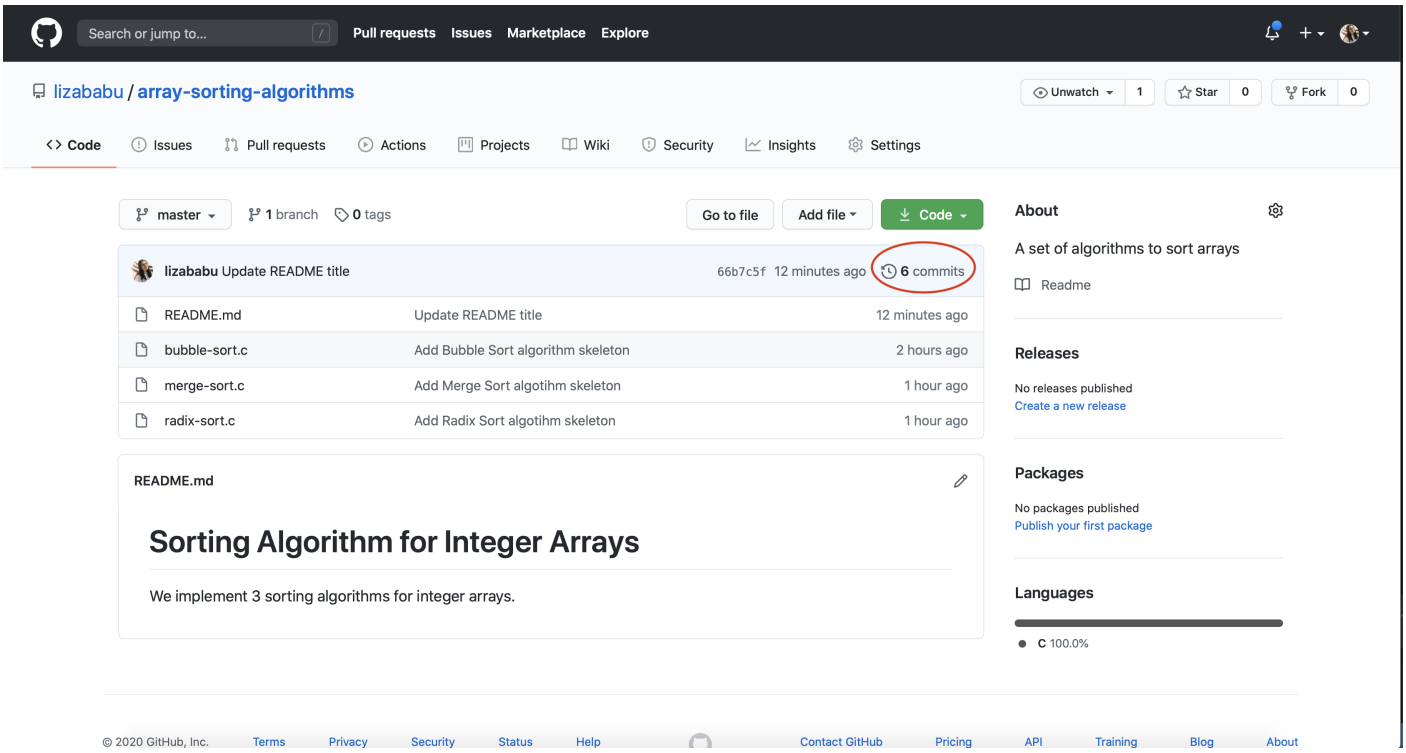
Vrem să publicăm pe GitHub toate schimbările făcute, pentru a fi vizibile și altor colaboratori ai proiectului. Publicăm commitul folosind comanda `git push`:

```
student@uso:~/array-sorting-algorithms$ git push origin master
Counting objects: 18, done.
Delta compression using up to 2 threads.
Compressing objects: 100% (16/16), done.
Writing objects: 100% (18/18), 1.95 KiB | 222.00 KiB/s, done.
Total 18 (delta 3), reused 0 (delta 0)
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/lizababu/array-sorting-algorithms.git
* [new branch]      master -> master
```

În felul acesta commiturile locale au fost publicate ("împinse", *push*) din repository-ul local în repository-ul remote identificat de `origin`. Commiturile locale se aflau pe branch-ul **master** din repository-ul **local** și au fost publicate tot în branch-ul **master** al repository-ului **origin**.

Folosiți pentru autentificare username-ul vostru și token-ul generat la secțiunea `Crearea unui token de autentificare (dacă nu aveți deja)`.

Ca să verificăm publicarea commiturilor, folosim interfața GitHub:



Obținerea commiturilor din repository-ul remote

În lucrul cu Git / GitHub, există bune practici pe care recomandăm să le urmăm.

Atunci când ne apucăm de lucru vrem să sincronizăm repository-ul local cu cel remote. Pot apărea diferențe în momentul în care altcineva a publicat schimbări remote după ce am făcut noi ultima sincronizare. În momentul în care cineva a publicat modificări asupra unei secvențe de cod pe care și noi o modificăm, apar conflicte. Conflictele trebuie rezolvate.

Facem acest lucru prin operația `pull`, care aduce local toate modificările și încearcă să rezolve conflictele în mod automat[5)]. Dacă rezolvarea conflictelor nu se poate face automat, trebuie să ne ocupăm de acest pas.

Note de subsol

Cuprins

- Laborator 06 (one page)
- Introducere
- Compilarea unui fișier cod sursă C
- Introducere în utilitarul Make și fișiere Makefile
- Introducere în Git și GitHub
- Primele commituri

---

[1] Exemple de soluții de build automation sunt Maven [https://maven.apache.org], Ant [https://ant.apache.org], Gradle [https://gradle.org]. https://en.wikipedia.org/wiki/Build_automation [https://en.wikipedia.org/wiki/Build_automation] https://www.softwaretestinghelp.com/best-build-automation-software-tools/ [https://www.softwaretestinghelp.com/best-build-automation-software-tools/]

[2] https://en.wikipedia.org/wiki/Source_code [https://en.wikipedia.org/wiki/Source_code]

[3] Mesajele de commit trebuie să fie punctuale și ușor de înțeles. Alte persoane care lucrează la același proiect software vor vrea să înțeleagă rapid ce am modificat printr-un anumit commit. Recomandări punctuale legate de crearea unor bune mesaje de commit găsiți aici: https://chris.beams.io/posts/git-commit/ [https://chris.beams.io/posts/git-commit/]

[4] Codul hash este calculat ca o sumă de control SHA-1 [https://en.wikipedia.org/wiki/SHA-1] a conținutului commitului.

[5] Noi am pornit de la un repository gol, așadar operația `pull` nu a fost necesară.

uso/laboratoare/ac/laborator-06.txt · Last modified: 2021/11/10 21:53 by andrei.tivga

Laborator 07 - Îmbunătățirea lucrului în linia de comandă

Îmbunătățirea lucrului în linia de comandă

Înainte să începeți laboratorul, dați comanda `cd ~/uso-lab && git pull`.

Principalul atu al utilizării aplicațiilor în linie de comandă, și nu în mediul grafic, este viteza cu care rezolvăm anumite sarcini. Această viteză este dată de posibilitatea de a combina utilitare în linia de comandă pentru a automatiza procese manuale, repetitive. Întotdeauna ne dorim să fim mai rapizi și să automatizăm cât mai mult din sarcinile noastre, deoarece cu cât ne terminăm treaba mai repede, cu atât avem mai mult timp liber la dispoziție.

De regulă, cu cât petrecem mai mult timp cu mâna pe tastatură și mai puțin pe mouse, cu atât suntem mai rapizi. Aliniate la această idee, aplicațiile ne pun la dispoziție scurtături pe care suntem încurajați să le folosim pentru a ne utiliza timpul mai eficient.

Scurtături în terminal

Tab completion

Funcția de **tab completion** este probabil una dintre cele mai utile funcții expuse de către terminal. Prin simpla apăsare a tastei `Tab` în timp ce scriem numele unei comenzi, al unei opțiuni a unei comenzi sau calea către un director sau fișier, terminalul va completa în mod automat textul. În cazul în care există mai multe opțiuni pentru auto-complete, prin dubla apăsare a tastei `Tab` ne va sugera opțiunile de auto-complete.

În imaginea de mai jos putem observa că pentru comanda `cd D` funcția de `Tab` completion a găsit mai multe opțiuni valide pentru auto-complete. În astfel de scenarii, cu mai multe opțiuni valide, apăsarea tastei `Tab` o singură dată nu produce niciun rezultat; trebuie să apăsăm tasta `Tab` de două ori consecutiv pentru a genera afișarea opțiunilor de auto-complete.

```
student@uso:~$ cd D
Desktop/   Documents/ Downloads/
student@uso:~$ cd D
```

Funcția de auto-complete este extrem de utilă și îmbunătățește semnificativ viteza cu care realizăm acțiuni în terminal.

Funcția este extrem de utilă atunci când lucrăm cu nume de fișiere, directoare și căi din sistem. În loc să scriem manual o cale către un nume foarte lung, lăsăm tasta `Tab` să facă asta pentru noi.

Atunci când avem o eroare în comandă (am scris greșit o anumită parte din numele comenzii sau al fișierului, fișierul nu există, etc.), tasta `Tab` nu produce nici un rezultat. Acesta este un alt motiv pentru care să folosim tasta `Tab`.

Folosiți funcția de `Tab` completion cât mai des cu putință[1)].

Navigarea în istoricul unei comenzi

Shellul implementează funcția de a reține istoricul comenzilor pe care le-am executat. Pentru a vedea istoricul curent putem rula comanda `history`. Vom obține un rezultat asemănător cu cel de mai jos:

```
student@uso:~$ history
[...]
21  ls
22  cd sorting/
23  ls -l
24  ls
25  cd
26  ls ~/Desktop ~/Documents ~/Downloads
27  ls ~/Desktop
28  ls Desktop/todos.txt
29  cp Desktop/todos.txt cp
30  ls
31  rm cp
[...]
```

Ciclăm prin comenzile date anterior folosind combinația de taste `Arrow Up` sau `Ctrl+p`, respectiv `Arrow Down` sau `Ctrl+n`.

Exercițiu: Ciclați prin istoricul de comenzi folosind combinația de taste `Ctrl+p`, respectiv `Ctrl+n`.

Căutarea în istoricul comenzilor

Terminalul ne pune la dispoziție un mod mai inteligent de a căuta în istoricul comenzilor prin combinația de taste `Ctrl+r`, ordinea căutării fiind de la cea mai recentă comandă la cea mai veche. Funcția este cunoscută sub numele de **history search**.

Pentru a porni căutarea, apăsați combinația de taste `Ctrl+r` și începeți să scrieți o parte din textul comenzii pe care o căutați, de exemplu `cd`.

```
(reverse-i-search)`cd': cd workspace/hello
```

De aici, avem următoarele opțiuni:

- Căutăm în continuare o comandă care conține șirul `cd`, folosind combinația de taste `Ctrl+r`
- Rulăm comanda pe care am găsit-o, folosind combinația de taste `Ctrl+o`
- Anulăm căutarea comenzii și revenim la starea inițială, folosind combinația de taste `Ctrl+g`

Căutarea este incrementală. Adică se rafinează pe măsură ce tastăm un caracter. Orice caracter apăsat rafinează căutarea.

Textul căutat se poate afla oriunde în interiorul comenzii; nu trebuie să fie primele litere din comandă. Pentru exemplul de mai sus, căutarea folosind textul `work` ar fi produs același rezultat

```
(reverse-i-search)`work': cd workspace/hello
```

Inspectarea sistemului de fișiere

Cea mai importantă comandă

Așa cum spuneam mai devreme, marele avantaj al utilizării terminalului este că ne ajută să rezolvăm sarcini foarte rapid. Rezolvăm sarcini folosind utilitarele pe care le avem disponibile în linia de comandă, fie că acestea fac parte din sistemul nostru sau le-am instalat.

Cel mai important utilitar pe care îl avem la dispoziție este `man`. Utilitarul `man` ne deschide pagina de manual în care este documentat un alt utilitar pe care dorim să-l folosim.

```
student@uso:~$ man
What manual page do you want?
```

Putem consulta însăși pagina de manual a utilitarului `man`

```
student@uso:~$ man man
```

Vom fi întâmpinați de următorul program interactiv:

```
MAN(1)                        Manual pager utils                        MAN(1)

NAME
       man - an interface to the on-line reference manuals

SYNOPSIS
       man  [-C  file]  [-d]  [-D]  [--warnings[=warnings]]  [-R encoding] [-L
       locale] [-m system[,...]] [-M path] [-S list]  [-e  extension]  [-i|-I]
       [--regex|--wildcard]   [--names-only]  [-a]  [-u]  [--no-subpages]  [-P
       pager] [-r prompt] [-7] [-E encoding] [--no-hyphenation] [--no-justifi-
       cation]  [-p  string]  [-t]  [-T[device]]  [-H[browser]] [-X[dpi]] [-Z]
       [[section] page[.section] ...] ...
       man -k [apropos options] regexp ...
       man -K [-w|-W] [-S list] [-i|-I] [--regex] [section] term ...
       man -f [whatis options] page ...
       man -l [-C file] [-d] [-D] [--warnings[=warnings]]  [-R  encoding]  [-L
       locale]  [-P  pager]  [-r  prompt]  [-7] [-E encoding] [-p string] [-t]
       [-T[device]] [-H[browser]] [-X[dpi]] [-Z] file ...
       man -w|-W [-C file] [-d] [-D] page ...
       man -c [-C file] [-d] [-D] page ...
       man [-?V]

 Manual page man(1) line 1 (press h for help or q to quit)
```

Observăm că ultima linie din terminal, **Manual page man(1) line 1 (press h for help or q to quit)**, ne oferă mai multe informații:

- Ne aflăm pe prima linie din prima pagină a manualului
- Putem apăsa tasta `h` pentru a acesa meniul de ajutor
- Putem apăsa tasta `q` pentru a ieși din manual

Navigarea prin paginile manualului

Navigăm cu câte o linie de terminal în joș și în sus folosind folosind tastele `Ctrl+n` și `Ctrl+p`. Putem folosi tastele `Ctrl+f` și `Ctrl+b` pentru a naviga, cu câte un ecran de terminal, în jos și în sus în pagină. Mai simplu, putem folosi tasta `Enter` pentru a naviga cu câte o linie în jos și tasta `Space` pentru a naviga cu câte un ecran în jos. Navigăm la începutul paginii folosind tasta `g`. Navigăm la sfârșit paginii folosind tasta `G`.

Putem folosi tastele `j` și `k` ca alternative pentru `Arrow Down` și `Arrow Up`. Astfel suntem mai rapizi pentru că nu ne mai mutăm mâna de pe tastele caractere.

Folosim `man` ca să vedem dacă un utilitar oferă o anumită funcționaltiate. Citim întreaga pagină de manual ca să vedem toate funcționalitățile sau căutăm o funcționalitate folosind cuvinte cheie. Pașii pentru căutarea unui cuvânt cheie sunt următorii:

1. Pentru a porni funcția de căutare apăsăm tasta `/` în sesiunea interactivă din `man`.
2. În continuare vom introduce textul pe care dorim să-l căutăm: poate să fie un cuvântul cheie pe care îl știm deja sau orice text care sperăm că ne duce la rezultatul dorit.
3. Acum apăsăm tasta `Enter`. Vom fi duși la primul rezultat care se potrivește căutării, dacă acesta există.
4. Dacă vrem să navigăm la următorul rezultat apăsăm tasta `n`. Dacă vrem să navigăm la un rezultat anterior apăsăm tasta `N`.

Căutarea[2)] are loc de la poziția curentă în pagină către sfârșitul paginii. Dacă am navigat deja în interiorul paginii, trebuie să avem în vedere că rezultatul de interes al căutării noastre se poate alfa undeva între începutul paginii și poziția noastră curentă[3)].

Selectarea multiplor fișiere folosind globbing

Înainte de a trece mai departe, mergeți în directorul `~/uso-lab/labs/05-cli/support/support-globbing`.

Întotdeauna când deschidem un terminal o facem pentru că vrem să realizăm o sarcină: de exemplu, vrem să redenumim rapid ultimele poze făcute cu telefonul de la genericul **DCIM1001** la ceva util **Excursie Sinaia, Ian 2020, 1001**, vrem să ne testăm proiectul și să urcăm modificările pe GitHub, etc.

Până acum am aplicat diferite comenzi fie pe fișiere individuale, fie pe întreg directorul. Foarte des vom avea nevoie de un mijloc prin care să putem selecta un număr variabil de fișiere care au un nume care corespunde unui tipar (*pattern*) comun.

Să revenim la scenariul prezentat anterior: vrem să selectăm pozele din excursia din Sinaia. În directorul în care avem pozele din excursie avem și alte poze de la alte evenimente. Știm că pozele din excursie încep toate cu numele **DCIM** și apoi sunt urmate de un număr. Ceea ce vrem să facem este să selectăm toate pozele al căror nume corespunde acestui tipar și să le mutăm într-un director separat. Pentru a face acest lucru, folosim **globbing**, ca în exemplul de mai jos:

```
student@uso:~/.../05-cli/support/support-globbing$ mv DCIM* excursie-Sinaia-2020/
```

Observăm argumentul pe care l-am dat comenzii `mv`, și anume `DCIM*`. Expresia `DCIM*` este un exemplu de globbing: adică o expresie care descrie un tipar prin folosirea unor caractere speciale, așa cum este caracterul `*`. În cazul de față, expresia `DCIM*` înseamnă orice fișier al cărui nume începe cu șirul de caractere `DCIM`.

Caracterul special *

În sintaxa globbing, caracterul * poate fi înlocuit cu orice caracter de oricâte ori, sau poate lipsi cu totul. În directorul nostru home (`~`), executăm următoarele comenzi:

```
student@uso:~$ ls
Desktop    Downloads  Pictures  Templates  examples.desktop  vm-actions-log.txt
Documents  Music      Public    Videos     uso.git           workspace
```

```
student@uso:~$ ls -d D*
Desktop  Documents  Downloads

student@uso:~$ ls -d Music*
Music
```

Observăm că în expresia `D*`, caracterul `*` înglobează toate caracterele care urmează literei **D**: "esktop", "ocuments" și "ownloads". Observăm că în cazul expresie `Music*`, `*` nu ține locul nici unui caracter.

Sintaxa specială []

În sintaxa globbing, folosim sintaxa `[]` pentru a defini o listă de caractere care pot fi folosite în înlocuire. Această sintaxă înlocuiește exact un caracter din lista oferită. În directorul nostru home (~), executăm următoarele comenzi:

```
student@uso:~$ ls -d Mus[ijk]c
Music

student@uso:~$ ls -d Mus[abc]c
ls: cannot access 'Mus[abc]c': No such file or directory
```

În expresia `Musi[ijk]c`, i-am "spus" shell-ului că al patrulea caracter poate să fie oricare din lista `[ijk]`. În acest context, folosind globbing s-a găsit cu succes numele fișierului **Music**. În expresia `Mus[abc]c`, i-am "spus" shell-ului că al patrulea caracter poate să fie oricare din lista `[abc]`. Deoarece nu avem niciun fișier numit **Musac**, **Musbc** sau **Muscc**, comanda ne-a afișat mesajul de eroare corespunzător.

Sintaxa `[]` nu ne limitează la a oferi enumarații de caractere, așa cum am făcut cu `[ijk]` sau `[abc]`. Sintaxa accepta și intervale, cum observăm în exemplul de mai jos:

```
student@uso:~$ ls -d Mus[A-Za-z0-9]c
Music
```

Citim expresia `[A-Za-z0-9]` în următorul mod: această expresie înlocuiește un caracter din intervalul `A-Z` sau din intervalul `a-z` sau din intervalul `0-9`; cu alte cuvinte înlocuiește un caracter *alfa-numeric*[4].

Tip

Folosind sintaxa `[]` putem rescrie mutarea pozelor a.î. să o facem mai precisă:

```
student@uso:~/.../05-cli/support/support-globbing$ mv DCIM[0-9][0-9][0-9][0-9].jpg excursie-Sinaia-2020/
```

Cu expresia de mai sus vom muta toate pozele din intervalul **DCIM0000** - **DCIM9999**.

Sintaxa specială {}

În sintaxa globbing, folosim sintaxa `{}` pentru a defini o listă de cuvinte (grupuri de caractere) care pot fi folosite în înlocuire. Această sintaxă înlocuiește exact un cuvânt din lista oferită. În directorul vostru home (~), executați următoarele comenzi:

```
student@uso:~$ ls -d {Downloads,Music}
Downloads  Music

student@uso:~$ ls -d {Down,Mus}*
Downloads  Music
```

Citim expresia `{Downloads,Music}`: în locul acestei expresii poate să existe cuvântul **Downloads** sau cuvântul **Music**. Observăm că putem să combinăm orice elemente de globbing, așa cum am făcut în expresia `{Down,Mus}*`.

Folosirea ad-litteram a caracterelor speciale

Există cazuri când numele fișierelor conțin caractere speciale. Unele fișiere pot fi prefixate cu o categorie din care fac parte, ca în exemplul de mai jos:

```
student@uso:~$ ls Documents/uni
'[PC] Course 01.pdf'  '[USO] Course 01.pdf'  '[USO] Course 02.pdf'
```

În exemplul de mai sus, fișierele pdf de curs sunt prefixate cu numele materiei: [PC], [USO]. Vrem să îi spunem sintaxei de globbing că în acest caz, șirul **[USO]** nu trebuie tratat ca o expresie, ci ca un șir de caracter normale. Pentru a face acest lucru, încadrăm șirul între **"**:

```
student@uso:~$ ls Documents/uni/"[USO]"*
'Documents/uni/[USO] Course 01.pdf'  'Documents/uni/[USO] Course 02.pdf'
```

Citim expresia `"[USO]"*`: orice fișier al cărui nume începe cu șirul de caractere **[USO]** și este urmat de orice caracter. Operația prin care eliminăm semnificația specială a unui caracter poartă numele de **escaping**; cu alte cuvinte, informal, spunem că am făcut escaping semnificației speciale a sintaxei `[]`. Termenul vine de la cuvântul **escape** (a scăpa), și exprimă că scăpăm de semnificația specială a unui caracter / set de caractere.

Exerciții

Pentru exercițiile următoare vom folosi fișierele din directorul de suport `~/uso-lab/labs/05-cli/support/support-globbing`.

1. Creați un director numit `pdfs`. Mutați toate fișierele cu extensia `.pdf` din directorul `~/uso-lab/labs/05-cli/support/support-globbing` în directorul `pdfs`.
2. Creați un director numit `Excursie Brasov, 2020-2021`. Mutați fișierele **DCIM** din intervalul 1400 - 1700 în directorul creat.
3. Creați un director numit `cursuri/anul-I`. Mutați toate fișierele care conțin cuvintele **curs** sau **slide** în directorul creat. Folosiți sintaxa `*{curs,slide}*`.

Căutarea unui fișier în sistem

De multe ori ne aflăm în situația în care căutăm un fișier pe disc: ex. doar ce am clonat un proiect de pe GitHub și vrem să inspectăm fișierul **Makefile** pentru a vedea cum compilăm și rulăm proiectul. Un alt exemplu poate fi că vrem să vedem cum arată fișierele de test existente în proiect. De multe ori, înțelegem mai bine proiectul doar prin simpla inspectare a testelor.

Există două utilitare care ne permit să căutăm în cadrul sistemului de fișiere: `locate` și `find`. Ele au fost prezentate și în `laboratorul 03: Lucrul cu fișiere (2)`. În continuare vedem câteva exemple de folosire ale utilitarului `find`.

Utilitarul find

Utilitarul `find` îndeplinește același scop: căuta în fișierele de pe sistem. `find` este un utilitar mai complex decât `locate`. Acesta ne permite să căutăm fișiere după nume, permisiuni, tipul fișierelor, data ultimei modificări și multe altele. Inspectăm pagina de manual a utilitarului pentru a vedea cum îl putem folosi.

```
student@uso:~$ man find

SYNOPSIS
       find  [-H]  [-L]  [-P]  [-D  debugopts]  [-Olevel]  [starting-point...]
       [expression]
```

La o primă vedere, `find` poate părea complex și intimidant, dar lucrurile stau foarte simplu. Folosim `find` cu sintaxa `find [starting-point] [expression]`, ca în exemplul de mai jos:

```
student@uso:~$ find . -name "*search*"
./C/searching
./C/searching/linear_search.c
./C/searching/other_binary_search.c
./C/searching/binary_search.c
./C/searching/modified_binary_search.c
./C/searching/jump_search.c
./C/searching/interpolation_search.c
./C/searching/fibonacci_search.c
./C/searching/ternary_search.c
./C/searching/pattern_searc h
./C/searching/pattern_search/naive_search.c
./C/searching/pattern_search/boyer_moore_search.c
./C/searching/pattern_search/rabin_karp_search.c
./C/data_structures/binary_trees/binary_search_tree.c
```

În exemplul de mai sus observă că am folosit ca **starting-point** `.` (căutarea pleacă din directorul curent), iar ca **expression** `-name "*search*"`.

Utilitarul `find` folosește o expresie compusă pentru căutare. În exemplul anterior am folosit opțiunea `-name PATTERN`. Exact ca în cazul utilitarului `locate`, **PATTERN** poate folosi sintaxa globbing, așa cum am făcut în exemplul de mai sus `"*search*"`.

Atunci când folosim sintaxa globbing, trebuie să fim atenți să încadrăm **PATTERN** între `"` (ghilimele), așa cum am făcut în exemplul de mai sus. Trebuie să facem asta pentru ca sintaxa globbing să fie interpretată de către utilitarul `find` și nu de către terminalul (`bash`) din care lansăm utilitarul.

Exerciții

1. Folosind `find` căutați fișierele care conțin șirul `tmp` în nume. Hint: [starting point] este `/`.
2. Folosind `find` căutați fișierele care conțin șirul `bin` în nume. Hint: [starting point] este `/`.

Note de subsol

Inspectarea fișierelor

Inspectarea rapida a conținutului fișierelor

În secțiunea anterioară, am văzut cum căutăm fișiere în sistem cu ajutorul utilitarelor `locate` și `find`. Căutăm un fișier cu un scop: vrem să găsim fișierul `README` pentru informații despre compilarea proiectului, vrem să ne amintim un detaliu de implementare din cod, etc.

De cele mai multe ori acțiunea noastră se poate grupa în una din următoarele două categorii:

- Ne dorim să inspectăm rapid conținutul fișierelor pentru a ne da seama dacă am găsit informația căutată.
- Ne dorim să afișăm pe ecran conținutul fișierelor pentru a extrage și prelucra informații din acestea.

Prelucrarea informației dintr-un fișier

Pentru a afișa pe ecran conținutul unui fișier folosim utlitarul `cat`. Rulăm comanda de mai jos, pentru a exemplifica:

```
student@uso:~$ cat ~/uso-lab/labs/05-cli/support/make-folder/hangman.c
#include <stdio.h>
#include <string.h>

int main() {
  // Get word to guess
  char answer[128];
  printf("Enter word to guess: ");

[...]
```

Observăm că pentru un fișier cu un număr mare de linii, așa cum este **binary_search.c**, afișarea întregului conținut pe ecran devine un impediment în a putea înțelege și urmări conținutul. De aceea vă încurajăm să folosiți `less` în loc de `cat` pentru a inspecta un fișier: vă este mult mai ușor să vă plimbați în interiorul fișierului și puteți folosi funcția search pentru a căuta în fișier. De asemeni, folosind `less` vă păstrați consola curată și puteți urmări mai ușor ce comenzi ați dat anterior și care au fost rezultatele acestora.

Folosim comanda `cat` în combinație cu alte comenzi pentru a extrage sau filtra conținutul anumitor fișiere. Comanda `cat` primește ca argumente calea către unul sau mai multe fișiere și afișează pe ecran conținutul concatenat al acestora.

Un exemplu uzual este faptul că vrem să extragem informațiile despre starea memoriei sistemului din fișierul `/proc/meminfo`. Pentru aceasta rulăm comanda de mai jos:

```
student@uso:~$ cat /proc/meminfo | grep "Mem"
MemTotal:        2041248 kB
MemFree:          236092 kB
MemAvailable:     874420 kB
```

În exemplul de mai sus folosim `cat` pentru a oferi ca intrare conținutul fișierului `/proc/meminfo` utilitarului `grep`; cu utilitarul `grep` filtrăm conținutul după textul `"Mem"`. Cu alte cuvinte, outputul comenzii `cat /proc/meminfo`, adică conținutul fișierului `/proc/meminfo` este textul pe care utilitarul `grep` îl prelucrează.

Exerciții: Plecând de la exemplul de mai sus, extrageți din fișierul `/proc/cpuinfo` dimensiunea memoriei cache a procesorului vostru; filtrați conținutul după textul `"cache"`.

Afișarea parțială a unui fișier

Am văzut că utilitarul `cat` afișează întreg conținutul unui fișier. Există scenarii în care suntem interesați doar de începutul sau sfârșitul unui conținut. Pentru aceste cazuri putem folosi utilitarele:

- `head` - afișează primele **10** linii din conținut
- `tail` - afișează ultimele **10** linii din conținut

Valoarea **10** este valoarea implicită a ambelor utilitare, dar putem specifica un alt număr de linii.

Așa cum am observat în capitolul despre procese, putem folosi utilitarul `ps` pentru a vedea care sunt procesele din sistem și ce resurse consumă acestea. Memoria sistemului este una dintre cele mai importante resurse; dacă sistemul nostru rămâne fără memorie disponibilă, tot sistemul este afectat: sistemul se va "mișca" mai greu, procesele se vor "mișca" mai greu sau pot chiar să își întrerupă activitatea. Știind acest lucru, suntem interesați să vedem care sunt primele zece procese care consumă cea mai multă memorie.

Folosim utilitarul `ps` pentru a afișa toate procesele din sistem:

```
student@uso:~$ ps -e -ouser,uid,pid,%mem,%cpu,rss,cmd --sort=%mem

USER       UID   PID %MEM %CPU   RSS CMD
root         0     2  0.0  0.0     0 [kthreadd]

[...]

student   1000  8338  3.0  0.0 61860 /usr/lib/evolution/evolution-calendar-factory-subprocess --factory all --bus-name org.gnome.evolution.dataserver.Subprocess.Backend.Calendarx830
student   1000  8307  3.1  0.0 64628 /usr/lib/evolution/evolution-calendar-factory
root         0  1338  3.8  0.0 78880 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
student   1000  7782  3.9  0.0 81312 /usr/lib/xorg/Xorg vt1 -displayfd 3 -auth /run/user/1000/gdm/Xauthority -background none -noreset -keeptty -verbose 3
student   1000  8437  8.4  0.0 171916 /usr/bin/gnome-software --gapplication-service
student   1000  7938 18.0  0.1 368304 /usr/bin/gnome-shell
```

Am folosit opțiunea `-ouser,uid,pid,%mem,%cpu,rss,cmd` pentru a selecta coloanele pe care să le afișeze `ps`.

Am folosit opțiunea `--sort` cu argumentul `%mem` pentru a sorta procesele după procentul de memorie folosită.

Folosiți comanda `ps -e -ouser,uid,pid,%mem,%cpu,rss,cmd --sort=%mem | less` pentru a vizualiza rezultatul comenzii `ps` într-o sesiune interactivă `less`.

Observăm că avem procesele sortate crescător după coloana `%MEM`. Folosim utilitarul `tail` pentru a extrage din rezultatul `ps` cele mai consumatoare zece procese:

```
student@uso:~$ ps -e -ouser,uid,pid,%mem,%cpu,rss,cmd --sort=%mem | tail
student   1000 12966  1.8  0.0 38216 /usr/lib/gnome-terminal/gnome-terminal-server
root         0  1074  2.2  0.0 45460 /usr/bin/containerd
student   1000  8274  2.3  0.0 48296 nautilus-desktop
root         0   336  2.6  0.0 53612 /lib/systemd/systemd-journald
student   1000  8338  3.0  0.0 61860 /usr/lib/evolution/evolution-calendar-factory-subprocess --factory all --bus-name org.gnome.evolution.dataserver.Subprocess.Backend.Calendarx830
student   1000  8307  3.1  0.0 64628 /usr/lib/evolution/evolution-calendar-factory
root         0  1338  3.8  0.0 78880 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
student   1000  7782  3.9  0.0 81312 /usr/lib/xorg/Xorg vt1 -displayfd 3 -auth /run/user/1000/gdm/Xauthority -background none -noreset -keeptty -verbose 3
student   1000  8437  8.4  0.0 171916 /usr/bin/gnome-software --gapplication-service
student   1000  7938 18.0  0.1 368248 /usr/bin/gnome-shell
```

În acest moment am găsit răspunsul căutat, dar avem două mici neajunsuri:

- Ne lipsește antetul, așa că nu știm ce informații avem pe coloane
- Procesele sunt sortate crescător, a.î. cel mai consumator este ultimul; vrem să fie sortate descrescător

Rezolvăm cele două probleme prin intermediul opțiunii `--sort`: dacă punem un `-` (minus) în fața argumentului după care sortăm, o să sortăm descrescător. Rulăm comanda:

```
student@uso:~$ ps -e -ouser,uid,pid,%mem,%cpu,rss,cmd --sort=-%mem | less
USER       UID   PID %MEM %CPU   RSS CMD
student   1000  7938 18.0  0.1 368248 /usr/bin/gnome-shell
student   1000  8437  8.4  0.0 171916 /usr/bin/gnome-software --gapplication-service
student   1000  7782  3.9  0.0 81312 /usr/lib/xorg/Xorg vt1 -displayfd 3 -auth /run/user/1000/gdm/Xauthority -background none -noreset -keeptty -verbose 3
root         0  1338  3.8  0.0 78880 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
student   1000  8307  3.1  0.0 64628 /usr/lib/evolution/evolution-calendar-factory

[...]
```

Observăm că acum avem formatul dorit. Ne mai rămâne să extragem primele **11** linii din rezultatul comenzii de mai sus; **11** deoarece prima este linia antetului iar următoarele zece sunt procesele de interes. Pentru aceasta utilizăm comanda `head` cu opțiunea `-11` ca în exemplul de mai jos:

```
student@uso:~$ ps -e -ouser,uid,pid,%mem,%cpu,rss,cmd --sort=-%mem | head -11
USER       UID   PID %MEM %CPU   RSS CMD
student   1000  7938 18.0  0.1 367952 /usr/bin/gnome-shell
student   1000  8437  8.4  0.0 171916 /usr/bin/gnome-software --gapplication-service
student   1000  7782  3.9  0.0 81312 /usr/lib/xorg/Xorg vt1 -displayfd 3 -auth /run/user/1000/gdm/Xauthority -background none -noreset -keeptty -verbose 3
root         0  1338  3.8  0.0 78880 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
student   1000  8307  3.1  0.0 64628 /usr/lib/evolution/evolution-calendar-factory
student   1000  8338  3.0  0.0 61860 /usr/lib/evolution/evolution-calendar-factory-subprocess --factory all --bus-name org.gnome.evolution.dataserver.Subprocess.Backend.Calendarx830
root         0   336  2.6  0.0 53612 /lib/systemd/systemd-journald
student   1000  8274  2.3  0.0 48296 nautilus-desktop
root         0  1074  2.2  0.0 45460 /usr/bin/containerd
student   1000 12966  1.8  0.0 38216 /usr/lib/gnome-terminal/gnome-terminal-server
```

Exerciții

1. Afișați primele zece procese sortate în funcție de memoria ocupată (Hint: RSS). Nu uitați să includeți antetul.
2. Afișați ultimele zece procese sortate în funcție de utilizarea procesorului (Hint: CPU). Nu uitați să includeți antetul.

Căutarea în fișiere

Așa cum am văzut până în acest punct din carte, majoritatea comenzilor Linux afișează o gamă largă de informații pe care apoi utilizatorul (adică noi) le filtrează pentru a extrage ceea ce îl intresează. La începutul acestei secțiuni, dar și de-a lungul cărții, am folosit utilitarul `grep` ca să filtrăm rezultatul unei comenzi.

Comanda `grep` este una dintre cele mai folosite în linie de comandă. Sintaxa de folosire a `grep` este următoarea:

```
SYNOPSIS
       grep [OPTIONS] PATTERN [FILE...]
```

`grep` caută **PATTERN** în lista de fișiere primită ca argument și afișează liniile care conțin **PATTERN**-ul căutat. Atunci când nu primește nici un fișier, citește text de la tastatură (intrarea standard) și afișează liniile care conțin **PATTERN**-ul căutat.

Până acum noi am utilizat `grep` după modelul de mai jos:

```
student@uso:~$ cat uso-lab/labs/05-cli/support/make-folder/hangman.c | grep gameover
  int gameover = 0;
  while (! gameover) {
    gameover = 1;
        gameover = 0;
```

În exemplul de mai sus, operatorul `|` trimite textul afișat de comanda `cat` către intrarea standard a comenzii `grep`. Acesta comandă este un exemplu de **oneliner**. Vom discuta mai multe despre oneliners în secțiunea următoare.

Comanda următoare este echivalentă cu cea de mai sus:

```
student@uso:~$ grep gameover uso-lab/labs/05-cli/support/make-folder/hangman.c
  int gameover = 0;
  while (! gameover) {
    gameover = 1;
        gameover = 0;
```

Observăm modul de folosire: `grep PATTERN cale/către/fișier`.

Exerciții

1. Căutați *patternul* "I" în fișierul `hangman.c`, pentru a vedea care sunt headerele incluse în acest fișiere (`#include <...>`). Observați cât de multe rezultate irelevante ați găsit datorită faptului că am căutat doar caracterul **I**. Folosiți *patternul* "include" în încercarea de a restrânge căutarea.

Opțiuni uzuale ale grep

Afișarea numărului liniei care conține patternul

Folosim opțiunea `-n` pentru a afișa și numărul liniei care conține patternul căutat:

```
student@uso:~$ grep -n gameover uso-lab/labs/05-cli/support/make-folder/hangman.c
18:  int gameover = 0;
19:  while (! gameover) {
46:    gameover = 1;
49:        gameover = 0;
```

Căutarea case-insensitive

Implicit, grep caută în mod case-sensitive patternul, așa cum putem observa din exemplul de mai jos:

```
student@uso:~$ grep Gameover uso-lab/labs/05-cli/support/make-folder/hangman.c
student@uso:~$
```

Pentru a efectua căutarea textului în mod case-insesnsitive, folosim opțiunea `-i`, ca în exemplul de mai jos:

```
student@uso:~$ grep -i Gameover uso-lab/labs/05-cli/support/make-folder/hangman.c
  int gameover = 0;
  while (! gameover) {
    gameover = 1;
        gameover = 0;
```

Excluderea unui pattern

Pentru a afișa toate liniile, mai puțin pe cele care conțin pattern, folosim opțiunea `-v`, ca în exemplul de mai jos:

```
student@uso:~$ grep -v gameover uso-lab/labs/05-cli/support/make-folder/hangman.c
#include <stdio.h>
#include <string.h>

int main() {
  // Get word to guess
  char answer[128];
  printf("Enter word to guess: ");
  fflush(stdout);
  scanf(" %s", answer);

  int N = strlen(answer);
  int mask[N];
  for (int i=0; i < N; ++i) {
    mask[i] = 0;
  }

[...]
```

Cu alte cuvinte, am afișat toate liniile din fișiere care nu conțin patternul `gameover`.

Căutarea recursivă a unui pattern

În căutările noastre de până acum, ca și în exemplele de mai sus, am presupus că știm în ce fișiere se găsește informația căutată de noi. Acest lucru este adevărat pentru fișiere din sistem cu informații bine cunoscute, cum ar fi `/proc/meminfo`, dar atunci când lucrăm cu un proiect nou nu vom ști în ce fișiere să căutăm informația dorită. De exemplu, în cazul proiectului cu algoritmi implementați în C, noi am făcut presupunerea că vom găsi linii care conțin patternul **gameover** în fișierul `uso-lab/labs/05-cli/support/make-folder/hangman.c`.

Atunci când nu știm în ce fișiere se află informația căutată putem să-i spunem lui `grep` să caute recursiv prin toată ierarhia de fișiere dintr-un anumit director. Pentru a efectua o căutare recursivă folosim opțiunea `-r`, ca în exemplul de mai jos:

```
student@uso:~$ grep -r gameover uso-lab/ | less
uso-lab/labs/05-cli/support/make-folder/hangman.c:  int gameover = 0;
uso-lab/labs/05-cli/support/make-folder/hangman.c:  while (! gameover) {
uso-lab/labs/05-cli/support/make-folder/hangman.c:    gameover = 1;
uso-lab/labs/05-cli/support/make-folder/hangman.c:        gameover = 0;
```

```
uso-lab/labs/04-appdev/support/simple-make/hangman.c:  int gameover = 0;
uso-lab/labs/04-appdev/support/simple-make/hangman.c:  while (! gameover) {
uso-lab/labs/04-appdev/support/simple-make/hangman.c:    gameover = 1;
uso-lab/labs/04-appdev/support/simple-make/hangman.c:        gameover = 0;
```

Best practice

De cele mai multe ori vom folosi opțiunile `-n`, `-i` și `-r` în aceeași timp. În cazul nostru de până acum, aceasta se traduce în:

```
student@uso:~$ grep -nri uso-lab | less
uso-lab/lectures/auto/demo/grades/script.wiki:3:Pentru acest scenariu, actualizați [[https://github.com/systems-cs-pub-ro/uso-lab|repository-ul public de USO]] și navigați în subdir
uso-lab/lectures/auto/demo/grades/script.wiki:6:student@uso:~$ cd uso-lab
uso-lab/lectures/auto/demo/grades/script.wiki:7:student@uso:~/uso-lab$ git pull
uso-lab/lectures/auto/demo/grades/script.wiki:8:student@uso:~/uso-lab$ cd lectures/auto/grades
uso-lab/lectures/auto/demo/grades/script.wiki:9:student@uso:~/uso-lab/lectures/auto/grades$ ls
uso-lab/lectures/auto/demo/grades/script.wiki:19:student@uso:~/uso-lab/lectures/auto/grades$ wc -l 2016-2017.csv
uso-lab/lectures/auto/demo/grades/script.wiki:21:student@uso:~/uso-lab/lectures/auto/grades$ wc -l 2017-2018.csv
uso-lab/lectures/auto/demo/grades/script.wiki:23:student@uso:~/uso-lab/lectures/auto/grades$ wc -l 2018-2019.csv
uso-lab/lectures/auto/demo/grades/script.wiki:25:student@uso:~/uso-lab/lectures/auto/grades$ wc -l 2019-2020.csv
uso-lab/lectures/auto/demo/grades/script.wiki:32:student@uso:~/uso-lab/lectures/auto/grades$ tail -n +2 2019-2020.csv | wc -l
uso-lab/lectures/auto/demo/grades/script.wiki:34:student@uso:~/uso-lab/lectures/auto/grades$ echo -n "2019-2020: " ; tail -n +2 2019-2020.csv | wc -l
uso-lab/lectures/auto/demo/grades/script.wiki:45:student@uso:~/uso-lab/lectures/auto/grades$ for i in *.csv; do echo -n "$(basename "$i" .csv): " ; tail -n +2 "
:
```

Astfel avem o căutare cât mai cuprinzătoare și putem folosi funcția de căutare în sesiunea interactivă `less` pentru a găsi linia și fișierul care ne interesează.

Exerciții

1. Găsiți toate fișierele din sistem care includ headerul `stdio.h`.

O înțelegere mai bună a shellului

Execuția comenzilor

Încheierea execuției unei comenzi

Atunci când rulăm o comandă aceasta își poate încheia execuția în două moduri: cu **succes** sau cu **eșec**. Atunci când își încheie execuția, orice proces întoarce un cod de eroare, care este un număr:

- Dacă numărul întors are valoarea `0`, procesul și-a încheiat execuția cu succes.
- Dacă numărul întors are orice altă valoare, procesul și-a încheiat execuția cu eroare, iar codul întors poate fi folosit pentru a afla mai multe informații despre eroarea pe care a întors-o procesul. În pagina `man` a utilitarului `ls` este specificat:

```
Exit status:
       0      if OK,


       1      if minor problems (e.g., cannot access subdirectory),


       2      if serious trouble (e.g., cannot access command-line argument).
```

Pentru a vedea codul cu care și-a încheiat execuția o comandă folosim sintaxa `$?`. Urmărim exemplul de mai jos:

```
student@uso:~$ ls Desktop/
todos.txt
student@uso:~$ echo $?
0
student@uso:~$ ls non-existent
ls: cannot access 'non-existent': No such file or directory
student@uso:~$ echo $?
2
```

Observăm că în cazul fișierului inexistet, comanda `ls non-existent` a întors valoarea 2, așa cum era specificat și în pagina de manual.

Înlănțuirea comenzilor în funcție de succes sau eșec

De multe ori vrem să executăm o succesiune de comenzi pentru a realiza o sarcină. De exemplu, atunci când vrem să instalăm o aplicație o rulăm trei comenzi:

- O să actualizăm indexul surselor de pachete folosind `apt update`
- O să instalăm pachetul care conține aplicația folosind `apt install`
- O să rulăm aplicația pentru a valida că instalarea a fost cu succes.

Preferăm să înlănțuim cele trei comenzi într-una singură pentru că astfel putem să pornim tot acest proces, să plecăm de la calculator, iar când ne întoarcem avem tot sistemul pregătit.

Pentru a înlănțui comenzi în terminalul bash avem trei operatori disponibili:

- Operatorul `;` care este folosit pentru separarea comenzilor. Urmăm exemplul de mai jos:

```
student@uso:~$ mkdir demo; cd demo; touch Hello; ls
Hello
```

În exemplul de mai sus am creat directorul `demo`, am navigat în interiorul său, am creat fișierul `Hello` și am afișat conținutul directorului. Am făcut toate acestea înlănțuind comenzile `mkdir`, `cd`, `touch` și `ls` cu ajutorul operatorului `;`.

Operatorul `;` este folosit pentru separarea comenzilor, dar nu ține cont dacă comenzile anterioare au fost executate cu succes sau nu. Urmăm exemplul de mai jos:

```
student@uso:~$ mkdir operators/demo; cd operators/demo
mkdir: cannot create directory ‘operators/demo’: No such file or directory
-bash: cd: operators/demo: No such file or directory
```

În exemplul de mai sus, comanda `mkdir` a eșuat deoarece nu a găsit directorul `operators` în care să creeze directorul `demo`. Cu toate acestea, operatorul `;` doar separă comenzile între ele, așa că și comanda `cd operators/demo` a fost executată, și și aceasta a eșuat deoarece nu există calea `operators/demo`.

Folosim operatorul `;` pentru a înlănțui comenzi care sunt independente unele de altele, și deci execuția lor nu depinde de succesul unei comenzi precedente.

- Operatorul binar `&&` (și logic) - execută a doua comandă doar dacă precedenta s-a executat cu succes. Exemplul anterior devine:

```
student@uso:~$ mkdir operators/demo && cd operators/demo
mkdir: cannot create directory ‘operators/demo’: No such file or directory
```

Observăm că din moment ce comanda `mkdir` a eșuat, comanda `cd` nu a mai fost executată.

- Operatorul binar `||` (sau logic) - execută a doua comandă doar dacă prima s-a terminat cu eșec. Urmărim exemplul de mai jos:

```
student@uso:~$ (ls -d operators || mkdir operators) && ls -d operators
ls: cannot access 'operators': No such file or directory
operators
student@uso:~$ (ls -d operators || mkdir operators) && ls -d operators
operators
operators
```

În exemplul de mai sus, prima comandă `ls` a eșuat, așa că a fost executată comanda `mkdir` și apoi a fost executată ultima comandă `ls`. La cea de-a doua rulare, a fost executată cu succes prima comandă `ls`, așa că comanda `mkdir` nu a mai fost executată, și apoi a fost executată ultima comandă `ls`.

Pentru a rezolva scenariul de la care am plecat inițial, putem rula:

```
sudo apt update && sudo apt install -y cowsay && cowsay "Howdy"
```

Comanda de mai sus va actualiza indexul pachetelor sursă, va instala pachetul `cowsay` și va rula comanda `cowsay` pentru a valida instalarea. O astfel de înlănțuire de comenzi este numită **oneliner**.

Exerciții

1. Scrieți un oneliner cu ajutorul căruia creați directorul `~/uso-lab/labs/05-cli/support/make-folder` și apoi copiați conținutul directorului `~~/uso-lab/labs/05-cli/support/redir` în el.
2. Actualizați onelinerul anterior astfel încât după copiere să pornească compilarea proiectului folosind comanda `make all`.

Înlănțuirea comenzilor folosind operatorul | (pipe)

Așa cum am descoperit în secțiunile și capitolele anterioare, în mediul Linux avem multe utilitare care rezolvă o nevoie specifică: `ls` afișează informații despre fișiere, `ps` despre procese, `grep` filtrează, etc. Toate acestea au la bază filozofia mediului Linux: "do one thing and do it well". Ca întodeauna, frumusețea stă în simplitate: avem o suită de unelte la dispoziție, fiecare capabilă să rezolve rapid o sarcină dată; pentru a rezolva o problemă mai complexă trebuie doar să îmbinăm uneltele.

Operatorul `|` (pipe) ne ajută să facem acest lucru. Atunci când folosim operatorul `|` preluăm rezultatul comenzii din stânga operatorului și îl oferim ca intrare comenzii aflate în dreapta operatorului.

Am folosit de mai multe ori operatorul `|` până acum:

- Am afișat informații despre procesele din sistem și am filtrat după numele unui proces:

```
student@uso:~$ ps -e | grep firefox
14912 pts/0    00:00:19 firefox
```

- Am extras primele zece procese care consumă cel mai mare procent de memorie:

```
student@uso:~$ ps -e -ouser,uid,pid,%mem,%cpu,rss,cmd --sort=-%mem | head -11
USER       UID   PID %MEM %CPU   RSS CMD
student   1000  7938 18.0  0.1 367952 /usr/bin/gnome-shell
student   1000  8437  8.4  0.0 171916 /usr/bin/gnome-software --gapplication-service
student   1000  7782  3.9  0.0 81312 /usr/lib/xorg/Xorg vt1 -displayfd 3 -auth /run/user/1000/gdm/Xauthority -background none -noreset -keeptty -verbose 3
root         0  1338  3.8  0.0 78880 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
student   1000  8307  3.1  0.0 64628 /usr/lib/evolution/evolution-calendar-factory
student   1000  8338  3.0  0.0 61860 /usr/lib/evolution/evolution-calendar-factory-subprocess --factory all --bus-name org.gnome.evolution.dataserver.Subprocess.Backend.Cale
root         0   336  2.6  0.0 53612 /lib/systemd/systemd-journald
student   1000  8274  2.3  0.0 48296 nautilus-desktop
root         0  1074  2.2  0.0 45460 /usr/bin/containerd
student   1000 12966  1.8  0.0 38216 /usr/lib/gnome-terminal/gnome-terminal-server
```

Alte exemple de prelucrări de text

Ne amintim de fișierul `/etc/passwd` conține informații despre toți utilizatorii din sistem.

```
student@uso:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
(...)
```

În Linux există filtrul de text `cut` prin care putem extrage doar anumite informații dintr-un output. Să zicem că vrem să extragem doar numele utilizatorilor, fără informațiile legate de grupuri sau home directory.

```
student@uso:~/uso-lab$ cat /etc/passwd | cut -f1 -d":"
root
daemon
bin
(...)
```

Argumentul `-f1` specifică faptul că vrem prima coloană, iar argumentul `-d:` specifică delimitatorul de coloane, în cazul nostru `:`.

1. Pornind de la comanda de mai sus, afișați numele utilizatorilor sortați alfabetic. (Hint: `man sort`)
2. Folosind utilitarul `wc`, obțineți numărul de utilizatori din sistem. (Hint: `man wc`)
3. Să afișeze cele mai consumatoare de memorie 10 procese din sistem. (Hint: folosiți `|` și `tail`)

Redirectări

Majoritatea utilitarelor pe care le folosim afișează rezultatele operațiilor pe care le aplică la ieșirea standard, adică pe ecran. În continuare vom aprofunda ceea ce am discutat despre redirectări în capitolul **Lucrul cu Fișiere**. Anterior am mai menționat și termenul de intrare standard; în această secțiune ne vom clarifica ce înseamnă, ce rol îndeplinesc și cum ne folosim de aceste cunoștințe. Acum, adăugăm câteva informații noi la ceea ce am învățat deja.

Putem folosi niște fișiere speciale pentru redirectarea outputului sau erorilor sau din care să citim input. Detaliem mai jos.

Fișiere speciale

Pe sistemele Linux găsim un număr de fișiere speciale pe care le putem folosim în diferite scopuri:

Fișierul `/dev/null` este un fișier care ignoră orice este scris în el. Este echivalentul unei găuri negre în sistemul nostru. Cu ajutorul său, putem rescrie exemplul de mai sus în modul următor:

```
student@uso:~$ firefox &> /dev/null &
[1] 10349
student@uso:~$ firefox > /dev/null 2>&1 &
[2] 10595
```

Acum orice va genera `firefox` va fi scris în `/dev/null`, care va consuma textul primit fără a ocupa spațiu pe disc. Fișierul `/dev/zero` este un generator de octeți. Acesta generează atâția octeți cu valoarea zero (**0**)[5] cât îi sunt ceruți. Urmăm exemplul:

```
student@uso:~$ cat /dev/zero | xxd
00000000: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000010: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000040: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000050: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000060: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000070: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000080: 0000 0000 0000 0000 0000 0000 0000 0000  ................
[...]
^C
```

Deoarece citim din generator, comanda `cat` va afișa o infinitate de octeți cu valoarea zero. Utilitarul `xxd` afișează în hexazecimal textul primit la STDIN. Trecem rezultatul lui `cat` prin `xxd` deoarece valoarea **0** nu este un caracter printabil. Cu alte cuvinte nu este un caracter obișnuit, ca cele de pe tastatură, deoarece nu are un echivalent grafic. Folosim `Ctrl+c` pentru a opri execția.**Exerciții**: Rulați comanda `cat /dev/zero` pentru a înțelege nevoia utilitarului `xxd` din exemplul de mai sus.

De reținut, fișierul `/dev/urandom` este un alt generator de octeți pe care-l putem folosi pentru extragerea de caractere. De exemplu, pentru a genera o parola de 30 de caractere alfanumerice.

Acesta generează atâția octeți cu valoare random cât îi sunt ceruți.

- **Exerciții**: Rulați comenzile din exemplul anterior, dar acum citiți din `/dev/urandom`.

Generatoarele de octeți sunt utile pentru a testa aplicațiile pe care le dezvoltăm. Majoritatea aplicațiilor pe care le vom scrie, ca și cele pe care le utilizăm, citesc și prelucrează informații. Testăm o aplicație pentru că vrem să verificăm că nu avem buguri. Pentru aceasta putem să folosim seturi de date de intrare cât mai variate și mai aleatoare, adică inputuri random. Folosim utilitarul `dd` pentru a genera un fișier de 100 MB cu octeți random, ca în exemplul de mai jos:

```
student@uso:~$ dd if=/dev/urandom of=rand-100mb count=100 bs=1M
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 1,11416 s, 94,1 MB/s
student@uso:~$ ls -lh rand-100mb
-rw-rw-r-- 1 student student 100M nov  8 17:49 rand-100mb
```

Am folosit următoarele opțiuni ale utilitarului `dd`:

- `if` - input file - calea către fișierul de unde citim
- `of` - output file - calea către fișierul unde scriem
- `bs` - block size - dimensiunea unui block citit din **if**
- `count` - block count - numărul de block-uri citite

- **Exerciții**: Creați un fișier numit `rand-50mb` folosind utilitarul `dd`.

Un caz uzual de utilizare a `dd` este suprascrierea unui disc cu informații aleatoare. Această metodă este utilizată ca o formă de securitate atunci când vrem să ștergem informații de pe un disc. Astfel suprascriem datele șterse pentru a preveni posibilitatea recuperării datelor de pe disc. Mai multe informații găsiți aici [https://uwnthesis.wordpress.com/2014/07/26/kali-how-to-use-dd-to-wipe-your-usb-pen-the-visual-guide/].

Exerciții

1. Afișați primele zece procese sortate în funcție de memoria ocupată (Hint: RSS). Nu uitați să includeți antetul. Redirectați rezultatul în fișierul **top10-rss-consumers**. Modificați comanda astfel încât rezultatul redirectării să nu șteargă conținutul existent.
2. Afișați ultimele zece procese sortate în funcție de utilizarea procesorului (Hint: CPU). Nu uitați să includeți antetul. Redirectați rezultatul în fișierul **top10-cpu-consumers**. Modificați comanda astfel încât rezultatul redirectării să nu șteargă conținutul existent.

Note de subsol

Cuprins

- Laborator 07 (one page)
- Îmbunătățirea lucrului în linia de comandă
- Inspectarea sistemului de fișiere
- Inspectarea fișierelor
- O înțelegere mai bună a shellului

- Sumar: Îmbunătățirea lucrului în linia de comandă

---

[1] Putem să ne găsim în situația în care ecranul terminalului nostru este plin cu rezultatele comenzilor rulate anterior sau cu opțiuni afișate de către auto-complete. Putem să curățăm ecranul folosind comanda `clear`. O alternativă mai rapidă este să folosim combinația de taste `Ctrl+l`. Aceasta va produce același rezultat (va curăța ecranul) și are avantajul că poate fi folosită în timp ce scriem deja o comandă.

[2] Căutarea este case-sensitive. Putem să schimbăm acest comportament prin introducerea opțiunii `-I` în sesiunea interactivă, înainte de a porni căutarea. Dacă doriți să aflați mai multe despre opțiunile pe care le putem introduce apăsați tasta `h` într-o sesiune interactivă și căutați textul "OPTIONS".

[3] Putem folosi tasta `?` pentru a porni o căutare de la poziția curentă către începutul paginii. Alternativ, putem naviga la începutul paginii prin apăsarea unei singure taste (`g`) și apoi pornim căutarea `/` de acolo.

[4] Folosim forma `A-Za-z` pentru a preciza orice caracter din alfabetul englez, indiferent dacă este majusculă sau nu. Nu putem folosi forma `A-z` datorită reprezentării caracterelor în tabelul ASCII. Caracterele **A-Z** sunt reprezentate în intervalul **65-90**, iar caracterele **a-z** în intervalul **97-122** în tabelul ascii. Dacă am folosi forma **A-z**, i-am indica expresiei globbing să includă și caracterele din intervalul **91-96** din tabelul ascii în expresia noastră.

[5] Valoarea **0** nu înseamnă cifra **0**. Valoarea **0** înseamnă caracterul **(null)** din tabelul ASCII [http://www.asciitable.com/]. Caracterul **0** are valoarea **48** în tabelul ASCII.

uso/laboratoare/laborator-07.txt · Last modified: 2021/11/16 09:33 by liza_elena.babu

Laborator 08 - Automatizarea în linia de comandă

Concepte

Comandă	Descriere scurtă
`cat`	afișare conținut fișier
`seq`	generare secvență de numere
`grep`	extrage linii care conțin o anumită expresie regulată
`cut`	extragere anumite coloane
`sed`	filtru de text avansat, folosit pentru substituții
`wc`	contorizare număr de linii, cuvinte sau caractere
`head`	afișare a primelor linii sau caractere
`tail`	afișare a ultimelor linii sau caractere
`for`	parcurgere listă de elemente pentru aplicarea de comenzi pentru fiecare element în parte
`if`	condiție de execuție a unor comenzi
`$(comm)`	expandarea unei comenzi (*command expansion*) - înlocuirea cu output-ul comenzii
`comm1 | comm2`	legarea a două comenzi: output-ul primei comenzi devine input pentru cea de-a doua comandă
`$var` sau `${var}`	valoarea variabilei `var`
`${#var}`	lungimea valorii variabilei `var` (în caractere)
`${var:2:3}`	un subșir al valorii variabilei `var`: de la indexul 2 se extrag 3 caractere

Pentru acest laborator vom folosi comenzi și noțiuni prezentate în laboratoarele de până acum în special laboratorul trecut (Laborator 07 - Îmbunătățirea lucrului în linia de comandă). Vor fi puține comenzi nou introduse; cel mai mult ne va interesa cum, dându-se o situație practică, vom putea folosi shell-ul pentru automatizare.

Automatizarea sarcinilor se referă la delegarea / degrevarea acțiunilor utilizatorilor către programe în sistemul de calcul. Un program va face automat, de sine stătător, acțiuni pe care le-ar face utilizatorul, rezultând în timp mai puțin consumat de utilizator.

Acțiunile care sunt candidate pentru automatizare sunt în general acțiuni repetitive, nu foarte simple și neinteractive. Automatizarea acestora va elibera utilizator de la executarea repetată, plictisitoare, manuală a acestora.

Perspective pentru automatizare

Automatizarea sarcinilor urmărește eficiență, în special temporală. Cu cât face mai rapid acțiuni repetitive, cu atât este mai bine. Acțiunile repetitive necesită interacțiune minimă cu utilizatorul, pentru a eficientiza timpul

Când automatizăm, folosim scripting și funcționalități existente. Un principiu esențial al automatizării este *don't repeat yourself* (nu reinventăm roata). Dacă există comenzi sau funcționalități existente, le folosim pe acelea. Altfel spus, urmărim atât eficiența execuției cât și eficiența dezvoltării.

Eficiența execuției o putem realiza nu doar prin scripting, ci și prin acțiuni precum:

- folosirea de scurtături de taste
- folosirea funcționalităților de tipul *command completion* sau *history search*
- folosirea de alias-uri de comenzi
- folosirea de șabloane: *globbing* sau expresii regulate (*regular expressions*)

Atunci când folosim scripting în shell, ne folosim de funcționalități existente ale shell-ului, pe care le vom descrie mai jos.

Scripting

Scripting presupune folosirea unui limbaj interpretat (numit limbaj de scripting). Acesta poate fi un limbaj precum Python, Perl sau Ruby sau poate fi chiar shell-ul CLI (interpretorul de comenzi). Limbajele de scripting au avantajul **dezvoltării rapide**: adăugăm și testăm rapid funcționalități noi.

Shell scripting are avantajul folosirii comenzilor existente în shell (comenzi de lucru cu fișiere, cu procese, comenzi de filtrare de text). Pentru acțiuni rapide (de tipul *quick'n'dirty*), shell scripting este o soluție foarte bună. Atunci când avem prelucrări numerice sau prelucrări de șiruri, shell scripting poate fi insuficient și este indicat să apelăm la un limbaj de scripting mai complet, precum Python, Ruby sau Perl.

Funcționalități shell

Funcționalitățile shell-ului sunt folosite în eficiența rulării comenzilor și în automatizare. Le-am folosit și amintit pe parcursul ultimelor laboratoare, în special în laboratorul trecut. Astfel de funcționalități sunt:

- redirectarea comenzilor în fișiere, folosind operatorii de redirectare: `>`, `<`, `>>`
- redirectarea ieșirii unei comenzi către intrarea altei comenzi: operatorul `|` (*pipe*)
- variabile shell
- escaping folosind *backslash*, ghilimele sau apostrofuri

Pe parcursul acestui laborator vom folosi funcțioalități ale shell-ului care sunt în special folosite în shell scripting:

- expresii regulate
- argumente în linia de comandă
- comenzi interne shell pentru controlul fluxului: `if`, `for`, `while`

Automatizarea prin shell scripting

Automatizarea prin shell scripting presupune, în mod uzual, **o secvență de comenzi**. Aceste comenzi sunt adăugate într-un script unde vor fi rulate la comun, fără a fi nevoie de rularea manuală a fiecărei comenzi.

Pentru a configura rulare unei secvențe de comenzi putem condiționa rularea scriptului folosind **variabile sau argumente în linia de comandă**. Acestea pot genera un comportament diferit al scriptului în funcție de valorile lor.

Pe baza variabilelor, argumentelor din linia de comanda sau datelor de intrare, un script poate lua **decizii condiționate**. Verificarea unei condiții poate duce la un comportament sau altul.

În rularea unui script adesea vom dori ca o acțiune să țintească mai multe fișiere sau procese sau șiruri. Pentru aceasta, vom urmări folosirea de **bucle pentru parcurgerea de liste de elemente**.

Comenzile folosite pot primi ca argument sau la intrarea standard ieșirea altor comenzi. În shell scripting folosim funcționalități de înlănțuire a comenzilor precum ***command expansion*** **sau** ***pipe-uri*** ducând la rezultate mai complexe.

Basics

Nu insistați pe această secțiune. Traversați-o în diagonală, ajungeți cât mai repede la secțiunea Exerciții. Reveniți la ea când aveți nevoie în rezolvarea exercițiilor.

Expresii regulate

Expresiile regulate sunt folosite pentru căutarea/selecția anumitor intrări în fișiere text și pentru parsarea informațiilor din fișiere text. În procesarea expresiilor regulate, matching-ul se face pe fiecare linie din text. Într-o expresie regulată, anumite caractere au rol special precum caracterele de mai jos.

Caracter	Efect în expresie regulată
^	început de linie
$	sfârșit de linie
.	orice caracter
*	expresia anterioară de oricâte ori, posibil niciodată
+	expresia anterioară de oricâte ori, cel puțin o dată
[ajt]	orice caracter din setul de caractere a, j, t
[^ajt]	orice caracter **mai puțin** caracterele a, j, t
|	expresia anterioară **sau** expresia de după (una dintre cele două expresii)
?	expresia anterioară o dată sau niciodată

Găsiți un tool online aici [http://regexr.com/] care va permite să vizualizați efectul expresiilor regulate în mod interactiv. Puteți să îl folosiți pentru a vă obișnui cu scrierea de regexp-uri.

Expresii regulate vs globbing

Construcție	RegExp	Glob
.	orice caracter	caracterul .
.*	orice caracter de oricâte ori, posibil niciodată	orice incepe cu .
?	expresia anterioară o dată sau niciodată	orice caracter
a+	caracterul a de oricâte ori, cel puțin o dată	caracterul a urmat de +

Expresii regulate & grep

În directorul din repository aferent laboratorului există un fișier `students.txt` pe care îl vom folosi ca suport pentru comenzi cu expresii regulate. Acest fișier conține o listă de studenți cu numele complet al studenților (prima coloană), grupa din care fac parte (a doua coloană) și diverse note la USO (nota finală - a treia coloană, nota la testul grilă - a patra coloană - și nota la testul practic - a cincea coloană), câmpuri separate prin caracterul tab.

Pentru căutarea și selectarea de linii din fișiere text folosim comanda `grep` care folosește, la rândul ei, expresii regulate. Astfel, dacă dorim să selectăm studenții care au litera `z` în numele lor, folosim comanda

```
student@uso:~/uso-lab/labs/06-scripting/support/00-basics$ grep 'z' students.txt
GHECENCO F. Răzvan      312CC   8       8.75    4.67
MARIN N. Răzvan 312CC   5       3.5     4.2
```

Este recomandat ca argumentul de tip expresie regulată transmis comenzii `grep` să fie plasat între apostrofuri (`'`) pentru a fi escapat. În felul acesta, caracterele din expresia regulată vor fi transmise întocmai comenzii `grep` și nu vor fi interpretate de shell.

În shell este recomandată folosirea ghilimelelor (”, *(double) quotes*) și a apostrofurilor (', *single quotes*) pentru escapare. **Escapare** înseamnă că nu vor fi interpretate special caracterele speciale din shell; adică vor fi interpretate drept caractere obișnuite. Exemple de caractere speciale în shell sunt:

- $: folosit pentru expandare: expandarea valorii unei variabile, expandarea unei comenzi, substituție aritmetică
- *: folosit în globbing: înseamnă orice caracter de oricâte ori
- >, <: folosite pentru redirectare
- &, |: folosite pentru trimiterea unei comenzi în background sau pentru înlănțuirea unei comenzi; sau ca parte a operatorilor de secvențiere condiționată (|| și &&)
- ;: folosit pentru secvențierea comenzilor
- (,): folosite pentru crearea de subshell-uri
- ”, ', \: folosite pentru escapare

Diferența dintre ghilimele și apostrofuri constă în interpretarea caracterului dolar ($, *dollar*). Între ghilimele, caracterul dolar își păstrează rolul special, pe când între apostrofuri este considerat un caracter ca oricare altul.

De exemplu, dacă dorim să afișăm valoarea variabilei USER, vom folosi construcția

```
student@uso.local:~$ echo "$USER"
student
```

Va fi afișată valoarea variabilei USER întrucât am folosit construcția $USER între ghilimele.

Dacă însă folosim construcția $USER între apostrofuri, ca în continuare

```
student@uso.local:~$ echo '$USER'
$USER
```

atunci va fi afișat chiar șirul $USER. Asta pentru că apostrofurile, spre deosebire de ghilimele, escapează inclusiv caracterul dolar.

Doar că expresia `z` nu se potrivește (nu face *match*) pe litera Z (majusculă). Pentru a selecta studenții al cărore nume conține atât litera `z`, cât și Z (majusculă) folosim expresia regulată `[zZ]` și comanda

```
student@uso:~/uso-lab/labs/06-scripting/support/00-basics$ grep '[zZ]' students.txt
PAJARCU Z.P. Raul-Constantin    312CC   2       0.5     1.7
GHECENCO F. Răzvan      312CC   8       8.75    4.67
MARIN N. Răzvan 312CC   5       3.5     4.2
ZINCULESCU C. Marius-Valentin   313CC   7       4.75    5.7
```

Expresia regulată `[zZ]` se potrivește pe setul de caractere compus din litera z cât și Z (majusculă). Cu aceasta au apărut două linii noi: o linie conține studentul cu numele de familie `ZINCULESCU` iar cealaltă linie conține inițialele `Z.P.` ambele cu litera Z (majusculă).

Dacă vrem să selectăm studenții al căror nume de familie începe cu litera F, atunci vom folosi expresia regulată ^F denotând litera F (majusculă) la început de rând. Vom folosi, așadar, comanda de mai jos:

```
student@uso:~/uso-lab/labs/06-scripting/support/00-basics$ grep '^F' students.txt
FLOREA N. Monica-Andreea        313CC   9       7.5     8.5
FULGER P. Alexandru-Gabriel     315CC   7       4.25    5
```

Dacă vrem, în schimb, să selectăm studenții al căror prenume începe cu litera F trebuie să privim lucrurile altfel. Urmărind textul putem observa că prenumele sunt precedate fie de caracterul spațiu (*blank*) fie de caracterul minus (`-`). Pentru început vom folosi expresia regulată `[ -]F` care face match pe caracterul spațiu sau minus urmat de caracterul F majusculă. Comanda aferentă este:

```
student@uso:~/uso-lab/labs/06-scripting/support/00-basics$ grep '[ -]F' students.txt
ONEA I. Flavia-Katilina 311CC   7       6.5     4.33
PLEȘEA Fl. Alexandru    311CC   9       9.25    7.9
GHECENCO F. Răzvan      312CC   8       8.75    4.67
EPURE P. Andi-Florin    314CC   8       9.5     3.67
NEACȘU C. Florin-Mărgărit       314CC   10      9       9
COSTEA I. Florin Traian 315CC   4       3.5     3.7
CHIȚESCU E. Bogdan-Florentin    315CC   9       7.75    6.89
```

Observăm, însă, că se face match și pe șirul `Fl.` și pe șirul `F.` reprezentând inițiale. Știm că un prenume conține doar litere mici și că **nu** se încheie cu punct (`.`, *dot*). Atunci extindem expresia regulată de mai sus la expresia `[ -]F[a-z]\+[^\.]`. Expresia regulată face match pe un șir care:

- începe cu unul dintre caracterele spațiu (*blank*) sau minus (`-`);
- continuă cu litera F (majusculă)
- continuă cu litere mici (folosind setul `[a-z]`)
- literele mici de oricâte ori cel puțin o dată (folosind caracterul plus (+))
- continuă cu orice caracter diferit de punct (`.`, *dot*), folosind expresia `[^\.]`. Folosim backslash (`\`) pentru a escapa caracterul punct (`.`, *dot*); altfel ar fi însemnat orice caracter, așa cum înseamnă într-o expresie regulată.

În expresiile regulate obișnuite (*basic regular expressions*), anumite caractere își pierd rolul lor special. Este cazul caracterului +, dar și al caracterelor ?, {, (,) și |. Pentru a-și păstra rolul special, acestea trebuie precedate de backslash în construcții de forma \+, \?, \{, \(, \) și \|.

Găsiți această precizare în pagina de manual a comenzii `grep`; căutați șirul `Basic vs Extended`.

Astfel, comanda `grep` pentru a extrage studenții al căror prenume începe cu litera F este

```
student@uso:~/uso-lab/labs/06-scripting/support/00-basics$ grep '[ -]F[a-z]\+[^\.]' students.txt
ONEA I. Flavia-Katilina 311CC   7       6.5     4.33
EPURE P. Andi-Florin    314CC   8       9.5     3.67
NEACȘU C. Florin-Mărgărit       314CC   10      9       9
COSTEA I. Florin Traian 315CC   4       3.5     3.7
CHIȚESCU E. Bogdan-Florentin    315CC   9       7.75    6.89
```

Dacă ați fi dorit să faceți match pe o literă (mică sau mare) ați fi folosit construcția `[a-zA-Z]`. Dacă ați fi dorit să faceți match pe o literă (mică sau mare) sau pe o cifră, ați fi folosit expresia `[a-zA-Z0-9]`.

tr & sed

Utilitarul **tr** permite translatarea, ștergerea și manipularea caracterelor primite la intrare. Practic, **sed** este un **stream editor** ce poate efectua transformări la nivel de string asupra unui text primit la intrare. În plus, sed poate primi expresii regulate ca argument de căutare.

Diferența dintre tr și sed este că primul efectuează transformări la nivel de caracter, pe când al doilea efectuează transformări la nivel de string. Din acest motiv putem spune că sed este un tr mai avansat, tr++ :).

Spre exemplu, folosind atât **tr**, cât și **sed**, să înlocuim caracterul ',' cu TAB în `student.csv`:

```
student@ubuntu:~/uso-lab/labs/06-scripting/support/00-basics$ cat students.csv | tr , "\t" > students.out
student@ubuntu:~/uso-lab/labs/06-scripting/support/00-basics$ cat students.out
VLĂDUȚU I. Liviu-Alexandru      311CC   6       3.5     5.22
GEORGIU V. Alexandra-Maria      311CC   10      10      9.67
PĂUNOIU N. Gabriel      311CC   7       6.5     3.5
BĂCÎRCEA A. Andrei      311CC   7       5.5     4.44
[...]

student@ubuntu:~/uso-lab/labs/06-scripting/support/00-basics$ sed 's/,/\t/g' students.csv > students.out
student@ubuntu:~/uso-lab/labs/06-scripting/support/00-basics$ cat students.out
VLĂDUȚU I. Liviu-Alexandru      311CC   6       3.5     5.22
GEORGIU V. Alexandra-Maria      311CC   10      10      9.67
PĂUNOIU N. Gabriel      311CC   7       6.5     3.5
BĂCÎRCEA A. Andrei      311CC   7       5.5     4.44
[...]
```

Observăm că cele două comenzi au același efect. Pentru a înlocui șiruri de caractere, tr nu mai oferă funcționalitatea dorită întrucât el face o mapare 1-la-1 între caracterele setul ce trebuie înlocuit și setul ce înlocuiește. În exemplul de mai jos, 1 este translatat în 2 indiferent unde apare, iar C în A:

```
student@ubuntu:~/uso-lab/labs/06-scripting/support/00-basics$ cat students.csv | tr "311CC" "312CA" > students.out
student@ubuntu:~/uso-lab/labs/06-scripting/support/00-basics$ cat students.out
VLĂDUȚU I. Liviu-Alexandru,322AA,6,3.5,5.22
GEORGIU V. Alexandra-Maria,322AA,20,20,9.67
PĂUNOIU N. Gabriel,322AA,7,6.5,3.5
BĂAÎRAEA A. Andrei,322AA,7,5.5,4.44
[...]

student@ubuntu:~/uso-lab/labs/06-scripting/support/00-basics$ sed "s/311CC/312CA/g" students.csv > students.out
student@ubuntu:~/uso-lab/labs/06-scripting/support/00-basics$ cat students.out
VLĂDUȚU I. Liviu-Alexandru,312CA,6,3.5,5.22
GEORGIU V. Alexandra-Maria,312CA,10,10,9.67
PĂUNOIU N. Gabriel,312CA,7,6.5,3.5
BĂCÎRCEA A. Andrei,312CA,7,5.5,4.44
[...]
```

cut & awk

Am văzut în laboratoarele precedente cum putem extrage informații structurate pe linii și coloane folosind utilitarul **cut**.

```
student@ubuntu:~/uso-lab/labs/06-scripting/support/00-basics$ cat /etc/passwd | cut -d':' -f1,6 | head -3
root:/root
daemon:/usr/sbin
bin:/bin
```

Utilitarul awk [https://linux.die.net/man/1/awk] permite aceleași acțiuni ca și **cut**, dar funcționalitatea sa este mai extinsă. Spre exemplu, awk poate folosi o expresie regulată ca delimitator, pe când cut acceptă un singur caracter:

```
student@ubuntu:~/uso-lab/labs/06-scripting/support/00-basics$ netstat -i
Kernel Interface table
Iface   MTU Met   RX-OK RX-ERR RX-DRP RX-OVR    TX-OK TX-ERR TX-DRP TX-OVR Flg
eth0       1500 0      1955      0      0 0           521      0      0      0 BMRU
lo        65536 0       359      0      0 0           359      0      0      0 LRU

student@ubuntu:~/uso-lab/labs/06-scripting/support/00-basics$ netstat -i | cut -d' ' -f1,2
Kernel Interface
Iface
eth0
lo

student@ubuntu:~/uso-lab/labs/06-scripting/support/00-basics$ netstat -i | awk -F' *' '{print $1,$2,$4}'
Kernel Interface
Iface MTU RX-OK
eth0 1500 1955
lo 65536 359
```

Awk [https://en.wikipedia.org/wiki/AWK] este considerat un fel de limbaj de programare ce vizează procesarea text. Există script-uri awk complexe ce se aseamănă programelor C. Printre altele, awk permite implementarea și apelarea de funcții.

sort & uniq

Dacă dorim să sortăm studenții din fișier în ordinea numelor vom folosi comanda `sort`:

```
student@uso:~/uso-lab/labs/06-scripting/00-basics$ sort students.csv
ALECU C. Ionuț-Gabriel,312CC,7,4.5,6.4
ASĂVOAEI P. Cătălin,315CC,8,6.75,7
BĂCÎRCEA A. Andrei,311CC,7,5.5,4.44
BADEA P. Bogdan,314CC,4,2.75,1.56
[...]
```

Cateva optiuni des folosite cu **sort** sunt:

- `-t` specifica separatorul
- `-k` specifica index-ul, sau cheia, coloanei dupa care vrem sa sortam intrarile
- `n` sortare numerică (implicit este alfabetica)
- `-r` sortare inversa (implicit crescator)

O sursă frecventă de greșeli este folosirea construcției `-k 3` în loc de `-k 3,3` atunci când folosim mai multe chei de sortare. Dacă folosim `-k 3` atunci cheia de sortare este dată de **toate** coloanele începând cu a 3-a coloană. În acest caz viitoare opțiuni de chei de sortare nu mai sunt luate în considerare.

Vezi și discuția de aici [http://unix.stackexchange.com/questions/52762/trying-to-sort-on-two-fields-second-then-first].

Dorim sa facem o sortare numerica descrescatoare după notă, urmata de o sortare alfabetică după grupă (adică să fie toate notele de 10 la început dar sortate în ordinea grupelor). Astfel, pentru a sorta intrările în ordinea notei, vom folosi separatorul **,** (virgulă, *comma*) și coloana a 3-a pentru cheie:

```
student@uso:~/uso-lab/labs/06-scripting/support/00-basics$ sort -t ',' -k 3,3nr -k 2,2 students.csv
GEORGIU V. Alexandra-Maria,311CC,10,10,9.67
MUȘATESCU V. Alexandru-Petrișor,311CC,10,8.5,9
RADU L. Alina,311CC,10,10,7.89
GONDOȘ I. Gabriel,312CC,10,9,7.33
[...]
```

În cazul în care există linii duplicate, putem folosi utilitarul **uniq** în conjuncție cu **sort** pentru a le elimina. În acest scop, adăugăm o linie duplicata în `students.csv`:

```
VLĂDUȚU I. Liviu-Alexandru,311CC,6,3.5,5.22
GEORGIU V. Alexandra-Maria,311CC,10,10,9.67
[...]
VLĂDUȚU I. Liviu-Alexandru,311CC,6,3.5,5.22
BENE D. Adrian,312CC,9,10,5
[...]
```

```
student@uso:~/uso-lab/labs/06-scripting/support/00-basics$ sort -t ',' -k 3,3nr -k 2,2 students.csv | wc -l
93
student@uso:~/uso-lab/labs/06-scripting/support/00-basics$ sort -t ',' -k 3,3nr -k 2,2 students.csv | uniq | wc -l
92
student@uso:~/uso-lab/labs/06-scripting/support/00-basics$ uniq students.csv | wc -l
93
```

Observăm că uniq elimină liniile duplicate adiacente. De aceea, dacă datele nu sunt sortate, comanda nu are nici un efect.

Shell scripting

Un script shell este un fișier text care conține comenzi și construcții specifice shell-ului. Un script shell începe cu construcția `#!/bin/bash`, denumită *shebang* care indică interpretorul scriptului; în cazul de față interpretorul este chiar shell-ul Bash. Dacă nu este specificat nici un shell prin shebang atunci implicit va fi luat shell-ul implicit (setat în `/etc/passwd`) asignat utilizatorului logat.

Spre exemplu, one-liners pe care le-am tot scris până acum ar putea fi puse într-un shell script și, mai mult, dacă sunt comenzi lungi și nu încap pe o linie le putem "sparge" în mai multe linii folosind separatorul `\`:

```
#!/bin/bash

export DATE=$(date +20%y%m%d) && \
mkdir -p /usr/share/snapshots && \
tar -cvpzf /usr/share/snapshots/$HOSTNAME_$DATE.tar.gz \
```

```
    --exclude=/proc --exclude=/lost+found \
    --exclude=/sys --exclude=/mnt \
    --exclude=/media --exclude=/dev \
    --exclude=/share/Archive /
```

ATENȚIE! În construcția de mai sus după \ trebuie să nu mai existe nici un caracter (nici măcar spațiu alb/trailing whitespaces)

Scriptul conține comenzi uzuale folosite în shell și alte comenzi care se regăsesc mai adesea în scripturi: `while`, `if`, `for`. Acestea nu sunt instrucțiuni, ci sunt tot comenzi shell; pot fi folosite, dacă sintaxa este corectă și în linia de comandă.

Argumente din linia de comandă

Un script moștenește variabilele de mediu ale shell-ului părinte precum `HOME`, `BASH`, `IFS`, `USER`. În script le putem folosi astfel:

```
#!/bin/bash

echo $HOME
```

Pe lângă aceste variabile, scriptul poate primi o serie de argumente din linia de comandă. Folosind următoarele variabile, putem referi argumentele primite de script din linia de comandă:

- `$*` este un string (`$1, $2 … $n`) format din toate argumentele primite de script
- `$@` este o listă formată din argumentele scriptului
- `$1, $2 … $n` reprezintă fiecare parametru din script
- `$0` este numele scriptului
- `$#` este numărul de argumente primite din linia de comandă

Spre exemplu, să luăm următorul script `arguments.sh`:

```
#!/bin/bash

echo There are $# arguments to $0: $*
echo first argument: $1
echo second argument: $2
echo third argument: $3
echo the list of arguments: $@
```

Să apelăm scriptul cu 4 argumente:

```
student@uso:~$ ./arguments.sh banane cirese caise castraveti
   There are 4 arguments to arguments.sh: banane cirese caise castraveti
   first argument: banane
   second argument: cirese
   third argument: caise
   the list of arguments: banane cirese caise castraveti
```

Constructia while read

Creați un nou script `extract-name` cu conținutul de mai jos:

```
#!/bin/bash

IFS=','
while read name group final_grade test_grade practical_grade; do
    echo "$name"
done < students.csv
```

Pentru parsare în shell folosim construcția `while read ...`. Construcția este urmată de numele variabilelor în care vom reține câmpurile parsate din cadrul fiecărei linii. Folosim la intrare fișierul `students.csv` aflat în directorul părinte; este un fișier în format CSV (*Comma Separated Values*) folosind ca separator caracterul virgulă (`,`, *comma*). Pentru a extrage doar numele studenților din fișierul de intrare vom rula scriptul `extract-name`:

```
student@uso:~/uso-lab/labs/06-scripting/support/00-basics$ ./extract-name
VLĂDUȚU I. Liviu-Alexandru
GEORGIU V. Alexandra-Maria
PĂUNOIU N. Gabriel
BĂCÎRCEA A. Andrei
[...]
```

Întrucât formatul de intrare folosește virgulă (`,`, *comma*) pe post de separator, am definit în script variabila internă `IFS` (*Internal Field Separator*) la valoarea `','`, așa cum observăm în linia 3 din scriptul `extract-name`:

```
IFS=','
```

Variabila internă shell-ului `IFS` (*Internal Field Separator*) definește caracterul sau caracterele care vor fi folosite pentru împărțirea unei linii în câmpuri (*splitting*).

Variabila internă este folosită ori de câte ori în shell este nevoie de împărțirea unei linii. Un caz uzual de folosire a variabilei `IFS` este în conjuncție cu construcția `while read` pentru citirea de linii de la intrarea standard sau dintr-un fișier și împărțirea acestora în câmpuri.

Informații și pe Wikipedia [http://en.wikipedia.org/wiki/Internal_field_separator].

Pașii de mai sus puteau fi realizați și cu ajutorul comenzii `cut`. Dar în cazul parsării folosind construcția `while read` avem două avantaje:

- putem afișa coloanele în ce ordine dorim; `cut` permitea afișarea de coloane doar în ordinea din fișierul de intrare;
- putem prelucra în continuare, în cadrul construcției `while read` informația parsată. Spre exemplu, afișarea poate avea forma "Studentul ... face parte din grupa ...".

if statement

Putem extinde script-ul de mai sus pentru a afișa doar studenții care au media mai mare decât 5.

```
#!/bin/bash

IFS=','
```

```
while read name group final_grade test_grade practical_grade; do
    if test "$final_grade" -gt 5; then
        echo "$name,$group,$final_grade"
    fi
done < students.csv
```

for loop

În bash există și construcții de tipul `for`. Un exemplu comun este acela de a itera prin conținutul unui director și de a face prelucrări asupra fișierelor. Spre exemplu, dorim să facem backup tuturor fișierelor dintr-un director trimis ca parametru în script:

```
#!/bin/bash

for file in $1/*
do
    if test -f $file; then
        stat --print="%a %F %n\n" $file
        cp $file $file.bkp
    fi
done
```

Se poate itera și pe output-ul unei comenzi:

```
#!/bin/bash

for file in $(find /usr/share/pixmaps/ -type f -iname '*.jpg')
do
    echo $file
done
```

Exerciții

Accesați subdirectorul `~/uso-lab/labs/06-scripting/support/` pentru exercițiile de laborator. Sunt și tutoriale (deja făcute / rezolvate) și exerciții. Tutorialele au rolul de a vă acomoda cu diferitele comenzi iar exercițiile sunt parte practică efectivă.

Realizați exercițiile în ordine. De la secțiunea 7 faceți echipe de 2-3 persoane, sub coordonarea asistentului și lucrați împreună la rezolvarea exercițiilor.

01. Alias-uri

Accesați subdirectorul `01-alias/`. Nu există fișiere de suport pentru aceste exerciții.

În acest set de exerciții / tutoriale vom vedea cum folosim alias-uri pentru eficientizarea acțiunilor.

Pentru început să folosim comanda `xdg-open` în formele de mai jos:

```
xdg-open .
xdg-open http://google.com
xdg-open ../07-extend-scripts/convert/Orar1CA.xls
```

Aceste comenzi deschid respectiv, un file browser, un web browser și LibreOffice.

Pentru a realiza mai rapid acțiunile, folosim un alias. Facem alias-urile `open` și `go` pentru `xdg-open` ca mai jos:

```
alias open='xdg-open'
alias go='xdg-open'
```

Exercițiu: Acum folosiți comanda `open` sau comanda `go` pentru cele trei acțiuni de mai sus. Am simplificat comanda `xdg-open` (mai greu de scris și mai greu de reținut).

02. One liners

Accesați subdirectorul `02-one-liners/` din directorul laboratorului.

Indexed Names

Intrați în subdirectorul `indexed-names/`. Dorim să realizăm operații automate în sistemul de fișiere, folosind `for` și one linere.

Rulăm cele două comenzi de mai jos:

```
for i in $(seq -f "%02g" 1 3); do touch test-"$i".txt; done
for i in $(seq -f "%02g" 0 12); do mkdir uso-curs-"$i"; done
```

La sfârșitul rulării acestor comenzi obținem 3 fișiere text (pentru cele 3 lucrări/teste) si 13 directoare, pentru cele 13 directoare. Am folosit `for` pentru parcurgerea unei liste și `seq` pentru crearea unei liste numerice.

Exercițiu: Realizați un one liner care creează cele 13 directoare și în cadrul fiecărui director creează un subdirector numit `slides/`, un fișier numit `note.txt` și un fișier numit `resurse.txt`.

Rename

Intrați în subdirectorul `rename/`. Copiați fișierele din subdirectorul `src/` în directorul curent:

```
cp src/* .
```

Vrem să redenumim toate fișierele cu extensia `.c` ca să avem prefixul `uso_` în nume. Pentru aceasta folosim one liner-ul:

```
for i in *.c; do mv "$i" uso_"$i"; done
```

După această comandă, vor apărea fișiere redenumite.

Apoi ștergeți fișierele nou generate și recopiați-le din directorul `src/`:

```
rm *.c
cp src/*.c .
```

Exercițiu: Realizați un one liner care redenumește toate fișierele cu extensia `.c` și adaugă sufixul `_uso` în nume. Adică fișierul `ptr.c` devine `ptr_uso.c`.

convert

În cadrul subdirectorului `convert/` avem mai multe fișiere `.svg`. Dorim să le convertim în fișiere PNG. Pentru aceasta folosim comanda `inkscape`.

Dacă nu este disponibilă comanda `inkscape` va trebui să o instalați folosind comanda:

```
sudo apt install inkscape
```

Pentru a converti fișierul `acl.svg` în fișierul `acl.png` folosim comanda:

```
inkscape --export-png=acl.png acl.svg
```

În urma comenzii a apărut fișierul `acl.png`. Verificăm conversia corectă folosind alias-ul `go` creat anterior:

```
go acl.png
```

Pentru a realiza conversia **tuturor** fișierelor din directorul curent folosim one liner-ul:

```
for i in *.svg; do inkscape --export-png=$(basename "$i" .svg).png "$i"; done
```

În urma rulării acestui one liner, toate fișierele `.svg` au fost convertite în fișiere PNG păstrându-se numele și adăugându-se extensia `.png`.

03. One liners vs commands

Accesați subdirectorul `03-one-liners-vs-commands/` din directorul laboratorului. Vom compara one linere cu comenzi mai simple.

Rulăm următoarele comenzi pentru a genera o parolă de 16 caractere și pentru a afișa PID-urile proceselor utilizatorului `student`:

```
tr -dc 'a-zA-Z0-9~!@#$%^&*_()+}{?></";.,[]=-' < /dev/urandom | fold -w 32 | head -n 1
ps -ef | grep student | tr -s ' ' | cut -d ' ' -f 2
```

Aceste acțiuni le putem realiza și folosind comenzi dedicate și opțiunile lor, fără a fi nevoie de trecerea prin operatorul | (*pipe*):

```
pwgen -y 16 1
ps -u student -o pid --no-header
```

În general, acolo unde o comandă oferă opțiuni, este de preferat, fiind mai robust, să folosim comandă în loc de înlănțuirea mai multor comenzi prin operatorul | (*pipe*).

Indexed Names

Intrați în subdirectorul `indexed-names/`. Vrem să realizăm operații similare exercițiului `indexed-names/` de la punctul anterior.

Astfel că, putem realiza același lucru precum comenzile de mai jos:

```
for i in $(seq -f "%02g" 1 3); do touch test-"$i".txt; done
for i in $(seq -f "%02g" 0 12); do mkdir uso-curs-"$i"; done
```

folosind comenzile:

```
touch $(seq -f "test-%02g.txt" 1 3)
mkdir $(seq -f "uso-curs-%02g" 0 12)
```

Avantajul celui de-al doilea set de comenzi este că vor crea un singur proces (`touch` sau `mkdir`). Avantajul primului set de comenzi este că putem adăuga mai multe (și mai diferite) comenzi care să fie executate pe fiecare element din bucla for.

04. Expresii regulate

Accesați subdirectorul `04-regex/` din directorul laboratorului. Vrem să folosim expresii regulate pentru a prelucra fișierele `students.csv` și `students.txt`.

Exerciții: Folosiți `grep` pentru a extrage, respectiv, din fișierul `students.csv`:

- studenții de la grupa 315CC
- studenții de la grupa 315CC și care au luat prima notă 9 (din cele 3 coloane)
- studenții de la grupa 315CC și care au luat a doua notă care începe cu cifra 7 (din cele 3 coloane)
- studenții de la grupa 315CC și care au luat a treia notă care începe cu cifra 6 (din cele 3 coloane)

Mai jos sunt rezultatele așteptate pentru cele patru comenzi de mai sus:

```
$ grep TODO students.csv
ȚIFREA C. Alexandru-Nicolae,315CC,7,7,6.25
BOGDAN O.Gh. Ana,315CC,5,4.5,1.89
COSTEA I. Florin Traian,315CC,4,3.5,3.7
OLTEANU Ș. Ionuț-Mihăiță,315CC,7,4.25,4.7
FULGER P. Alexandru-Gabriel,315CC,7,4.25,5
DUȘE-VASILIU V. Andra,315CC,8,7,6.4
CHIȚESCU E. Bogdan-Florentin,315CC,9,7.75,6.89
MAVRODIN I. Andrei,315CC,9,7,6.4
DRĂGOI M. Ovidiu-Alexandru,315CC,8,8,3.44
MANEA A. Mihail-Alin,315CC,7,6.75,6.22
TRACHE M. Ana-Maria,315CC,7,4.75,4.9
IORDACHE M. Tiberiu-Constantin,315CC,8,8.25,4.56
BARBU I. Ștefan,315CC,10,10,7.56
MĂNOIU I. Ioana-Veronica,315CC,10,10,8.6
STOICA I.D. Aurel-Octavian,315CC,7,3.75,5.4
IUȘAN V. Bogdan-Cosmin,315CC,9,7.25,8
BURCEANU D.N. Radu-Gabriel,315CC,8,8.75,7
```

```
ASĂVOAEI P. Cătălin,315CC,8,6.75,7
ȘTIRBĂȚ A. Steliana,315CC,10,10,10

$ grep TODO students.csv
CHIȚESCU E. Bogdan-Florentin,315CC,9,7.75,6.89
MAVRODIN I. Andrei,315CC,9,7,6.4
IUȘAN V. Bogdan-Cosmin,315CC,9,7.25,8

$ grep TODO students.csv
ȚIFREA C. Alexandru-Nicolae,315CC,7,7,6.25
DUȘE-VASILIU V. Andra,315CC,8,7,6.4
CHIȚESCU E. Bogdan-Florentin,315CC,9,7.75,6.89
MAVRODIN I. Andrei,315CC,9,7,6.4
IUȘAN V. Bogdan-Cosmin,315CC,9,7.25,8

$ grep TODO students.csv
ȚIFREA C. Alexandru-Nicolae,315CC,7,7,6.25
DUȘE-VASILIU V. Andra,315CC,8,7,6.4
CHIȚESCU E. Bogdan-Florentin,315CC,9,7.75,6.89
MAVRODIN I. Andrei,315CC,9,7,6.4
MANEA A. Mihail-Alin,315CC,7,6.75,6.22
```

05. find

Accesați subdirectorul `05-find/` din directorul laboratorului.

Pentru început vom crea o ierarhie de fișiere pe care să o folosim în continuare folosind comanda:

```
tar xf etc.tar.gz
```

În urma rulării acestei comenzi obținem un subdirector `etc/`. Folosim comanda `find` pentru a parcurge fișiere din ierarhia de fișiere `etc/`.

Pentru început rulăm comenzile de mai jos pentru a afișa anumite intrări:

```
find etc/
find etc/ -type f
find etc/ -type f -name 'a*'
find etc/ -type f -name 'a*.*'
```

Exercițiu: Afișați intrările de tip link simbolic din ieraria `etc/`. Apoi afișați intrările de tip link simbolic care încep cu litera `a`.

Dacă dorim să rulăm anumite informații despre un fișier dat, precum dimensiunea sa, putem folosi comenzi precum cele de mai jos:

```
find etc/ -type f -exec stat -c "%s" {} \;
find etc/ -type f -exec stat -c "%s,%n" {} \;
find etc/ -type f -exec stat -c "%s,%n" {} \; | sort -n
find etc/ -type f -exec stat -c "%s,%n" {} \; | sort -n | tail -10
```

Exercițiu: Ștergeți toate intrările de tip link simbolic din ierarhia `etc/` al căror nume începe cu litera `a`.

Putem selecta anumite fișiere după permisiunile prezente. De exemplu, pentru a afișa fișierele care au permisiuni de execuție pentru toți utilizatorii (adică bitul `x` este prezent pentru `user`, `group` și `others`) folosim comanda:

```
find etc/ -type f -perm /111
find etc/ -type f -perm /111 -exec ls -l {} \;
```

Exercițiu: Scoateți permisiunile de execuție de pe toate fișierele care au permisiuni de execuție pentru toți utilizatorii.

Putem selecta fișiere după timpul de access, precum comenzile de mai jos:

```
find etc/ -type f -mtime -25
find etc/ -type f -newer etc/apt/trusted.gpg
```

Exercițiu: Afișați toate fișierele cu dimensiune `0`. Apoi toate fișierele cu dimensiune mai mare de `20` kiloocteți. Folosiți opțiunea `-size` a comenzii `find` (puteți vedea detalii în pagina de manual).

06. Scripturi simple

Accesați subdirectorul `06-simple-scripts/` din directorul laboratorului. În acest director avem exemple de scripturi practice. Urmăriți toate scripturile din acest director (nu e nevoie să le rulați) și vedeți ce face fiecare.

Exercițiu: Pentru dezactivarea montării stick-urilor USB pe sistem putem folosi comenzile:

```
sudo rmmod uas
sudo rmmod usb_storage
```

Apoi, pentru a reactiva, folosim comenzile:

```
sudo modprobe usb_storage
sudo modprobe uas
```

Creați, respectiv, scripturile `disable-usb` și `enable-usb` care să dezactiveze și reactiveze montarea stick-urilor USB. Rulați-le pentru verificare.

Exercițiu: Creați un script numit `system-info` care afișează informații despre sistem pe modelul de mai jos:

```
date: 2019-11-05
kernel: 5.0.0-32-generic
version: Ubuntu 18.04.3 LTS
num_processes: 336
```

07. Extindere scripturi

Accesați subdirectorul `07-extend-scripts/` din directorul laboratorului.

convert

Intrați în subdirectorul `convert/`. Avem 4 fișiere `.xls` reprezentând orarele de anul 1. Convertim aceste fișiere în format PDF folosind scriptul `solution`:

```
./solution
```

Vom obține câte un fișier PDF. Putem folosi aliasul `go` pentru a deschide un fișier:

```
go Orar1CA.pdf
```

PDF Stitching

Intrați în directorul `pdf-stitch/`. Vrem să obținem fișierele de tip orar; pentru aceasta folosim scriptul `download`:

```
./download
```

Apoi rulați scriptul anterior pentru a converti fișierele `.xls` descărcate în fișiere PDF:

```
../convert/solution
```

Ca să unim mai multe fișiere PDF folosim comanda `pdftk`. De exemplu, dacă vrem să unim fișierele de anul 1 CTI folosim comanda:

```
mkdir out
pdftk Orar1*.pdf cat output out/Orar1.pdf
```

Acum în fișierul `out/Orar1.pdf` avem toate orarele unificate și e mai ușor de printat. Putem verifica folosind aliasul `go`:

```
go out/Orar1.pdf
```

Exerciții: Creați un script `solution` care generează 6 fișiere PDF pentru orarele pentru fiecare dintre cei 6 ani (4 licență și 2 master).

Wiki Concatenate

Intrați în directorul `wiki-cat/`. Avem mai multe directoare cu fișiere de tip wiki pe care dorim să le concatenăm.

Ca să obținem un fișier de tip wiki pentru laboratorul 1, folosim comanda:

```
cat 01-fs/{concepts,demo,recap,basics,need-to-know,nice-to-know,get-a-life,summary}.wiki > 01-fs.wiki
```

Exerciții: Creați un script `do-all` care generează fișiere de tip wiki pentru toate cele 4 laboratoare din director. Folosiți `for` pentru a parcurge toate 4 directoarele.

Diacritice

Intrați în directorul `diacritics/`. În directorul `in/` avem fișiere care conțin diacritice în format necorespunzător (*cedilla below*); vrem să fie în format corespunzător (*comma below*), așa cum sunt în directorul `expected/`. Lista completă de diacritice este în fișierul `diacritice_ro.txt`.

Dacă doriți să vedeți codul UTF-8 pentru diacritice folosiți comanda:

```
xxd diacritice_ro.txt
```

În scriptul `solution` aveți comanda `sed` pentru a converti diacritica ş (*cedilla below*) în diacritice ș (*comma below*).

Exerciții: Extindeți scriptul `solution` pentru a converti și celelalte trei diacritice: ţ, Ş și Ţ.

Exerciții: Creați un script `do-all` care convertește toate fișierele din directorul `in/` în diacriticele de tip *comma below*. Folosiți `for` pentru a parcurge toate fișierele. Pentru verificare folosiți comanda `cmp` și comparați fișierele obținute cu cele din directorul `expected/`.

08. Exemple de scripturi

Accesați subdirectorul `08-sample-real-scripts/` din directorul laboratorului. Aici sunt exemple de scripturi pe care le folosim în practică. Parcurgeți scripturile și înțelegeți-le. Acolo unde sunt lucruri neclare, apelați la asistent.

09. Argumente pentru scripturi

Accesați subdirectorul `09-args/` din directorul laboratorului.

Orar

Intrați în directorul `timetable/`. Scriptul `download` descarcă orarele din anul universitar curent, semestrul 1. Rulați scriptul pentru a-l testa:

```
./download
```

Va descărca fișierele pentru orar în format `.xls`.

Exerciții: Creați un script numit `solution` care primește două argumente: primul este anul universitar, altul este semestrul și descarcă orarele pentru acel an universitar și pentru acel semestru de pe site-ul de orare: http://acs.pub.ro/~cpop/ [http://acs.pub.ro/~cpop/]

Rularea fără argumente a scriptului `solution` ar trebui să afișeze un mesaj de forma:

```
$ ./solution
Usage: ./solution <academic-year> <semester>
  Example: ./solution 2017_2018 sem1
```

Medie

Intrați în directorul `average/`.

Scriptul `compute` calculează media la a doua notă pentru **toți** studenții din fișierul `../../04-regex/students.csv`:

```
$ ./compute
Average: 6.72
```

Exercițiu: Creați un script numit `solution` care primește ca argument un nume de grupă și calculează media doar pentru aceștia. Un exemplu de rulare este

```
$ ./solution 314CC
Average: 7.56
```

10. Înlocuiri de șiruri

Accesați subdirectorul `10-replace/` din directorul laboratorului.

Fișierul `template_reader.c` conține template-ul unui fișier C. În acest fișier construcția `TEMPLATE` trebuie înlocuită cu un "mesaj secret". Acest mesaj este generat dintr-un șir în care fiecare literă este înlocuită cu valoarea sa ASCII din care scădem valoarea `0x41` (în hexazecimal). Pentru a face înlocuirea folosim scriptul `generate`:

```
./generate
```

În urma rulării scriptului `generate` a fost creat fișierul `reader.c` pornind de la fișierul `template_reader.c`. Acum putem compila fișierul folosind fișierul `Makefile` cu ajutorul comenzii:

```
make
```

În urma rulării comenzii `make` a rezultat executabilul `reader` pe care îl putem acum lansa:

```
./reader
```

Parcurgeți scriptul `generate` și înțelegeți-l. Acolo unde aveți nelămuriri, adresați-vă asistentului.

11. Dezvoltare de exerciții cu scripting

Accesați subdirectorul `11-practical/` din directorul laboratorului.

În fiecare dintre cele 3 subdirectoare aveți niște provocări. Fiecare provocare are un fișier `flag` care conține un mesaj ascuns. Va trebuie să "descifrați" acel mesaj. Ca să înțelegeți cum a fost generat acel mesaj ascuns, vedeți scriptul `encode`. Scriptul encode este scriptul folosit pentru generarea fișierului `flag` pornind de la un mesaj inițial (pe care trebuie să îl aflați).

Exercițiu: Alegeți o provocare (sau mai multe) și creați un script `decode` care decodifică mesajul ascuns în fișierul `flag` și afișează mesajul inițial.

Exercițiu: Gândiți-vă voi la o provocare și creați pentru aceasta scriptul `encode` și `decode` și fișierele auxiliare.

12. Shell vs Python

Accesați subdirectorul `12-shell-vs-python/` din directorul laboratorului. Vrem să vedem cum se compară scripturi scrise în shell față de scripturi scrise în Python.

Scriptul `generate` este un script shell pe care l-am folosit în secțiunea 10. Scriptul `generate.py` are aceeași funcționalitate, dar scris în Python, iar mesajul inițial este citit din fișieru `in`.

În general, pentru acțiuni simple, folosim shell scripting. Pentru prelucrări de date și acțiuni mai complexe, este recomandat să folosim un limbaj de scripting mai puternic, precum Python.

Cuprins

- Laborator 08 (one page)
- Concepte
- Basics
- Exerciții

uso/laboratoare/laborator-08.txt · Last modified: 2021/10/12 19:40 (external edit)

Laborator 09 - Utilizatori și permisiuni

În acest laborator discutăm despre rolul utilizatorilor în sistemul de operare, utilizatorul privilegiat și permisiuni de access.

Accesul privilegiat

În sistemul de operare există acțiuni critice (*critical / sensitive actions*): configurarea rețelei, instalarea de aplicații, adăugarea de noi conturi de utilizatori, configurarea serviciilor sistemului. Aceste acțiuni afectează funcționarea sistemului de operare și pot cauza probleme (de funcționare, de securitate) la o utilizare necorespunzătoare; de exemplu, configurarea greșită a rețelei poate duce la lipsa de conectivitate la Internet; adăugarea unui cont nou poate permite accesul unor persoane nedorite la nivelul sistemului; instalarea unei aplicații poate duce la rularea de malware la nivelul sistemului. De aceea aceste acțiuni pot fi realizate doar într-un **mod privilegiat**.

Modul privilegiat este activat cel mai adesea prin accesarea unui **cont de utilizator privilegiat**. Spunem, așadar, că pe un sistem avem conturi de utilizator obișnuite (*regular users*) și conturi de utilizator privilegiat (*privileged user*). Utilizatorul privilegiat se mai numește **administrator**, **admin** sau **superuser**. În Linux, utilizatorul privilegiat se numește **root**.

Un utilizator obișnuit nu poate realiza acțiuni critice; doar utilizatorul privilegiat poate. De aceea, va trebui să accesăm utilizatorul privilegiat pentru realizarea acțiunilor critice.

Folosirea sudo

Accesarea contului privilegiat se realizează cel mai simplu, în Linux, cu ajutorul comenzii `sudo`. De exemplu, dacă dorim să instalăm o aplicație din contul obișnuit, vom primi eroare:

```
student@uso:~$ apt install nmap
E: Could not open lock file /var/lib/dpkg/lock-frontend - open (13: Permission denied)
E: Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontend), are you root?
```

Această acțiune va reuși, însă, dacă prefixăm comanda cu `sudo`:

```
student@uso:~$ sudo apt install nmap
Reading package lists... Done
Building dependency tree
Reading state information... Done
nmap is already the newest version (7.60-1ubuntu5).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

Contul privilegiat (**root**) accesat prin intermediul comenzii `sudo` are putere deplină asupra sistemului. Poate realiza toate acțiunile posibile (critice sau necritice). De aceea, este riscantă accesarea acestui cont: o comandă rulată greșit poate duce la probleme grave la nivelul sistemului.

O comandă celebră, destinată începătorilor, este `sudo rm -rf /`. În mod amuzant, se spune că această comandă citește e-mailuri foarte rapid (*read mail really fast*). În realitate comanda șterge întregul sistem de fișiere, de la rădăcină (`/`), făcând sistemul nefuncțional și ducând la pierderea datelor.

De aceea, contul privilegiat trebuie accesat cât mai rar, **doar atunci când este nevoie**. În mod implicit, un utilizator se autentifică în sistem cu un cont neprivilegiat (de exemplu `student`), urmând să acceseze contul privilegiat (**root**) doar la nevoie.

Exercițiu: Acțiuni privilegiate

Realizați următoarele acțiuni din contul de utilizator obișnuit `student` și vedeți care are nevoie de acces privilegiat. Explicați de ce unele au nevoie de acces privilegiat.

- `touch /home/student/a.txt`
- `apt install strace`
- `ip address show`
- `cat /etc/passwd`
- `cat /etc/shadow`
- `touch /etc/test.conf`
- `touch /tmp/my.data`
- `service ssh restart`
- `dmesg`
- `dmesg -c`

Erori de permisiuni și alte erori

O confuzie frecventă apare între erorile de permisiuni și erorile de altă natură. De exemplu, atunci când afișăm conținutul unui fișier, putem avea următoarele situații:

```
student@uso:~$ cat /var/log/boot.log
cat: /var/log/boot.log: Permission denied
```

```
student@uso:~$ cat /var/log/kill/them/all.log
cat: /var/log/kill/them/all.log: No such file or directory
```

Prima comandă afișează o eroare de permisiune (*Permission denied*). A doua afișează o eroare de sistem de fișiere: fișierul care se dorește afișat nu există (*No such file or directory*).

Doar erorile de permisiune sunt "rezolvabile" folosind contul privilegiat, care are acces complet în sistem. Celelalte tipuri de erori nu sunt rezolvate astfel, trebuie găsită sursa problemei. Astfel că, dacă prefixăm cele două comenzi de mai sus cu `sudo`, pentru folosirea contului privilegiat, vom obține:

```
student@uso:~$ sudo cat /var/log/boot.log
[  OK  ] Started Show Plymouth Boot Screen.
[  OK  ] Started Forward Password Requests to Plymouth Directory Watch.
[  OK  ] Reached target Local Encrypted Volumes.
[...]
student@uso:~$ sudo cat /var/log/kill/them/all.log
cat: /var/log/kill/them/all.log: No such file or directory
```

Observăm că prima comandă, care afișase eroare de permisiune, funcționează acum. În vreme ce a doua comandă, care afișase eroare de sistem de fișiere, nu își schimbă comportamentul: afișează în continuare aceeași eroare.

Exercițiu: Erori de permisiune și erori de alte tipuri

Rulați următoarele comenzi fără `sudo`. Aceste comenzi vor afișa mesaje de eroare. Fără a folosi `sudo`, stabiliți care comenzi generează erori de permisiuni (rezolvabile cu `sudo`) și alte tipuri de erori (nerezolvabile folosind `sudo`). Apoi rulați folosind `sudo` pentru a confirma presupunerile.

- `apt install sports-illustrated`
- `ip address show ether114`
- `kill $(pidof chainsaw)`
- `netcat -l -p 700`
- `chown student:student /home/student`
- `chown student:student /home/search/and/destroy`
- `userdel hariseldon`

În cazul comenzilor care afișează alte tipuri de erori, presupunând rezolvarea lor, este posibil, în continuare să fie nevoie de `sudo`. Care dintre comenzile de mai sus intră în această categorie?

Redirectare folosind sudo

Fie o situație în care dorim să scriem informație într-un fișier. Să presupunem că dorim să scriem informația *All for the empire* în fișierul `/home/student/zealot.txt`. Avem două opțiuni:

1. Editâm fișierul folosind `nano /home/student/zealot.txt`, scriem informația *All for the empire** și apoi salvăm.
2. Folosim comanda `echo "All for the empire" > /home/student/zealot.txt`.

Ambele opțiuni vor funcționa, utilizatorul curent `student` (neprivilegiat) având permisiuni de editare / creare.

Dacă alegem un fișier la care utilizatorul curent nu are acces, intuiția ar fi să prefixăm comenzile de mai sus `sudo`. Adică, în cazul fișierului `/etc/zealot.txt`, am realiza acțiunile:

1. Editâm fișierul folosind `sudo nano /etc/zealot.txt`, scriem informația *All for the empire* și apoi salvăm.
2. Folosim comanda `sudo echo "All for the empire" > /etc/zealot.txt`.

Prima comandă (folosirea editorului) funcționează. A doua comandă însă nu funcționează.

A doua comandă nu funcționează pentru că doar comanda `echo` rulează în modul privilegiat. Redirectarea (dată de operatorul >) este realizată de proces shell, care aparține utilizatorului neprivilegiat `student`. Adică se deschide, cu ajutorul redirectării, fișierul `/etc/zealot.txt` cu permisiunile utilizatorului `student`; pentru că fișierul este deschis de un utilizator neprivilegiat, acțiunea eșuează.

Sunt două soluții:

1. Folosirea unui proces shell privilegiat.
2. Folosirea unei comenzi prefixată de comanda `sudo`, comandă care să deschidă fișierul `/etc/zealot.txt` fără redirectare. În felul acesta fișierul va fi deschis în modul privilegiat.

Pentru prima variantă, folosim comenzile:

```
student@uso:~$ sudo su
root@uso:/home/student# echo "All for the empire" > /etc/zealot.txt
```

În această situație, a doua comandă este rulată cu permisiunile contului privilegiat (`root`).

Pentru a doua variantă, putem folosi comanda `tee` care primește ca argument un fișier în care scrie informație. Prefixarea cu `sudo` va duce la rularea comenzii `tee` în mod privilegiat, adică și la deschiderea fișierului `/etc/zealot.txt` în mod privilegiat. Comanda folosită este:

```
student@uso:~$ echo "All for the empire" | sudo tee /etc/zealot.txt
All for the empire
```

Comanda `tee` are ca efect scrierea informației în fișierul primit ca argument **și** la ieșirea standard. Din acest motiv mesajul `All for the empire` apare **și** în fișierul `/etc/zealot.txt` **și** la ieșirea standard.

Comanda `tee` în modul append

În forma implicită de mai sus, comanda `tee` suprascrie conținutul fișierului primit ca argument. Dacă dorim să adăugăm conținut în fișier (*append*), folosim opțiunea `-a` a comenzii `tee`:

```
student@uso:~$ echo "All for the empire" | sudo tee -a /etc/zealot.txt
```

Shell privilegiat

Adesea dorim realizarea mai multor acțiuni critice, în forma rulării mai multor comenzi. Pentru aceasta, cel mai simplu este să folosim un proces shell care rulează în mod privilegiat.

Așa cum am văzut mai sus, folosim comanda `sudo su` pentru obținerea unui shell privilegiat:

```
student@uso:~$ sudo su
root@uso:/home/student#
```

Promptul ne indică prezența shellului privilegiat:

- utilizatorul este `root`, utilizatorul privilegiat
- promptul se încheie în caracterul # (*diez*), caracter care denotă, prin convenție, un shell privilegiat

Același efect poate fi obținut prin rularea comenzii `sudo su root`:

```
student@uso:~$ sudo su root
root@uso:/home/student#
```

Fiind mai mult de tastat, preferăm folosirea comenzii `sudo su` în loc de `sudo su root`.

Un alt mod de a obține un shell de root este cu ajutorul comenzii `sudo bash`, cu aceleași efecte ca mai sus:

```
student@uso:~$ sudo bash
root@uso:/home/student#
```

Ambele comenzi (`sudo su`, respectiv `sudo bash`), duc la rularea unor comenzi în mod privilegiat: comanda `su` este rulată privilegiat, comanda `bash` este rulată privilegiat. Efectul este crearea unui proces shell Bash care rulează privilegiat; și, deci, un shell în cadrul căruia putem rula comenzi în mod privilegiat.

Pentru a închide procesul shell care rulează în mod privilegiat folosim comenzile uzuale de închidere a shellului: `exit`, `logout` sau combinația de taste `Ctrl+d`. După încheierea procesului shell privilegiat, vom reveni în procesul shell inițial.

Vom prezenta mai multe despre procese și utilizatori în secțiunea users_perms_processes.

Vom prezenta detalii despre comanda `su` în secțiunea users_perms_privileged_su.

Exerciții: Shell privilegiat

1. Folosiți, pe rând, comenzile `sudo su` și `sudo bash` pentru a obține un shell privilegiat. Apoi folosiți comanda `exit` sau comanda `logout` sau combinația de taste `Ctrl+d` pentru a închide shellul curent.
2. Folosiți comanda `sudo su` pentru a obține un shell privilegiat. În noul shell privilegiat folosiți comanda `sudo su` pentru a obține un nou shell privilegiat. Folosiți două instanțe de comenzi de închidere a shellului (`exit` sau `logout` sau `Ctrl+d`) pentru a reveni la shellul inițial (neprivilegiat).

Identificarea utilizatorului

Promptul shellului este configurabil și poate să nu ofere informații despre utilizatorul curent. Pentru siguranță, folosim comenzi dedicate pentru identificarea utilizatorului curent.

whoami

Cea mai directă comandă, pentru afișarea numelui utilizatorului curent (*username*) este comanda `whoami`:

```
student@uso:~$ whoami
student
student@uso:~$ sudo whoami
root
```

Rularea comenzii `whoami` duce la afișarea utilizatorului curent (`student`). Prefixarea comenzii cu `sudo` duce la afișarea numelui utilizatorului privilegiat (`root`).

id

O comandă care permite afișarea de informații despre utilizatori este comanda `id`. O utilizare simplă a comenzii afișează informații extinse despre utilizatorul curent:

```
student@uso:~$ id
uid=1000(student) gid=1000(razvan) groups=1000(student),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),116(lpadmin),126(sambashare),128(kvm),130(docker)
```

Pentru utilizatorul curent se afișează:

- identificatorul utilizatorului (*user id*, `uid`), o valoare numerică, aici `1000`
- numele utilizatorului (*username*), șirul pe care îl folosim, aici `student`
- identificatorul grupului utilizatorului (*group id*, `gid`) și numele grupului (*group name*), aici `1000` și `student` (poate fi considerată o coincidență egalitatea cu identificatorul utilizatorului și numele utilizatorului)
- alte grupuri din care face parte utilizatorul (*groups*)

Relevante în acest moment sunt informațiile legate de identificatorul și de numele utilizatorului. Identificatorul utilizatorului (**UID**) este folosit de sistemul de operare pentru verificarea permisiunilor. Numele utilizatorului este folosit de noi, oamenii, care reținem mai ușor șiruri / nume în loc de numere. Similar se întâmplă și pentru procese: sistemul de operare le identifică după un număr numit **PID** (*process identifier*).

Orice utilizator are un identificator (pentru sistemul de operare) și un nume (pentru oameni). De obicei, primul utilizator obișnuit (neprivilegiat) de pe un sistem Linux are identificatorul `1000`; este cazul utilizatorului `student` de mai sus.

Putem folosi comanda `id` pentru a afișa informații și despre alți utilizatori ai sistemului, de exemplu despre utilizatorul privilegiat (`root`):

```
razvan@uso:~$ id root
uid=0(root) gid=0(root) groups=0(root)

razvan@uso:~$ id -u root
0

razvan@uso:~$ id -un 0
root
```

În prima comandă rulată am afișat informații despre utilizatorul `root`. Utilizatorul are UID-ul `0`; acesta este modul în care este recunoscut de sistemul de operare. Practic, numele `root` este o convenție; ceea ce oferă privilegii unui proces este prezența UID-ului `0` ca atribut al procesului.

Putem afișa doar UID-ul unui utilizator prin folosirea opțiunii `-u` a comenzii `id`, ca în a doua comandă de mai sus. La fel putem afișa doar numele corespunzător unui UID prin folosirea opțiunii `-un`, ca în a treia comanda de mai sus.

Exerciții: Identificarea utilizatorilor

1. Obțineți un shell privilegiat. Folosiți comenzile `whoami` și `id` pentru a afișa informații despre utilizatorul curent (`root`).

   Folosiți comanda `id` pentru a afișa identificatorul utilizatorului `student`.

2. Dintr-un shell de orice fel (privilegiat sau neprivilegiat), afișați:

   - UID-ul utilizatorului `daemon`.
   - UID-ul utilizatorului `syslog`.
   - Numele utilizatorului cu UID-ul `100`.
   - Numele utilizatorului cu UID-ul `2`.

   Acești utilizatori sunt utilizatori de sistem (*system users*), neinteractivi - adică nu sunt utilizatori cu care ne putem autentifica în sistem.

Schimbarea utilizatorului

Mai sus am folosit comanda `sudo su` pentru a obține un proces shell privilegiat. Comanda duce, de fapt, la rularea privilegiată a comenzii `su`.

Comanda `su` este folosită pentru schimbarea utilizatorului. În forma sa simplă (fără argumente) schimbă utilizatorul în utilizatorul privilegiat (`root`). Schimbarea unui utilizator va cere parola utilizatorului schimbat. Astfel, în cazul comenzii de mai jos, se va cere parola contului `root`:

```
student@uso:~$ su
Password:
su: Authentication failure
```

Întrucât nu știm parola contului `root`, am apăsat `Enter` și a fost afișat mesaj de eroare de autentificare.

Comanda `su` poate fi folosită pentru a schimba utilizatorul în cel al cărui nume a fost primit ca argument. De exemplu, comanda de mai jos ar avea ca efect schimbarea utilizatorului curent în utilizatorul `daemon`. Din nou, necunoscând parola, nu vom reuși autentificarea:

```
student@uso:~$ su daemon
Password:
su: Authentication failure
```

Atunci când prefixăm comanda `su` de comanda `sudo` se rulează `su` în mod privilegiat și nu mai este necesară parola contului `root`:

```
student@uso:~$ sudo su
root@uso:/home/student#
```

O opțiune frecventă a comenzii `su` este `-` (semnul minus sau cratimă). Această opțiune duce la crearea unui mediu de lucru specific unui utilizator care se autentifică (*login*). Cel mai vizibil efect este schimbarea directorului curent în directorul home al utilizatorului către care dorim schimbarea. De aceea, în comanda de mai jos, noul director este directorul home al utilizatorului privilegiat (`root`), adică `/root/`:

```
razvan@uso:~$ sudo su -
root@uso:~# pwd
/root
```

sudo, su și parole

Se poate întâmpla ca în cazul folosirii `sudo` să fie solicitată o parolă. Este vorba de parola utilizatorului curent, cel care rulează comanda `sudo`. Pentru siguranță, este nevoie de confirmarea parolei utilizatorului pentru a realiza acțiunea presupus critică prefixată de comanda `sudo`. Solicitarea sau nu a parolei utilizatorului curent la rularea comenzii `sudo`, ține de configurarea acesteia.

Sumarizând:

- Rularea comenzii `su` duce la solicitarea parolei contului de utilizator către care dorim schimbarea.
- Rularea comenzii `sudo` (urmată de altă comandă) **poate duce** (depinde de configurație) la solicitarea parolei contului utilizatorului curent.

Exerciții: Folosirea su

1. Obțineți un shell privilegiat. În shellul privilegiat, folosiți comanda `su` pentru a schimba utilizatorul în `student`. Închideți shellul utilizatorului `student`. Apoi închideți și shellul privilegiat. Ați revenit la punctul inițial.

2. Obțineți un shell privilegiat. În shellul privilegiat, folosiți comanda `su` pentru a schimba utilizatorul în `student` inclusiv mediul de login: directorul să fie `/home/student/`. Închideți shellul utilizatorului `student`. Apoi închideți și shellul privilegiat. Ați revenit la punctul inițial.

3. Obțineți un shell privilegiat. În shellul privilegiat, folosiți comanda `su` pentru a schimba utilizatorul în `daemon`. Primiți eroare.

   Încercați să schimbați utilizatorul în `bin`. La fel, primiți eroare.

   Aceste erori să întâmplă întrucât utilizatorii `daemon` și `bin` sunt utilizatori de sistem (*system users*) care nu sunt gândiți să fie utilizatori interactivi: adică se ne putem autentifica în sistem ca acei utilizatori și să rulăm procese shell (interactive).

Utilizatori și procese

Acțiunile unui cont de utilizator în cadrul sistemului de operare sunt realizate de procesele deținute de acel utilizator. Un cont de utilizator nu există ca entitate de sine stătătoare, ci este un atribut al unui proces.

Așa cum am indicat și anterior, un cont utilizator este identificat de un număr (**UID**); numele este folosit doar de oameni pentru identificarea facilă; sistemul de operare este preocupat doar de număr / UID. Un proces aparține unui cont utilizator dacă acel proces are atributul UID corespunzător acelui cont de utilizator. Ceea ce înseamnă că, în momentul creării, un proces primește un UID. Spunem, informal, că acel utilizator deține acel proces.

Un caz particular sunt procesele shell, procese interactive. Spunem, informa, că un utilizator poate rula comenzi sau că are acces în sistem, atunci când există un shell cu UID-ul acelui cont de utilizator. Cel mai adesea, shellul este primul proces care aparține unui utilizator. Apoi shellul creează alte procese aparținând acelui utilizator. Shellul este cel mai adesea creat la autentificarea utilizatorului.

Autentificarea utilizatorilor

Autentificarea unui utilizator (*login*, *authentication*) în sistem presupune următoarea secvență de pași:

1. Furnizarea unor elemente de autentificare (numite și credențiale - *login credentials*), în mod tipic un **nume de utilizator** (*username*) și o **parolă** (*password*)

2. Verificarea credențialelor de sistemul de operare
3. În cazul verificării cu succes, crearea unui proces cu UID-ul utilizatorului astfel autentific, cel mai adesea un proces shell

Furnizarea credențialelor

Primul pas se realizează într-o interfața de autentificare (grafică sau în linia de comandă). Se furnizează un prompt unde utilizatorul transmite credențialele.

Un mod de a simula acest prompt în linia de comandă este cu ajutorul comenzii `login`, comandă care este apelată în momentul autentificării CLI:

```
student@uso:~$ sudo login
uso login: student
Password:
student@uso:~$
```

Avem creat astfel, un nou proces shell. E o abordare strict didactică, având acum un shell creat dintr-un shell existent, pentru același utilizator.

Comanda `login` poate fi folosită și transmițând ca argument numele de utilizator:

```
student@uso:~$ sudo login student
Password:
student@uso:~$
```

Pentru a închide shellul nou creat folosim comanda `exit` sau comanda `logout` sau combinația de taste `Ctrld+d`.

Este nevoie de folosirea `sudo` în fața comenzii `login` pentru că această comandă este privilegiată: are acces la baza de date autentificare, folosită în pasul de verificare a credențialelor.

Verificarea credențialelor

Verificarea credențialelor este realizată de interfața de autentificare, în general printr-un proces de autentificare, precum `login`. Acest proces citește credențialele și apoi le validează cu o bază de date de autentificare (*authentication database*) disponibilă în sistem. Această baza de date de autentificare este o resursă critică a sistemului; accesul la această bază de date, chiar și de citire, este considerat o acțiune critică ce poate fi realizată doar privilegiat. De aceea, comanda `login` trebuie rulată în mod privilegiat și trebuie prefixată de `sudo`.

Baza de date de autentificare este o resursă critică pentru că reține credențialele tuturor utilizatorilor disponibili în sistem. Faza de verificare presupune compararea credențialelor furnizate cu cele din baza de date de autentificare. Dacă aceasta nu s-ar realiza privilegiat, orice utilizator ar putea avea acces la credențialele celorlalți utilizatori (posibil inclusiv `root`); ceea ce ar fi o problemă de securitate în sistem.

În Linux, baza de date de autentificare este în mod tipic fișierul `/etc/shadow`:

```
razvan@uso:~$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1334 Jul  2  2020 /etc/shadow
```

Observăm că acest fișier poate fi accesat doar de utilizatorul `root` și de grupul `shadow`. Vom discuta despre permisiuni de acces și grupuri în secțiunile următoare.

Conturi de utilizator fără parolă

Anumite conturi de utilizator pot să nu aibă parolă. Ceea ce înseamnă că acele conturi nu pot fi accesate în acest mod: autentificare cu parolă. Pot fi însă autentificate în alte moduri, cum ar fi schimbarea utilizatorului.

Crearea unui proces

Ulterior autentificării, se creează un prim proces care are UID-ul utilizatorului autentificat. Acest prim proces este configurabil, cel mai adesea este shellul (grafic sau CLI - depinzând de modul de autentificare).

În cazul autentificării în linia de comandă, procesul creat este descris în fișierul `/etc/passwd`:

```
student@uso:~$ grep student /etc/passwd
student:x:1000:1000:Student User,,,:/home/student:/bin/bash
```

Acest fișier conține informații despre un utilizator: numele de utilizator, UID-ul său, directorul home și primul proces pornit în cazul autentificării (adesea shellul). În cazul utilizatorul `student`, conform așteptărilor, primul proces este creat din executabilul `/bin/bash`, corespunzător shellului Bash.

După autentificare există un proces în sistem care aparține unui cont de utilizator. Dacă acest proces este shellul, spunem adesea că, un utilizator are acces în sistem.

Anumite conturi de utilizator (`daemon`, `bin` etc.) au configurat în fișierul `/etc/passwd` ca executabil pentru pornirea primului proces `/bin/false` sau `/usr/sbin/nologin`. Acest lucru înseamnă că aceste conturi de utilizator nu pot fi accesate prin autentificare. Ci doar prin crearea, în moduri specifice, a unor procese cu UID-ul acelor utilizatori. Nu intrăm în detalii aici despre aceste moduri.

Vizualizarea și gestiunea informațiilor despre utilizatori

În Linux, fișierul `/etc/passwd` reține informațiile despre conturile de utilizatori. Fișierul `/etc/shadow` reține credențialele (parolele) despre conturile de utilizatori.

Fișierul `/etc/passwd` nu este critic și poate fi citit de oricine. Fișierul `/etc/shadow` este critic și poate fi citit doar de utilizatorul privilegiat. În continuare, fișierul `/etc/passwd` poate fi modificat doar de utilizatorul privilegiat: acțiunea de citire este necritică, dar cea de scriere este critică.

În mod obișnuit, principiile de securitate într-un sistem de operare, relativ la accesarea informațiilor despre conturile de utilizatori, sunt:

- Un utilizator neprivilegiat (obișnuit) poate să vizualizeze informațiile proprii, mai puțin parola.
- Un utilizator neprivilegiat (obișnuit) poate să modifice informațiile proprii, inclusiv parola.
- Un utilizator neprivilegiat (obișnuit) poate să vizualizeze informațiile altor utilizatori, mai puțin parola.
- Un utilizator neprivilegiat (obișnuit) **NU** poate să modifice informațiile altor utilizatori.
- Un utilizator (de orice tip) **NU** poate vizualiza parolele *plain-text* ale utilizatorilor. Parolele sunt ținute criptat.
- Un utilizator privilegiat poate să modifice orice informație a altor utilizatori, inclusiv parola.

Vizualizare

Așa cum am precizat anterior, un utilizator poate folosi comenzile `whoami` și `id` pentru a afișa informații despre sine sau alți utilizatori. Un alt mod este de a parcurge direct conținutul fișierului `/etc/passwd`, fișier care este consultat și de aceste comenzi. Un mod mai simplu este folosirea comenzii `finger` care afișează informații detaliate:

```
student@uso:~$ finger student
Login: student                  Name: Student User
Directory: /home/student                Shell: /bin/bash
```

Gestiune

Un utilizator își poate modifica 3 informații:

- Shellul de login, folosind comanda `chsh`
- Informațiile legate de nume, folosind comanda `chfn`
- Parola, folosind comanda `passwd`

Nu insistăm pe primele două, doar pe modificarea parolei. Un utilizator își poate schimba parola prin simpla invocare a comenzii `passwd`:

```
student@uso:~$ passwd
Changing password for student.
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

Acum utilizator are o nouă parolă.

Evident, utilizatorul privilegiat poate schimba parolele tuturor utilizatorilor.

Exerciții: Gestiunea utilizatorului curent

1. Din contul utilizatorului curent (`student`), folosiți comanda `passwd` pentru a schimba parola utilizatorului. **GRIJĂ MARE** să nu uitați parola schimbată. Folosiți într-un nou shell comanda `sudo login student` pentru a testa noua parolă.
2. Dintr-un shell de root folosiți comanda `passwd student` pentru a schimba parola utilizatorului student **GRIJĂ MARE** să nu uitați parola schimbată. Folosiți într-un nou shell comanda `sudo login student` pentru a testa noua parolă.

Vizualizarea proceselor

Putem vizualiza evoluția proceselor (privilegiate, neprivilegiate) folosind comanda `ps` pentru a înțelege funcționarea comenzilor și apartenența acestora la un anumit utilizator.

Comanda simplă `ps` afișează procesele din terminalul curent, în vreme ce comanda `ps -f` afișează inclusiv numele de utilizator:

```
student@uso:~$ ps
  PID TTY          TIME CMD
18900 pts/2    00:00:00 ps
28892 pts/2    00:00:00 bash
student@uso:~$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
student   18907 28892  0 22:38 pts/2    00:00:00 ps -f
student   28892 20845  0 14:02 pts/2    00:00:00 /bin/bash
```

Se afișează procesul shell (`bash`) și procesul `ps` însuși, ambele aparținând utilizatorului `student`.

Putem personaliza afișarea pentru a prezenta și UID-ul:

```
student@uso:~$ ps -o uid,user,pid,ppid,cmd
  UID USER       PID  PPID CMD
 1000 student   19083 28892 ps -o uid,user,pid,ppid,cmd
 1000 student   28892 20845 /bin/bash
```

Folosirea comenzii `ps` ajută la înțelegerea comenzilor `sudo` sau `su`.

Atunci când creăm un shell privilegiat folosind `sudo bash`, avem următoarea situație:

```
student@uso:~$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
student   19153 28892  0 22:46 pts/2    00:00:00 ps -f
student   28892 20845  0 14:02 pts/2    00:00:00 /bin/bash

student@uso:~$ sudo bash

root@uso:~# ps -H
  PID TTY          TIME CMD
19154 pts/2    00:00:00 sudo
19155 pts/2    00:00:00   bash
19163 pts/2    00:00:00     ps

root@uso:~# ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
root     19154 28892  0 22:46 pts/2    00:00:00 sudo bash
root     19155 19154  0 22:46 pts/2    00:00:00 bash
root     19173 19155  0 22:46 pts/2    00:00:00 ps -f

root@uso:~# pstree -Aup 28892
bash(28892,student)---sudo(19154,root)---bash(19155)---pstree(19198)

root@uso:~# exit
exit

student@uso:~$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
student   19205 28892  0 22:47 pts/2    00:00:00 ps -f
student   28892 20845  0 14:02 pts/2    00:00:00 /bin/bash
```

Observăm că există un proces `sudo` și un proces `bash` care aparțin utilizatorului `root`. Aceste procese sunt create în urma comenzii `sudo bash`. Procesul părinte (`PPID`) pentru procesul `sudo` este `28892` adică shellul inițial al utilizatorului `student`, cel care a rulat comanda `sudo bash`. Practic dintr-un shell al utilizatorului `student`, folosind `sudo` se generează un shell al utilizatorului `root`. Comanda `pstree` afișează ierarhia de procese și schimbările de conturi de utilizator.

O situație similară avem în cazul `sudo su`:

```
student@uso:~$ sudo su

root@uso:/home/student# ps -H
  PID TTY          TIME CMD
19432 pts/2    00:00:00 sudo
19433 pts/2    00:00:00   su
19434 pts/2    00:00:00     bash
19445 pts/2    00:00:00       ps

root@uso:/home/student# ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
root     19432 28892  0 22:54 pts/2    00:00:00 sudo su
root     19433 19432  0 22:54 pts/2    00:00:00 su
root     19434 19433  0 22:54 pts/2    00:00:00 bash
root     19455 19434  0 22:54 pts/2    00:00:00 ps -f

root@uso:/home/student# pstree -Aup 28892
bash(28892,student)---sudo(19432,root)---su(19433)---bash(19434)---pstree(19460)

root@uso:/home/student# exit

student@uso:~$
```

Exerciții: Vizualizarea proceselor

Realizați un scenariu în care accesați contul `root` apoi contul `student` și apoi iar contul de `root`. Afișați procesele create la fiecare pas. Folosiți comanda `pstree` ca să afișați arborescența de procese astfel create.

Apoi închideți procesele astfel create (folosind `exit`, `logout` sau `Ctrl+d`). Afișați, la fiecare pas, ierarhia de procese.

Permisiuni de acces

Atât utilizatorii cât și grupurile de utilizatori sunt folosiți în Linux pentru controlul accesului. Controlul accesului presupune restricționarea operațiilor pe care utilizatorii le pot executa asupra resurselor sistemului (crearea de fișiere și directoare, modificarea și ștergerea acestora, execuția de programe, etc.).

Astfel, niciun utilizator nu poate să modifice/folosească un fișier dacă el, grupul din care face parte sau grupul `others` nu au acest drept (vom detalia imediat).

În Linux, exista două concepte ce sunt folosite pentru a oferi securitate la nivelul sistemului de fișiere:

- file ownership
- permisiuni

Fiecare fișier este deținut de un singur utilizator și de un singur grup, și are anumite permisiuni. Fiecare utilizator sau grup poate să dețină mai multe fișiere.

Fiecare resursă din sistemul de fișiere are permisiuni pentru trei categorii de utilizatori:

- utilizator (owner): utilizatorul care este deținătorul fișierului/directorului
- grup (group): grupul de utilizatori care deține fișierul/directorul
- ceilalți (others): utilizatorii care nu intră în niciuna dintre categoriile anterioare

Pentru fiecare dintre cele trei categorii există trei permisiuni, care oferă următoarele drepturi:

- de citire: utilizatorul/grupul poate citi conținutul fișierului, respectiv poate lista conținutul directorului
- de scriere: utilizatorul/grupul poate modifica/șterge conținutul fișierului, respectiv poate crea/șterge/redenumi fișiere din director
- de execuție: utilizatorul/grupul poate lansa fișierul în execuție, respectiv poate parcurge directorul

Ordinea în care se aplică aceste permisiuni este următoarea:

1. dacă utilizatorul este deținătorul fișierului, atunci se aplică permisiunile pentru owner
2. dacă face parte din grupul deținător, atunci se aplică permisiunile pentru grup
3. dacă niciuna dintre condițiile precedente nu este adevărată, atunci se aplică permisiunile pentru others

Ownership și permisiuni

Pentru a inspecta permisiunile și deținătorii unui anumit director sau fișier folosim comanda `ls`, împreună cu argumentul `-l`.

```
student@uso:~$ ls -l /home/student
total 56
drwxr-xr-x  2 student student 4096 aug  6 17:41 Desktop
drwxr-xr-x  3 student student 4096 aug 20 21:00 Documents
drwxr-xr-x  2 student student 4096 aug  6 17:41 Downloads
drwxr-xr-x  2 student student 4096 aug  6 17:41 Music
drwxr-xr-x  2 student student 4096 aug  6 17:41 Pictures
drwxr-xr-x  2 student student 4096 aug  6 17:41 Public
drwxr-xr-x  2 student student 4096 aug  6 17:41 Templates
drwxr-xr-x  2 student student 4096 aug  6 17:41 Videos
-rw-r--r--  1 student student 8980 aug  6 17:37 examples.desktop
drwxr-xr-x 14 student student 4096 aug 20 20:57 uso.git
-rw-r--r--  1 student student 4827 aug 21 14:37 vm-actions-log.txt
```

Dacă rulăm comanda pe un director, implicit se vor lista fișierele din director și nu vom vedea permisiunile acestuia. Pentru a vedea permisiunile directorului, putem folosi argumentul `-d` pentru comanda `ls`, împreună cu calea către director.

```
student@uso:~$ ls -ld uso-lab/
drwxr-xr-x 14 student student 4096 aug 20 20:57 uso-lab/
```

Semnificația fiecărui câmp din outuput-ul anterior este prezentată în tabelul de mai jos, folosind ca exemplu informațiile afișate pentru directorul `Desktop`.

Tip	Permisiuni	Număr de referințe	Owner	Grup	Dimensiunea în octeți	Data și ora ultimei modificări	Nume
d	rwxr-xr-x	2	student	student	4096	aug 6 17:41	Desktop

Coloanele din tabel au următoarea semnificație:

- Tip: indică dacă intrarea este un fișier (`-`) sau director (`d`)
- Permisiuni: specifică ce drepturi au asupra fișierului cele trei categorii de utilizatori (owner, grup și others). Fiecărui grup îi corespund 3 caractere (r – read; w – write, x – execute). În acest caz, owner-ul fișierului (utilizatorul `student`) are toate cele trei permisiuni (`rwx`), iar utilizatorii din grupul ce deține fișierul (grupul `student`) și toți ceilalți utilizatori din sistem (`others`) au permisiuni de citire și execuție (`r-x`).
- Owner/Grup - utilizatorul/grupul care dețin fișierul

Permisiunile se pot specifica și folosind cifre (notație în baza 8 – *octal*):

- `r` = 4
- `w` = 2
- `x` = 1

Pentru a determina permisiunile unei anumite categorii, adunăm cele 3 valori. Astfel, pentru exemplul de mai sus, permisiunile sunt `755` (7 pentru owner, 5 pentru grup și 5 pentru others).

Modificarea permisiunilor

La crearea unui fișier sau director se atribuie valori implicite pentru permisiuni. Acestea se pot modifica ulterior.

Pentru a demonstra modul de lucru cu permisiunile, vom crea un fișier nou, după care îi vom modifica permisiunile. Permisiunile noi se pot specifica folosind comanda `chmod`, folosind oricare din cele doua notații:

1. În octal: 777 (drepturi depline: rwx), 740 (drepturi depline pentru owner, de citire pentru utilizatorii din grup și nicio permisiune pentru alți utilizatori).
2. Alfabetică: +/-[rwx] – o folosim de obicei atunci când dorim să modificăm o singură permisiune. De exemplu, `+r` permite accesul de citire, iar `-w` șterge accesul de scriere.

```
student@uso:~$ touch testfile.sh
student@uso:~$ ls -l testfile.sh
-rw-r--r-- 1 student student 0 sep 16 16:47 testfile.sh
```

După cum observăm, nu vom putea executa scriptul nou creat, deoarece utilizatorul student nu are drepturi de execuție asupra lui. Ne propunem să permitem execuția, folosind ambele metode.

#1

```
student@uso:~$ chmod +x testfile.sh
student@uso:~$ ls -l testfile.sh
-rwxr-xr-x 1 student student 0 sep 16 16:47 testfile.sh

student@uso:~$ chmod -x testfile.sh
student@uso:~$ ls -l testfile.sh
-rw-r--r-- 1 student student 0 sep 16 16:47 testfile.sh
```

#2

```
student@uso:~$ ls -l testfile.sh
-rw-r--r-- 1 student student 0 sep 16 16:47 testfile.sh
student@uso:~$ chmod 755 testfile.sh
student@uso:~$ ls -l testfile.sh
-rwxr-xr-x 1 student student 0 sep 16 16:47 testfile.sh
```

Exercițiu: Adăugați utilizatorul `andrei` folosind comanda `sudo adduser andrei`. Folosiți o parolă la alegere și țineți-o minte.

Exercițiu: Adăugați utilizatorul `maria` în sistem folosind comanda `sudo adduser maria`. Folosiți o parolă la alegere și țineți-o minte. Autentificați-vă ca `maria` folosind `su` sau `login`.

Exercițiu: Din contul utilizatorului `maria`, creați fișierul `/home/maria/todo_list` și scrieți în el `Maria's TODO list`.

Exercițiu: Configurați permisiunile acestui fișier astfel încât doar utilizatorul `maria` să aibă drepturi de citire, scriere și execuție pe el. Verificați că ați configurat permisiunile corect, folosind contul utilizatorului `andrei`.

Exercițiu: Modificați configurările făcute anterior astfel încât toți utilizatorii din grupul `maria` să poată vizualiza conținutul fișierului `/home/maria/todo_list`. În continuare, utilizatorii al căror nume nu este `maria` și care nu se află în grupul `maria` nu trebuie să aibă niciun drept pe acest fișier.

Exercițiu: Verificați configurarea făcută la exercițiul anterior. Pentru aceasta, adăugați utilizatorul `student` în grupul `maria` folosind comanda `sudo adduser student maria`, autentificați-vă ca utilizatorul `student` și încercați să vizualizați conținutul fișierului. Autentificați-vă apoi și ca utilizatorul `andrei` și încercați să vizualizați conținutul fișierului. `andrei` nu ar trebui să aibă drept să citească fișierul `/home/maria/todo_list`.

Cuprins

- Laborator 09 (one page)
- Accesul privilegiat
- Utilizatori și procese
- Permisiuni de acces

uso/laboratoare/laborator-09.txt · Last modified: 2021/12/06 23:20 by razvan.deaconescu

Laborator 10 - Investigarea securității sistemului

Concepte

Prezentarea și securizarea datelor (encodare, criptare, hashing)

Encodare

Encodarea este folosită când dorim să prezentăm o serie de date sub o altă formă. De exemplu, când există caractere neprintabile în datele noastre, nu le putem afișa sub forma unui string. **Base64** este un cunoscut exemplu de algoritm de encodare pentru că transformă orice vector de octeți într-un string de caractere printabile.

Este important de precizat faptul că encodarea **nu ascunde/securizează** informații, ci doar le prezintă sub o altă formă.

Folosind comanda **base64** fără niciun parametru putem să encodăm un text.

```
student@uso:~$ echo -n "Base64 is not encryption!" | base64
QmFzZTY0IGlzIG5vdCBlbmNyeXB0aW9uIQ==
```

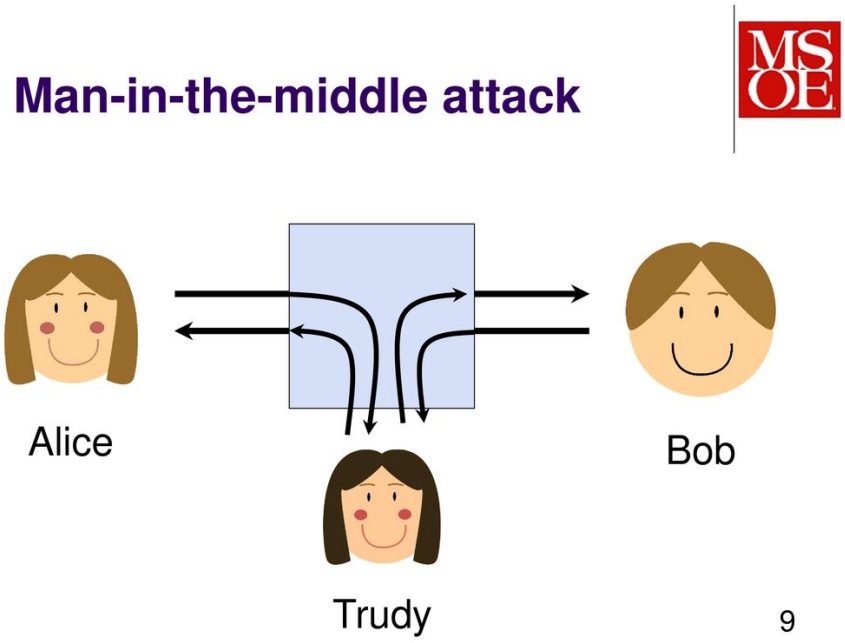
Utilizând parametrul **-d** putem decoda un text encodat.

```
student@uso:~$ echo -n "QmFzZTY0IGlzIG5vdCBlbmNyeXB0aW9uIQ==" | base64 -d
Base64 is not encryption!
```



Criptarea

Să presupunem că Alice și Bob doresc să comunice pe Internet. Pentru început, cei doi folosesc un canal simplu prin care transmit mesajele în clar. Problema care apare este că Trudy, care este rău intenționat, poate asculta ce vorbesc Alice și Bob și va și înțelege mesajele deoarece ele sunt transmise în clar.



Criptarea înseamnă ascunderea datelor folosind un algoritm public și o cheie secretă. În cazul nostru, Alice **criptează** mesajul pe care dorește să îl transmită, iar Bob îl **decriptează**. Alice și Bob au aceeași cheie secretă negociată într-un moment de timp anterior comunicării. **Trudy** vede în continuare mesajele, știe algoritmul de criptare (deoarece e public), dar nu cunoaște cheia de decriptare, deci nu poate vedea conținutul lor. Un exemplu de algoritm de criptare este **AES**.

Criptarea presupune ascunderea/securizarea datelor într-o manieră recuperabilă. Adică ele vor putea fi ulterior **decriptate**.

Folosind utilitarul **openssl** putem să criptăm conținutul unui fișier.

```
student@uso:~$ echo -n "plaintext content" > plaintext_file.txt
student@uso:~$ openssl aes-256-cbc -in plaintext_file.txt -out encrypted_file.enc -pass pass:"uso rules"
student@uso:~$ ls
encrypted_file.enc  plaintext_file.txt
```

Utilizând parametrul **-d** putem decripta fișierul `encrypted_file.enc`.

```
student@uso:~$ openssl aes-256-cbc -d -in encrypted_file.enc -out decrypted_file.txt -pass pass:"uso rules"
student@uso:~$ cat decrypted_file.txt
plaintext content
```

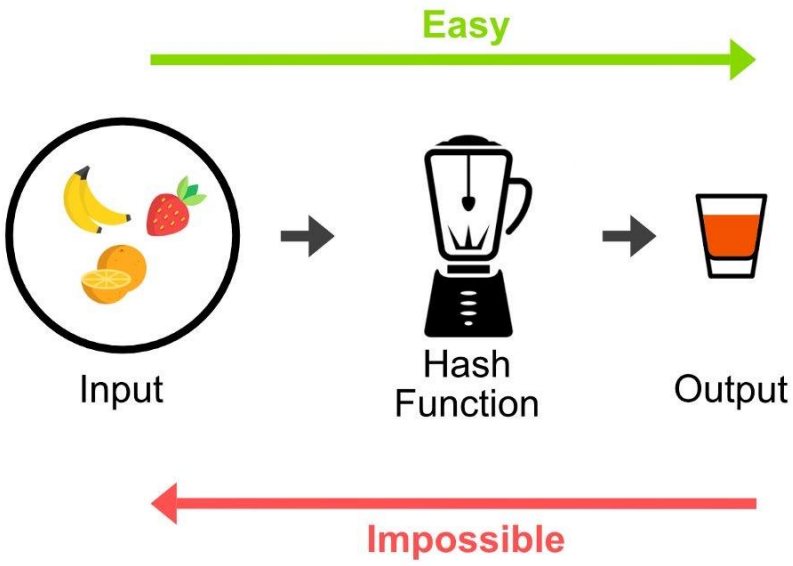
Hashing

Hash-ul este o funcție one-way [https://en.wikipedia.org/wiki/One-way_function]. Ea primește un input oricât de mare (1 byte, un cuvânt, un fișier, un întreg HDD, etc) și returnează un string de o dimensiune fixă în funcție de tipul de hash folosit. Hash-urile sunt rezistente la **coliziuni**, adică este extrem de greu de găsit 2 input-uri care să producă același hash. Mai mult, un singur bit schimbat în input va genera un output complet diferit. Exemple de algoritmi de hashing: md5 (învechit), SHA-1, SHA-256.

Hash-urile sunt folosite pentru verificarea integrității unui fișier (dacă chiar și un bit diferă, atunci va avea alt hash) și la stocarea parolelor.

Folosind comanda **md5sum** fără niciun parametru putem să generăm hash-ul unui text.

```
student@uso:~$ echo -n "plaintext content" | md5sum
f88d6c9eab9c6a2ef3cfd3c59832b4d6  -
```



Stocarea parolelor

1. Cea mai naivă abordare de stocare a parolelor într-o bază de date este în clar. Dacă un atacator reușește să obțină acces la baza de date respectivă, nu doar că obține acces la toate conturile, ci și la orice alt cont al unui utilizator de pe alt site unde a fost folosită aceeași parolă. Este evident că această abordare nu este **sigură**.

User	Password
Mihai	mereverzi
Radu	usoemisto
Andrei	usoemehmateemisto

2. O altă variantă (mai sigură) ar fi să nu stocăm parolele în clar, ci să stocăm **hash-ul** parolei în baza de date. Astfel, când un utilizator se loghează, facem hash-ul parolei introduse de el și îl comparam cu cel din baza de date.

User	Password
Mihai	43eff133cceacbf1354369568b486450
Radu	4491fea305731261a83f75672e25bb88
Andrei	96f3e4decfcd7cd02c3027f0b6416b6e

Abordarea este într-adevăr mai bună, dar încă nu complet sigură. Deși hash-urile nu pot fi sparte direct (dintr-un hash să obținem mesajul inițial), ele pot fi atacate folosind dicționare. Un dicționar este o mulțime de corespondențe `(hash_mesaj) → (mesaj)`. Pentru a sparge un hash, putem folosi dicționare mari online precum acesta [https://crackstation.net/]. Dacă avem norocul ca hash-ul să se afle în dicționar, atunci obținem parola în clar. De obicei, acest lucru se întâmplă pentru toate parolele ce folosesc cuvinte cunoscute sau au o dimensiune/complexitate mică.

Enter up to 20 non-salted hashes, one per line:

```
43EFF133CCEACBF1354369568B486450
96f3e4decfcd7cd02c3027f0b6416b6e
```

Nu sunt robot reCAPTCHA Confidențialitate - Termeni

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
43EFF133CCEACBF1354369568B486450	md5	mereverzi
96f3e4decfcd7cd02c3027f0b6416b6e	Unknown	Not found.

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

Pentru baza de date de mai sus, parola lui Mihai a putut fi spartă, dar cea a lui Andrei nu. Andrei a avut o parola mai lungă, dar acest lucru nu asigură neapărat siguranța ei.

3. O abordare finală ar fi să folosim un salt pentru hash-uirea parolei. Salt-ul este un string random de câteva caractere random ce se folosește în felul următor la construirea unui hash: Hash(salt | parolă), unde "|" înseamnă concatenare. Astfel, salt-ul previne atacul cu dicționar de mai sus deoarece este foarte puțin probabil ca hash-ul respectiv să existe într-un dicționar. Salt-ul se stochează în clar în baza de date.

	Salt	Password
Mihai	dfd454ddf	f98ad20d2f42dcd4e53a4d1f40f9c1a5

Pentru Mihai, este: Hash(dfd454ddfmereverzi).

Hash	Type	Result
f98ad20d2f42dcd4e53a4d1f40f9c1a5	Unknown	Not found.

Liste de cuvinte

John the Ripper este un utilitar de spargere al parolelor. În cadrul acestui demo dorim sa aflăm parolele utilizatorilor din sistem. Pentru a reduce timpul de brute-force al utilitarului **John** vom crea o listă de cuvinte.

Instalare **John The Ripper**

```
student@uso:~$ sudo apt-get update
student@uso:~$ sudo apt-get install john
```

Copiem hash-urile parolelor în fișierul mypasswd.txt.

```
student@uso:~$ sudo unshadow /etc/passwd /etc/shadow > mypasswd.txt
```

Creăm o listă de cuvinte care ne-ar putea ajuta sa ghicim parola unui calculator dintr-o universitate.

```
student@uso:~$ echo -ne "school\nuniversity\nstudent" > wordlist.txt
```

Rulăm utilitarul **John the Ripper**.

```
student@uso:~$ john --wordlist=wordlist.txt mypasswd.txt
Loaded 3 password hashes with 3 different salts (crypt, generic crypt(3) [?/64])
Remaining 2 password hashes with 2 different salts
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:00 100% 0g/s 50.00p/s 100.0c/s 100.0C/s school..student
Session completed
```

Pentru a vizualiza rezultatele rulăm următoarea comandă

```
student@uso:~$ john --show mypasswd.txt
student:student:1000:1000:Student User,,,:/home/student:/bin/bash

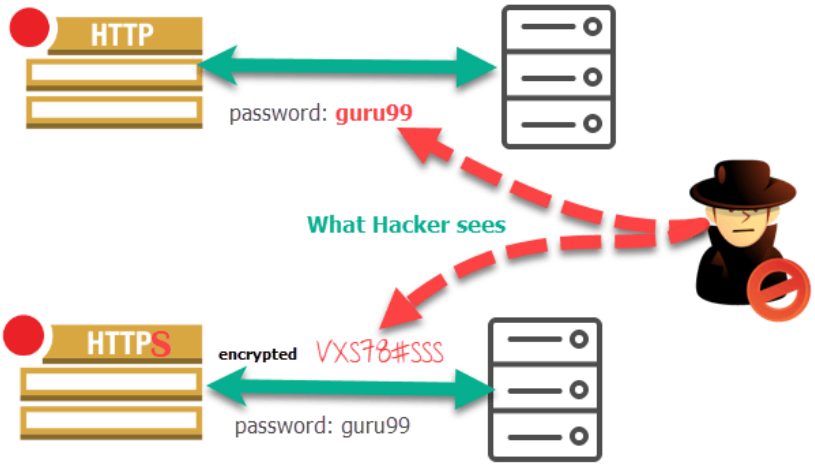
1 password hash cracked, 1 left
```

Utilitarul ne indică faptul că parola **student** este parola utilizatorului `student`.

HTTP vs HTTPS

Protocolul de rețea **HTTP** aduce beneficii din multe puncte de vedere. Unul dintre dezavantajele lui este faptul că toate datele sunt trimise în clar. Astfel, un atacator precum Trudy va putea intercepta și înțelege un pachet HTTP.

Acest start suplimentar necesar de criptare este adus de **HTTPS**, detaliat mai pe larg aici [https://ro.wikipedia.org/wiki/HTTPS].



Least privilege

Principle of least privilege se referă la faptul că un utilizator, program, modul, etc va putea accesa doar informațiile și resursele necesare pentru desfășurarea activității **intenționate**.

Cu alte cuvinte, utilizatorul **student** va putea accesa doar fișierele și directoarele care îl privesc, nu și cele ce conțin date private ale altor utilizatori, de exemplu directorul `/home/gion`, care este home-ul utilizatorului `gion`.

În Linux, acest lucru se definește folosind permisiuni. Ne amintim de suita de comenzi `chown`, `chgrp`, `chmod` din laboratoarele trecute. Dorim de exemplu să îi dăm drepturi doar utilizatorului student pe directorului sau de home:

```
student@uso:~$ sudo chown student /home/student/
student@uso:~$ sudo chgrp student /home/student/
student@uso:~$ sudo chmod 700 /home/student/
student@uso:~$ ls -l /home/
total 4
drwx------ 23 student student 4096 dec  8 23:30 student
```

Cu `chown` schimbăm **ownerul** directorului, cu `chgrp` schimbat **group ownerul**, iar cu `chmod` îi permitem doar utilizatorului `student` să vadă sau modifice conținutul directorului `/home/student`.

Principle of least privilege se aplică foarte mult în zona de web. Se asigură astfel faptul ca dacă o pagină a unui site a fost compromisă, nu va fi compromis întreg site-ul sau chiar întreg server-ul.

Exercitii

1. Hash me

[1a] Realizați un md5 pe string-ul `whoareyou` folosind md5sum.

Hash-ul obținut trebuie să fie `a46601d9f660704eb28d1ce17ac1fae4`. Asigurați-vă că obțineți același rezultat.

[1b] Intrați pe CrackStation [https://crackstation.net/] și introduceți hash-ul vostru. Veți observa că el a fost deja spart.

Hash	Type	Result
a46601d9f660704eb28d1ce17ac1fae4	md5	whoareyou

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

[1c] Generați un string de 10 caractere random ca în exemplul de mai jos:

```
student@uso:~$ head /dev/urandom | tr -dc A-Za-z0-9 | head -c 10 ; echo ''
oargwZRgEv
```

[1d] Folosiți string-ul generat mai devreme ca **Salt** pentru mesajul `whoareyou`. În cazul din exemplu, s-ar fi realizat md5-ul pe `oargwZRgEvwhoareyou`

[1e] Încercați pe CrackStation [https://crackstation.net/] să spargeți acest nou md5. Observați că un **Salt** întărește securitatea unui hash.

Hash	Type	Result
f98ad20d2f42dcd4e53a4d1f40f9c1a5	Unknown	Not found.

2. Generare de parole

Pe Linux, putem genera parole folosind utilitarul `pwgen`:

```
student@uso:~$ pwgen
ceigie4E phie3fuG ooQua8ch Quochu7A Wi2iesha fiena3Qu xohWah7w nahte9Ao
Caem6Eis Jeiquo2f phahM1je oLou6ohb aishae2O bo1Eishu PhugeS1z Tang6oox
Gaif4aij xoh1Evoa Eipoax7A ADi5Hohx ahJ8aida Aef0och4 iuD7oa3y Tatoo8po
Aephuit2 Segh9gah usaif4Ah Oex5quoo Thei9kae aoyuPh0F iu2vi3Ph aipai3Ow
Oof4Tei9 Naing5Ru nee9geiN noosh7Fo aF7ae8va zoo0IT7w uoj2Sie8 Zooghee0
........
```

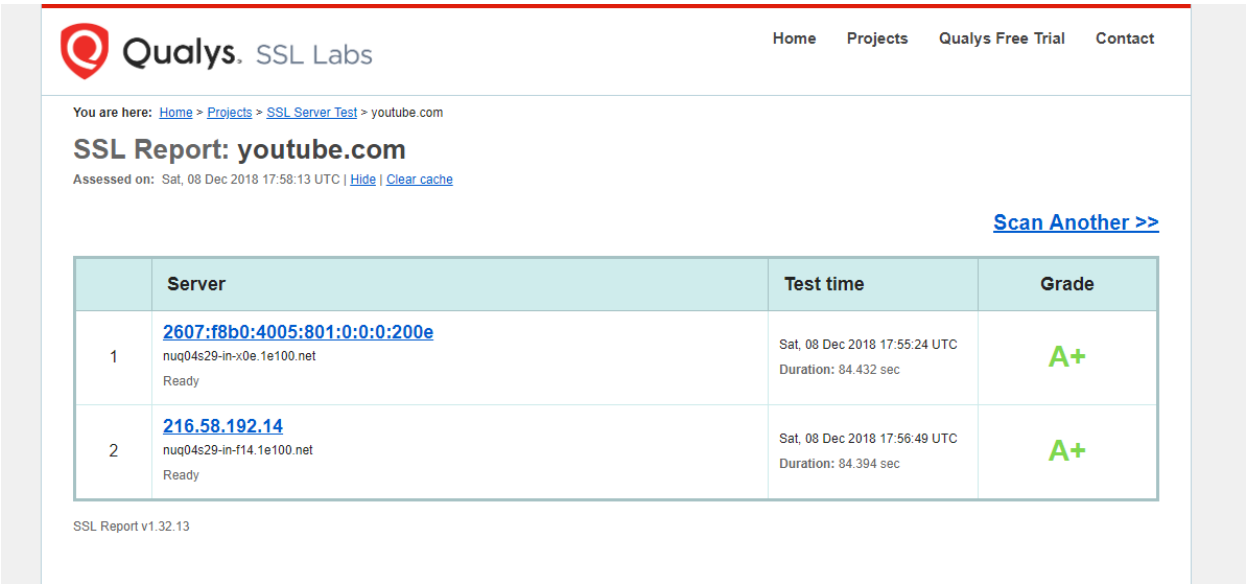
[2a] În modul default, **pwgen** generează parole ușor de memorat. Generați o parolă greu de memorat, sigură. `Hint: man pwgen`

[3a] Folosind pwgen, generați o parolă de **80 de caractere**, **singură pe linie**, **fără vocale**, cu **cel puțin o majusculă** și fără să conțină **numere**. `Hint: man pwgen`

3. Site-uri considerate sigure

Putem folosi ssllabs [https://www.ssllabs.com/ssltest/] pentru a verifica certificatul [https://www.techopedia.com/definition/29743/website-security-certificate] unui site HTTPS.

Introduceți **Hostname-ul** `youtube.com` și observați ce notă primește:



Introduceți apoi **Hostname-ul** `expired.badssl.com` ce are certificatul expirat:



ssllabs [https://www.ssllabs.com/ssltest/] este un bun utilitar pentru a verifica dacă un site este sigur sau nu.

Pentru exercițiile următoare, mergeți în directorul `/home/student/uso-lab/` și rulați comanda `git pull`. Mergeți în directorul `/home/student/uso-lab/labs/08-security/support`.

4. Spargerea parolei prin brute-force

Tehnica de **brute-force** presupune încercarea oricăror combinatii posibile de caractere în încercarea de a sparge o parolă. În cazul nostru, vom lucra cu un **zip**.

Dacă parola este suficient de lungă (peste 10-12 caractere) atunci ea este considerată relativ sigură pentru un atac de acest tip întrucât va dura mult prea mult găsirea parolei corecte, chiar cu cele mai puternice CPU-uri din prezent.

Vom încerca să spargem parola arhive zip `secret_brute_force.zip`. Pentru a face acest lucru, vom instala utilitarul **fcrackzip**:

```
student@uso:~$ sudo apt-get install fcrackzip
```

Știm faptul că arhiva are o parolă de maxim 4 caractere, deci este fezabil să rulăm un atac de tip brute-force:

```
student@uso:~/.../labs/08-security/support$ fcrackzip -v -l 1-4 -u secret_brute_force.zip
found file 'flag.txt', (size cp/uc     34/    22, flags 1, chk 6543)


PASSWORD FOUND!!!!: pw == fd7
student@uso:~/.../labs/08-security/support$ unzip -P fd7 secret_brute_force.zip
Archive:  secret_brute_force.zip
 extracting: flag.txt
student@uso:~/.../labs/08-security/support$ cat flag.txt
USO{viața_pe_internet}
```

Argumentele comenzii **fcrackzip** sunt următoarele:

- `-v` → verbose
- `-l 1-4` → atac de tip brute-force cu lungimea parolei între 1 și 4 caractere
- `-u` → verifică parola prin încercarea de a dezarhiva arhiva

5. Spargerea parolei cu wordlist

Asemănător exemplului de mai sus, avem în același director arhiva zip `secret_wordlist.zip`. Deoarece parola este suficient de lungă, ea nu poate fi spartă folosind un atac de tip brute-force.

Poate fi însă spartă folosind un dicționar de cuvinte. **fcrackzip** are opțiunea de a încerca parole dintr-o listă (numite wordlist). Dacă parola arhivei se află în lista de cuvinte, atunci ea va putea fi spartă.

Deși sună greu de realizat în practică (Ce wordlist va avea în ea parola mea `nuraspundatelefoncandstauinbrigada` ??), multe persoane folosesc parole simple pentru a-și proteja datele personale.

[2a] Spargeți parola arhivei `secret_wordlist.zip` folosind dicționarul `wordlists.txt`. (HINT: `man fcrackzip` sau `Google`)

Extra

1. SUID

1. Citiți despre **SUID** aici [https://www.linux.com/blog/what-suid-and-how-set-suid-linuxunix].
2. Compilați programul de mai jos.
3. Setați bitul de SUID pe binarul creat.
4. Este posibil să fie nevoie să setați utilizatorul **root** ca owner pe binar.
5. Afișați /etc/shadow folosind binarul creat fără a folosi sudo. (./binar)

```
#include <stdio.h>
#include <stdlib.h> // For exit()

int main()
{
        FILE *fptr;

        char filename[100], c;

        printf("Enter the filename to open \n");
        scanf("%s", filename);

        // Open file
        fptr = fopen(filename, "r");
        if (fptr == NULL)
        {
                printf("Cannot open file \n");
                exit(0);
        }

        // Read contents from file
        c = fgetc(fptr);
        while (c != EOF)
        {
                printf ("%c", c);
                c = fgetc(fptr);
        }

        fclose(fptr);
        return 0;
}
```

2. Iptables

Folosiți comanda **iptables** pentru a bloca traficul SSH către sistemul fizic. Puteți urmări exemplu de aici [https://unix.stackexchange.com/questions/397481/use-iptables-to-block-ssh].

Pentru a testa, porniți mașinile virtuale `tom` și `jeryy`. Mai întâi conectați-vă prin `ssh` pe mașina `tom`.

1. Dacă acest lucru nu funcționează, reveniți la pasul anterior și depanați problema.
2. Dacă funcționează, deconectați-vă și încercați acum să vă conectați de pe mașina virtuală pe mașina fizică prin `ssh`.
3. Dacă nu funcționează, atunci totul este în regulă.

3. XSS

XSS sau **Cross-site scripting** este o vulnerabilitate web listată în `OWASP TOP 10`. Citiți mai multe despre XSS, precum și niște exemple, aici [https://www.owasp.org/index.php/Cross-site_Scripting_(XSS)].

Parcurgeți câte nivele puteți din XSS GAME [https://xss-game.appspot.com/].

4. Python encrypt

Scrieți un program în `python` ce criptează și apoi decriptează un text folosind **AES** în modul **CBC**. Puteți citi mai multe despre AES aici [https://thebestvpn.com/advanced-encryption-standard-aes/]. Pentru program, puteți urmări indicațiile de aici [https://docs.python-guide.org/scenarios/crypto/].

5. Obținere certificat

Alegeți un site care să folosească **HTTPS**. Obțineți certificatul site-ului din CLI folosind `openssl`.

Cuprins

- Laborator 10 (one page)
- Concepte
- Exercitii
- Extra

uso/laboratoare/laborator-10.txt · Last modified: 2021/10/12 19:40 (external edit)

Laborator 11 - Planificarea sarcinilor administrative

Folosirea unui workstation remote

Înainte de începerea laboratorului rulați următoarele comenzi:

```
cd uso-lab
git pull
cd ~/uso-lab/labs/09-task-admin/lab-container/
./lab_prepare.sh cleanup
```

Înainte de începerea laboratorului rulați următoarele comenzi:

```
cd ~/uso-lab/labs/09-task-admin/lab-container
./lab_prepare.sh install remote
./lab_prepare.sh install ssh-server
./lab_prepare.sh install local
```

Dacă nu mai aveți spațiu pe disc, rulați comanda `docker image prune --all`.

Noi ca dezvoltatori software sau administratori de sisteme, avem nevoie de multă putere de procesare pentru a ne realiza obiectivele și accesibilitatea necesară pentru a folosi sistemele oricând, oricum.

Un avantaj la folosirea unei stații la distanță este faptul că este mereu disponibilă. Aceasta nu trebuie să fie oprită, nu are nevoie de restart, dacă ne deplasăm undeva, aceasta nu se schimbă cu nimic. De oriunde de oriunde l-ai accesa, vei avea la dispoziție sistemul la modul "Pickup and Play".

Unele lucruri pe care vrem să le rezolvăm care necesită putere de procesare sunt compilarea programelor, antrenarea modelelor de machine learning, testarea la limită (stress testing) a aplicațiilor, sau simularea infrastructurilor complexe folosind mașini virtuale, rularea de servere complexe pentru anumite aplicații.

În mod obișnuit, sistemele care oferă multă putere de procesare nu vin în pachete compacte, ci acestea sunt sub formă de calculatoare de timp stații de lucru (workstation), servere, sau laptopuri grele. O altă abordare pentru a folosi un workstation este să obținem o mașină virtuală într-un cloud, cum ar fi AWS sau Microsoft Azure.

În același timp, nu este un lucru la îndemână pentru toți să lucreze pe astfel de sisteme din considerente de generare de căldură, consum de curent, spațiu la locul de muncă lipsă de mobilitate a sistemelor. Astfel, dacă vrem să lucrăm la o problemă care necesită multă putere de procesare, avem nevoie să fim în apropierea unui sistem care ne oferă multă putere de procesare. Acest lucru nu este mereu posibil din motive precum faptul că nu ai acces la locul de muncă sau că lucrezi într-un mediu la distanță.

În acest capitol prezentăm situația lucrului pe o stație la distanță. Cum putem să ne adaptăm modul de lucru pentru a folosi sisteme la care nu avem acces fizic și cum putem să monitorizăm și să folosim cât mai bine aceste sisteme.

Conectarea la workstation

Pentru a parcurge această secțiune este recomandat să descărcați ultima versiune a respository-ului laboratorului. Pentru a descărca ultima versiune a repository-ului, rulați comanda `git pull` în directorul `~/uso-lab/labs/09-task-admin/lab-container/`.

Infrastructura laboratorului este bazată pe containere Docker ale căror imagini vor fi generate pe propriul calculator. Dacă nu aveți deja instalat Docker Engine pe sistem, scriptul `~/uso-lab/labs/09-task-admin/lab-container/lab_prepare.sh` va instala aplicația.

După ce ați terminat de lucrat, vă recomandăm să opriți containerele rulând comanda `./lab_prepare.sh delete` în directorul `~/uso-lab/labs/09-task-admin/lab-container/`.

Prima problemă cu care ne vom confrunta este conectarea la stație, deoarece, în majoritatea cazurilor, nu vom lucra în aceeași rețea cu sistemul la distanță și stația nu va avea o adresă IP publică. Așadar, nu ne vom putea conecta direct la sistem folosind un protocol de comunicare la distanță, cum ar fi SSH fără să facem pași suplimentari.

Câteva soluții de conectare la sistem pe care le vom aborda sunt:

- VPN;
- tunel SSH;
- DDNS (Dynamic Domain Name System).

Nu vom aborda soluția bazată pe DDNS, deoarece aceasta presupune că avem o adresă IP publică configurată pe ruterul primit de la furnizorul de Internet, dar acest lucru nu mai este valabil pentru majoritatea stațiilor.

Problema adreselor IP private

Pentru rularea acestui demo, rulați în directorul `~/uso-lab/labs/09-task-admin/lab-container/` comanda `./lab_prepare.sh install remote` și comanda `./lab_prepare.sh install local` Pentru a ne conecta la infrastructura pentru această secțiune, vom folosi comanda `./lab_prepare.sh connect` având ca parametru numele mașinii la care vrem să ne conectăm, `local` sau `remote`.

Adresele IPv4 au fost contruite pentru a fi identificatori unici în Internet pentru stații. Acestea pot să identifice maxim 2^{32} stații în Internet. Însă acum există mai multe dispozitive conectate la Internet decât există adrese IPv4 care se pot asigna pe dispozitive.

Pentru a se rezolva această problemă, au fost alese anumite intervale de adrese IP care pot fi atribuite pe mai multe dispozitive în același timp. Astfel, s-a rezolvat problema suprapunerii adreselor IP. Dezavantajul este că aceste adrese, nemaifiind unice, nu mai pot să fie accesibile de oriunde din Internet.

Intervalele adreselor IPv4 private sunt următoarele:

- `10.0.0.0` - `10.255.255.255`;
- `172.16.0.0` - `172.31.255.255`;
- `192.168.0.0` - `192.168.255.255`

Nicio adresă IP din intervalele de mai sunt nu poate fi accesată direct din Internet.

În această subsecțiune vom lucra cu 3 stații care sunt distribuite în felul următor:

- `local`, reprezintă stația "locală", adică laptopul de pe care ne-am conecta, dacă ar fi vorba de un scenariu real; are o singură interfață de rețea cu adresa IP `10.11.11.3`
- `remote`, reprezintă workstationul la care vrem să ne conectăm; are o singură interfață cu adresa IP `10.10.10.3`.

Vom verifica conectivitatea cu stația `remote` de pe stația `local` folosind comanda `ping`:

```
root@local:~# ping -c 2 10.10.10.3
PING 10.10.10.3 (10.10.10.3) 56(84) bytes of data.

--- 10.10.10.3 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1010ms

root@local:~# ping -c 2 141.85.241.99
PING 141.85.241.99 (141.85.241.99) 56(84) bytes of data.
64 bytes from 141.85.241.99: icmp_seq=1 ttl=61 time=9.02 ms
64 bytes from 141.85.241.99: icmp_seq=2 ttl=61 time=8.58 ms

--- 141.85.241.99 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 8.588/8.806/9.024/0.218 ms
```

Rezultatul comenzii primei rulări a comenzii `ping` demonstrează faptul că nu avem conectivitate între stația `local` și stația `remote`. Totuși, am testat și conectivitatea cu o altă adresă IP din Internet, în cazul acesta folosind adresa IP a stației `fep.grid.pub.ro` (cu adresa IP 141.85.241.99), cu care putem să comunicăm, deci nu este o problemă de conectare la Internet. Pentru a recapitula funcționalitatea utilitarului `ping`, revizitați capitolul Configurarea sistemului.

Conectarea prin VPN

Pentru acest demo vom lucra pe stația `remote`. Pentru a ne conecta la stația `remote` vom rula comanda `./lab_prepare.sh connect remote` în directorul `~/uso-lab/labs/09-task-admin/lab-container/`.

Prima soluție pentru conectarea la o stație care folosește o adresă IP privată o reprezintă serviciile de tip VPN (*Virtual Private Network*). Acestea conectează două stații care în mod fizic nu sunt conectate la aceeași rețea.

Pentru această soluție avem două moduri de organizare: folosind un server public pe care îl configurăm noi drept server de VPN, sau folosirea unui serviciu public de VPN cum ar fi:

- Hamachi [https://vpn.com/]
- ZeroTier [https://zerotier.com/].

Am folosit ca exemplu serviciile Hamachi sau ZeroTier, deoarece acestea pot fi folosite gratuit și sunt ușor de configurat.

Soluții VPN

Nume soluție	Avantaje	Dezavantaje
Hamachi	* Ușor de setat * Aplicație client disponibilă pe Linux și pe Windows * Nu necesită înregistrarea	* Latență mai mare decât alte soluții * Soluția gratis nu este optimă pentru un număr mare de calculatoare
ZeroTier	* Ușor de instalat și setat * Permite 50 de stații într-o rețea	* Necesită înregistrare pentru folosire
FreeLAN	* Soluție gratis * Nu limitează numărul de stații dintr-o rețea	* Configurare dificilă * Nu funcționează pe toate stațiile, este nevoie de anumite condiții speciale

Folosirea serviciului Hamachi

Pentru început, recomandăm folosirea aplicației Hamachi, deoarece acesta nu presupune înregistrarea unui cont pentru folosirea aplicație. Hamachi vine cu dezavantajul că putem să conectăm maxim cinci stații între ele și viteza conexiunii este mai mică decât dacă am folosi unele servicii plătite, cum ar fi OpenVPN.

Instalarea Hamachi

Nu vom instala pachetul folosind aplicația `apt`, cum am învățat până acum, deoarece repository-ul de pachete folosit de Ubuntu nu conține pachetul pentru Hamachi. Pentru instalarea aplicației Hamachi vom descărca pachetul.

Vom folosi comanda `wget` pentru a descărca pachetul:

```
root@remote:~# wget https://www.vpn.net/installers/logmein-hamachi_2.1.0.203-1_amd64.deb
--2021-01-05 15:49:15--  https://www.vpn.net/installers/logmein-hamachi_2.1.0.203-1_amd64.deb
Resolving www.vpn.net (www.vpn.net)... 64.95.128.197, 64.95.128.199
Connecting to www.vpn.net (www.vpn.net)|64.95.128.197|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1613102 (1.5M) [application/vnd.debian.binary-package]
Saving to: 'logmein-hamachi_2.1.0.203-1_amd64.deb'

logmein-hamachi_2.1.0.203-1_amd64 100%[=========================================================>]   1.54M  4.14MB/s    in 0.4s

2021-01-05 15:49:16 (4.14 MB/s) - 'logmein-hamachi_2.1.0.203-1_amd64.deb' saved [1613102/1613102]

root@remote:~# dpkg -i logmein-hamachi_2.1.0.203-1_amd64.deb
Selecting previously unselected package logmein-hamachi.
(Reading database ... 12216 files and directories currently installed.)
Preparing to unpack logmein-hamachi_2.1.0.203-1_amd64.deb ...
Unpacking logmein-hamachi (2.1.0.203-1) ...
Setting up logmein-hamachi (2.1.0.203-1) ...
mknod: /dev/net/tun: File exists
Starting LogMeIn Hamachi VPN tunneling engine logmein-hamachi
starting - success
Processing triggers for systemd (245.4-4ubuntu3.3) ...
```

Am folosit comanda `dpkg` care gestionează aplicații sub forma pachetelor `.deb`, specifice distribuțiilor Debian, Ubuntu și Mint. Opțiunea `-i` specifică comenzii `dpkg` că vrem să instalăm o aplicație.

Verificăm că am instalat aplicația Hamachi folosind comanda `hamachi` în felul următor:

```
root@remote:~# hamachi
  version    : 2.1.0.203
  pid        : 42
  status     : offline
  client id  :
  address    :
  nickname   :
  lmi account:
```

Pentru a ne autentifica la serverele Hamachi, folosim comanda `hamachi login`:

```
root@remote:~# hamachi login
Logging in .......... ok
root@remote:~# hamachi
  version    : 2.1.0.203
  pid        : 42
  status     : logged in
  client id  : 253-932-022
  address    : 25.114.254.180    2620:9b::1972:feb4
  nickname   : remote
  lmi account: -
```

Această comandă generează un identificator unic per stație și stabilește un nickname. Rulând comanda `hamachi` vor fi afișate identificatorul, nickname-ul sistemului, adresa IP din VPN și nickname-ul sistemului.

Exercițiu: Instalare Hamachi

Instalați Hamachi pe stația `local`.

Crearea unei rețele private

Odată ce am instalat Hamachi pe ambele stații pe care vrem să le conectăm, trebuie să creăm o rețea virtuală prin care acestea două să comunice. Această rețea va conecta cele două calculatoare, cu toate că ele nu sunt fizic în aceeași rețea.

Vom crea rețeaua virtuală de pe stația `remote`. Pentru a crea o rețea virtuală folosim comanda `hamachi create` împreună cu numele rețelei și parola acesteia. Stația `remote` va avea drepturi de administrare a rețelei. Doar având drepturi administrative se poate scoate sisteme din rețea, sau se poate șterge rețeaua. Pentru demo-ul pe care îl urmăriți, folosiți în loc de șirul de caractere `nume-prenume` numele și prenumele vostru.

```
root@remote:~# hamachi create nume-prenume 12345667890
Creating nume-prenume .. ok
root@remote:~# hamachi list
 * [nume-prenume]  capacity: 1/5, subscription type: Free, owner: This computer
```

Am folosit comanda `hamachi list` pentru a verifica faptul că a fost creată rețeaua. Comanda `hamachi list` afișează toate rețelele din care face parte stația.

Conectarea la o rețea

Pentru a conecta stația `local` la rețeaua privată vom folosi comanda `hamachi join` urmată de numele rețelei la care vom conecta stația și parola rețelei.

```
root@local:~# hamachi join nume-prenume 12345667890
Joining nume-prenume .. ok
root@local:~# hamachi list
 * [nume-prenume]  capacity: 2/5, subscription type: Free, owner: remote (253-932-022)
 * 253-932-022   remote                    25.114.254.180    alias: not set          2620:9b::1972:feb4                          via server  TCP
```

Rulând comanda `hamachi list`, am afișat rețelele la care este conectată stația `local` și stațiile cu care împarte rețelele. Adresa IP a stației stației `remote` din rețeaua privată este `25.114.254.180`, conform rezultatului comenzii.

Pentru a testa conexiunea dintre stațiile `local` și `remote` vom rula comanda `ping`.

```
root@local:~# ping -c 2 25.114.254.180
PING 25.114.254.180 (25.114.254.180) 56(84) bytes of data.
64 bytes from 25.114.254.180: icmp_seq=1 ttl=64 time=76.5 ms
64 bytes from 25.114.254.180: icmp_seq=2 ttl=64 time=92.3 ms

--- 25.114.254.180 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 76.592/84.493/92.394/7.901 ms
```

Exercițiu: Crearea și folosirea unei rețele private

Creați o nouă rețea privată numită `prenume-nume`, cu parola de acces `anaaremere`. Conectați stațiile `local` și `remote` la noua rețea.

Conectarea folosind un tunel SSH

Pentru rularea acestui demo, rulați în directorul `~/uso-lab/labs/09-task-admin/lab-container/` comanda `./lab_prepare.sh install ssh-server`. Pentru a ne conecta la infrastructura pentru această secțiune, vom folosi comanda `./lab_prepare.sh connect` având ca parametrul numele mașinii la care vrem să ne conectăm, `local`, `remote` sau `ssh-server`.

Abordările pe care le-am discutat mai sus presupun accesul la un server central și drepturi de administrator din partea utilizatorului pentru instalarea aplicațiilor (cum ar fi Hamachi sau OpenVPN). Dacă nu avem acces la un cont de administrator, nu putem să ne conectăm la stația de la distanță.

O alternativă care nu presupune drepturi administrative sunt tunelele SSH. Un tunel SSH reprezintă o conexiune între două stații facilitată de protocolul SSH. Astfel, în loc să trimitem comenzi prin SSH către o stație, putem trimite orice tip de mesaj, pentru orice aplicație.

Dezavantajul acestei abordări este că necesită accesul la un server terț care să fie accesibil de ambele stații.

În această subsecțiune vom lucra cu 3 stații care sunt distribuite în felul următor:

- `local`, reprezintă stația "locală", adică laptopul de pe care ne-am conecta, dacă ar fi vorba de un scenariu real; are o singură interfață de rețea cu adresa IP `10.11.11.3`;
- `remote`, reprezintă workstationul la care vrem să ne conectăm; are o singură interfață cu adresa IP `10.10.10.3`;
- `ssh-server`, reprezintă serverul terț prin care ne vom conecta ca să ajungem la workstation; această stație are două interfețe conectate la ea, cu adresele IP `10.11.11.2` și `10.10.10.2`, dar în realitate aceasta ar avea o singură placă de rețea.

Stațiile `local` și `remote` nu au conectivitate între ele, dar au conectivitate la stația `ssh-server`, deoarece sunt în aceeași rețea.

Inițializarea tunelului SSH

Primul pas necesar pentru conectarea la o stație prin tunel SSH este inițializarea tunelului. Când inițializăm tunelul acesta va deschide portul `4242` pe stația `ssh-server` de la adresa `10.10.10.2`, astfel încât mesajele trimise către portul `4242` vor fi trimise către portul `22` de pe stația `remote` , stația de pe care inițializăm tunelul SSH. Vom folosi utilizatorul `root` cu parola `root` pentru a ne conecta.

Pentru a deschide tunelul vom folosi comanda următoare:

```
root@remote:~# ssh -N -R 4242:localhost:22 root@10.10.10.2
The authenticity of host '10.10.10.2 (10.10.10.2)' can't be established.
ECDSA key fingerprint is SHA256:xV1orHYj4fhkc5HE91sfh8QhaVqke/AEMa8mYI423HY.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.10.10.2' (ECDSA) to the list of known hosts.
root@10.10.10.2's password:
```

Opțiunile comenzii `ssh` folosite sunt următoarele:

- `-N` este folosită atunci când deschidem tunele pentru a nu deschide shelluri în care să dăm comenzi;
- `4242` este portul pe care vrem să îl deschidă pe stația `ssh-server`;
- `localhost` este stația către care vor fi trimise mesajele primite pe portul `4242`. În cazul acesta mesajele vor fi trimise către `localhost`, adică stația `remote`, cea de pe care rulăm comanda de tunelare;
- `22` este portul de pe stația `localhost` către care vrem să fie trimise mesajele primite pe portul `4242` de pe stația `ssh-server`;
- `root@10.10.10.2`, utilizatorul și adresa IP a stației către care vrem să deschidem tunelul; în cazul acesta este adresa IP a stației `ssh-server`.

După ce am pornit tunelul, comanda `ssh` nu va mai afișa nimic la terminal. Tunelul va rămâne activ cât timp comanda `ssh` de mai sus rulează.

Putem să verificăm că a fost deschis portul `4242` pe stația `ssh-server` rulând comanda `netstat -tlpn`:

```
root@ssh-server:~# netstat -tlpn
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      24/sshd
tcp        0      0 127.0.0.1:4242          0.0.0.0:*               LISTEN      99/sshd: root
tcp        0      0 127.0.0.11:42991        0.0.0.0:*               LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      24/sshd
```

Cât timp această fereastră rămâne deschisă, tunelul va fi activ. Vom învăța în secțiunea `task_admin_services_config_custom` cum să rulăm această comandă în afara terminalului și cum să ne asigurăm că tunelul este mereu deschis.

Folosirea tunelului SSH

Odată creat tunelul, vrem să îl folosim de pe stația `local`, pentru a putea rula comenzi pe stația `remote`.

Pentru a face acest lucru, trebuie să ne conectăm la stația `server`. Serverul `server` poate fi accesat la adresa `10.11.11.2`.

```
root@local:~# ssh root@10.11.11.2
root@10.11.11.2's password:
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 5.4.0-52-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage
This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Tue Jan  5 23:08:53 2021 from 10.11.11.3
root@ssh-server:~#
```

După cum am observat mai sus, în rezultatul comenzii `netstat`, tunelul SSH este deschis pe portul 4242 al stației `ssh-server` și redirectează mesajele către stația `remote`.

Pentru a ne conecta la acest port folosind clientul SSH, rulăm următoarea comandă:

```
root@ssh-server:~# ssh root@localhost -p 4242
[...]
root@remote:~#
```

Am folosit opțiunea `-p` pentru a ne conecta folosind SSH pe un alt port decât portul predefinit (22). În comanda de mai sus ne-am conectat la stația `localhost`, adică stația `ssh-server` dar, deoarece portul 4242 este de fapt un tunel, conexiunea a fost redirectată la stația `remote`.

Observăm că promptul s-a schimbat în `root@remote:~#`, deci ne-am conectat la stația `remote`.

Exercițiu: Crearea și folosirea tunelelor SSH

Creați un tunel SSH care să ducă de la stația `local` la `ssh-server` folosind portul 6970 și conectați-vă la stația `local` din stația `remote` folosindu-vă de stația `ssh-server` ca mai sus.

Folosirea stației la distanță

Odată ce am reușit să realizăm o conexiune persistentă între stația locală și workstationul remote pe care vrem să lucrăm și ne-am configurat mediul de lucru la distanță, vrem să folosim stația.

Pentru folosirea stației vom presupune următorul flux de lucru:

- ne conectăm la sistem
- editățm fișiere sursă sau fișiere de configurare
- compilăm un program, sau rulăm o aplicație
- ne deconectăm de la sistem

Folosirea tmux

Pentru acest demo vom lucra local, pe stația `uso`.

Spre deosebire de un laptop unde, dacă închidem ecranul și îl băgăm în ghiozdan, acesta intră în hibernare, stația de lucru poate să lucreze în continuare. Acest lucru ne oferă persistență mediului de lucru, astfel nu mai este nevoie să repornim IDE-ul, mediul de lucru, taburile pentru debugging și așa mai departe.

Atunci când ne conectăm la un calculator printr-un client SSH și rulăm comenzi, acestea vor rula în foreground.

Dacă avem o aplicație care rulează mult timp, cum ar fi o căutare intensă în sistemul de fișiere folosind `updatedb`, și conexiunea SSH se întrerupe, se va întrerupe și execuția comenzii `updatedb`.

Pentru a rezolva această problemă și a ne folosi de disponibilitatea oferită de un sistem distanță, vom folosi utilitarul `tmux`. Acest utilitar pornește o sesiune de shell care este independentă de terminalul în care rulează, astfel, putem să ne conectăm și să ne deconectăm de la ea.

Crearea unei sesiuni tmux

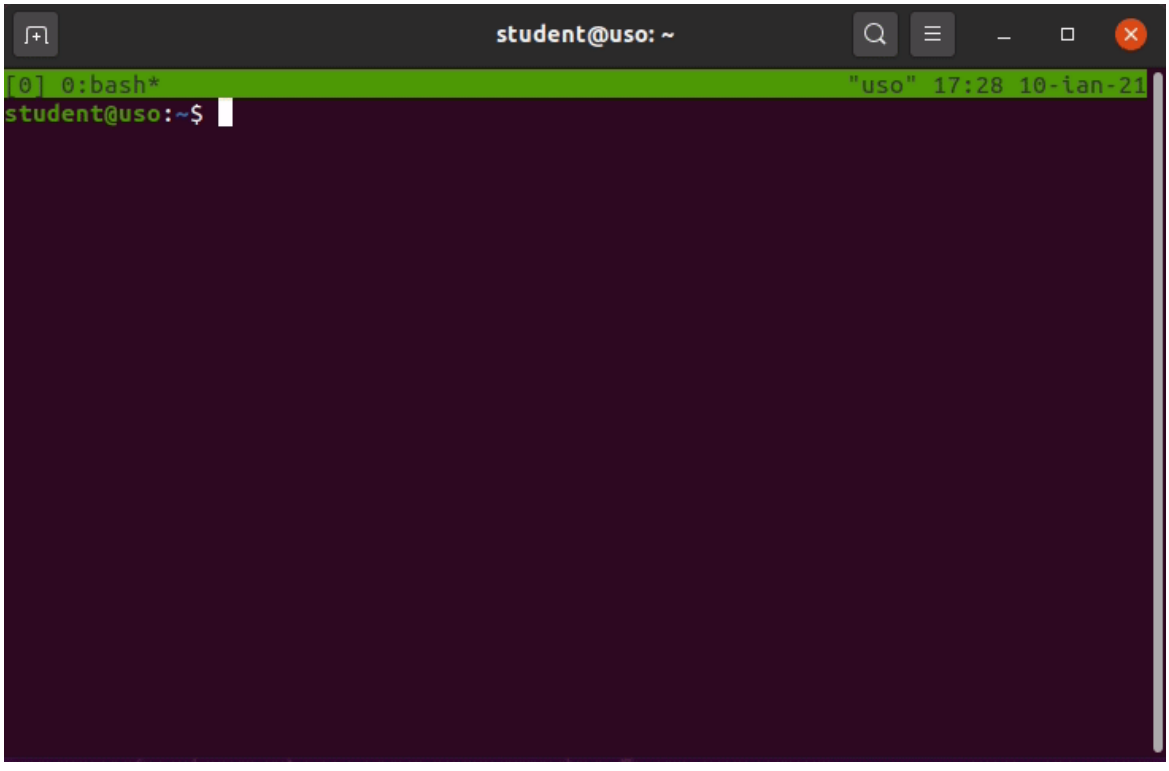
Fiecare pornire a aplicației `tmux` deschide o nouă sesiune. Putem considera fiecare sesiune ca o fereastră a unui browser. De regulă nu este nevoie să avem mai multe ferestre de browser, sau de `tmux` pornite, deoarece avem alte moduri de organizare a spațiului de lucru cu care putem să lucrăm mai ușor folosind scurtături. În plus, pentru fiecare sesiune de `tmux` ar fi nevoie să pornim un nou client SSH, lucru care, în funție de modul de conectare la stație, adaugă pași în plus.



Detașarea și reatașarea la o sesiune tmux

Aplicația `tmux` permite detașarea de la o sesiune folosind combinația de taste `Ctrl+v d`

Vom vedea pe parcursul acestei subsecțiuni că toate scurtăturile de taste ale aplicației `tmux` folosesc ca prefix combinația `Ctrl+v`. Această combinație este predefinită pe mașina virtuală `USO.ova`, dar pe sistemele obișnuite combinația va fi `Ctrl+b`. Putem modifica prefixul în fișierul de configurare `tmux.conf`.



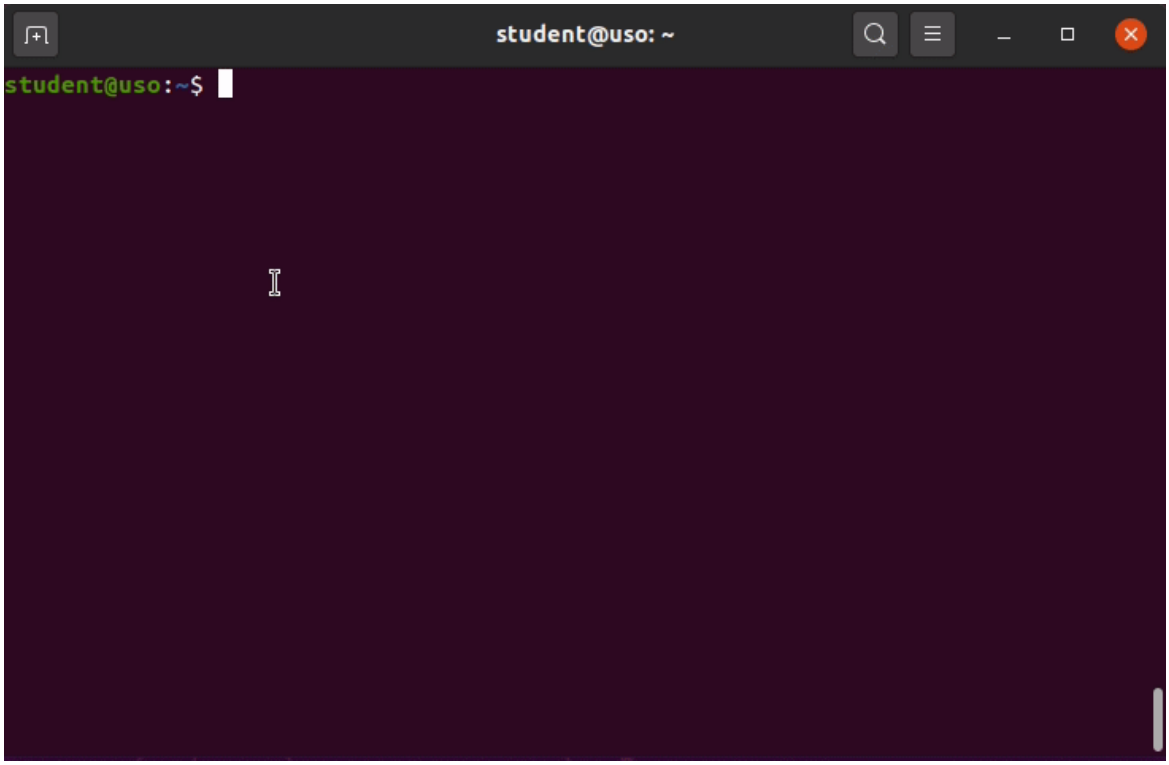
Pentru a ne reatașa la o sesiune trebuie să ne dăm seama la care sesiune să ne atașăm. Fiecare sesiune `tmux` are asociat un identificator.

Vom afișa toate sesiunile deschise folosind comanda `tmux ls`:

```
student@uso:~$ tmux ls
0: 1 windows (created Wed Jan  6 02:44:06 2021)
```

În momentul de față există o singură sesiune cu identificatorul `0`.

Pentru a ne reatașa la sesiune folosim comanda `tmux attach-session -t 0`.



Taburi în tmux

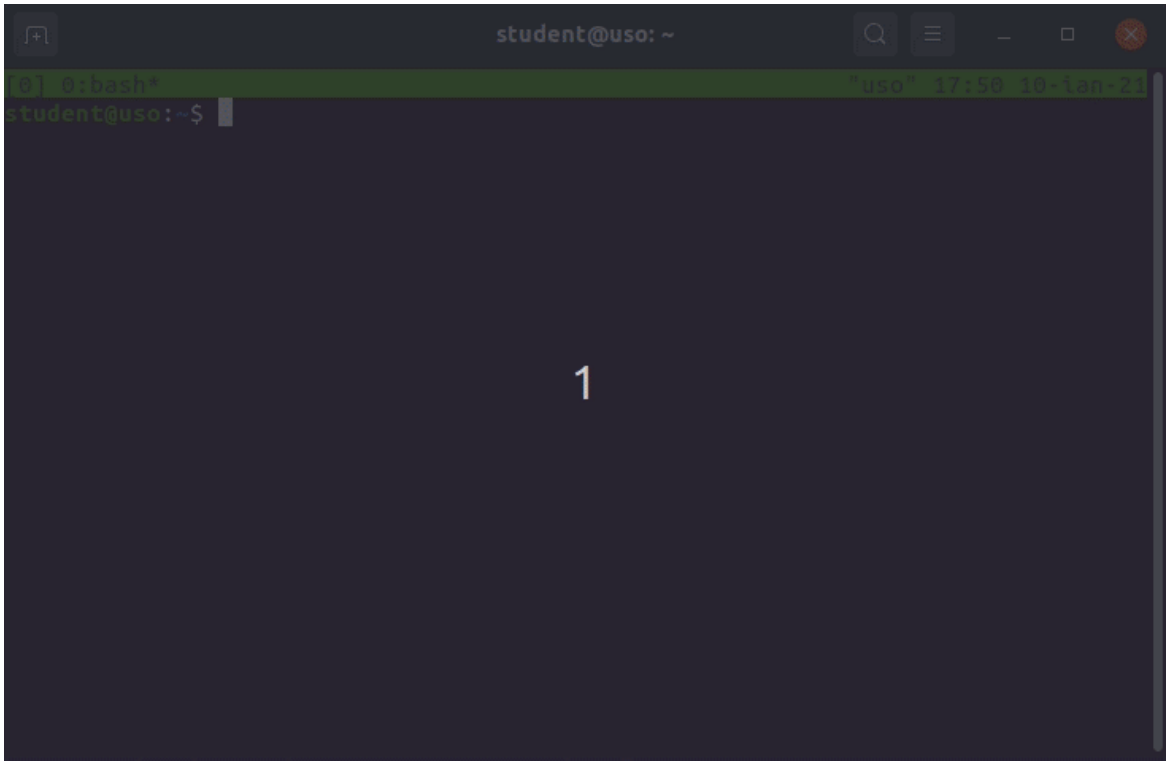
Pentru a ne organiza terminalele deschise pe stația de la distanță, recomandăm folosirea taburilor (numite ferestre - *window* în `tmux`), deoarece acestea sunt deschise pe durata sesiunii și permit crearea și gestiunea facilă.

De exemplu, pentru a lucra la o aplicație, avem nevoie să deschidem într-un tab documentul sursă pe care îl gestionăm, într-un tab avem un shell în care compilăm aplicația și într-un tab rulăm comenzi de verificare a aplicației. Având fiecare tab cu sarcina sa desemnată, păstrăm un istoric mai curat și mai ușor de interpretat. Astfel vom reduce numărul de comenzi pe care le rulăm pentru a reporni editorul de text, pentru a schimba directul de lucru etc.

Pentru a porni un nou tab folosim combinația de taste `Ctrl+v c`.

În bara de taburi din terminal a apărut un nou tab cu denumirea `1:bash*`.

Fiecare tab are propriul identificator și propriul nume. De exemplu, `Ctrl+v 1` ne va plasa pe primul tab, iar `Ctrl+v 2` ne va plasa pe al doilea tab.



Odată ce se schimbă tabul activ se schimbă și sublinierea tabului activ din bara de taburi.

Exerciții

1. Creați două sesiuni de `tmux`.
2. În prima sesiune creată deschideți două taburi. Rulați în primul tab comanda `htop` și în al doilea tab deschideți fișierul `/etc/passwd` folosind editorul de text `nano`.
3. În a doua sesiune creată deschideți trei taburi. Rulați în primul tab comanda `sudo apt-get update`, rulați în al doilea terminal comanda `iostat -x 2 5` și în al treilea tab rulați comanda `tail -f /var/log/syslog`. Reveniți la primul tab din sesiune.

Configurarea stației la distanță

După ce am realizat o conexiune către stația pe care vom lucra, trebuie să ne pregătim mediul de lucru. Configurația mediului de lucru depinde de propriile gusturi și aplicațiile pe care le folosește fiecare.

Configurarea mediului de lucru este recomandată pentru a adapta sistemul la propriile nevoi. De exemplu, o persoană care va lucra foarte mult cu repository-uri de tip Git, probabil va avea nevoie de un prompt care să îi afișeze pe ce branch lucrează, astfel încât să gestioneze mai ușor branchurile. Aceste modificări au rolul de a reduce acțiunile repetitive pe care le facem sau de a pregăti aplicații pe care le vom folosi mai departe.

Această secțiune cuprinde recomandări de configurare a sistemului de la distanță. Au formă de sugestii, nu sunt obligații, pe baza cărora fiecare poate decide pentru configurarea propriului mediu de lucru.

Configurarea shellului

Pentru acest demo vom lucra pe stația locală `uso`.

Primul aspect pe care îl vom modifica la mediul de lucru este shellul în care rulăm comenzi, deoarece acesta este cea mai folosită unealtă. Fie că că editâm cod, sau administrăm sisteme, acesta este locul unde rulăm comenzi.

Modificările la nivelul shellului se fac schimbând variabile de mediu sau rulând comenzi înainte de rularea efectivă a shellului.

Modificarea mediului shellului o realizăm într-un fișier de configurare. Vom folosi fișierul `~/.profile`, deoarece acesta este citit și rulat de toate implementările de shell majorore, cum ar fi `dash`, `csh`, `bash` sau `zsh`, astfel oferă intercompatibilitate între shelluri.

Modificarea shellului predefinit

Ne dorim să modificăm shellul predefinit pe care îl folosește sistemul, deoarece fiecare implementare de shell oferă propriile funcționalități. De exemplu, fiecare dintre shellurile `csh`, `bash` și `zsh` implementează un mecanism de auto-completion diferit, astfel completarea se face în mod diferit atunci când apăsăm tasta `Tab`.

Pentru a modifica shellul predefinit al unui utilizator folosim comanda `usermod` cu opțiunea `-s` în felul următor:

```
student@uso:~$ sudo apt-get install zsh
[...]
```

```
student@uso:~$ sudo usermod -s /bin/zsh student
student@uso:~$ su - student
This is the Z Shell configuration function for new users,
[...]
--- Type one of the keys in parentheses --- 2
/home/student/.zshrc:15: scalar parameter HISTFILE created globally in function zsh-newuser-install
(eval):1: scalar parameter LS_COLORS created globally in function zsh-newuser-install
student@uso ~ % echo $SHELL
/bin/zsh
```

Rulând comanda de mai sus, am modificat shellul predefinit al utilizatorului `student` în `/bin/zsh`. Pentru verificare, ne-am autentificat ca utilizatorul `student`. La prima autentificare `zsh` ne-a întrebat ce fel de configurare vrem, iar noi am folosit opțiunea `2`, pentru generarea automată a unei configurări. Am afișat valoarea variabilei `SHELL` pentru a afișa calea către shellul folosit.

Exercițiu: Modificarea shellului predefinit:

Modificați shellul predefinit al utilizatorului `student` cu shellul `/bin/bash`. Încercați să folosiți funcționalitatea de auto-complete. Ce observați?

Configurarea aliasurilor

Există comenzi pe care le rulăm frecvent. Totuși, unele comenzi sunt lungi și durează mult timp să le tastăm de fiecare dată atunci când vrem să le rulăm. Pentru a rezolva această problemă, putem folosi aliasuri. Un alias este o prescurtare pentru o comandă.

Vom folosi comanda `alias` pentru defini un alias.

```
student@uso:~$ alias gs="git status"
student@uso:~$ cd uso-lab/
student@uso:~/uso-lab$ gs
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
student@uso:~/uso-lab$ alias
[...]
alias gs='git status'
alias gti='git'
alias l='ls -CF'
alias la='ls -A'
alias ll='ls -alF'
alias ls='ls --color=auto'
```

Am definit aliasul `gs` pentru comanda `git status` și am verificat-o în repository-ul `uso-lab`.

Pentru a verifica ce aliasuri sunt definite, vom folosi comanda `alias` fără parametri.

Definirea de mai sus a unui alias nu este persistentă, ci acesta va fi definit cât timp shellul curent este deschis. Pentru a defini un alias în mod persistent, trebuie să îl definim, folosind comanda `alias` într-un fișier de configurare, cum ar fi `.profile`.

Exercițiu: Configurarea aliasurilor

1. Configurați aliasul `gcs` pentru comanda `git commit --signnoff`.
2. Configurați aliasul `glog` pentru comanda `git log --oneline`.

Modificarea dimensiunii istoricului de comenzi

Fișierul de istoric al unui shell este locul unde sunt salvate comenzile rulate într-un shell până la închiderea sesiunii de shell. Acest fișier este folositor pentru repetarea acțiunilor.

De exemplu, funcționalitatea de căutare inversă din shell bazată pe `Ctrl+r` caută în istoricul shellului, așa că este de preferat să avem un istoric complet în care să căutăm pentru a avea mai multe intrări în care să căutăm.

Pentru a mări dimensiunea istoricului shellului, este suficient să setăm variabila de mediu `HISTSIZE` într-un fișier de configurare. Vom folosi fișierul de configurare `.profile` din motivele enumerate mai sus, aflat în directorul home al utilizatorului `student`. Această modificare va fi valabilă doar pentru utilizatorul `student`.

```
student@uso:~$ echo HISTSIZE=20000 >> /home/student/.profile
```

Exercițiu: Modificarea dimensiunii istoricului de comenzi

Modificarea de mai sus nu este suficientă, deoarece aceasta schimbă doar dimensiunea istoricului din shell, care este salvat în fișierul de istoric la închiderea sesiunii de shell. La pornirea shellului, dimensiunea fișierului de istoric este concatenată folosind variabila `HISTFILESIZE`. Valoarea predefinită a acestei variabile este `500`.

Faceți modificările necesare astfel încât fișierul de istoric să fie concatenat la `20000` de comenzi la pornirea shellului.

Configurarea promptului

Promptul este o sursă importantă de informații. Acesta ne poate oferi mai multă informație și ne eliberează din a rula anumite comenzi. Practic, noi putem obține mai mult informații rulând mai puține comenzi.

De exemplu, dacă lucrăm foarte des cu repository-uri de Git, vrem să avem un mod cât mai facil de a vedea pe ce branch lucrăm. Această informație poate fi adăugată în prompt.

Promptul shellului `bash` este setat folosind variabila `PS1`. Orice șir de caractere va fi scris în variabila `PS1` și va fi afișat înainte de zona în care introducem comenzi. Dacă modificăm variabila `PS1` vom vedea că promptul se modifică:

```
student@uso:~$ echo $PS1
\[\e]0;\u@\h: \w\a\]${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$
student@uso:~$ PS1="president@white-house:~$ "
president@white-house:~$ hostname
uso
president@white-house:~$ id
uid=1000(student) gid=1000(student) groups=1000(student),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),997(docker)
president@white-house:~$
```

După cum observăm în rezultatul primei comenzi rulate mai sus, valoarea variabilei `PS1` este un șir de caractere complex, dar noi putem să îl suprascriem. Odată suprascrisă variabila `PS1`, promptul se schimbă în valoarea din variabilă. Totuși, am rulat comenzi de verificare pentru a vedea că utilizatorul cu care suntem autentificați este în continuare `student` și stația la care suntem conectați este `uso`

Putem să generăm propriul prompt complex folosind utilitare online. Recomandăm EZPrompt [https://ezprompt.net/] . Acest site are funcționalitatea de a genera un prompt modificat. Vrem să generăm un prompt de forma `username@hostname:path_to_current_dir-git_branch`. EZPrompt a generat următoarele comenzi pentru a modifica promptul, pe care le vom adăuga la finalul fișierului `.profile`:

```
# get current branch in git repo
function parse_git_branch() {
    BRANCH=`git branch 2> /dev/null | sed -e '/^[^*]/d' -e 's/* \(.*\)/\1/'`
    if [ ! "${BRANCH}" == "" ]
    then
        STAT=`parse_git_dirty`
        echo "[${BRANCH}${STAT}]"
    else
        echo ""
    fi
}
# get current status of git repo
function parse_git_dirty {
    status=`git status 2>&1 | tee`
    dirty=`echo -n "${status}" 2> /dev/null | grep "modified:" &> /dev/null; echo "$?"`
    untracked=`echo -n "${status}" 2> /dev/null | grep "Untracked files" &> /dev/null; echo "$?"`
    ahead=`echo -n "${status}" 2> /dev/null | grep "Your branch is ahead of" &> /dev/null; echo "$?"`
    newfile=`echo -n "${status}" 2> /dev/null | grep "new file:" &> /dev/null; echo "$?"`
    renamed=`echo -n "${status}" 2> /dev/null | grep "renamed:" &> /dev/null; echo "$?"`
    deleted=`echo -n "${status}" 2> /dev/null | grep "deleted:" &> /dev/null; echo "$?"`
    bits=''
    if [ "${renamed}" == "0" ]; then
        bits=">${bits}"
    fi
    if [ "${ahead}" == "0" ]; then
        bits="*${bits}"
    fi
    if [ "${newfile}" == "0" ]; then
        bits="+${bits}"
    fi
    if [ "${untracked}" == "0" ]; then
        bits="?${bits}"
    fi
    if [ "${deleted}" == "0" ]; then
        bits="x${bits}"
    fi
    if [ "${dirty}" == "0" ]; then
        bits="!${bits}"
    fi
    if [ ! "${bits}" == "" ]; then
        echo " ${bits}"
    else
        echo ""
    fi
}
export PS1="\u@\h:\w-\`parse_git_branch\` "
```

Pornind un nou shell, vedem că promptul s-a schimbat, acum nu mai este colorat. Când schimbăm directorul curent într-un repository Git, în prompt va apărea și branch-ul pe care este setată replica repository-ului

```
student@uso:~- cd uso-lab/
student@uso:~/uso-lab-[master !?] check-language-support ^C
```

Exercițiu: Configurarea promptului

Se întâmplă ca atunci când lucrăm cu ierarhii de fișiere cu multe directoare, să nu încapă comenzile pe un singur rând. Acest lucru este deranjant, deoarece comenzile devin greu de urmărit.

Pentru a rezolva această problemă, vrem să avem promptul pe un rând și spațiul unde introducem textul pe următorul rând.

Modificați promptul astfel încât comenzile rulate să apară pe următorul rând față de prompt.

Gestionarea spațiului de stocare partajat

Pentru a parcurge această secțiune este recomandat să descărcați ultima versiune a respository-ului laboratorului. Pentru a descărca ultima versiune a repository-ului, rulați comanda `git pull` în directorul `~/uso-lab/labs/09-task-admin/lab-container/`.

Infrastructura laboratorului este bazată pe containere Docker ale căror imagini vor fi generate pe propriul calculator. Dacă nu aveți deja instalat Docker Engine pe sistem, scriptul `~/uso-lab/labs/09-task-admin/lab-container/lab_prepare.sh` vă va instala aplicația.

După ce ați terminat de lucrat, vă recomandăm să opriți containerele rulând comanda `./lab_prepare.sh delete` în directorul `~/uso-lab/labs/09-task-admin/lab-container/`.

O componentă importantă a mediului de lucru este spațiul de stocare. Cu toate că vom rula aplicații pe serverul de la distanță, avem nevoie de acces la spațiul de stocare al acestuia, deoarece vrem ca, într-un final, să urmărim rezultatul procesării și, eventual, să îl analizăm folosind utilitare grafice dedicate. O altă nevoie pe care o avem este editarea codului la distanță, deoarece majoritatea programatorilor folosesc IDE-uri în mediu grafic, care nu pot rula mereu eficient de la distanță. Soluția la această nevoie este să partajăm spațiul de stocare între serverul `remote` și stația `local` de pe care lucrăm.

Stocare partajată folosind SSHFS

Pentru rularea acestui demo, rulați în directorul `~/uso-lab/labs/09-task-admin/lab-container/` comanda `./lab_prepare.sh install remote` și comanda `./lab_prepare.sh install ssh-server`. Pentru a ne conecta la infrastructura pentru această secțiune, vom folosi comanda `./lab_prepare.sh connect local`.

SSHFS este o soluție de stocare partajată care permite montarea unui sistem de fișiere care nu este legat fizic la stația `local`, ci se folosește de protocolul SSH pentru a transmite operațiile asupra fișierelor prin rețea către un sistem de fișiere conectat prin rețea.

Avantajul folosirii SSHFS este că nu necesită descărcarea sistemului de fișiere de la distanță, deci nu duce la duplicarea fișierelor. Dezavantajul acestei abordări este că dacă pierdem conexiunea la sistemul de fișiere de la distanță, nu mai avem acces la fișiere.

Montarea temporară a unui sistem de fișiere

Pentru a monta sistemul de fișiere de pe un alt sistem, vom folosi comanda `sshfs`

```
root@local:~# ls /mnt/

root@local:~# sshfs root@10.11.11.2:/ /mnt/
The authenticity of host '10.11.11.2 (10.11.11.2)' can't be established.
ECDSA key fingerprint is SHA256:xV1orHYj4fhkc5HE91sfh8QhaVqke/AEMa8mYI423HY.
Are you sure you want to continue connecting (yes/no)? yes
root@10.11.11.2's password:

root@local:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
overlay          16G   14G  539M  97% /
tmpfs            64M     0   64M   0% /dev
tmpfs           2.0G     0  2.0G   0% /sys/fs/cgroup
/dev/sda5        16G   14G  539M  97% /etc/hosts
shm              64M     0   64M   0% /dev/shm
root@10.11.11.2:/   16G   14G  539M  97% /mnt

root@local:~# ls /mnt/
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
```

Comanda de mai sus a montat ierarhia de fișiere cu rădăcina în directorul `/` de pe sistemul de la adresa IP `10.11.11.2` în directorul `/mnt` de pe stația locală, cu numele `local`, autentificându-se ca utilizatorul `root`.

Am folosit comanda `df` pentru a afișa informații despre toate sistemele de fișiere montate pe stația locală. Observăm că pe ultima linie apare conexiunea către stația de la adresa `10.11.11.2`.

Sintaxa comenzii `sshfs` se aseamănă cu sintaxa comenzii `scp`.

Acest mod de montare a sistemului de fișiere este temporară. Atunci când vom opri stația, sistemul de fișiere va fi demontat.

Exercițiu: Montarea temporară a unui sistem de fișiere

1. Montați temporar sistemul de fișiere cu rădăcina în directorul `/` de pe stația `10.11.11.2` în directorul `/mnt/vol1`.
2. Montați temporar sistemul de fișiere cu rădăcina în directorul `/home/student` de pe stația `10.11.11.2` în directorul `/mnt/vol2`.

Montarea persistentă a unui sistem de fișiere

Deoarece nu ne dorim să rulăm comanda `sshfs` atunci când vrem să folosim un sistem de fișiere la distanță, vrem să montăm persistent sistemul de fișiere de la distanță, astfel încât această montare să persiste după oprirea stației locale.

Ca să montăm persistent sistemul de fișiere avem nevoie să copiem cheia SSH pe stația de la distanță, deoarece montarea se va face în mod neinteractiv, deci nu vom avea posibilitatea de a introduce parola, așa cum am descoperit în laboratorul Configurarea avansată a sistemului.

Pentru a monta persistent sistemul de fișiere, vom scrie o intrare în fișierul `/etc/fstab`, care va conține detalii despre sistemul de fișiere pe care vrem să îl montăm. Pentru a monta sistemul de fișiere `/` de pe sistemul de la adresa IP 10.11.11.2 în directorul /mnt

de pe stația locală, autentificându-ne ca utilizatorul root`, vom folosi următoarele comenzi:

```
root@local:~# ssh-keygen
[...]
root@local:~# ssh-copy-id root@10.11.11.2
[...]
root@local:~# echo "root@10.11.11.2:/  /mnt  fuse.sshfs  defaults  0  0" >> /etc/fstab

root@local:~# mount -a

root@local:~# df -h
Filesystem         Size  Used Avail Use% Mounted on
overlay             16G   14G  539M  97% /
tmpfs               64M     0   64M   0% /dev
tmpfs              2.0G     0  2.0G   0% /sys/fs/cgroup
/dev/sda5           16G   14G  539M  97% /etc/hosts
shm                 64M     0   64M   0% /dev/shm
root@10.11.11.2:/   16G   14G  539M  97% /mnt
```

Am scris în fișierul `/etc/fstab` folosind comanda `echo`, iar pentru a monta sistemul de fișiere am folosit comanda `mount` cu opțiunea `-a` pentru montarea sistemelor de fișiere descrise în fișierul `/etc/fstab`.

Atenție!:

În mod normal am scrie în fișierul `/etc/fstab` folosind un editor de text. Vă recomandăm să nu scrieți în fișiere critice sistemului folosind redirectări, deoarece orice greșeală poate șterge conținutul fișierului.

Exercițiu: Montarea persistentă a unui sistem de fișiere

1. Montați persistent sistemul de fișiere cu rădăcina în directorul `/` de pe stația `10.11.11.2` în directorul `/mnt/vol1`.
2. Montați persistent sistemul de fișiere cu rădăcina în directorul `/home/student` de pe stația `10.11.11.2` în directorul `/mnt/vol2`.

Stocare partajată folosind aplicații online

SSHFS nu este o soluție bună pentru a face backup fișierelor deoarece, existând o singură replică, ștergerea locală a unui fișier duce la ștergerea sa și de pe stația remote și, astfel, la pierderea sa. Pe lângă aceasta, dacă stația locală are conexiune slabă la Internet, accesul la fișiere este greoi și neresponsiv. Suplimentar, trebuie configurată o conexiune SSH, care poate necesita la rândul ei existența unui tunel etc.

O alternativă la folosirea SSHFS sunt soluții cum ar fi GoogleDrive, Dropbox, ownCloud sau One Drive. Aceste soluții stochează o replică a fișierului pe toate calculatoarele autentificate de pe un anumit cont. Un alt avantaj al acestora este că oferă suport pentru controlul versiunilor pentru a șterge modificarea anterioară. Cu dezavantajul că trebuie configurate și cu riscul apariției conflictelor la modificări simultane pe noduri diferite. Și cu dezavantajul că acum informația este duplicată: dublu spațiu ocupat și pot apărea conflicte la modificări.

Stocarea partajată folosind Dropbox

Pentru rularea acestui demo vom folosi direct mașina virtuală `uso`.

Dropbox este o soluție care se folosește de un server în Internet care stochează fișierele, ca apoi acestea să fie replicate pe fiecare calculator client.

Este necesară crearea și activarea un cont pentru a folosi serviciul Dropbox.

Dropbox oferă o aplicație care rulează în linie de comandă pe care o vom descărca pentru a sincroniza sistemul de fișiere de pe serverele Dropbox într-un director local.

Vom descărca aplicația Dropbox folosind comanda `wget` și o vom instala folosind comanda `dpkg` împreună cu parametrul `-i`.

```
student@uso:~$ wget https://www.dropbox.com/download?dl=packages/ubuntu/dropbox_2020.03.04_amd64.deb -O dropbox.deb
[...]
student@uso:~$ sudo dpkg -i dropbox.deb
Starting Dropbox...
Dropbox is the easiest way to share and store your files online. Want to learn more? Head to https://www.dropbox.com/

In order to use Dropbox, you must download the proprietary daemon. [y/n] y
[...]
student@uso:~$ dropbox start
To link this computer to a Dropbox account, visit the following url:
https://www.dropbox.com/cli_link_nonce?nonce=ffd7d648a2ca2302d1177c0c389e87bd
```

Observație

Am folosit opțiunea `-O` a comenzii `wget` pentru a salva fișierul descărcat cu numele `dropbox.deb`.

Pentru a instala ultimele componente ale clientului Dropbox este necesar să rulăm comanda `dropbox start -i`. După ce am ponit aplicația, este necesar să înregistrăm stația online la contul nostru. Vom face acest lucru accesând linkul afișat de aplicație.

Exercițiu: Stocarea partajată folosind Dropbox

Pentru rezolvarea acestui exercițiu, rulați în directorul `~/uso-lab/labs/09-task-admin/lab-container/` comanda `./lab_prepare.sh install dropbox`. Pentru a ne conecta la infrastructura pentru această secțiune, vom folosi comanda

`./lab_prepare.sh connect dropbox`.

1. Conectați-vă la stația `dropbox` și porniți aplicația Dropbox pe aceasta.
2. Creați un fișier numit `hello.txt` în directorul `~/Dropbox`, partajat de cele două mașini. Scrieți în fișier mesajul `Hello from remote` pe stația `dropbox`. Verificați că există fișierul `hello.txt` în directorul `~/Dropbox` pe stația `uso`.

Configurarea și administrarea serviciilor

Scopul serviciilor este să ruleze în continuu și să primească cereri de la aplicații client, cum ar fi un client SSH, și să răspundă la aceste cereri. Serviciile sunt aplicații care rulează în continuu pe stație, spre deosebire de comenzi, cum ar fi `find` sau `ls`, care rulează cât timp se execută comanda. Deoarece ne dorim să avem majoritatea timpului conectivitate la mașină, sau ne dorim să avem acces la paginile web folosite, serverele rulează fără oprire și servesc cereri venite de la clienți. Un sever / serviciu este oprit explicit de utilizator prin comenzi specifice sau este oprit la oprirea sistemului. Fără o intervenție explicită și fără oprirea sistemului, un server / serviciu va rula nedefinit.

Vrem să pornim și să configurăm servicii instalate în sistem, precum:

- un server web;
- un server SSH, pentru a ne conecta la stație;
- un serviciu care ține un tunel SSH deschis către stație

Avem nevoie de o interfață unică, cu o sintaxă minimală, care ne permite să gestionăm serviciile pe sistem, și care ne permite sa generăm noi propriile servicii.

Lucrul cu serviciile în Linux

Comanda folosită pentru gestionarea serviciilor este `systemctl`. Aceasta se regăsește pe majoritatea distribuțiilor moderne de Linux.

Verificarea stării unui serviciu

Atunci când instalăm o aplicație care rulează sub forma unui serviciu, vom putea să îi verificăm starea folosind comanda `systemctl status`.

```
student@uso:~$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
     Active: active (running) since Sat 2021-01-09 04:22:11 EET; 1min 17s ago
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 639 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 657 (sshd)
      Tasks: 1 (limit: 4656)
     Memory: 4.8M
     CGroup: /system.slice/ssh.service
             └─657 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups

ian 09 04:22:11 uso systemd[1]: Starting OpenBSD Secure Shell server...
ian 09 04:22:11 uso sshd[657]: Server listening on 0.0.0.0 port 22.
ian 09 04:22:11 uso sshd[657]: Server listening on :: port 22.
ian 09 04:22:11 uso systemd[1]: Started OpenBSD Secure Shell server.
ian 09 04:23:25 uso sshd[1821]: Accepted password for student from 192.168.56.1 port 33714 ssh2
ian 09 04:23:25 uso sshd[1821]: pam_unix(sshd:session): session opened for user student by (uid=0)
```

Mai sus am afișat starea serviciului SSH. Observăm că acesta este activ și putem urmări ce procese a generat și ce mesaje de diagnosticare afișează.

Programul `systemd` se ocupă de pornirea, oprirea și gestionarea serviciilor folosite în sistem pe baza unor fișiere numite fișiere unitate. Fișierul care gestionează serviciul SSH este `/lib/systemd/system/ssh.service`. Îl putem identifica din rezultatul comenzii `systemctl status`.

Atunci când un serviciu este instalat, acesta vine la pachet cu fișierul cu extensia `.service` cu care acesta va fi gestionat de `systemd`, după cum putem vedea în rezultatul comenzii de mai jos:

```
student@uso:~$ sudo apt install ntp
[...]
Setting up ntp (1:4.2.8p12+dfsg-3ubuntu4) ...
Created symlink /etc/systemd/system/network-pre.target.wants/ntp-systemd-netif.path → /lib/systemd/system/ntp-systemd-netif.path.
Created symlink /etc/systemd/system/multi-user.target.wants/ntp.service → /lib/systemd/system/ntp.service.
ntp-systemd-netif.service is a disabled or a static unit, not starting it.
[...]
```

Serviciul `ntp` este folosit pentru sincronizarea ceasului cu surse precise de timp din Internet. Acesta este un serviciu important pentru buna funcționare a aplicațiilor în Internet, deoarece o configurare a timpului pentru o stație poate duce la o funcționare incorectă a acesteia în comunicare.

Exercițiu: Afișarea stării serviciilor

Afișați starea serviciului `thermald`, care gestionează senzorii de temperatură ai sistemului.

Oprirea unui serviciu

Pentru a opri un serviciu vom folosi comanda `systemctl stop` în felul următor:

```
student@uso:~$ sudo netstat -tlpn
[sudo] password for student:
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      501/systemd-resolve
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      4425/sshd: /usr/sbi
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN      541/cupsd
tcp6       0      0 :::22                   :::*                    LISTEN      4425/sshd: /usr/sbi
tcp6       0      0 ::1:631                 :::*                    LISTEN      541/cupsd
student@uso:~$ sudo systemctl stop ssh
[sudo] password for student:
student@uso:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: e>
     Active: inactive (dead) since Sat 2021-01-09 04:58:28 EET; 9s ago
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 640 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
    Process: 660 ExecStart=/usr/sbin/sshd -D $SSHD_OPTS (code=exited, status=0/>
   Main PID: 660 (code=exited, status=0/SUCCESS)

ian 09 04:57:29 uso systemd[1]: Starting OpenBSD Secure Shell server...
ian 09 04:57:29 uso sshd[660]: Server listening on 0.0.0.0 port 22.
ian 09 04:57:29 uso sshd[660]: Server listening on :: port 22.
ian 09 04:57:29 uso systemd[1]: Started OpenBSD Secure Shell server.
ian 09 04:58:28 uso sshd[660]: Received signal 15; terminating.
ian 09 04:58:28 uso systemd[1]: Stopping OpenBSD Secure Shell server...
ian 09 04:58:28 uso systemd[1]: ssh.service: Succeeded.
ian 09 04:58:28 uso systemd[1]: Stopped OpenBSD Secure Shell server.
```

Pentru a verifica dacă mai funcționează serviciul SSH vom folosi comanda `netstat` pentru a verifica dacă mai ascultă serverul SSH pe portul 22.

```
student@uso:~$ sudo netstat -tlpn
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      501/systemd-resolve
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN      541/cupsd
tcp6       0      0 ::1:631                 :::*                    LISTEN      541/cupsd
```

Observăm că nu ascultă niciun program pe portul 22.

Recapitulare: Afișarea stării serviciilor

Afișați starea serviciului `ssh`.

Pornirea unui serviciu

Pornirea unui serviciu se face folosind comanda `systemctl start` în felul următor:

```
student@uso:~$ systemctl start ssh
student@uso:~$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
     Active: active (running) since Sat 2021-01-09 05:05:30 EET; 2min 30s ago
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 4408 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 4425 (sshd)
      Tasks: 1 (limit: 4656)
     Memory: 3.4M
     CGroup: /system.slice/ssh.service
             └─4425 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups

ian 09 05:05:30 uso systemd[1]: Starting OpenBSD Secure Shell server...
ian 09 05:05:30 uso sshd[4425]: Server listening on 0.0.0.0 port 22.
ian 09 05:05:30 uso sshd[4425]: Server listening on :: port 22.
ian 09 05:05:30 uso systemd[1]: Started OpenBSD Secure Shell server.
ian 09 05:05:54 uso sshd[4477]: Accepted password for student from 192.168.56.1 port 34852 ssh2
ian 09 05:05:54 uso sshd[4477]: pam_unix(sshd:session): session opened for user student by (uid=0)
```

Dacă serviciul nu pornește cu succes, aceasta va afișa un mesaj de avertizare.

Repornirea unui serviciu

Pe parcursul rulării serviciilor pe un sistem, vom dori să schimbăm configurația unui serviciu sau să adăugăm parametri de rulare în plus. Majoritatea aplicațiilor citesc fișierele de configurare o singură dată, atunci când sunt pornite, deci orice modificare ulterioară a fișierelor de configurare nu va fi sesizată de serviciu. Din acest motiv, avem nevoie să repornim servicii.

De exemplu, vrem să permitem utilizatorilor să se autentifice ca utilizatorul `root` pe sistemul nostru. Acest lucru este dezactivat în mod predefinit de serviciu din motive de securitate.

```
student@uso:~$ ssh root@localhost
root@localhost's password:
```

```
Permission denied, please try again.
[...]
```

Observăm că inițial nu puteam să ne conectăm la mașina locală ca utilizatorul `root`, cu toate că introduceam parola corectă.

```
student@uso:~$ sudo su -c 'echo "PermitRootLogin yes" >> /etc/ssh/sshd_config'
student@uso:~$ systemctl restart ssh
student@uso:~$ ssh root@localhost
root@localhost's password:
[...]
root@uso:~#
```

Pentru a face acest lucru, avem nevoie să reconfigurăm serviciul SSH. Odată ce am adăugat opțiunea `PermitRootLogin yes` în fișierul de configurare al serviciului și am repornit serviciului, am reușit să ne autentificăm ca utilizatorul `root`.

Pornirea unui serviciu la startup

Atunci când configurăm un sistem și vrem să definim servicii care rulează pe el, ne dorim ca serviciile să fie `setup and forget`, să nu fie necesar să le supraveghem și să le monitorizăm prea mult. Un mod de a ne ușura lucrul cu serverul este pornirea serviciilor la startup, ca să nu fie nevoie sa intervenim noi după secvența de pornire a sistemului, ca să le pornim de mână, folosind comanda `systemctl start`

Putem să vedem dacă un sistem pornește automat la startup din rezultatul comenzii `systemctl status` pe linia care începe cu `Loaded:`. Dacă rezultatul conține șirul de caractere `enabled`, atunci serviciul pornește la startup, iar dacă aceasta conține șirul `disabled`, atunci serviciul nu va porni la startup.

Pentru a dezactiva un serviciu la startup vom folosi comanda `systemctl disable` în felul următor:

```
student@uso:~$ sudo systemctl disable ntp
Synchronizing state of ntp.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install disable ntp
Removed /etc/systemd/system/multi-user.target.wants/ntp.service.
student@uso:~$ sudo systemctl status ntp
    ● ntp.service - Network Time Service
     Loaded: loaded (/lib/systemd/system/ntp.service; disabled; vendor preset: enabled)
     Active: active (running) since Sat 2021-01-09 05:18:49 EET; 27min ago
       Docs: man:ntpd(8)
   Main PID: 9756 (ntpd)
      Tasks: 2 (limit: 4656)
     Memory: 1.3M
     CGroup: /system.slice/ntp.service
             └─9756 /usr/sbin/ntpd -p /var/run/ntpd.pid -g -u 127:134

ian 09 05:18:55 uso ntpd[9756]: Soliciting pool server 185.173.16.132
ian 09 05:24:31 uso ntpd[9756]: kernel reports TIME_ERROR: 0x2041: Clock Unsynchronized
ian 09 05:28:41 uso ntpd[9756]: 91.189.91.157 local addr 10.0.2.15 -> <null>
ian 09 05:29:00 uso ntpd[9756]: 85.204.240.2 local addr 10.0.2.15 -> <null>
ian 09 05:29:04 uso ntpd[9756]: 188.213.49.35 local addr 10.0.2.15 -> <null>
ian 09 05:29:05 uso ntpd[9756]: 86.124.75.41 local addr 10.0.2.15 -> <null>
ian 09 05:29:08 uso ntpd[9756]: 85.204.240.1 local addr 10.0.2.15 -> <null>
ian 09 05:29:09 uso ntpd[9756]: 195.135.194.3 local addr 10.0.2.15 -> <null>
ian 09 05:29:52 uso ntpd[9756]: 91.189.89.198 local addr 10.0.2.15 -> <null>
ian 09 05:38:03 uso ntpd[9756]: 93.190.144.28 local addr 10.0.2.15 -> <null>
```

Observăm că pe linia care conține șirul de caractere `Loaded`, mesajul este `disabled`, dar serviciul funcționează în continuare.

Pentru a activa un serviciu la startup vom folosi comanda `systemctl enable`:

```
student@uso:~$ sudo systemctl enable ntp
Synchronizing state of ntp.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable ntp
Created symlink /etc/systemd/system/multi-user.target.wants/ntp.service → /lib/systemd/system/ntp.service.
student@uso:~$ sudo systemctl status ntp
● ntp.service - Network Time Service
     Loaded: loaded (/lib/systemd/system/ntp.service; enabled; vendor preset: enabled)
     Active: active (running) since Sat 2021-01-09 05:18:49 EET; 30min ago
       Docs: man:ntpd(8)
   Main PID: 9756 (ntpd)
      Tasks: 2 (limit: 4656)
     Memory: 1.3M
     CGroup: /system.slice/ntp.service
             └─9756 /usr/sbin/ntpd -p /var/run/ntpd.pid -g -u 127:134

ian 09 05:18:55 uso ntpd[9756]: Soliciting pool server 185.173.16.132
ian 09 05:24:31 uso ntpd[9756]: kernel reports TIME_ERROR: 0x2041: Clock Unsynchronized
ian 09 05:28:41 uso ntpd[9756]: 91.189.91.157 local addr 10.0.2.15 -> <null>
ian 09 05:29:00 uso ntpd[9756]: 85.204.240.2 local addr 10.0.2.15 -> <null>
ian 09 05:29:04 uso ntpd[9756]: 188.213.49.35 local addr 10.0.2.15 -> <null>
ian 09 05:29:05 uso ntpd[9756]: 86.124.75.41 local addr 10.0.2.15 -> <null>
ian 09 05:29:08 uso ntpd[9756]: 85.204.240.1 local addr 10.0.2.15 -> <null>
ian 09 05:29:09 uso ntpd[9756]: 195.135.194.3 local addr 10.0.2.15 -> <null>
ian 09 05:29:52 uso ntpd[9756]: 91.189.89.198 local addr 10.0.2.15 -> <null>
ian 09 05:38:03 uso ntpd[9756]: 93.190.144.28 local addr 10.0.2.15 -> <null>
```

Observăm că pe linia care conține șirul de caractere `Loaded`, mesajul s-a schimbat în `enabled`.

Configurarea unui serviciu

În majoritatea cazurilor, serviciile sunt configurate prin fișiere de configurare. Acestea se pot găsi în mai multe locuri:

- `/etc/default/`, unde sunt fișiere care permit modificarea opțiunilor de rulare a unei comenzi, ca și cum am adăuga noi opțiuni la rularea unei comenzi. Aceste fișiere sunt citite de `systemd`, înainte să pornească serviciul;
- directorul `/etc/nume_serviciu/` , unde se află fișierele de configurare sau direct fișierul `/etc/nume_serviciu`. Aceste fișiere sunt citite și interpretate de servici. De exemplu, pentru configurarea serviciului NTP, există fișierul `/etc/ntp.conf`, iar pentru configurarea serviciului SSH folosim fișierele din directorul `/etc/ssh/`.

Exerciții: Gestiunea serviciilor

1. Ordered List ItemConfigurați serviciul SSH pentru a permite autentificarea ca utilizatorul `root` folosind numai autentificare bazată pe chei.
2. Instalați serviciul `vsftpd`. Acesta este un serviciu de transfer de fișiere.Realizați modificările necesare astfel încât acest serviciu să **NU** pornească la startup; Dezactivați funcționalitatea bazată pe IPv6 a serviciului. **Hint**: `listen_ipv6`, `listen`. Asigurați-vă că serviciul rulează. Acest serviciu ascultă pe portul `21`.

Definirea unui serviciu personalizat

Atunci când vrem să ne conectăm la Internet printr-un proxy, avem nevoie ca proxy-ul să fie deschis în permanență și să redeschidă conexiunea în cazul în care aceasta este întreruptă. Putem să facem acest lucru folosind servicii `systemd` pe care să le gestionăm noi folosind suita de comenzi `systemctl`.

Ne propunem să găzduim propriul proxy care va trimite mesaje criptate prin SSH către o altă stație, de unde vor fi trimise mesaje în Internet. Astfel vom putea trece peste anumite filtre care țin cont de poziția geografică.

Un serviciu în `systemd` se definește printr-un fișier de configurare în directorul `/lib/systemd/system/libvirtd.service`. Pentru serviciul nostru vom genera fișierul la calea `/lib/systemd/system/libvirtd.service/auto-proxy.service`. Acest fișier va avea următorul conținut:

```
[Unit]
Description=Keeps a proxy to 'fep.grid.pub.ro' open
After=network-online.target ssh.service

[Service]
User=student
ExecStart=/usr/bin/ssh -D 1337 -q -C -N <username-acs>@fep.grid.pub.ro
ExecStop=/usr/bin/pkill -f '/usr/bin/ssh -D 1337 -q -C -N'
KillMode=process
Type=simple
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Acesta este un șablon pe care putem să îl folosim pentru multe tipuri de servicii. Opțiunile relevante pentru înțelegerea formatului sunt:

- `ExecStart`, comanda care se va executa pentru a porni serviciul;
- `ExecStop`, comanda care se va executa pentru a opri serviciul

Atenție

Pentru ca acest serviciu să funcționeze, este necesar să copiați cheia stației `uso` pe mașina `fep.grid.pub.ro`.

Putem să verificăm dacă a pornit proxy-ul verificând dacă ascultă vreun serviciu pe mașina locală pe portul `1337`:

```
student@uso:~$ sudo netstat -tlpn
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:1337          0.0.0.0:*               LISTEN      32177/ssh
tcp        0      0 0.0.0.0:21              0.0.0.0:*               LISTEN      15794/vsftpd
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      7088/systemd-resolv
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      10459/sshd: /usr/sb
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN      541/cupsd
tcp6       0      0 ::1:1337                :::*                    LISTEN      32177/ssh
tcp6       0      0 :::22                   :::*                    LISTEN      10459/sshd: /usr/sb
tcp6       0      0 ::1:631                 :::*                    LISTEN      541/cupsd
```

Exercițiu: Definirea unui serviciu personalizat

Realizați configurația necesară astfel încât să creați un tunel deschis permanent de pe stația `uso`, care primește mesaje din portul `4242` al stației de la adresa `10.11.11.3` și le trimite către portul `22` al mașinii `uso`.

Cuprins

- Laborator 11 (one page)
- Folosirea unui workstation remote
- Conectarea la workstation
- Configurarea stației la distanță

- Folosirea stației la distanță
- Gestionarea spațiului de stocare partajat
- Configurarea și administrarea serviciilor

uso/laboratoare/laborator-11.txt · Last modified: 2021/10/12 19:40 (external edit)